

Instructing Machines

Table of contents

How it works	1
Week 1	4
1 Course introduction	5
What the course is for	5
How the material teaches	5
The assistant and the arc it follows	6
Badges	7
The logbook	10
The projects, and who writes what	11
What to expect from yourself	11
2 Getting started	13
The course folder	13
The terminal	13
Installing pixi	16
2.1 Mac	17
2.2 Windows	17
Installing the course environment	17
The text editor	18
Opening the folder	18
When something goes wrong	19
You are all set	20
3 Hello world	21
Your first program	21
Error Messages	23
Strings	24
Comments	25
4 Notebooks	27
Scripts versus notebooks	27
The script you already know	27
Notebooks in VScode	27
Getting notebooks from the terminal	28
The notebook you are reading	29
Restart and run all	30
Python and IPython	30
When the kernel's memory gets in the way	32
The sandbox	32
The other magics in this book	34
Which one to use, and when	34

Two kinds of cells	35
Running a code cell	36
Order on the page and order in time	36
Adding, moving, copying and deleting cells	37
Choosing the kernel	38
Restarting the kernel	39
What a notebook is on disk	40
5 Meet the assistant	41
6 About Microsoft Copilot	43
Logging in	43
Keeping it in the browser	43
What to expect of it	44
7 Values, operators, logic	45
Logic	48
Variables	51
Different types of values	54
Mixed exercises	57
8 Precedence	61
The order of things	61
Statements and Expressions	63
Substitution and Reduction	64
General exercises	67
9 Insides	76
It's switches all the way down	76
The three parts that do the work	76
A machine that was only ever imagined	77
What the imagined machine has to do with yours	78
The operating system, the manager in the middle	78
Software is just files full of instructions	79
10 Instruction	80
Layers of instruction	80
AI	80
Week 2	84
11 Control flow	85
The <code>if</code> statement	85
Else-statement	87
Blocks of code	88
Elif-statement	89
General exercises	91
12 Functions	94
Functions	94
Functions can take arguments	99

Functions and variables	101
Builtin functions	103
General exercises	104
13 The AI predicts, the widget proves	111
Part 1 — Warm-up: arithmetic precedence (30 min)	111
Part 2 — Comparisons and boolean logic (35 min)	112
Part 3 — Short-circuit values, not just True/False (35 min)	113
Part 4 — Mixed types and strings (30 min)	114
Part 5 — Break the AI (30 min)	115
Wrap-up (10 min)	115
Week 3	116
14 How a model produces code, and why it can be wrong	117
Two new rungs on the ladder	118
15 Objects	120
Methods	120
Using the Python documentation	122
String formatting	122
Indexing and slicing strings	125
General exercises	130
16 Lists	132
Lists	132
Indexing and slicing lists	134
General exercises	139
17 Dictionaries	140
Dictionaries	140
General exercises	143
18 Tuples	145
Grouping values	145
Tuple assignment	146
19 Turtle, week 3: an object that keeps its own state	148
Getting a turtle on the screen	148
Commands and questions	150
Asking the turtle where it is	151
Two names, one turtle	153
Lists and dictionaries that describe a drawing	153
Week 4	155
20 Ask it to explain, then confirm by running	156
Part 1 — Syntax errors (30 min)	156
Part 2 — Runtime errors (40 min)	157
Part 3 — Explaining code that runs fine (35 min)	158

Part 4 — Catch the wrong explanation (35 min)	159
Part 5 — Bring your own (20 min)	159
Wrap-up (10 min)	160
21 Iteration	161
The for-loop	161
22 Testing	167
Why test your code?	167
Basic testing	167
The project testing utility	168
23 Turtle, week 4: loops, and the first tests	171
The same thing, several times	171
Loops inside loops	173
Checking with the machine instead of your eyes	174
 Week 5	 178
24 Practical AI use	179
25 Data structures	181
General exercises	187
26 Files	191
Writing files	191
Reading files	193
General exercises	195
27 Turtle, week 5: drawings described by data	196
A drawing as a list of steps	196
A drawing as a dictionary of instructions	197
A drawing as a matrix	198
Saving a drawing and getting it back	200
 Week 6	 203
28 Ask for another way	204
Part 1 — Warm-up, done together (20 min)	204
Part 2 — Two solutions, three problems (50 min)	204
Part 3 — Clever versus readable (30 min)	205
Part 4 — Defend your choice, in writing (30 min)	206
Part 5 — Ask for a third way (25 min)	206
Wrap-up (10 min)	206
29 Translating ORFs	207
Project files	207
Translating a single codon	207
Splitting an open reading frame into codons	208
Translating an open reading frame	209

30 Worked example: solving a problem by composition	211
31 Turtle, week 6: building a drawing out of pieces	215
Making the pieces	215
Judging between two versions	217
Two pens on one canvas	219
Comparing designs rather than code	220
 Week 7	 221
32 Let it draft a small piece you can read and test	222
Part 1 — Warm-up, done together (25 min)	222
Part 2 — Three small functions (60 min)	223
Part 3 — Catch the silent bug (35 min)	223
Part 4 — Drafting a method, not just a function (35 min)	224
Wrap-up (15 min)	224
 33 Classes	 225
You have been using objects all along	225
The type knows what an object can do	225
Classes define types	227
A point in a plane	227
Writing your first class	228
What is <code>self</code> and why do we need it?	229
Making an object	229
Getting at the data	231
How should it look as a string?	232
A method of your own	233
<code>self</code> and method calls	235
Distance to another point	235
Peeking behind the curtain, again	237
Making your own type feel like a real one	239
General exercises	240
What you learned	242
 34 Primer analysis	 243
Count the number of bases in your candidate primer	243
Compute the melting temperature	244
Reverse complement the sequence	244
Check for hairpins	245
 35 Debugging	 247
 36 Turtle, week 7: your own kind of object, and finding the bug	 250
An object of your own	250
Two bugs, on purpose	253
 Week 8	 257
 37 Reading and judging	 258

38 Tests are the contract	263
39 Pairwise alignment	268
Filling in the dynamic programming matrix	269
Reconstructing the optimal alignment	273
40 Turtle, week 8: retiring the turtle	276
The two ways looking fails	276
Looking is a filter, not a verdict	280
 Week 9	 284
41 The AI invents functions; the docs are the test	285
A method that does not exist	285
What actually caught it	286
The same failure outside libraries	286
42 Codon usage	287
Read an open reading frame and count its codons	288
Group codon counts by amino acid	290
Turn counts into frequencies	291
Compute the codon usage	293
43 Code in other files	294
A module is a file	294
What <code>import</code> actually does	295
The three ways to write it	297
You have been importing your own code all term	298
Modules that came with Python	298
A package is a folder of modules	299
Splitting a project of your own	300
What comes next	301
44 Documentation and resources	303
 Week 10	 306
45 Plan before you prompt	307
46 HIV sub-groups	311
Compute the similarity of two sequences	312
Read the HIV sequences into your program	313
Compare your HIV sequence to HIV sequences of known subtype	314
Compute maximum similarity to each subtype	315
Identify the HIV subtype	316
47 Recursion	317
What happens when you call a function	317
Recursion	318
Factorial	318

Why it works	319
Divide and conquer	322
Counting the leaves	323
What you learned	326
48 Worked example: decomposition modeled live	327
 Week 11	 330
49 Directing to resources: clinic	331
50 Delegate the full problem	334
51 Sequence trees	338
Measuring sequence distance	338
Lower triangular distance matrices	339
Generate a distance matrix	341
Clustering	341
Perform the clustering	343
 Week 12	 347
52 Learning with an AI; how you know a fact is true	348
53 Finding genes	350
Finding Open Reading Frames	351
Translation of open reading frames	355
Put everything together	355
54 Dataframes: a table you can plot	357
The table is just a text file	357
Handing the file to pandas	358
Pulling out a column	359
Asking a column a question	361
Counting the categories	362
More than one column at a time	362
Putting the table in order	363
Your own function, applied to a column	364
From a table to a picture	366
What we have deliberately left out	368
 Week 13	 369
55 Finale: formulating and planning your own tool	370
56 Finale build clinic	371
Take the pieces in dependency order	371
The cycle for a single piece	371
Prompting for one piece and only one piece	372
When a piece will not converge	372

Verification follows authorship	372
The journal as you go	373
57 Finale finishing and interface testing	374
The function that faces you	374
Running it end to end on real data	375
What you actually built	375
The limits that do not go away	376
58 Genome assembly	378
Read and analyze the sequencing reads	380
Compute overlaps between reads	381
Find the correct order of reads	384
Reconstruct the genomic sequence	387
 Week 14	 389
59 Surviving AI	390
60 The AI-off test: how you know you have learned it	394
 Cheatsheets	 396
61 The tools you check your work with	397
Seeing the substitution steps with <code>%%steps</code>	397
Rebuilding a program with <code>%%puzzle</code>	398
Watching memory with <code>%%codelens</code>	399
A fresh interpreter with <code>%%sandbox</code>	399
Drawing with <code>Turtle</code>	400
Checking a project with <code>im_pytest</code>	401
Narrating code with <code>snippet-cast</code>	401
Exploring a table with <code>iplot</code>	402
Which tool answers which question	402
62 Widget demo	403
Sandbox cell magic	403
Codelens cell magic	403
Puzzle cell magic	404
Steps cell magic	404
Turtle cell magic	405
Interactive plotting with Seaborn	405

How it works

> instructing machines

Bioinformatics · Aarhus University · 2026

Instructing Machines is a fourteen-week introduction to programming for molecular biology, written for students who will have a model at their side from the first line they type. Every week pairs a piece of Python with a way of working alongside an AI and a way of checking what it hands back.

[Start here](#) [Download the course folder](#)

14 weeks · 65 chapters · 8 projects on real data · 7 interactive widgets · MIT licensed

Brain

Your training is what lets you ask a questions about molecular biology and judge answers you are handed.

Code

Python lets you write machine instructions in something close to the plain English you think in.

AI

An AI can turn a problem posed in English into Python code that you can test and verify.

CPU

Once your question is expressed in as Python code, running it on you computer produces the answer.

AI in programming

The course design page describe the reasoning behind how AI is taught and integrated in different ways across the course but in a nutshell the philosophy goes like this:

An AI is useful to you up to the limit of what you can specify and verify. The course spends fourteen weeks raising that limit.

What the material looks like

The notes are executable notebooks. Exercises come with widgets that step through code line by line, so a claim about what the machine does can be settled on the spot.

screenshot: notebook with the step-through widget

A live demo of the same material runs in the browser through JupyterLite — no install.

```
def gc_content(seq):
    """Fraction of G and C in a DNA sequence."""
    gc = seq.count("G") + seq.count("C")
    return gc / len(seq)

# The test is the contract. Ask the AI for the body, keep this.
assert gc_content("GCGC") == 1.0
assert gc_content("ATAT") == 0.0
```

For students

Fifteen minutes to your first line

The course folder, Python, the editor and the terminal are all set up in one pass, most of which is waiting. You only do it once — see Chapter 2.

By the end you can:

- Read code the way the machine does.
- Write Python to solve small problems.
- Break a problem into sub-problems and write them as testable functions.
- Verify code written by an AI, or by someone else.
- Judge how much of a task to hand over, and how much to keep.
- Explain why an AI is only as useful to you as your ability to check it.

Getting started

For teachers

Take it and make it yours

Built for undergraduates in Molecular Biology and Molecular Medicine at Aarhus University, and MIT licensed. If you teach a similar course, fork it.

- Notes, slides, exercises and projects, all in the repository
- Seven notebook widgets, installable as packages
- A JupyterLite demo, so students can try before installing
- A written account of the course design and the AI arc

[Fork on GitHub](#) [Widget demo](#)

Kasper Munch · Bioinformatics Research Centre, Aarhus University

[Course catalogue](#) [GitHub](#) [MIT license](#)

Week 1

1 Course introduction

You are about to learn to program at a moment when a machine can already write the program for you. That is an odd position to be in, and pretending otherwise would waste your time. So this course is arranged around it. You will learn Python the way anyone learns Python, by writing small programs and watching them run, and alongside that you will learn what it takes to hand work to an assistant and know whether what comes back is any good. Those two things turn out to be the same skill approached from two directions, and this chapter explains the arrangement before you meet it week by week.

What the course is for

By the end of the term you should be able to look at a problem from your own field, decide what smaller problems it breaks into, say what each of those smaller pieces must do, and check whether a piece does it. Some of the code that fills those pieces you will have written yourself. Some of it an assistant will have written for you. In either case you are the one who decides whether it is correct, and the whole course is built to make you capable of that decision.

The hard part of programming is almost never the individual piece. Students who understand variables, loops, functions and dictionaries perfectly well can still sit in front of a new problem with no idea where to start, because putting the pieces together is a separate skill that only comes from doing it many times. That is why the first five weeks introduce the language and the eight weeks after that are projects. The concepts are the smaller half of the work. The composition is the larger half, and it is built by practice.

Everything the course asks you to do rests on one idea, which is that plausible and correct are different states, and moving from the first to the second is called verification. Code is verified by tests and by reading it. A claim about biology is verified by evidence and by what you already know. Your own learning is verified by doing the thing with the assistant switched off. Three situations, one idea, applied three times.

How the material teaches

The notes introduce things by doing them first and naming them afterward. You write and run something, it works, and only then does the text tell you that what you just wrote is called a for-loop and explain why it did what it did. This is deliberate. A name attached to something you have already made work sticks; a name attached to nothing does not.

Almost every exercise asks you to predict before you run. Decide what you think will appear on the screen, write it down or say it out loud, then run the code and

find out. The gap between your prediction and the result is the only reliable signal of what you do not yet understand, and you get that signal for free every time, provided you actually commit to a prediction first. Skipping the prediction and going straight to the run turns an exercise into a demonstration and teaches you very little.

Underneath all of it is a way of reading code that the course calls substitution and reduction. A variable is replaced by its value. An expression is replaced by the simpler expression it evaluates to, and then by a simpler one again, until only a single value is left. A function call is replaced by the value it returns. If you can carry out those replacements in your head, in order, for a piece of code you have never seen, then you can read code, and everything else follows. Several of the course tools exist for no other reason than to show you the substitution steps the machine actually took, so you can compare them against the ones you took.

The course also asks you to swear three oaths, and it asks in earnest. The first is that you type code and never paste it, because the typing is where the syntax settles into your hands. The second is that lines run from top to bottom, which sounds obvious until the day it stops feeling obvious. The third is that you trace every substitution and reduction consciously rather than guessing at what a line probably does. They are numbered vows, taken with a raised hand, and students who keep them do better than students who do not.

The assistant and the arc it follows

An assistant is present from the first week, and its part in the course grows across the term along a fixed sequence of roles. Each role licenses a different kind of request, and each new role is added to the ones you already have rather than replacing them. By week fourteen you have all of them and choose between them.

The sequence opens with Explainer. You put something in front of the assistant that already exists, an error message or a few lines of code, and ask what it means. You are not asking it to produce anything, so there is nothing yet to be wrong about beyond the explanation itself, which you check by running the code.

Translator comes next. You ask it to turn code into plain language, or a plain-language description into code, and you check the translation in both directions. Illustrator asks for examples and analogies for something you half understand. Comparer puts two versions of the same thing side by side and asks which is better and why, after which you make the decision yourself; the assistant supplies the argument and you supply the judgment.

Drafter is the first role that produces code you intend to keep. You specify a small function, the assistant writes a first version, and you test it before it goes anywhere near the rest of your program.

The last three roles hand over progressively more. As a Worker the assistant fills in a piece you have already specified and already written a test for, so the check exists before the code does. As a Collaborator it works with you on something where you set the decomposition and it contributes pieces and opinions. As a Developer it takes a goal and a specification and returns something you then check against acceptance criteria you wrote in advance.

One rule holds at every level. The assistant predicts, and the machine proves. Whenever the assistant tells you what a piece of code does, you check with a tool that runs it. Whenever it hands you a function, you check with a test. The prediction is cheap and the proof is what makes it worth anything.

Badges

Every exercise in the course carries a badge naming what the assistant is licensed to do for that particular exercise. The badge sits on its own line just below the exercise heading, in small capitals, and it is a link. Click it and you land on the description of that rung below, which tells you what you may ask for, what you may not, and what you have to do with whatever comes back. So an exercise that permits explaining opens like this: `AI: EXPLAINER` and one that permits drafting says: `AI: DRAFTER` and one you must do **no** AI assistance says: `SOLO`. The assistant lives in your browser, in Microsoft 365 Copilot, and it includes a Study and Learn agent that guides your reasoning instead of handing over the answer. VS Code stays free of it. Keeping the two apart is a working arrangement rather than a moral position: when the help is one deliberate window away, using it becomes a decision you make rather than something that happens to your code while you are typing.

The badges below are in the order the course grants them, and they are cumulative: once a rung is yours it stays yours, so an exercise badged `AI: WORKER` in week eight also permits everything from the six rungs before it. The badge names the *highest* thing you may do, not the only thing.

`SOLO · AI: EXPLAINER · AI: TRANSLATOR · AI: ILLUSTRATOR · AI: COMPARER · AI: DRAFTER · AI: WORKER · AI: COLLABORATOR · AI: DEVELOPER`

SOLO

The assistant is closed. Not open and unconsulted. Closed.

This is the only badge that takes something away, and it is the one that carries the most weight. A `SOLO` exercise means the browser tab is shut, you are not rereading an answer from earlier, and you are not reaching for a hint you half remember from a chat this morning. You type, you predict, you run, you fix. It is there for your benefit rather than as a rule to be policed. Nobody can tell whether you kept it. What a `SOLO` exercise buys you is evidence about yourself: if you can do it alone, you have learned the thing, and if you cannot, you have found that out now rather than in week twelve when everything depends on it. Doing a `SOLO` exercise with help does not get you in trouble. It just deletes the only measurement you had. Roughly three quarters of the exercises in this book are `SOLO`, and that proportion is the course working as designed rather than an oversight waiting to be corrected.

AI: Explainer

Ask it what something that already exists means. Then run the thing and see.

The first rung, and the safest, because you are not asking the assistant to produce anything. You put in front of it something that is already there — a traceback,

a line of code you did not write, a word in the documentation you do not know — and you ask what it means. Ask it in the shape of a question about a specific object: *what does this error message mean?*, pasting the whole traceback, or *what does 'ATG' in dna do here?* Do not ask *why doesn't my code work?*, which is a request for a fix wearing the costume of a question. If the answer is longer than the thing you asked about, ask again more narrowly. Then prove it. An explanation is a prediction about what the code does, and you have a machine that settles predictions: run the line, or put it through `%steps` and watch it reduce. When the explanation and the machine agree, you have learned something. When they disagree, you have learned something considerably more interesting, and you should write it in your logbook.

AI: Translator

Code into English, English into code, and you check the journey both ways.

Here you ask the assistant to say the same thing in the other language. Point at a loop and ask for one plain English sentence describing what it does. Or describe a small operation in plain English — *go through the string and count how many of the characters are G or C* — and ask for the Python. The check is what makes this a rung rather than a shortcut, and it is symmetric. Going from code to English, translate the loop yourself first, in one sentence, before you read the assistant's sentence; then compare the two and decide which is more precise about what actually happens, including the part where the loop does something surprising on an empty string. Going from English to code, read every line of what comes back and run it on an input where you already know the answer. Translation is where a vague understanding gets caught, because a sentence you cannot write yourself is a piece of code you do not yet understand, however comfortable it looks on the page.

AI: Illustrator

Ask for more examples of a pattern you half understand.

Sometimes the problem is not that an explanation was wrong but that one example was not enough. This rung licenses you to ask for more of them: *give me three more examples of slicing a string with a negative step*, or *show me the same dictionary lookup written for three different kinds of key*. Analogies count too, as long as you remember that an analogy is a teaching aid and not a fact about Python. The discipline is simple and it is not optional: run every example. Examples are exactly the kind of thing an assistant produces fluently and gets subtly wrong, because a plausible-looking example is cheap and a correct one requires knowing what the code actually does. Predict what each one prints before you run it, which turns three free examples into three exercises. Ask for illustrations of something you have already met and are unsure of, not of something you have not been taught yet. Used the second way, this rung is how you fall behind while feeling productive.

AI: Comparer

Two versions, one judgment, and the judgment is yours.

Now the assistant argues and you decide. You have a working function; you ask for a second way of writing it, or you show it two versions you already have and ask what the trade-offs are. What comes back is an argument, and arguments are useful. What does not come back is a decision. Ask it to be specific: “which of these two is easier to read, and point at the lines that make the difference”, rather than “which is better”. A general question gets a general answer, and a general answer about code is almost always flattering to whichever version was mentioned last. Then do the part that is actually yours. Make the choice, and write down one sentence saying why, naming the reason: this one is shorter but hides the empty-sequence case; that one repeats itself but I can see what it does without thinking. If you cannot write that sentence, you have not compared the two versions, you have only read about them. And run both, on the same inputs, including the awkward ones. Two functions that look equivalent frequently are not.

AI: Drafter

It writes a small piece. You read every line, and a test decides.

This is the first rung that produces code you intend to keep, and it is safe for exactly two reasons: the piece is small enough to read in full, and something stands behind it before you are allowed to believe it. The order matters more than anything else here, and it is: specify, draft, read, test. Specify first — say what the function is called, what it takes, what it returns, and what it should do on the awkward inputs — because a request with no specification has nothing to be right or wrong about. Draft second. Read third, every line, and if there is a line you cannot explain, you do not own it yet and the fix is to ask AI: EXPLAINER questions about that line until you do. Test fourth, and let the test decide rather than your impression of how the code looks. Keep the piece small. A function you can read in one screen is a function whose faults you can find; a hundred lines that look right is exactly the situation this course exists to keep you out of.

AI: Worker

You write the contract. It fills in the body. The test says whether it kept the contract.

At this rung the assistant writes code you have already specified in a form the machine can check. That is the whole of it, and the order is not negotiable: the tests exist before the code does. So you write the `assert` statements first, from your own understanding of what the function must do, including the decisions nobody else can make for you — what `is_gc_rich` should return on a strand that is exactly half GC, what it should do with an empty strand. Then you ask for the implementation, in prose, without showing it your tests. Then you run the tests you already had. Not showing it the tests is the part students most want to skip, and it is the part that does the work. Code written against a visible test suite can satisfy the suite and nothing else. Code written against a description and then measured by tests it never saw is being genuinely checked. And when you throw

the implementation away and ask for a different one, and the unchanged tests still pass, you have discovered the thing this rung is really teaching: the implementation is replaceable and the specification is not.

AI: Collaborator

You own the plan. It contributes pieces and opinions, one at a time.

Here the assistant works on something with you rather than for you, and the division of labour is exact: you set the decomposition, it fills pieces and offers opinions on the plan, and every one of those opinions is yours to accept or reject. Do the planning with the assistant closed. Break the task into named pieces, say what each one takes and returns, and write two or three worked examples for each — real inputs, real expected outputs. Only then open the browser, show it the plan, and ask a narrow question: *is there a case my examples do not cover, or a piece I have not named?* Not *write this for me*, which is a different rung and a later one. Whatever it suggests, write down *accept* or *reject* and one line of reason for each. That record is the difference between collaborating and being led, and you will want it later, because the failure mode at this rung is not bad code. It is a plan that drifted into being the assistant's plan while you were agreeing with things one at a time.

AI: Developer

Hand over the whole job, against criteria you wrote before you asked.

The last rung, and the one that only works because of everything under it. You give a goal and a specification and get back something whole, which you then check. Write the acceptance criteria first, in a file, before you send a single prompt. What must the thing do, on what inputs, producing what, and what would count as it having failed? Criteria written afterwards are not criteria; they are a description of whatever you got. When the result does not meet them, do not send *this doesn't work, fix it* — say which criterion failed, on which input, with what output, and what you expected instead. Vague reports get vague repairs. Two rules keep this rung honest. If the loop has not converged in two rounds of that, take the keyboard back and write the piece yourself, because a third round is almost always slower than doing it. And before you keep anything, close the assistant and explain the most-delegated function in it, line by line, out loud or in writing. You cannot own a result you cannot explain, and by the end of this course owning the result is the entire job.

The logbook

Once a week you write one entry in a logbook you keep all term. An entry records something the assistant got right, something it got wrong, and how you knew which was which. It takes a few minutes and it is the single best record of your own development that the course produces, because by week twelve you will be reading week-two entries where you could not tell a plausible answer from a correct one, and the difference will be visible on the page.

The projects, and who writes what

From week six to week thirteen the course is eight weekly projects, each one a small piece of working biology code. Every project has three parts, and it helps to name them separately because the course moves them between hands at different times. The decomposition is the list of functions the project consists of, with what each one takes and what each one returns. The tests are the code that decides whether a function does what it is supposed to. The bodies are the actual lines inside each function. In the first project all three parts start in familiar places. The decomposition is given to you, the tests are given to you, and you write the bodies. Over the following weeks each part changes hands on its own schedule, and once a part has moved it does not move back.

The bodies go first, and they go to the assistant. In week seven you nominate one function and let the assistant draft it, then test what comes back. In week eight it drafts again, but only after you have written a test for the function yourself, which is the point at which the order of operations becomes the lesson: the check exists before the code exists. From week nine the assistant does more of the routine work, the glue between your code and a library, while you keep the parts that carry the actual biology. The tests move second, and they move to you. Week eight ships you every test but one, and you write the missing one. Week nine ships half. By week eleven you get a single end-to-end test that says whether the whole thing works, and nothing that tells you which part is broken. By week twelve you get none, and what you can check is what you thought to check. The decomposition moves last, because it is the hardest of the three. Through week nine the full list of functions is given. In week ten most of it is given and you name two pieces yourself. In week eleven you get only the top-level function and decide everything under it. In week twelve you get the goal and the data. In week thirteen you formulate the problem, build it, and only afterward see how we would have broken it up, which is the closest thing the course has to a final examination of the skill it is actually teaching.

Two rules keep this from becoming unfair. A project never asks for anything the notes have not already taught, and it asks for it the week after they taught it, so there is always a week between meeting an idea and having to use it under your own direction. And each transfer happens once. A part that has become yours stays yours, so the ground never shifts back under you.

What to expect from yourself

There is a fear worth addressing directly, which is that learning to program is pointless now that a machine can do it. The opposite is closer to the truth, and the reason is arithmetic. What you can safely get out of an assistant is limited twice over by what you know: once by your ability to ask for the right thing, and again by your ability to tell whether what came back is right. Both limits are your own knowledge, so the value the assistant has for you grows roughly as the square of what you know. At zero knowledge it is worth nothing, or less than nothing, because you cannot catch its mistakes. And every new thing you learn is now worth more than it used to be, because it pays you twice, once directly and once by unlocking more of the assistant that you can use without danger.

That is also where the line about cheating falls, and it is a sharper line than it first appears. Producing something you did not formulate, do not understand, and cannot check is cheating, and you will feel it. Producing something you formulated, understand, and validated is tool use, no different in kind from using a microscope. The same habit of never trusting what you have not checked keeps your work correct and tells you which side of that line you are standing on.

Exercise 1-1

SOLO

Make a file called `logbook.md` somewhere you will find it again, and write the first entry today, before you have used the assistant for anything in this course. Write down what you currently expect it to be good at, what you expect it to be bad at, and how you imagine you would tell the difference. You will read this again in week fourteen. Nobody else will, so make it truthful rather than impressive.

2 Getting started

This chapter will get your computer set up with what you need for the course. You need to carefully read the instructions and follow them to the letter. If a step does not do what it says it will, that is information rather than failure, read **?@sec-when-something-goes-wrong** at the bottom, and bring the exact error message to class. Do **not** try to complete the steps by some alternative means. That will only sink you deeper into the quicksand.

The course folder

Everything you need for the whole term lives in one folder. Download it now:

i Download the course folder

`instructing-machines.zip` — about two megabytes.

Unzip it, and put the folder somewhere plain and short that belongs to you. Your user/home folder is the best place. Do **not** put it in OneDrive or iCloud Drive. They sync files in and out from under you while Python is reading them, and that produces errors nobody can explain.

Inside the folder you will find a file called `START-HERE.md`. It says the same things this chapter says, only shorter, so you have them at hand when the book is not open.

The terminal

On **Mac**, the terminal is an application called *Terminal*. You can find it by typing “Terminal” in Spotlight Search. When you start it, you will see something like Figure 2.1. On **Windows**, it is called *Windows PowerShell*, and you should be able to find it from the Start menu — make sure you open *Windows PowerShell* and **not** some other shell (the name should be exactly that). When you do, you should see something like Figure 2.2.

What are *Windows PowerShell* and this *Terminal* thing? Both are what we call *terminal emulators*. They are used to run other programs, like the ones you will write yourself. I will informally refer to both as “the terminal”. So if I write something like “open the terminal”, you should open *Windows PowerShell* if you are running Windows and the *Terminal* application if you are on a Mac.

The terminal is a very useful tool. However, to use it, you need to know a few basics. First of all, a terminal lets you execute programs on your computer. You type the command you want and then hit Enter. The first word of commands is always the name of a program. The place where you type is called a prompt (or



Figure 2.1: The Terminal app on Mac

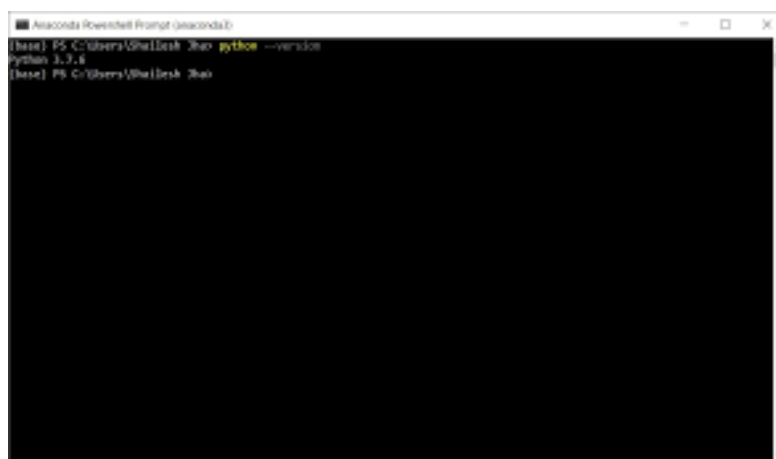


Figure 2.2: The Windows PowerShell app on Windows

command prompt), and it may look a little different depending on which terminal emulator you use. In this book, we represent the prompt with the character `>`. So a command in the examples below is the line of text to the right of the `>`.

You can navigate your folders in terminal the same way you do in Finder on mac and File Explorer on Windows. When you open the terminal, you are put in your user folder. You can see which folder you are in by typing `pwd`, and then pressing `Enter` on the keyboard. When you press `Enter`, you tell the terminal to execute the command you have written to run the `pwd` program. In this case, the command you typed tells you the path to the folder we are in. If I do it, I get:

Terminal

```
> pwd
/Users/kasper/instructing-machines
```

So, right now, I am in the `instructing-machines` folder. `/Users/kasper/instructing-machines` is the folder's path or "full address", with slashes (or backslashes on Windows) separating nested folders. So `instructing-machines` is a subfolder of `kasper`, which is a subfolder of `Users`. That way, you know which folder you are in and where that folder is.

Let us see what is in this folder. You can use the `ls` command (l as in Lima and s as in Sierra). When I do that and press `Enter` I get the following:

Terminal

```
> ls
course.py    get.py       pixi.toml    START-HERE.md week1
data         pixi.lock    projects     update.py
```

There seem to be three folder: `data`, `projects`, and `week1`. If you are curious about what is inside the `week1` folder, you can "walk" into the folder with the `cd` command. To use this command, you must specify which folder you want to walk into (in this case, `week1`). We do this by typing `cd`, then a space, and then the folder's name. When I press `Enter` I get the following:

Terminal

```
> cd week1
>
```

It seems that nothing really happened, but if I run the `pwd` command now to see which folder I am in, I get the following:

Terminal

```
> pwd
/Users/kasper/instructing-machines/week1
```

Just to keep track of what is happening: before we ran the `cd` command, we were in the `/Users/kasper/instructing-machines` folder, and now we are in `/Users/kasper/instructing-machines/week1`. This means that we can now use the `ls` command to see what is in the `week1` folder:

Terminal

```
> ls
notebooks-in-vscode.ipynb script-vs-notebook.ipynb
```

So `week1` contains two notebook files (you can tell by their `.ipynb` suffix). Don't try to open them yet. Let us go back to where we came from. To walk "back" or "up" to `/Users/kasper/instructing-machines`, we again use the `cd` command, but we do not need to name a folder this time. Instead, we use the special name `..` to say that we wish to go to the parent folder, i.e. the folder we just came from:

Terminal

```
> cd ..
> pwd
/Users/kasper/instructing-machines
```

Hopefully, you can now navigate your folders and see what is in them. You will need this later to reach the folders with the code you write for the exercises and projects during the course. There are many other commands that you may learn along the way, but for now, the only ones you need are:

Action	Windows	Mac
Show current folder	<code>pwd</code>	<code>pwd</code>
List folder content	<code>ls</code>	<code>ls</code>
Go to subfolder "notes"	<code>cd notes</code>	<code>cd notes</code>
Go to parent folder	<code>cd ..</code>	<code>cd ..</code>

Installing pixi

Pixi is the program that installs Python for you along with all the packages you need for the course. To avoid conflicts between multiple projects that use

Python, Pixi installs one self-contained Python *per project folder* (in your case the `instructing-machines` folder). It lives in a hidden `.pixi` folder inside the course folder, nothing is installed anywhere else, and deleting the course folder removes every trace of it. So the Python we use cannot collide with a Python you install next year for something else, and if your installation ever ties itself in a knot, the cure is to delete the `.pixi` folder and start again.

2.1 Mac

Open your Terminal application, paste this line, and press Enter:

```
curl -fsSL https://pixi.sh/install.sh | sh
```

2.2 Windows

Download and run the installer:

https://github.com/prefix-dev/pixi/releases/latest/download/pixi-x86_64-pc-windows-msvc.msi

Double-click it and click through. That is all.

If you have one of the rare ARM-based Windows machines — a Surface Pro X, for instance — use `pixi-aarch64-pc-windows-msvc.msi` instead.

Now, whichever machine you are on: **close the terminal window and open a new one..** A program that has just been installed is invisible to terminals that were already open. A terminal reads the list of available programs once, when it starts, and does not look again. So the terminal you installed pixi from does not know pixi exists. This one detail is responsible for more confusion than everything else on this page combined.

Check that it worked by typing:

Terminal

```
$ pixi --version
pixi 0.60.0
```

You will probably see a different version number, and that is fine. If you instead see something about the command not being recognized, see [?@sec-when-something-goes-wrong](#).

Installing the course environment

Now you can install Python and the packages we need for the course. Open the terminal and navigate to the `instructing-machines` folder (run `pwd` to make sure). Then run this command:

Terminal

```
> pixi install
```

When it is done there is a new hidden `.pixi` folder in the course folder containing your very own Python.

Then check it:

Terminal

```
> pixi run check  
Everything is installed. Python 3.13.5
```

If you see that line, you are finished with the hard part.

The `check` in that command is not a built-in pixi command — it is a little errand we wrote for you and stored in the folder's `pixi.toml` file. You will meet a second one, `pixi run get`, in [?@sec-getting-the-notebooks](#).

The text editor

You will also need a *text editor*. A text editor is where you write your Python code. For this course, we will use *Visual Studio Code* — or *VS Code* for short. You can download it from code.visualstudio.com. If you open VS Code, you should see something like Figure 2.3.

You may wonder why we cannot use Word to create and edit files with programming code. The reason is that a text editor made for programming, such as VS Code, only saves the actual characters you type. So, unlike Word, it does not silently save all kinds of formatting, like margins, boldface text, headers, etc. With VS Code, what you type is *exactly* what is in the file when you save it. In addition, where Word is made for prose, VS Code is made for programming and has many features that will make your programming life easier.

Opening the folder

Start VS Code, then choose **File → Open Folder...** and select the `instructing-machines` folder — the folder itself, not a file inside it.

This matters more than it looks like it does, so it is worth one sentence of explanation. The course folder carries a small settings file that tells VS Code where the course Python lives, and VS Code only reads those settings when the folder is *what you opened*. Open a single file instead and VS Code has no idea any of this exists.

VS Code will show a small notification offering to install the *recommended extensions* for this folder. Say yes. It is installing Python support, notebook

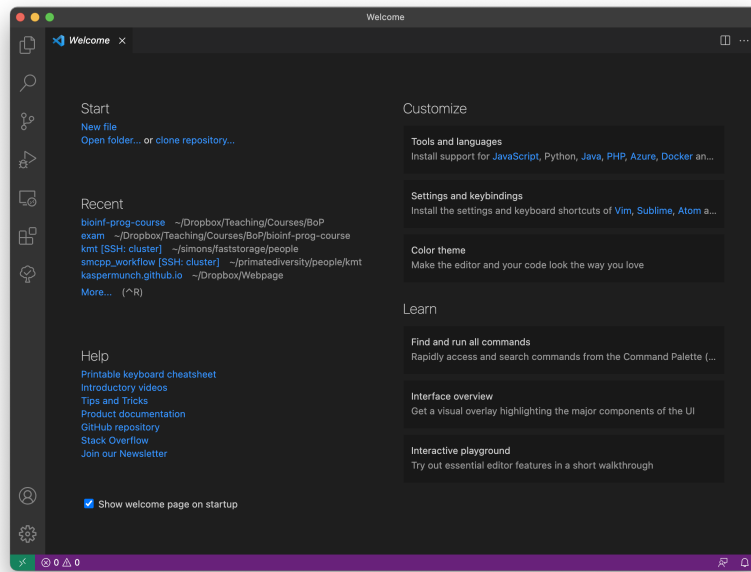


Figure 2.3: Visual Studio Code

support, and pixi support. If you miss the notification, open the Extensions panel on the left and search for `ms-python.python`, `ms-toolsai.jupyter` and `renan-r-santos.pixi-code` and install each one.

When something goes wrong

- **“pixi is not recognized as an internal or external command” (Windows) or “command not found: pixi” (Mac).** The terminal you are typing in was opened before pixi was installed. Close it and open a new one. If you are inside VS Code, quit VS Code entirely and start it again.
- **Windows says a script cannot be run, or mentions an execution policy.** You have found a PowerShell install script somewhere. Do not fight with it — use the `.msi` installer linked in `?@sec-installing-pixi` instead. It needs no permissions of that kind.
- **pixi install fails part-way, or complains about paths being too long.** The folder is almost certainly too deep, or inside OneDrive. Move the whole folder to `C:\bioprogr\` and run `pixi install` again.
- **VS Code will not offer the right kernel.** Make sure you opened the *folder* (`?@sec-opening-the-folder`), that `pixi install` has finished, and that the Python and Jupyter extensions are installed. Then run **Developer: Reload Window** from the command palette (`Ctrl+Shift+P`, or `Cmd+Shift+P` on a Mac).
- **A widget shows as blank space, or as raw text.** You are running the notebook on the wrong Python. Click **Select Kernel** in the top right of the notebook and choose the one whose path contains `.pixi`.
- **Nothing works and the lecture has already started.** Open the course website and use the in-browser notebooks. They run Python inside the browser with nothing installed, they are slower and slightly limited, and they will get you

through the next hour. Then come to the study café and we will fix your installation properly.

If you see red text you cannot place, take a screenshot — or better, copy the text itself — and send it to kaspermunch@birc.au.dk. Bring the exact message, not a description of it. Reading error messages is one of the things this course is actually about.

You are all set

Well done! You are all set to start the course. Have a cup of coffee, and look forward to your first program. While you sip your coffee, I need you to take an oath (one of three you will take during this course). Raise your right hand! (put your coffee down first).

! Oath 1

I swear *never* to copy and paste code. I will always *read* the examples and then *type* them into my editor.

This serves three purposes (as if one was not enough):

1. You will be fully aware of every bit of each example.
2. You will learn to write code correctly and without omissions and mistakes.
3. You will get Python “into your fingers”. It sounds silly, but it *will* get into your fingers.

You can take your hand down now.

3 Hello world

Let's get you started...

Your first program

Dive in and make your first program. Create a new file in your editor (*VScode*) and save it as `hello.py`. The `.py` suffix tells your editor that this file contains Python code. As you will see, this makes your life a whole lot easier. Such a file with Python code is usually called a *script*, but we can also call it a program.

Now write *exactly* this in the file (`hello.py`):

```
print("Hello world")
```

Your editor will color your code differently, but that is unimportant. Save your file with the added code and have your first program! Of course, there is not much point in having a program if it just sits there on your computer. To run your program, do the following:

1. Open the terminal and navigate to the folder (directory) where you saved `hello.py`. Use the `cd` command to do so. If you do not remember how, reread the previous chapter.
2. Type `python hello.py` in the terminal and hit Enter.

This is where you shout, "It's alive!" toss your head back, and do the insane scientist laugh.

Okay, what just happened? You wrote a program by creating a file in which you wrote one line of code. You then ran the program using Python, and it wrote (printed) `Hello world` in the terminal. Do not worry about the parentheses and quotes for now; just enjoy your new life as a programmer.

Maybe you wonder why we write `print` and not `write` or something else. That goes back to when computers were enormous, clunky things with no screens attached. They could only interact with the user by *printing* things on an actual physical paper printer. Back then, the output you now see on the screen was printed onto a piece of paper that the programmer could then look at. These days, `print` shows up in the terminal, but the story should help you remember that `print` spits text *out of your program* just like a printer.

Now try to add another line of code like this:

```
print("Hello world")
print("I am your first program")
```



Figure 3.1: Hello world

Save the file and run it again by typing `python hello.py` into your terminal and hitting Enter.

You should see this:

```
Hello world
I am your first program
```

Now your program prints `Hello world` and *then* prints `I am your first program`. The *then*-part is important. That is how a Python program works. Python (the `python` you write in front of `hello.py`) reads your `hello.py` file and then executes the code, one line after another, until it reaches the end of the file. This is essential, so reread from the beginning of the paragraph. Now, read it once more. It may seem trivial, but it is fundamental to remember that this is how Python runs your code. So here is Oath 2:

! Oath 2

I swear to always remember that each line of code in my script is executed one after another, starting from the first line and ending at the last line.

When you write Python code, you always follow this workflow:

1. Change the code in the file.
2. Save the file.
3. Execute the code in the file using the terminal.
4. Start over.

Make sure you get the hang of this in the following exercises.

Important: The examples and exercises in this course are designed to work if you execute your scripts from their stored folder. So, you'll need to go ahead and navigate the relevant folder before you're done with your script. If your script is called `hello.py`, you must *always* execute it precisely like this: `python hello.py`. If you do it any other way, you may use a different Python version than you think. On some computers, it is possible to type `hello.py` without `python` in front of it. Don't do that. Do *not* "drag" the script file into the Terminal either. In VScode, you can press a small play icon on the top right to execute the code. Don't do that either. There are many other things you should not do, but you get the drift.

Exercise 3-1

SOLO

Try swapping the two lines of code in the file and rerun the program. What does it print now?

Exercise 3-2

SOLO

Try to make the program print a greeting to yourself. Something like this:

Hello Sarah!

– if your name is Sarah, of course.

Exercise 3-3

SOLO

Add more lines of code to your program to make it print something else. Can you make your program print the same thing ten times?

Error Messages

Did you get everything right with your first program, or did you get error messages when executing your code with `python`? Maybe you wrote the following code (adding an extra closing parenthesis):

```
print("Hello world"))
```

and then got an error like this:

```
Cell In[1], line 1
    print("Hello world"))
                        ^
SyntaxError: unmatched ')'
```

This is Python's way of telling you that the `hello.py` script has an error in line 1. If you write something that does not conform to the proper syntax for Python code, you will get a `SyntaxError`. Python will do its best to figure out where the problem is and point to it with a `^` character.

You will see many error messages in your new life as a programmer. So you must practice reading them. At first, they will be hard to decipher, but once you get used to them, they will help you quickly identify where the problem is. If there is an error message that you do not understand, the internet is your friend. Just paste the error message into Google's search field, and you will see that you are not the only one on the internet getting started on Python programming. It is okay if you do not know how to fix the problem right now, but it is essential to remember that these error messages are Python's way of helping you understand what you did wrong.

Exercise 3-4

AI: EXPLAINER

Try to break your new shiny program and make it produce an error message when you run it. An easy way of doing this is to remove or change random characters from the program. If you run this (with a missing end-parenthesis:

```
print("Hello world"  
print("I am your first program")
```

You will get this error:

```
Cell In[2], line 1  
    print("Hello world"  
      ^  
SyntaxError: '(' was never closed
```

The `^` character tells you when your code stops making sense to Python. Sometimes, that is a bit after where you made your mistake.

Try to make other kinds of errors. Which error messages do you see? Do you always see the same error message, or are they different? Try googling the error messages you get. Can you figure out why the change you made broke the program? How many other error messages can you produce?

Before you fix anything, paste one of your error messages into the assistant and ask what it means and which character is missing. Then put the character back and run the program again. The assistant tells you a story about your error; your own screen is what settles whether the story was true.

Strings

In programs, text values are called *strings*, and you have already used strings a lot in your first program. A string is simply a text, but we call it a string because it is a “string of characters”. In Python, we represent a string like this:

```
"this is a string"
```

or like this:

```
'Hello world'
```

That is, we take the text we wish to use as our value and put quotes around it. We can use double quotes (the first example) or single quotes (the second example). We can mix them like this:

```
print('this is "some text" with a quote')
```

but not like this:

```
print("this is a broken string')
```

Can you see why and how handy it is to have single and double quotes? If we did not have both, we could not have text with quotes. However, you must use the same kind of quotes at each end of the string. Running the latter example gives an error message telling you that Python cannot find the quote that was supposed to end the string:

```
Cell In[3], line 1
    print("this is a broken string')
      ^
SyntaxError: unterminated string literal (detected at line 1)
```

It is Python's way of saying, "The string you started never ended"

Comments

You have already learned that Python reads and executes one line of code at a time until your program has no more lines of code.

However, we can make a line invisible to Python by putting a `#` symbol in front of it, like this:

```
# print("Hello world")
print("Greetings from your first program")
```

When you do that, Python does not read that line. It is not part of the program. Rerun the program. You will notice that now only the second line is part of your program:

This is useful in two ways:

1. It lets you disable certain lines of your code by keeping Python from reading them. For example, see what happens if that line of code is not executed to understand how your program works.
2. It allows you to write notes in your Python code to help you remember how it works.

Lines with a `#` in front are called "comments" because we usually use them to write comments about our code. If you ask for help with some problem, you will often hear your instructor say: try to "comment out" in line two. When your instructor says that, it simply means that you should add a `#` in front of line two to see what then happens.

Exercise 3-5

SOLO

What happens if I put a `#` in the middle of a line of code? Try it out!

Exercise 3-6

SOLO

Try this:

```
# Note to self: the lines below print stuff
print("Hello world")
print("Greetings from your first program")
```

Exercise 3-7

SOLO

Try this:

```
print("Hello world") # actually, everything after a # is ignored
print("Greetings from your first program")
```

What did you learn? Which parts of each line are considered part of the program?

Exercise 3-8

SOLO

Now try this:

```
print("Hello # world")
print("Greetings from your first program")
```

What did you learn about `#` characters in strings? ##### Exercise

SOLO

Try this:

```
print("Hello world"#)
print("Greetings from your first program")
```

Did you expect this to work? Why? Why not? What error message did you get?

4 Notebooks

Scripts versus notebooks

You have written a program, saved it in a file, and run it from the terminal. That is one way of running Python, and it is the way most Python in the world gets run. This chapter shows you the other way, the one you are looking at right now, because this page is itself a running Python program. The two ways are not the same, and the places where they differ are exactly the places where beginners get confused. So let us take them apart.

The script you already know

Your `hello.py` is a script. It is a plain text file sitting on the disk with some Python code written in it. To run it, you go to the terminal and type:

Terminal

```
python hello.py
```

The program called `python` is the interpreter. It opens your file, reads the first line, does what it says, reads the next line, does what that one says, and carries on until there are no more lines. Then it stops and exits.

Two things follow from that, and both of them matter more than they look.

The first is Oath 2 again: the order the lines appear in the file is the order in which they run. Nothing jumps around. If you want to know what a script does, you read it downward.

The second is that every run starts from nothing. When the interpreter exits, everything it was holding is thrown away, because it was all in memory, and memory is emptied the moment a program ends. Run the script again tomorrow and it starts from a blank slate once more. A script therefore carries everything it needs inside itself. If it works today, it works tomorrow, on your machine or on someone else's, and that reliability is bought entirely by starting from nothing every single time.

Notebooks in VScode

The notes you are reading are also notebooks you can run. Reading this page in a browser will teach you nothing, because this chapter is about clicking things. The `week1` notebooks are already in the course folder, in the `week1` subfolder. Open

the course folder in vscode and expand the week1 folder to show its contents. Then double-click the `notebooks-in-vscode.ipynb` file to open it and work through it there. Everything below assumes you are looking at the real notebook with a cursor in it.

Every other chapter you fetch when we get to it, so that the notebook you work in is always the current version of the chapter you are reading. There is a **Download notebook** button in the margin of every chapter on the website so you can download the notebook and put it in the course folder.

Downloading Notebooks

The notes you are reading are also notebooks you can run. Reading this page in a browser will teach you nothing, because this chapter is about clicking things. The week1 notebooks are already in the course folder, in the `week1` subfolder. Open the course folder in vscode and expand the week1 folder to show its contents. Then double-click the `notebooks-in-vscode.ipynb` file to open it and work through it there. Everything below assumes you are looking at the real notebook with a cursor in it.

Every other chapter you fetch when we get to it, so that the notebook you work in is always the current version of the chapter you are reading. There is a **Download notebook** button in the margin of every chapter on the website so you can download the notebook and put it in the course folder.

Getting notebooks from the terminal

I also made you an easier way to get the notebooks. Say you want the notebook for the “Iteration” chapter, all you need to do is navigate to the course folder and run `pixi run get iteration`:

[H]
Terminal

```
> pixi run get iteration
Fetching https://munch-group.org/instructing-machines/notebooks/iteration.ipynb

Saved iteration.ipynb. Open it in VS Code and pick the .pixi kernel.
```

To see what you can ask for other than `iteration`, run it without a name:

[H]
Terminal

```
> pixi run get
```

If you already have a notebook of that name, yours is left exactly as it is and

the fresh copy arrives beside it as `iteration-2.ipynb`. Nothing you have written is ever overwritten, so it is always safe to ask for a clean copy.

The notebook you are reading

Now look at where you are. This page is a notebook, a file whose name ends in `.ipynb`. You can download it using the link in the margin and open it in VScode. Instead of one long stream of code, a notebook is built from cells. Some cells are text, like this one. Other cells hold code and can be run one at a time.

Enough description. Run the cell below. Click it and press Shift and Enter together.

```
dna = "ATGGTGCATCTGACTCCTGAGGAGAAG"  
len(dna)
```

Two things just happened. A small number appeared in brackets to the left of the cell, and the length of the string appeared underneath it even though you never wrote `print`. Hold on to both of those; we will come back to them.

Now run the next cell.

```
dna.count("A")
```

That cell never mentions what `dna` is, and yet it worked. This is the whole difference in one line. Behind this page there is a Python process that has been running the entire time, and it is still running now. It is called the kernel. When you run a cell, the code in that cell is handed to that same living process, which does what it says and then waits for whatever you hand it next. The first cell put a string into that process and gave it the name `dna`. The second cell asked the same process about a name it already knew.

So a notebook is not a file that the interpreter reads from top to bottom. It is a pile of small pieces of code that you hand, one piece at a time, to a session that remembers everything you have handed it so far.

The number in brackets is the execution count, and it tells you the order in which you handed things over. The first cell you run in a session gets a `1`, the next gets a `2`, and so on. Run the same cell again and its number goes up. This is the notebook quietly telling you something important, which is that the kernel remembers the order you ran cells in, not the order the cells sit on the page. You can run the bottom cell first, then the top one, then the bottom one again, and the kernel will happily oblige. You can even delete a cell after running it, and the names it defined will still be sitting in the kernel, alive and usable, with nothing on the page left to explain where they came from.

That is the notebook's great convenience and its great trap. It is a convenience because you can try one small thing, look at the answer, and try the next small thing, without rerunning your whole program each time. It is a trap because what you see on the page can stop matching what the kernel actually holds.

The other loose end was the length appearing without `print`. When the last line of a cell is a bare expression, something that reduces to a value, the notebook displays that value for you. Only the last line, and only if it is an expression. An assignment like `dna = "ATG"` produces no value, so nothing is shown. This is a courtesy of the notebook, not a rule of Python. A script will print nothing at all unless you ask it to with `print`, which is one of the small ways that code moved out of a notebook stops behaving the way you remember.

Exercise 4-1

SOLO

Decide what will happen before you touch anything. Scroll back up to the cell that defines `dna`, change the string to `"GGGGG"`, and run only that cell. Then run the counting cell again. What number do you expect, and what do you get? Now change the string back and run the counting cell without rerunning the definition. What do you get, and why?

Exercise 4-2

SOLO

Find the command in your editor that restarts the kernel and runs all the cells from the top. In VScode it sits at the top of the notebook. Predict what will happen to the execution counts before you use it, then use it and check.

Restart and run all

The exercise you just did is the single most useful habit you can form about notebooks, so let us give it a name. Restarting the kernel throws away the running process and starts a fresh one that knows nothing. Running all the cells afterward feeds them back in, from the top, in page order. In other words, restarting and running all makes your notebook behave, for one run, like a script.

Do this before you believe a notebook works, and certainly before you hand one to anybody else. A notebook that only works because of something you ran twenty minutes ago in a cell you have since deleted is a notebook that will fail for the next person who opens it, and that next person is often you, next week.

Python and IPython

There is a second thing going on that you have not been told about yet, and now is the time.

If you type `python` in the terminal with no file name after it, you do not get an error. You get a prompt that looks like this:

Terminal

```
Python 3.13.0  
>>>
```

That is the interpreter waiting for you to type one line of Python at a time. You type a line, it runs it, it shows you the answer, and it waits again. It is the same interpreter that runs your scripts, just taking its instructions from your keyboard instead of from a file. Type `exit()` to leave.

There is also a second such prompt, from a program called IPython, which you start by typing `ipython`. It looks like this:

Terminal

```
In [1]:
```

IPython is not a different language. It is the same Python, with a friendlier shell wrapped around it: it completes names when you press Tab, it colors your code as you type, it prints much nicer error messages, and if you put a question mark after a name it shows you the documentation for it. It also adds something Python itself does not have, which we are about to meet.

And here is the point of this section. The kernel behind a Python notebook is IPython. That bracketed number next to your cells is IPython's counter, the same one that says `In [1]` at its terminal prompt. You have been using IPython since the first cell of this chapter without knowing it.

Magic commands

Run this cell.

```
%whos
```

What you just ran is not Python. It is a magic command, one of IPython's own commands, and `%whos` asks it to list every name the kernel is currently holding. Look at that list and notice that `dna` is in it. That table is the kernel's memory made visible, and it is a good thing to run whenever a notebook starts behaving strangely.

Magic commands come in two shapes. A line magic starts with one percent sign and affects only its own line, so `%whos` above is a line magic. A cell magic starts with two percent signs, must be the very first line of the cell, and takes charge of everything below it in that cell. Every widget in this course is a cell magic, which is why they all look like `%steps`, `%puzzle`, `%code lens` and, in a moment, `%sandbox`.

Because magics belong to IPython and not to Python, they only work where IPython is running, which means in a notebook or at the `ipython` prompt. Copy `%whos` into `hello.py`, run `python hello.py`, and you will get a `SyntaxError`, because the plain interpreter has never heard of it. Keep that in mind the next time you move code out of a notebook and into a script and it stops working for no visible reason.

Exercise 4-3

AI: EXPLAINER

Ask the assistant to explain the difference between a line magic and a cell magic, and to give you three examples of each. Then take each example it gives you and try it in a cell here. Some will work, some will not exist, and some will turn out to be the wrong shape. Note in your logbook which of its examples survived contact with the machine.

When the kernel's memory gets in the way

The kernel's long memory is a gift when you are exploring and a menace when you are learning.

Suppose an exercise asks you to run a piece of code that should fail, so that you can read the error message. If you happen to have defined that name in some other cell half an hour ago, the code will not fail at all, and the lesson evaporates. Or suppose you write a small piece of code, run it, see it work, and conclude that it is correct. It may be correct. It may also be leaning on a name that a cell somewhere above quietly left lying around, in which case it will collapse the moment it is moved anywhere else.

What we want, for a lot of the exercises in this book, is a cell that behaves like a script: one that starts from nothing, contains everything it needs, and leaves nothing behind. That is what the sandbox is for.

The sandbox

Run this cell once. It brings in the tool and registers the magic with IPython.

```
import sandbox_widget
```

Now run this one.

```
%%sandbox  
print("Hello from a fresh interpreter")
```

The output turned up in a shaded box below the cell instead of in the usual place, which is your visual cue that this cell was not run the way the others were.

Here is what makes it different. Predict what the next cell will do before you run it. Remember that `dna` has been sitting in the kernel since the top of the chapter, and that the cell below asks for it by name. This cell is meant to fail.

```
%%sandbox
print("The sandbox is about to look for dna.")
print(dna)
```

It printed the first line and then raised a `NameError` on the second, and both the printed line and the full traceback landed in the box. The notebook itself carried on untroubled.

Now name what you saw. `%%sandbox` is a cell magic that takes everything below it and runs it in a brand new Python interpreter, a separate process started for that one cell and thrown away the moment it finishes. Nothing that the notebook knows about is visible inside it. The kernel has a `dna`, and the sandbox has never heard of it, because the sandbox is not the kernel.

The barrier runs both ways. Run the next two cells in order.

```
%%sandbox
codon_count = 64
print(codon_count)

print("codon_count" in globals())
```

The sandbox cell made a `codon_count` and printed it quite happily. The ordinary cell that followed asked the kernel whether it has ever heard of that name, and the answer is `False`. Whatever the sandbox does stays in the sandbox, and when its process ends, all of it goes.

So a `%%sandbox` cell is a tiny script that happens to live inside a notebook. It runs from nothing, top to bottom, and it has to carry everything it needs, including its own `import` lines. Anything it defines dies with it, and any mistake it makes is caught and shown to you rather than left to poison the rest of the session.

That last part is why so many exercises in this book use it. A cell that is supposed to break can be relied upon to break, every time, for everybody, no matter what you ran before it.

Exercise 4-4

SOLO

The two cells below have the same last line. Decide what each one will show before you run either of them, and write your two predictions down. Then run them from the top.

```
message = "ATG"  
print(message * 3)
```

```
%%sandbox  
print(message * 3)
```

Exercise 4-5

SOLO

Write a new cell that starts with `%%sandbox` and that prints `ATGATGATG` successfully. You may not change any cell above it. There is only one way to do this, and working out what it is tells you that you have understood the section.

Exercise 4-6

AI: EXPLAINER

Copy the traceback from the failing sandbox cell above, paste it to the assistant, and ask it to explain what went wrong and why. Read its answer carefully. It has not seen this chapter and does not know that the cell was run in a separate process, so watch for the moment it explains the error confidently in terms it has invented. Write down in your logbook what it got right, what it got wrong, and how you knew.

The other magics in this book

Every widget you will meet in this course works the same way as the sandbox: you import it once near the top of the notebook, and after that a cell magic is available for the rest of the session. `%%steps` shows you the substitution and reduction steps inside a single statement. `%%puzzle` gives you shuffled lines to put back in order. `%%codelens` walks through a piece of code one statement at a time and draws what is in memory as it goes. And `%%sandbox` runs a cell as though it were a script.

If a magic reports that it does not exist, the answer is almost always that you have not run its import in this kernel, or that you restarted the kernel since you did.

Which one to use, and when

Use a notebook when you are exploring, when you want to see an answer before deciding on the next step, and when the text and the code belong together, which is why the notes in this book are notebooks. Use a script when you want something to run the same way every time, when you want to hand it to somebody else, and when the point is the result rather than the journey.

And use the two habits from this chapter to keep yourself out of trouble. When you want to know whether a notebook really works, restart the kernel and run everything from the top. When you want to know whether a piece of code stands on its own, put it in a `%%sandbox` cell and watch it either run or fall over.

That second habit is going to matter a great deal once the assistant starts writing code for you. Code pasted into a notebook that appears to work may only be working because of something the kernel already had lying around from an earlier cell, and that is a very easy way to accept a piece of code that is broken. Dropping it into a sandbox forces it to prove that it stands up by itself. The assistant predicts that its code works. The sandbox is one of the ways you make it prove it.

Exercise 4-7

SOLO

Make this week's logbook entry. Describe in a few sentences, in your own words and without looking back at this chapter, what the kernel is and why a `%%sandbox` cell cannot see it. If you cannot do it without looking, read the chapter again and then try once more.

Two kinds of cells

A notebook is a stack of cells, and there are two kinds.

Text cells hold prose, written in a simple formatting language called Markdown. The paragraph you are reading is one of them. When a text cell is finished it is rendered, meaning you see the formatted result rather than the raw text you typed. Double-click one and it turns back into editable text; press Shift and Enter together and it renders again.

Code cells hold Python. They have a run button on the left and a slot on the left of that where a number appears after they have been run. Everything in a code cell is Python, which means a line of prose typed into a code cell by mistake will produce a `SyntaxError`, and a line of Python typed into a text cell by mistake will simply sit there looking like a sentence and never run. Both mistakes are common and both are worth recognizing on sight.

Exercise 4-8

SOLO

Double-click this paragraph. You should see the raw Markdown, with the exercise heading above it starting with four hash marks. Add a sentence of your own at the end. Press Shift and Enter to render it back. Now double-click it again and remove your sentence. You have just done the two things you will do to every text cell for the rest of the term.

Running a code cell

There are three ways to run the cell your cursor is in, and the difference between them is only where you end up afterward.

Shift and Enter together runs the cell and moves the cursor to the next one, which is what you want when you are working down a page. Control and Enter together, or Command and Enter on a Mac, runs the cell and leaves the cursor where it is, which is what you want when you are editing one cell repeatedly until it works. Alt and Enter together, or Option and Enter on a Mac, runs the cell and makes a new empty one below it, which is what you want when you are building something up a piece at a time.

Run the cell below with Shift and Enter.

```
dna = "ATGGTGCATCTGACTCTGAGGAGAAG"  
len(dna)
```

Two things happened. A number appeared in the brackets to the left of the cell, and the value `27` appeared underneath it.

The number is the execution count. It says which run this was, counting every code cell you have run since the kernel started. The first cell you run gets `1`, the next gets `2`, and so on. If you run the same cell three times its number changes three times, because the count records runs and not cells.

The value underneath appeared because the last line of the cell was an expression sitting on its own. A notebook shows you the value of the final expression as a courtesy, without you asking. This is a notebook habit rather than a Python rule: put `len(dna)` on its own line in a script and nothing at all appears on your screen.

Exercise 4-9

SOLO

Run the cell above two more times and watch the number in the brackets. Predict what it will say before each run. Then add a second line `dna.count("A")` below the `len(dna)` line, run it once more, and see which of the two values gets displayed. Work out the rule from what you observe.

Order on the page and order in time

This is the part of notebooks that catches everyone, so it is worth meeting it deliberately rather than by accident at midnight.

A script runs from top to bottom, always. A notebook runs in whatever order you clicked. The cells are laid out down the page, but the kernel remembers them in the order you ran them, and those two orders have nothing to do with each other. A notebook where you have been jumping around can show a page that reads perfectly and produces results that nobody could reproduce, including you,

because the state that produced them came from a sequence of clicks that is not written down anywhere.

The execution counts are the only record you have of that sequence. When you open somebody's notebook and the counts down the page read 4, 1, 9, 2, you are looking at a page that has never once been run in the order it is written.

```
codon_count = 64
codon_count
```

Exercise 4-10

SOLO

Click into the `dna` cell further up the page and run it again. Then come back down here and run the `codon_count` cell. Look at the execution counts on the two cells and confirm that the higher number is now on the cell that appears earlier on the page. That is a notebook telling you the truth about itself, and it is the only warning you will get.

Adding, moving, copying and deleting cells

Hover in the space between two cells and a small toolbar appears offering a new code cell or a new text cell at that spot. That is the easiest way to add one, and it is the way you will use most of the time.

Everything else is faster from the keyboard, and to use the keyboard you need to know about the two modes a cell can be in. When your cursor is inside a cell, typing changes its contents, and that is edit mode. Press Escape and the cursor leaves the cell; the cell is still selected, shown by a bar down its left side, but now the keys are commands rather than text. That is command mode. Press Enter to go back into the cell.

In command mode, `A` makes a new cell above the selected one and `B` makes one below. `M` turns the selected cell into a text cell and `Y` turns it back into a code cell. `C` copies the selected cell, `X` cuts it, and `V` pastes what you copied below the selection. Pressing `D` twice, quickly, deletes the selected cell, and `Z` undoes that, which is a good pair to know in that order.

To move a cell up or down, use the two arrow buttons in the toolbar that appears at the top right of the selected cell, or pick the cell up by its left edge and drag it. Every one of these commands also lives in the Command Palette, which you open with Control and Shift and P together, or Command and Shift and P on a Mac. Typing part of a command name there shows you the command along with whatever keyboard shortcut your copy of VScode has assigned to it, which is the reliable way to answer any shortcut question rather than trusting a list in a book.

Exercise 4-11

SOLO

Do all of this without touching the mouse after the first click. Click on this cell and press Escape to get into command mode. Press **B** to make a new cell below, press Enter to get into it, and type `print("first")`. Press Escape, then **B** again, then Enter, and type `print("second")`. Run both. Now select the `print("first")` cell and move it below the other one using the arrows in its toolbar, and run both again in page order to confirm the output changed. Finally select each of your two cells in turn and press **D** twice to remove them, leaving the page as you found it. If you delete one cell too many, press **Z**.

Choosing the kernel

The cells are the notebook. The thing that actually runs your code is a separate program called the kernel, and it is a real Python interpreter living in the background with all your variables in it. The notebook sends a cell to the kernel, the kernel runs it and sends back whatever came out.

Look at the top right of the notebook window. There is a button there naming the kernel this notebook is attached to. If it says something like Select Kernel, no interpreter is attached yet, and nothing will run until you attach one. Click it and choose the environment for this course. If you pick a different Python installation, your code will run, but the course tools will not be installed in it, and every `import` of a course package will fail with a `ModuleNotFoundError` that has nothing to do with your code. A large fraction of the confusing failures in the first weeks are a wrong kernel.

You can ask the kernel what it is from inside the notebook.

```
import sys
sys.version.split()[0]
```

That is the version of Python answering. To find out which installation it came from, run `sys.executable` in a cell of your own; the path it returns should sit inside the course environment. Comparing that path with the name on the kernel button is how you settle an argument about which Python is actually running your code.

Exercise 4-12

SOLO

Make a new cell below this one, put `sys.executable` in it, and run it. Read the path. Then click the kernel button at the top right and read the name it shows. Satisfy yourself that they describe the same installation. Delete your cell when you are done.

Restarting the kernel

Because the kernel keeps everything you have run, it will happily keep a variable you have since deleted from the page, a function whose definition you have changed, and a value from an experiment you abandoned twenty minutes ago. Code that leans on any of those looks like it works and will not work for anyone else.

Restarting the kernel throws all of it away and gives you a fresh interpreter with nothing in it. Your cells and their text are untouched; only the memory is emptied. The Restart button is in the toolbar along the top of the notebook, next to a Clear Outputs button that wipes the displayed results without restarting anything, and next to Restart and Run All, which restarts and then runs every cell down the page in order.

Restart and Run All is the one that matters. It is the only way to find out whether your notebook does what it appears to do, because it is the only way to run it the way a stranger would. Anything that survives that has been proved to work in page order. Anything that fails was depending on something that is not written down.

Exercise 4-13

SOLO

Run the cell below, so that `virus` exists in the kernel.

```
virus = "HIV"  
virus
```

Now press Restart. Then run the cell below, and only that cell. Predict first what will happen, and be specific about the error you expect.

```
virus.lower()
```

You should get a `NameError` saying that `virus` is not defined, because the restart emptied the kernel and the cell that creates `virus` has not been run since. Now use Restart and Run All, and watch the whole page run from the top with the counts numbering themselves neatly downward. Both cells work again, because this time they ran in the order they are written.

If a notebook of yours is behaving strangely and you cannot see why, restart and run all before you do anything else. Most of the time the strangeness is stale memory, and most of the rest of the time the restart turns a mystery into an error message you can read.

What a notebook is on disk

The `.ipynb` file holds your cells, in page order, along with whatever output each cell produced the last time it ran. That is why you can send someone a notebook and they see your results before running anything, and it is also why the file changes every time you run a cell even if you typed nothing. It is worth knowing when you start putting notebooks into a repository and wonder why a file you only looked at is showing up as modified.

A notebook is a good place to work something out, with prose and code and results interleaved on one page. A `.py` file is a better place for code that other code depends on, because it starts from nothing every time and therefore says exactly what it needs.

Exercise 4-14

SOLO

Make a new notebook of your own, from the Command Palette or from File and New File. Attach the course kernel to it. Give it a text cell with a heading and a sentence saying what it is for, then a code cell that stores a DNA string in a variable, then another that prints its length. Restart and run all, and confirm that the counts read 1 and 2 down the page. Save it somewhere you will find it again. That notebook is where you should try things out this week, and getting it set up now means the next time you have a question you can answer it in thirty seconds instead of ten minutes.

5 Meet the assistant

The week-one introduction to working with an AI assistant in this course.

You have just read how an assistant produces its answers: it guesses a plausible next word, over and over, with no separate sense of whether the sentence it is building is true. That is the last thing you needed to know about the machine before you actually start talking to it. This note is about the practical arrangement: where the assistant lives, what you are and are not allowed to ask of it at each point in the term, and the one habit that everything else in this course is built to protect.

Where the assistant lives

For this course, the assistant lives in your browser, as Microsoft 365 Copilot, which you already have free access to through your university account. Inside it sits a second mode worth knowing about from day one, the **Study and Learn agent**, which behaves less like a machine that answers and more like a patient teaching assistant that asks you questions back. You will meet it properly in a later week; for now, know that it exists and that “the assistant” from this point on can mean either mode depending on what a given exercise asks for.

VS Code, by contrast, stays an AI-free space for the whole course. No Copilot extension, no inline suggestions, nothing typing over your shoulder while you write. This is not nostalgia for a harder way of working. You cannot learn to judge code you did not write until you have spent real time writing code yourself, one substitution and one reduction at a time, and an assistant humming along inside your editor is exactly the thing that would let you skip that part. Keep the two spaces separate: the browser is where you consult the assistant, the editor is where you think.

Badges: what you are allowed to ask of it, and when

Not every exercise permits the same amount of help, and the course marks this explicitly rather than leaving it to guesswork. Exercises that involve the assistant carry a small badge naming the **role** it is allowed to play, and the roles form a ladder that gets more permissive as the term goes on and your judgment gets sharper:

Early on, a badge such as **AI: Explainer** means the assistant may explain something to you, an error message, a line of code, but you still do the deciding and the running yourself. Much later, a badge such as **AI: Developer** means you may hand it a real piece of work, provided you have already learned to specify it exactly and verify what comes back. You never get a badge you have not earned the skills for yet; the ladder is cumulative, and each step assumes everything below it. Some exercises carry no AI badge at all but instead a lock, **SOLO**, which means exactly

The role ladder

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · Worker ·
Collaborator · Developer

what it says: no assistant, do it yourself first. SOLO exercises are not a punishment. They are where you build the very judgment the badges later ask you to spend.

The logbook

Once a week, you will write one entry in your **logbook**: a short, candid note about something the assistant got right, something it got wrong, and how you knew which was which. You already wrote the first one, in the exercise at the end of last note. Keep the habit going. By the end of the term the logbook will be a record of your own growing skill at telling a plausible answer from a correct one, which is a more useful thing to have at the end of a course than almost anything else you will produce.

! The rule the whole course turns on

The AI predicts; the machine, or a test, proves. Every badge, every SOLO marker, and every widget in this course exists to put that rule into practice. Whatever the assistant tells you, in whatever role, treat it as a claim to be checked, not a fact to be filed away. Nothing that follows makes sense without this.

Exercise 5-1

AI: EXPLAINER

Ask the assistant, in the browser, what a **license badge** and a **SOLO marker** are for in a course, without telling it anything about this course specifically. Read its guess. Then compare it against what this note just told you. Where did it match, and where did it just produce something plausible-sounding that happened to be wrong for our purposes? This is the smallest possible version of the whole term: ask, read, check against the real thing.

For your logbook this week, note which badge you are most looking forward to earning, and which one makes you the most nervous, and why.

6 About Microsoft Copilot

The assistant this course uses is Microsoft Copilot, which the university provides to you as a student. It is worth being clear about which Copilot this is, because Microsoft has attached the name to several quite different products. The one you want is the chat assistant that runs in a browser tab. It is not GitHub Copilot, which is a separate tool that suggests code inside an editor as you type, and which this course deliberately does not use, because the whole discipline being taught here depends on the assistant living somewhere you have to go to rather than appearing unasked beside your cursor. Your editor stays a place where nothing writes code but you.

The ladder so far

Explainer

Still to come: Translator,
Illustrator, Comparer,
Drafter, Worker,
Collaborator, Developer

Logging in

Go to the Copilot landing page at copilot.microsoft.com and choose Work rather than Web or Personal, then sign in with your university credentials. The Work option matters. It is what gives you access through the university's agreement, and it is what keeps what you type covered by that agreement rather than by a consumer account's terms.

If you find yourself in a session that does not show you as signed in with your university account, sign out entirely and start again from the landing page, because a half-signed-in session will sometimes work well enough to be confusing and then fail on something you needed.

Keeping it in the browser

Keep Copilot in a browser tab, separate from your editor, for the whole course. This is not an arbitrary restriction. Every habit this material builds depends on there being a visible boundary between the code you wrote and the code the assistant proposed, and on your having to make a deliberate decision to cross it. An assistant that completes your lines while you type erases that boundary, and with it the moment at which you would have read what you were about to accept.

You will move text across the boundary in both directions, pasting a function or an error message into the chat and copying a suggestion back out, and every one of those crossings is a point at which you read what you are moving. That is the point.

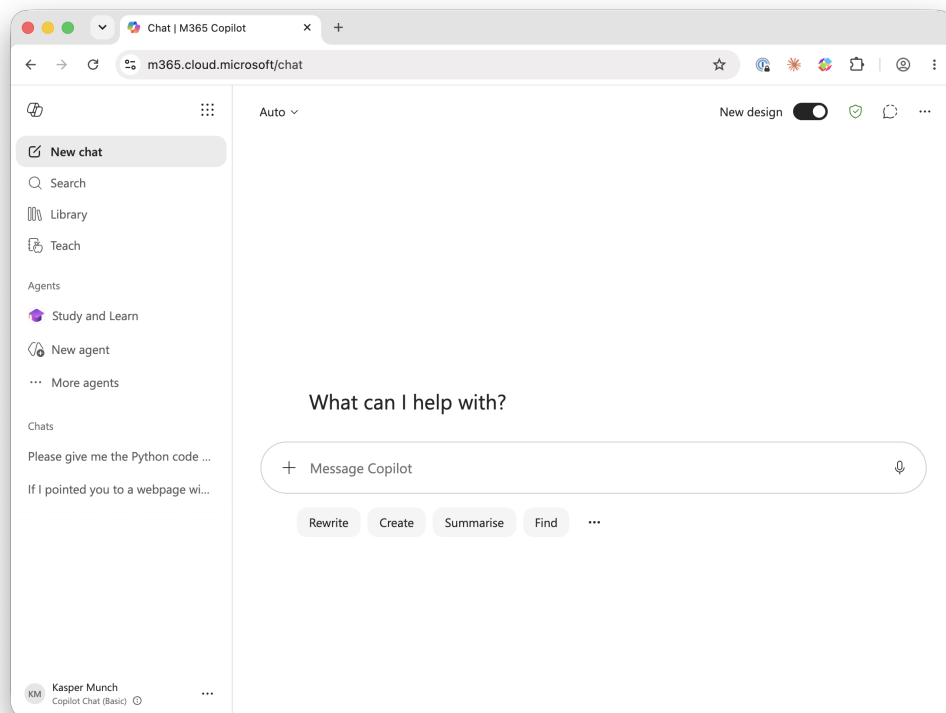


Figure 6.1: Choosing Work on the Copilot landing page before signing in with university credentials.

What to expect of it

Copilot is a conversation. It keeps the earlier turns of that conversation in view, which is useful when you are refining a piece of code, and misleading when you have moved on to something else, because it will happily carry an assumption from twenty minutes ago into an answer about a different problem. When you change topic, start a new conversation. It costs nothing and it removes a whole class of confusing answers.

It will also give you different answers to the same question asked twice, and this is not a fault to be worked around but a property to be used. Two answers that agree tell you a little. Two answers that disagree tell you a great deal, because at least one of them is wrong and you now know to find out which.

The version of the underlying model behind Copilot changes over time, without announcement, and answers you get this term may differ from answers a classmate got last term or that you get next month. Nothing in this course depends on a particular version, because nothing in this course asks you to trust an answer without checking it. That is precisely why the material is built the way it is.

7 Values, operators, logic

This chapter is about values and variables, the two most central concepts in programming.

```
print("Four times two is", 4 * 2)
```

Notice how you can print more than one thing at a time if you put commas between the values you want to print? We can group computations using parentheses, just like in normal math. Try this:

```
print("10 / (2 + 3) is", 10 / (2 + 3))
print("(10 / 2) + 3 is", (10 / 2) + 3)
```

In addition to the regular math operators, there are a few extra operators that we call *comparison operators* because they are used to compare two values, e.g., two numbers.

Try this:

```
print("Is 5 greater than -2?", 5 > -2)
print("Is 5 greater or equal to -2?", 5 >= -2)
print("Is 5 less or equal to -2?", 5 <= -2)
print("Is 5 less than 7 - 2?", 5 < 7 - 2)
print("Is 5 equal to 7 - 2?", 5 == 7 - 2)
```

Operator	Operation
<	less-than
>	greater-than
<=	less-than-or-equal
>=	greater-than-or-equal
==	equal
!=	not equal

As you may have noticed, running this code and comparing things using these operators, we always produce either `True` or `False`. E.g., the following

```
print(5 < 7)
```

prints the value `True` because 5 is smaller than 7. `True` and `False` are special values in Python that we can use (and print if we like) just like any other Python value:

```
print(True)
print(False)
```

Exercise 7-1

SOLO

Try to write and run the code below. Compare each line to what is printed when you run the code. Make sure you understand why.

```

print("I have", 25 + 30 / 6, "of something")
print("I have", 100 - 25 * 3 % 4, "of something else")

print("Is it true that 3 + 2 < 5 - 7?")
print(3 + 2.1 < 5.4 - 7)

print("3 + 2.1 is", 3 + 2.1)
print("5.4 - 7 is", 5.4 - 7)

print("Oh, that's why it's False.")

```

Exercise 7-2

SOLO

An additional comparison operator even tests if something is a part of something else. That operator is called `in`. One use of it is to test if one string is part of another string. Try this to figure out how it works:

```

print("Hell" in "Hello world")
print("Hello world" in "Hello world")
print("Hello world" in "Hell")
print("lo wo" in "Hello world")
print("Artichoke" in "Hello world")

```

Exercise 7-3

SOLO

Say the supermarket has chocolate bars for 7 kr. Write a small Python program (in a file called `chocolate.py`) that prints how many chocolate bars you can get for your 30 kr. You should run it like this;

For example, it could output something like this:

```
$ python chocolate.py
```

to have it print something like this:

I can buy 4.285714285714286 chocolate bars!

Exercise 7-4

AI: EXPLAINER

We mentioned a special operator called *modulo*. Google it if you do not remember what it does. How about *integer division*. Explain both to a fellow student (or to yourself out loud).

Then ask the assistant to explain modulo and integer division to a first-week student, and compare its explanation to yours. Where the two differ, settle it by writing the calculation in Python and running it: `17 % 5` and `17 // 5` take three seconds to check, and no amount of confident prose beats them.

Exercise 7-5

SOLO

You obviously cannot go buy 4.3 chocolate bars in a store. You will have to settle for 4. Can you change the program you made in exercise 7-3 to print the number of bars you can buy and the change you then have left? Use the modulo and integer division operators. Something like:

I can buy 4 chocolate bars, leaving me with 2 kr in change.

Exercise 7-6

SOLO

What happens if you try to run the following program?

```
print( 1 / 0 )
```

If you get an error? What kind of error? Why do you think you get that error? Do you think it makes sense?

Exercise 7-7

SOLO

You probably know the Pythagorean theorem for computing a right-angled triangle's hypotenuse (the longest side). The Pythagorean theorem looks like this: $a^2 + b^2 = c^2$. Here c is the length of the hypotenuse, and a and b are the lengths of the triangle's two legs. So if we have a triangle where $a = 5$ and $b = 2$ and we want to find c^2 we can do this in Python:

```
print("The squared length of the hypotenuse is:", 5**2 + 2**2)
```


Exercise 7-8

SOLO

However, we are rarely interested in the *squared* length of the hypotenuse. Can you modify the code you wrote in exercise 7-7 so you compute c instead of c^2 ? Taking the square root of a number is the same as taking that number and exponentiating it to 0.5, so the square root of x is $x^{0.5}$. Do you know of a Python operator that does exponentiation?

Logic

Now you know how to use the comparison operations to produce a `True` or `False` value. There are three additional operators that let you express more elaborate “True/False” statements than with the comparison operators alone. These are the *logical* operators: `and`, `or` and `not`.

Exercise 7-9

SOLO

Go through the code below and see if you can figure out what each line does. Then, write the code into your editor and run it to see what happens.

```
print(2 < 3)
print(10 < 12)
print(8 > 100)
print(2 < 3 and 10 < 12)
print(8 > 100 and 2 < 3)
print(8 > 100 or 2 < 3)
print(not 8 > 100 and 2 < 3)
print(not 8 > 100 and not 2 > 3)
```

Did it do what you expected? Can you explain each line?

Exercise 7-10

SOLO

When exposed to the operators `and`, `or` and `not`, some values are considered true and others are considered false. What happens when you put `not` in front of something that is considered true or false? Decide what you think and why before you write the code and try it out.

```
print(not True)
print(not False)
print(not 0)
print(not -4)
print(not 0.000000)
print(not 3.14159265359)
print(not "apple")
print(not "")
```

From the code above, try to find out which values Python considers true and which it considers false. Can you come up with a rule?

Exercise 7-11

SOLO

The logical operator `and` takes two values (the one to the left of the operator and the one to the right) and figures out whether *both* the left and the right expression are true. It boils down to this:

Left expression	Right expression	Result
True	True	True
True	False	False
False	True	False
False	False	False

Write some code to confirm that the table above is correct using Python. For example, to test the first case, do this:

```
print(True and True)
```

Exercise 7-12

SOLO

Python will only do the necessary work to determine if a logical expression is true. That means that if the value left of `and` is considered false by Python, then there is no reason to look at the right value since it is already established that they are not *both* considered true. In this case the expression reduces to the *left* value. I.e. `False and True` reduces to `False`.

If Python considers the value left of `and` true, then It needs to look at the right value, too, to establish if they are both considered true. In this case, the expression reduces to the value on the right. I.e., `True and False` reduces to `False`.

A rule of thumb is that the whole expression reduces to the last value that Python needs to consider to decide if the whole expression is true or false. Use that rationale to explain how the two last combinations in exercise 7-11 are evaluated.

Exercise 7-13

SOLO

Like the `and` operator, the `or` operator also takes two values. However, the `or` operator determines whether *one* of the two values is true. Thus, the `or` operator boils down to this:

Left expression	Right expression	Result
True	True	True
True	False	True
False	True	True
False	False	False

Write some code using Python to confirm that the table above is correct. For example, to test the first case, do this:

```
print(True or True)
```

Exercise 7-14

SOLO

As with the `and` operator, Python will not do any more work than absolutely necessary when evaluating an expression with 'or'. So if the value left of `or` is considered true by Python, then there is no reason to look at the right value since it is already established that at least *one* of them is considered true. In this case the expression reduces to the *left* value. I.e. `True or False` reduces to `True`.

If the value left of `or` is considered false by Python, then Python still needs to look at the right value to establish if at least one of them is considered true. In this case, the expression reduces to the right value. I.e., `False or True` reduces to `True`.

Again, the whole expression reduces to the last value that Python needs to consider to decide if the whole expression is true or false. Use that same rationale to explain to yourself how the two last combinations in exercise 7-13 are evaluated.

Exercise 7-15

SOLO

Remember what you learned in exercise 7-10 about which values are considered true and which are considered false. Combine that with what you learned in exercise 7-11 and exercise 7-13 about what logical expressions reduce to and see if you can figure out what is printed below and why. Use the rule-of-thumb from exercise 7-14. Decide what you think before you write the code and try it out.

```

print(True and 4)
print(0 and 7)
print(-27 and 0.5)
print(42 and 0)
print("apple" and "orange")
print("apple" and "")
print(42 or 0)
print("apple" and "")
print("apple" or "")

```

If you were surprised by what was printed, maybe go back and look at exercise 7-11 and exercise 7-13 again.

Exercise 7-16

SOLO

Recall the `in` operator from exercise 7-2? There is also an operator called `not in`. I guess you can imagine what that tests. Try it out.

Variables

“ By now, you probably feel the first signs of brain overload. If you do not take breaks, your brain may overheat and explode - we have seen that happen. One of the nice things about the brain is that it works when you rest. Archiving and understanding a lot of new information takes time, and force-feeding your brain will not help. The last part of this chapter is very important so now might be a good time for a good long break. ”

This section is about *variables* and is where the fun begins. A variable is a way of assigning a name to a value. `8700000` is just a value, but if we assign a name to it, then it gets a special meaning:

```

number_of_species = 8700000
print(number_of_species)

```

In this case, the variable `number_of_species` represents the estimated number of eukaryotic species on the planet, which is 8700000. So `8700000` is the *value*, and “`number_of_species`” is the variable name. Write the code above into a file and run it. Notice how this lets us refer to the *value* using the *variable name*. What appears in the terminal when you do that? Do you see `number_of_species` or `8700000`?

As you can see in the small program above, one of two different things happens when a variable name appears in Python code:

- **Assignment:** When a variable name appears to the left of an equal sign, a value is assigned to the variable. This happens in the first line where `number_of_species` is *assigned* the value `8700000`.
- **Substitution:** In all other contexts, the variable is substituted for its value. This happens in the second line where Python *substitutes* the variable name `number_of_species` for its value `8700000` and then prints that.

That is it, but let us take the example further and create another variable to which we assign the value `1200000`. That is the number of species discovered so far. Now, let us add this to the program and use it to compute the number of species we have yet to identify. Start by reading the code below super carefully. Remember that a variable is *either* assigned a value or substituted for the value it represents. For each occurrence of the variables below, determine if they are being assigned a value or if they are substituted for their value.

```
number_of_species = 8700000
number_discovered = 1200000
number_unidentified = number_of_species - number_discovered
print(number_unidentified)
```

Now write the code into a file and run it. Take some time to let it sink in that variables are extremely useful for two reasons:

1. Variables give meaning to a value. Without the variable name, the value of `1200000` could just as well be the number of people that live in Copenhagen. However, by giving the value a meaningful name, it becomes clear what it represents.
2. We can assign new values to variables (that is why they are called variables). For example, we can change the value of `number_discovered` as new species are discovered.

Your variable names can be pretty much anything, but they have to start with a letter or an underscore (`_`), and the rest of the name has to be either letters, numbers, or underscores. To be clear, a space is *not* any of those things, so do not use spaces in variable names. Above all, be careful in your choice of variable names. Variable names are case-sensitive, meaning that `count` and `Count` are different variables. Stick to lowercase variable names. That makes your code easier to read.

Exercise 7-17

SOLO

For each occurrence of the variables below, determine if they are being assigned a value or if they are substituted for their value.

```
breeding_birds = 4
print(breeding_birds)
breeding_birds = 5
print(breeding_birds)
```

Exercise 7-18

SOLO

For each occurrence of the variables below, determine if they are being assigned a value or if they are substituted for their value.

```
breeding_birds = 4
print(breeding_birds)
breeding_birds = breeding_birds + 1
print(breeding_birds)
```

Exercise 7-19

SOLO

What happens if you take the first example in this section and swap the two lines? So, going from this:

```
number_of_species = 8700000
print(number_of_species)
```

to this:

```
print(number_of_species)
number_of_species = 8700000
```

Explain to yourself what happens in each case. What kind of error do you get with version two, and why? Remember @important-oath2!

Exercise 7-20

SOLO

Write the following code in a file, save it, and run it.

```
income = 45000
tax_percentage = 0.43
tax_amount = tax_percentage * income
income_after_tax = income - tax_amount
print('Income after tax is', income_after_tax)
```

You should get an error that looks at lot like this one:

```
Traceback (most recent call last):
  File "tax.py", line 3, in <module>
    tax_amount = tax_percentage * income
NameError: name 'tax_percentage' is not defined
```

It says that the error is on line 3. Can you figure out what is wrong? Hopefully, you will now appreciate how much attention to detail is required when programming. Every tiny, little symbol or character in your code is *essential*.

Different types of values

By now, you probably have a pretty good idea of what a value in Python is. So far, you have seen text like `'Banana'`, integers like `7`, and numbers with a fractional part like `4.25`.

In Python, a text value is a *type* of value called a *string*, which Python denotes as `str` (abbreviation for “string”). So `'Banana'` is a string, and so is `'Banana split'`. There are two types of numbers in Python. Integers (7, 42, and 3) are called `int`. Numbers with a fractional part (like 3.1254 and 4.0) are that are called `float` (an abbreviation for “floating-point number”).

As I mentioned earlier, `True` and `False` are Python values too. They are called booleans or `bool`, named after an English mathematician called George Boole famous for his work on logic.

So the different *types* of values we know so far are:

Name	Type in Python	Examples
String	<code>str</code>	<code>"hello"</code> , <code>'9'</code>
Integer	<code>int</code>	<code>0</code> , <code>2721</code> , <code>9</code>
Floating-point	<code>float</code>	<code>1.0</code> , <code>4.4322</code>
Boolean	<code>bool</code>	<code>True</code> , <code>False</code>
None	<code>NoneType</code>	<code>None</code>

In case you did not notice, I added a special type at the end that can only have the value `None`. I may sound a little weird, but in programming, we sometimes need a value representing nothing or `None`. For now, just make a mental note that `None` is also a Python value.

When you do computations in Python, it is no problem to mix integers and floating-point numbers. Try this:

```
print("What is 0.5 * 2?", 0.5 * 2)
print("What is 3 / 2?", 3 / 2)
```

As you can see we can also make computations using only integers that result in floating-point numbers.

Some of the math operators not only work on numbers, but they also work on strings. That way, you can add two strings together. It is no longer math, of course - but quite handy.

```
fruit = 'Ba' + 'na' + 'na'
print(fruit)
```

Exercise 7-21

AI: EXPLAINER

If you try to combine different types of values in ways that are not allowed in Python, you will get an error. Try each of the following weird calculations, and read each error message carefully.

```
x = 3 - '1.5'  
print(x)
```

```
x = None - 4  
print(x)
```

For each one, paste the error into the assistant and ask which of the two values Python is unhappy about, and why. Then decide whether you agree before you move on. These messages name types, so the assistant's answer is easy to check: if it says Python objected to the string, the message should be saying so too.

Exercise 7-22

SOLO

Write these two examples and compare the resulting values of `x`

```
x = '9' + '4'  
print(x)
```

```
x = 9 + 4  
print(x)
```

Exercise 7-23

SOLO

Try these two examples. What happens in each case? Does it make sense?

```
x = '72' * 3
```

```
x = '72' * '3'
```

Exercise 7-24

SOLO

Will this work? Use what you have learned from the other exercises and try to predict what will happen here. Then, write the code and try it out.


```
x = 'Ba' + 'na' * 2
print(x)
```

Exercise 7-25

SOLO

Sometimes, you may need to change a string to a number. You can do that like this:

```
some_value = "42"
other_value = int(some_value)
```

Write some code that converts strings to numbers and numbers to strings. Remember that numeric values are either integers or float. Use `int`, `float` as in the example above. You will notice that only meaningful conversions work. E.g., this will not work: `number = int('four')`. To convert a number to a string, you can use `str`.

Having completed the above exercises, you should take note of the following four important points:

1. All Python values have a type. You know about strings, integers, floating-points, and booleans so far.
2. Math operators let you do cool things like concatenating two strings by adding them together.
3. The flip side of that cool coin is that Python will assume you know what you are doing if you add two strings (`'4' + '4'` is `'44'` not `8`) or multiply a string with an integer (`'4' * 4` is `'4444'` not `16`).
4. You can change the type of a value, e.g., `'4'` to `4` or `1` to `1.0`.
5. Python will throw a `TypeError` if you try to combine types values of values in ways that are not allowed.

“ **Escape characters:** An escape character is a backslash `\` followed by a single character. `\n` and `\t` are the most commonly used ones. ”

Exercise 7-26

SOLO

What do you think is printed here?

```
main_course = 'Duck a la Banana\n'
dessert = 'Banana split\n'
menu = main_course + dessert
print(menu)
```

Can you figure out what the special character `\n` represents?

Exercise 7-27

SOLO

What do you think is printed here?

```
dish_one = 'Banana\t\tsplit'
dish_two = 'Chocolate\tcake'
print(dish_one)
print(dish_two)
```

Can you figure out what the special character `\t` represents?

Mixed exercises

Each chapter in the book ends with a set of mixed exercises meant to allow you to combine what you have learned so far. In this case, they are meant to train your familiarity with the following topics:

- Strings
- Math
- Logic
- Types of values
- Variables

Exercise 7-28

SOLO

What happens if you try to run the following program?

```
print("What happens now?", 1 / )
```

If you get an error, why do you think you get that error?

Exercise 7-29

SOLO

What happens if you try to run the following program?

```
print("What happens now?", 1 / 3
```

If you get an error, why do you think you get that error? Can you fix it? (Hint: EOF is short for End Of File)

Exercise 7-30

SOLO

Determine, for each of the eight occurrences of the variable `x` below, where it is being assigned a value and when it is substituted for its value:

```
x = 1
x = x + 1
x = x + 1
x = x + 1
print(x)
```

Then, figure out what is printed and why (remember @important-oath2). What value does `x` represent at each occurrence in the code?

Exercise 7-31

SOLO

Some comparison operators also work with strings. Consider this code:

```
print("apples" == "pears")
```

What is printed here? Write the code and see for yourself once you think you know. If you were wrong, make sure you understand why.

Exercise 7-32

SOLO

Consider this code:

```
print('aaaaaa' < 'b')
print('a' < 'b')
print('aa' < 'ab')
print('99' > '100')
print('four bananas' > 'one banana')
```

What is printed here? Write the code and see for yourself once you think you know. By what rule does Python decide if one string is smaller than another? You may have a clue if you have looked something up in an encyclopedia recently. Also, try to google “ASCII table”.

Exercise 7-33

SOLO

Consider this code:

```
print('banana' < 'Banana')
```

What is printed here? Write the code and see for yourself once you think you know.

Exercise 7-34

SOLO

Do you think it is allowed to use relational operators on values of different types? Try these out and see for yourself:

```
print('Banana' > 4)
```

```
print('42' == 43) # this one is dangerous...
```

```
print(4 in '1234')
```

Practice reading this kind of error (`TypeError`).

Exercise 7-35

SOLO

Can you use the `in` operator to test if this mini gene is part of the DNA string?

```
mini_gene = 'ATGTAG'  
dna_string = 'GCTATGTAGGTA'
```

Exercise 7-36

SOLO

Say you have two strings `"4"` and `"2"`. What happens if you add them like this: `"4" + "2"`. Can you convert each one to integers so you get `6` when you add them? (have a look at exercise 7-25 if you do not remember).

Exercise 7-37

SOLO

What happens if you run this code? Do you get an error? Do you remember why?

```
1value = 42
```

Exercise 7-38

SOLO

What happens if you run this code?

```
print('Hi')  
print('Hi')  
print('Hi')
```

Compare this to what happens when you run this code:

```
print('Hi\nHi\nHi')
```

Do you remember what `\n` represents? What does it tell about what is added at the end every time you print something?

Exercise 7-39

SOLO

Make three exercises for your fellow students. See if you can make them so they test the understanding of (almost) all you have learned so far.

8 Precedence

This chapter is about how Python interprets (or evaluates) the code you write. It has a few fancy long words that may seem foreign to you. Do not let that throw you off. They are all just fancy names for something straightforward.

The order of things

Fear not. Precedence is just a nasty word for something we have already talked about. Precedence just means that some things are done before others or, more correctly, that some operations are performed before others. You already know that multiplication is done before addition. Another way of saying that is that multiplication *takes precedence* over addition. The expression below obviously reduces to 7 in two steps:

$$1 + 3 * 2$$

First, $3 * 2$ reduces to 6, and then $1 + 6$ reduces to 7. If we wanted to add 1 and 3 first, we would need to enforce this by adding parentheses:

$$(1 + 3) * 2$$

This is because the multiplication operator (`*`) has higher precedence than the addition operator (`+`). Here is the list of the most common operators and their precedence in Python:

Level	Category	Operators
Highest	exponent	<code>**</code>
	positive / negative	<code>+x</code> , <code>-x</code>
	multiplication	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>
	addition	<code>+</code> , <code>-</code>
	relational	<code>!=</code> , <code>==</code> , <code><=</code> , <code>>=</code> , <code><</code> , <code>></code> , <code>in</code> , <code>not in</code>
	logical	<code>not</code>
	logical	<code>and</code>
Lowest	logical	<code>or</code>

Sometimes, a statement contains adjacent operators with the same precedence. In this case, Python evaluates the expression from left to right. I.e., This following expression first reduces to $0.5 * 2$ and then to 1

```
2 / 4 * 2
```

The following one first reduces to $1 * 4$ and then to 4:

```
2 / 2 * 4
```

If you want Python to order the operations in any other way, you need to use parentheses (E.g., `2 / (2 * 4)`).

Exercise 8-1

SOLO

Look at each expression in the exercises below and use the table above to decide if it evaluates to `True` or `False` . Then, write the code and test if you were right. If not, figure out why.

```
2 + 4 * 7 == 2 + (4 * 7)
```

Exercise 8-2

SOLO

Does this reduce to `True` or `False` ?

```
4 > 3 and 2 < 1 or 7 > 2
```

Exercise 8-3

SOLO

Does this reduce to `True` or `False` ?

```
4 > 3 and (2 < 1 or 7 > 2)
```

Exercise 8-4

SOLO

Does this reduce to `True` or `False` ?

```
2 * 4 ** 4 + 1 == (2 * 4) ** (4 + 1)
```

Statements and Expressions

To talk concisely about programming (and to receive more useful help from your instructors), you need to have a bit of vocabulary. *Statements* and *expressions* are two words you need to know. Distinguishing between statements and expressions will help us discuss our code.

- A **statement** is a line of code that performs an action. Python evaluates each statement until it reaches the end of the file (remember @important-oath2?). `print(y * 7)` is a statement, and so is `x = 14`. They each represent a full line of code and perform an action.
- An **expression** is any code that reduces to one value. `y * 7` is an expression, and so are `y * 7 + 14 - x` and `4 > 5`.

We will talk more about how Python handles expressions in the next section, but right now, you must understand that statements *do something* while expressions are things that reduce to a value. Hopefully, this distinction will be more clear after completing the following exercises.

Exercise 8-5

SOLO

Did you notice in the above examples that `print(y * 7)` is a statement and `y * 7` is an expression? Yes, expressions can be part of statements. In fact, they most often are. Similarly, expressions are often made up of other smaller expressions. E.g., `y * 7` is part of the longer expression `y * 7 + 14 - x`.

Take a look at this code:

```
x = 5
y = 20
z = (x + y) / 2 + 20
print(z * 2 + 1)
h = 2 * x - 9 * 48
print(h)
```

Write down the code on a piece of paper. Now, mark all statements and all expressions. Remember that expressions are often made up of smaller expressions so that you can find a *lot* of them. E.g. `(x + y)`, `2 + 20`, and `(x + y) / 2 + 20` are all expressions. A single variable (like `x`) is also a small expression. Discuss with a fellow student. Do you agree on what to find?

Exercise 8-6

SOLO

Consider the following code:


```
greeting = 'Hello' + ' my '  
print(greeting + 'friend')
```

How many statements are there in this piece of code? How many expressions?

Substitution and Reduction

Although *substitution* and *reduction* may not sound like your new best friends, they truly are! If you remember to think about your Python code in terms of substitution and reduction, then programming will make a lot of sense. Understanding and using these simple rules will allow you to read and understand any code. If you do not, you may get by for a while - only to find yourself in big trouble later when things become more complicated.

You should remember, from the section on variables in the previous chapter, that variables in Python are either assigned a value or substituted for the value they represent.

In the first two lines of code below, the variables `x` and `y` are each assigned a value. Now consider the last line in the example:

```
x = 4  
y = 3  
z = x * y + 8
```

Here, `x` is *substituted* by the value `4`, and `y` is *substituted* by the value `3`. So now the expression after the equals sign reads `4 * 3 + 8`. Because we multiply before we add, `4 * 3` *reduces* to `12` so that the expression now reads `12 + 8`. Finally, this *reduces* to the value `20`. The very *last* thing that happens is that the variable `z` is assigned the value `20`.

You should do these steps every time you see an expression. You may think this is overdoing things a bit, but it is *not*. This kind of explicit thinking is what programming is all about, and it will become increasingly important as the course progresses. So make sure you make it a habit while it still seems trivial. Then, over time, it will become second nature.

Now raise your right hand and read the third and last oath out loud:

! Oath 3

I swear to consciously consider every single substitution and reduction in every Python expression that I read or write from this moment on.

This was the last of the three oaths, but it is *by far* the most important one. You can take your hand down now.

NB: You may not realize this at this point, but the last two subsections are the most important ones in the book. Go back and read them many times as you proceed through the course. If you explicitly think in terms of substitution and reduction, you will be fine. If you do not, you are entering a minefield with snowshoes on.

Exercise 8-7

SOLO

Do the substitution and reduction steps with pen and paper, then run it to check yourself by inserting a `print` statement at the end.

```
x = 7
y = 4 + x
2 + x * x**2 + y - x
```

Exercise 8-8

SOLO

Do the substitution and reduction steps with pen and paper, then run it to check yourself by inserting a `print` statement at the end.

```
a = 4
b = a
c = 2
c = a + b + c
```

Exercise 8-9

SOLO

Do the substitution and reduction steps with pen and paper, then run it to check yourself by inserting a `print` statement at the end.

```
x = 1
x = x
```

Exercise 8-10

SOLO

Do the substitution and reduction steps with pen and paper, then run it to check yourself by inserting a `print` statement at the end.

```
microsatellite = "GTC" * 41
```

Surprised?

Exercise 8-11

SOLO

Do the substitution and reduction steps with pen and paper, then run it to check yourself by inserting a `print` statement at the end.

```
mini_gene = "ATG" + "GCG" + "TAA"
```

What did you do first here? Does the order of additions matter? What operations does Python perform first when operators have the same precedence? (left to right or right to left)

Exercise 8-12

SOLO

Do the substitution and reduction steps with pen and paper, then run it to check yourself by inserting a `print` statement at the end.

```
number = 1 / 1 * 4
```

In what order are reductions made? Does the order of operations matter, and in what order does Python do the reductions?

Exercise 8-13

SOLO

Do the substitution and reduction steps with pen and paper, then run it to check yourself by inserting a `print` statement at the end.

```
x = 4
y = x + x
```

Exercise 8-14

SOLO

Do the substitution and reduction steps with pen and paper, then run it to check yourself by inserting a `print` statement at the end.

```
x = 4
x = x + 1
```

Exercise 8-15

SOLO

Do the substitution and reduction steps with pen and paper, then run it to check yourself by inserting a `print` statement at the end.

```
x = 4
x += 1
```

Compare the final value of `x` to that in exercise 8-14. Can you see what `+=` is a shorthand for? Nifty, right?

General exercises

The following exercises are meant to train your familiarity with the topics we have treated so far – in this case especially:

- Substitution
- Reduction
- Assignment
- Simple precedence rules
- Comparison operators
- Logical operators
- Distinction between text and numbers

Read each exercise and think hard about the questions before you code anything. *Then* write the code and try it out. Remember that it is crucial that you type it in – as super dull as it may be (remember oath one). This trains your accuracy and attention to detail, and it builds programming into your brain. Play around with each bit of code. Make small changes and see how it behaves.

There is a reason why there are lots of questions in this exercise but no answers. You are supposed to find them yourself if it takes you quite a while. That is the way you build understanding. Some of the questions may seem trivial, but do them anyway. If you only understand these concepts superficially, they will come back and bite you in the ass when things get more complicated.

Exercise 8-16

SOLO

Consider this code:

```
1.2 * 3 + 4 / 5.2
```

What does that expression evaluate to? Try to explicitly make all the reductions on paper before you write and run the code.

Exercise 8-17

SOLO

Consider this code:

```
1.2 * (3 + 4) / 5.2
```

What does that expression evaluate to? Try to explicitly make all the reductions on paper before you write and run the code.

Exercise 8-18

SOLO

Consider this code:

```
10 % 3 - 2
```

What does that expression evaluate to? Try to explicitly make all the reductions on paper before you write and run the code.

Exercise 8-19

SOLO

Consider this code:

```
11 % (7 - 5)**2
```

What does that expression evaluate to? Try to explicitly make all the reductions on paper before you write and run the code.

Exercise 8-20

SOLO

Consider this code:

```
a = 5
x = 9
banana = 7
x + 4 * a > banana
```

What does the last expression evaluate to? Try to explicitly make all the substitutions and reductions on paper before writing the code. What happens if you write and run the code? Why?

Exercise 8-21

SOLO

Consider this code:

```
dance = 'can'
dance = dance + dance
print("Do the", dance)
```

What is printed? Try to explicitly make all the substitutions and reductions on paper before writing the code. What happens if you write and run the code? Why?

Exercise 8-22

SOLO

Consider this code:

```
foo = 30
bar = 50
baz = bar + foo
print(baz)
bar = 10
print(baz)
```

There are two print statements. The first print statement prints `80`. But what about the second print statement? Does that print `80` or `40`? Find out and make sure you understand why it prints what it prints. If not, reread the section on substitution.

Exercise 8-23

SOLO

Consider this code:

```
1 == '1'
```

and this:

```
1 == 1.0
```

What does this reduce to? Try printing it and seeing once you think you know. If you were wrong, make sure you figure out why.

Exercise 8-24

SOLO

Consider this code:

```
a = '1'
b = '2'
c = a + b
print(a, b, c)
print(a + b == 3)
```

What is printed here? Write the code and see for yourself once you think you know. If you were wrong, make sure you understand why.

Exercise 8-25

SOLO

Consider this code:

```
a = 1
b = 2
c = a + b
print(a, b, c)
print(a + b == 3)
```

What is printed here? Compare to exercise 8-24. Write the code and see for yourself once you think you know. If you were wrong, make sure you understand why.

Exercise 8-26

SOLO

Consider this code:

```
x = 4
print(x + 2 and 7)
print(x + 2 or 7)
x = -2
print(x + 2 and 7)
print(x + 2 or 7)
```

What is printed here? Write the code and see for yourself once you think you know. If you were wrong, make sure you understand why.

Exercise 8-27

SOLO

I have shuffled the statements in the code below. Put them in the right order to make the code print `100`.

```
x = x + 4
print(x)
x = x * 5
x = x * x
x = 4
```

Exercise 8-28

SOLO

I have shuffled the statements in the code below. Put them in the right order to make the code print the string `'Banana'`.

```
y = 'n'
x = 'B' + y + x
print(x)
x = 'a'
y = (x + y) * 2
```

Exercise 8-29

SOLO

Make a puzzle exercise, like the two previous ones, for a fellow student.

Exercise 8-30

SOLO

Remind yourself of the different types of Python values you know. For example, one of them is an integer (`int`). Make a list.

Exercise 8-31

SOLO

You already know about several types of data values in Python. Two are integers called `int`, and decimal numbers (or floating points) called `float`. When you use an operator like `+` or `>` it produces a value. No matter what you put on either side of `>` it produces a boolean value (`bool`), `True` or `False`. For other operators, the type of value produced depends on which values the operator works on. Try this and see if you print an integer or a float (`8` or `8.0`):


```
x = 4
y = 2
result = x * y
print(result)
```

Now try to replace `4` with `4.0`. What type is the result now? Try to also replace `2` with `2.0`. What type is the result now? Can you extract a rule for what the `*` operator produces depending on what types the two values have?

Exercise 8-32

SOLO

In exercise 8-31, you investigated what types of values the `*` operator produces. Redo that exercise with the operators: `+`, `-`, `/`, `**`, `//`, and `%`. What are the rules for what is returned if both values are integers, one value is a float, or both values are floats?

Exercise 8-33

SOLO

Make a list of all the operators you know so far in order of precedence (without looking in the notes). Then check yourself.

Exercise 8-34

SOLO

What does his expression reduce to, and what type of value is it?

```
3 > 2
```

Exercise 8-35

SOLO

What does his expression reduce to, and what type of value is it? Do all the reduction steps in your head.

```
2 - 4 * 5 - 2 * 1/3
```

Exercise 8-36

SOLO

What does his expression reduce to, and what type of value is it? Do all the reduction steps in your head.

```
3 > 2 and 2 - 4 * 5 - 2 * 9
```

Exercise 8-37

SOLO

What is printed here and why?:

```
print(True and "banana" or "orange")
```

Try to change the `True` value to `False` and see what happens. Can you explain it? If not, look at exercise 7-14 again.

Exercise 8-38

SOLO

What does his expression reduce to? Do all the reduction steps in your head.

```
0 and 1 or 2
```

Exercise 8-39

SOLO

What does his expression reduce to? Do all the reduction steps in your head.

```
4 and 1 or 2
```

Exercise 8-40

SOLO

If you understood exercise 8-37, then you should also understand this one:

```
weather = 'rain'
what_to_do = weather == 'rain' and 'watch movies' or 'go swimming'
print(what_to_do)
```

What happens if you change `'rain'` in the first line to something else (like `'sun'`)?

Exercise 8-41

SOLO

What is the value of `results` once the code below has run? Do the substitutions, reductions, and assignments in your head before you run the code.

```
x = 7
y = 13
z = x + y
x = 0
result = x + y + z
```

Exercise 8-42

SOLO

What is the value of `results` once the code below has run? Do the substitutions, reductions, and assignments in your head before you run the code.

```
x = 5
y = x + 1
x = y + 1
y = x + 1
result = x + y
```

Exercise 8-43

SOLO

In the code below I have shuffled the statements. Put them in the right order to make the code print `9`. To do that you must think about which values each variable will in each statement depending on the how you order the statements.

```
x = x + 1
y = 5
y = y - 1
print(y)
x = 1
y = y * x
```

Exercise 8-44

SOLO

In the code below, I have shuffled the statements. Put them in the right order to make the code print `'Mogens'`

```
c = b
print(c)
a = b + a
b = 'og'
b = c + a
c = 'M'
a = 'ens'
```

Exercise 8-45

SOLO

Make three exercises that require the knowledge of programming so far. Have your fellow students solve them.

9 Insides

Since this course is all about instructing machines, some idea about what a computer is will be helpful.

<https://youtu.be/d86ws7mQYIg>

You are about to spend fourteen weeks giving instructions to a machine, and part of that time telling an AI to give instructions to the same machine on your behalf. It helps enormously to know, in broad strokes, what is actually sitting under your fingers. You do not need to become an electrical engineer. What you need is a mental model, a small and sturdy picture you can return to whenever the words pile up.

The ladder so far

Explainer

Still to come: Translator,
Illustrator, Comparer,
Drafter, Worker,
Collaborator, Developer

It's switches all the way down

Deep down, a computer is millions of tiny switches, and each switch is either off or on. That is it. There is no secret third thing. We write “off” as **0** and “on” as **1**, and one such off-or-on is called a bit, the smallest possible piece of information. One bit on its own cannot say much. But line eight of them up and you get a byte, and a byte can be in 256 different patterns, enough to stand for a number from 0 to 255, or a single letter, or one dot of color. Group bytes together and you can represent anything at all: this sentence, a photograph, a song, an entire genome, or the program that is reading them. The number **01100100** does not know whether it is the number 100, the letter **'d'**, or a shade of gray. Its meaning comes entirely from the piece of code that decides how to read it. When you learn later that in Python **'d'** and **100** are different “types”, and a type is nothing more than an agreement about how to read some bits.

The three parts that do the work

Inside the box, the three most important components are the hard disk, the memory and the CPU. The hard disk is where your files live: your code, your data, everything. Crucially, the disk is permanent, so what you save there survives when you switch the computer off. When you save a file, you are writing it to the disk. It is roomy but slow. The memory, usually called RAM, holds whatever is running right now, meaning the program that is currently going and the values it is working on. Memory is temporary. The instant a program stops, or the computer restarts, whatever was in memory is gone. This is the single most common source of beginner heartbreak, so let me say it plainly: nothing in memory is saved unless you write it to the disk. A value your program computed but never saved vanishes when the program ends. It was in the blue, not the amber. The CPU, the central processing unit, is the worker. It does the actual computing, one tiny step at a time, but staggeringly fast, billions of steps a second. And it has one important limitation:

it only ever works on what is currently in memory. So the normal rhythm of a running program goes from disk to memory to CPU. A copy of your file is loaded from the disk into memory, and the CPU works through it from there.

This gives you a habit worth forming now. Whenever something surprising happens, a value that seems to have disappeared or a change that did not stick, your first question should be whether that thing was kept on the disk or was only ever temporary in memory.

Exercise 9-1

SOLO

You run a program that calculates the number of **A** bases in a DNA sequence and prints the answer to the screen. You then close the program. Is that count still anywhere on the computer? Where was it while the program ran, kept or temporary? Where would it need to go for you to still have it tomorrow? Before you read on, decide what you think and write it down.

A machine that was only ever imagined

The CPU does one tiny simple step at a time, just extremely fast. The idea that anything and everything can be computed computer as a long chain of tiny mechanical steps is older than the computer chips that drive modern computers. In 1936, a young mathematician named Alan Turing asked a beautifully simple question: what is the simplest possible machine that is able can compute anything? The solution he came up with became one of the most famous ideas in all of science, and it is small enough to fit in your head.

The Turing machine was a thought experiment, a machine Turing built on paper and in his head in order to reason about what computers could and could not do, never a device he bolted together in a workshop. People have since built working models for the fun of it, and Turing himself went on to help design very real machines, including ones that broke wartime codes (Benedict Cumberbatch in *The Imitation Game*). However, that his machine was imaginary is exactly what made it powerful, because Turing would then prove things about every possible computer by studying only his simple made-up one.

Here is the Turing machine: There is a long paper tape divided into squares, and each square holds a single symbol, say a **0**, a **1**, or a blank. There is a head parked over one square that can do only three things: read the symbol under it, write a new symbol in its place, and move one square left or right. There is a state, a little sticky note reminding the machine roughly where it is in the task. And there is a fixed table of rules which says, for every combination of current state and symbol under the head, what to write, which way to move, and which state to switch to next. That is all it does. It reads the square under the head, looks up the one rule that matches the current state and symbol, writes, moves one step, changes state, and then does it again, and again, until it reaches a rule that says stop. No screen, no keyboard, no apps. Just a head shuffling along a tape, following a table doing a long sequence of things *instructed* by rules.

Exercise 9-2

SOLO

Grab a scrap of paper and draw three squares holding `0 1 1`, put the head on the leftmost square, and set the state to `A`. Now follow three rules, one step at a time until a rule tells you to stop:

- In state `A` reading a `0`: write `1`, move right, stay in state `A`.
- In state `A` reading a `1`: write `0`, move right, switch to state `B`.
- In state `B` reading a `1`: write `1`, move left, and then stop.

Before you start, try to predict what the three squares will say when the machine stops, and where the head will end up. Then trace it slowly and check.

If you got `1 0 1`, halted on the middle square, you traced it correctly. If not, find the step where your prediction and the rules parted ways, because that hunt is the real exercise.

What the imagined machine has to do with yours

Now the astonishing part. This ridiculous little tape shuffler can compute anything that any computer can compute, your laptop included.

Give it the right table of rules and this simple method will compute anything that any computer can compute, your laptop included. Turing went on to show that you can make a set of rules that lets the machine read a description of any other machine written on its tape and then act it out. One machine that can become any machine, just by being fed the right instructions. That is the theoretical birth certificate of the thing on your desk, a general-purpose computer that runs any program you give it.

So how does this paper fantasy line up with the chips of modern computers? Both grind through a set of instructions one tiny step at a time, and the head's cycle of reading, writing, and moving is mechanical version of the CPU doing one small operation after another. The Turing machine keeps storage (the tape) separate from the worker (head and its rules), just as your computer keeps memory separate from the CPU. Notice that on the tape the rules and the data sit together as plain symbols, which is exactly the modern truth that a program is just more data in memory. When Python reads your `hello.py` and acts it out, it is playing the role of a universal machine reading instructions off a tape.

The operating system, the manager in the middle

You never actually talk to the disk, memory, and CPU directly. Sitting between your programs and the hardware is the operating system. macOS, Windows, and Linux are all operating systems. The operating system sits between your programs and the hardware, handing out memory, finding files, and sharing the CPU so your programs do not have to fight over the electronics.

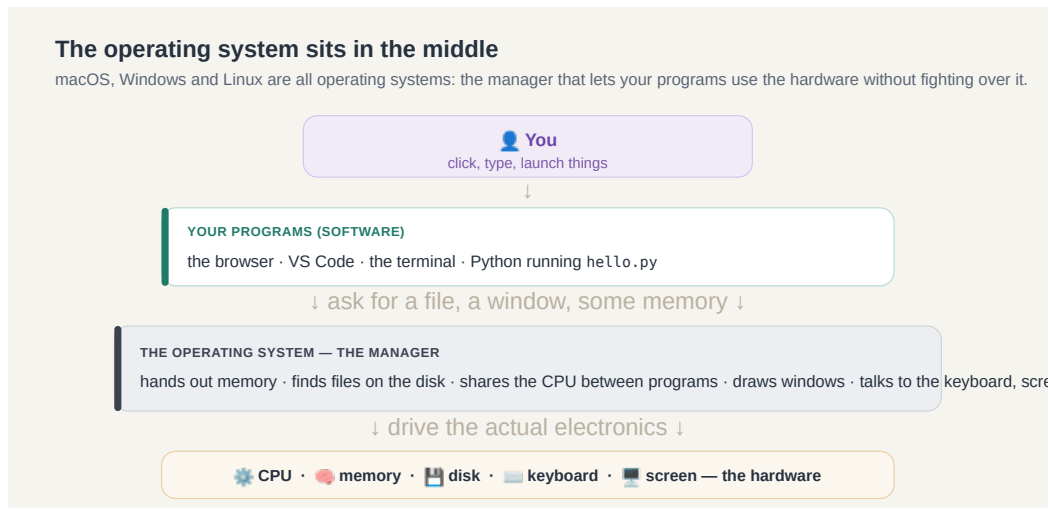


Figure 9.1: The operating system sits between your programs and the hardware: it hands out memory, finds files, shares the CPU between programs, and talks to the keyboard and screen.

As Figure 9.1 shows, when your program wants to open a file it does not go rummaging through the disk itself. It asks the operating system, and the operating system finds the file and hands it over. When a program needs free memory to work data, the operating system parcels some out. When you have a browser, an editor, and a music player all running at once, the operating system is the one sharing the single CPU between them, giving each one slices of time so quickly that they all seem to run at the same moment. It is also the operating system that listens to your keyboard and draws the windows on your screen.

Software is just files full of instructions

So what is a program, or a piece of software? It is, in the end, a file on the disk full of instructions for the CPU. A game, your browser, VS Code, Python itself, every one of them is a file of instructions that the operating system loads into memory and lets the CPU carry out. There is a catch, though. The CPU only understands one language, which is a brutally simple one. It is called machine code, and it consists of raw numbers representing the simplest possible actions (or operations). E.g., move this number here, add these two, compare them, jump to that instruction. This roughly means “put the number 4 into a register”:

```
10111000 00000100 00000000
```

Machine code is made to make CPUs fast, not to be read by humans. The entire history of programming is the story of building friendlier and friendlier languages (like Python) that then translates to machine code under the hood.

10 Instruction

Programmers face a trade-off. The closer your instructions are to the machine's own language, the more direct control you have, and the more painful they are to write. The closer your instructions are to plain human language, the easier they are to write, and the more you are trusting something else to bridge the communication with the CPU. The history of programming is basically a history of adding new layers to the stack of translations between you and the CPU.

The ladder so far

Explainer

Still to come: Translator, Illustrator, Comparer, Drafter, Worker, Collaborator, Developer

Layers of instruction

At the bottom is machine code, the raw bits the CPU runs. One step up is assembly, which swaps the bit patterns for short words like `MOV` and `ADD`, still one word per machine step. Higher up are compiled languages like C: now you write recognizable words and arithmetic, and a program translates the whole thing down to machine code before it runs. Higher still are interpreted languages, and this is where Python lives, the language you are about to learn.

In the diagram, notice that every level is a translator down to the level below it. Assembly is translated to machine code; C is translated to assembly code; Python code is handled by the Python *program*, which turns each instruction into machine steps as it goes. This makes Python an *interpreted* language. Unlike the C language where a compiler takes your entire code file and translates it all into machine code before it runs, Python it reads your file with code and runs its instructions, one line at a time and produce the machine code to make the CPU execute the instruction. This is also, quietly, why we make you run scripts from the terminal before we ever open a notebook. Typing `python hello.py` forces four things apart that a notebook glues together: the file on the disk, the interpreter that reads it, the CPU that does the work, and the terminal where you launch it and watch the output. Once you have felt those four as separate things, the rest of the course is much less mysterious.

An interpreter is a translator you can trust to mean exactly what your code says. Hold that thought right up against the AI, because the AI is about to look like just one more rung on the ladder, and the most important lesson in this course is the exact way in which it is not.

AI

For the purpose of producing code, an AI assistant looks like it belongs at the very top of the ladder, closer to human language than the levels below it: If you ask an AI in plain English how to solve a problem, like “How do I find open reading frames in this DNA sequence”, it will offer to hand Python code back to you. It *looks like*

Figure 10.1: The layers of instruction, which translates human language into assembly, and so on.

just the next translator, from a description even nearer to how you actually think, down toward code the machine can run. However, where as the levels described above were exact 1:1 *translations*, what you get from an AI is a basically a really qualified guess.

So the framing is useful right up until the moment it breaks. And where it breaks is the whole point. To see the break, you have to look at what the AI is actually doing under the hood, which is nothing like what a compiler does. In simple terms, a modern AI language model does one small thing over and over: it guesses the next word. You give it some text, your question, your request. It reads all of it and then scores every possible next token (a token is a word, or a small piece of a word), by how likely that token is to come next. It picks one, adds it to the text, and then does the whole thing again to choose the word after that, and the word after that, until it has produced a full answer. If a AI where to continue “the stop codon in this sequence is” and the model rates **TAG** as the most likely next word and picks it. It would chose **TAG** because, across the mountains of text it has seen, that word tends to follow in sentences like this, not because it went and checked any actual sequence. It is producing a plausible continuation. Often the plausible answer is also the correct one, and sometimes it is fluent, confident, convincing - and completely wrong. That gap, between plausible and true, is the most important thing to understand about these tools.

The AI learns, in a process called training, done before the model is released for use. During training, the model is shown an enormous amount of text, books, websites, articles, code, much of the public internet. Over and over it plays a game: hide the next word, let the model guess it, then compare the guess to the word that was really there. Every time, it nudges the values of millions of internal variables, called weights, to make it a little better than guessing. Repeat that billions of times and the weights settle into a configuration that makes startlingly good guesses about what word comes next. Those weights are everything the model knows. There is no library of facts inside it, no lookup table, only the model and its fitted weights. It is not completely unreasonable to compare it to a linear regression. That is also just a model that can give you a plausible Y value if you hand it an X value, but only once it has been “trained” on lots of x,y data to fit its slope and intercept variables.

Although an AI arguably makes for better conversation than a linear regression model, but once trained, its behavior is also completely fixed. Only the input you provide determines the what reply you get, just like the only way to get a different Y value from the regression is to pass it a different X value. Whatever it knows is baked into those weights from training, which is why it can be out of date, and why it does not base its responses on external evidence unless you explicitly ask it to, and even then, it may still be guessing a plausible continuation, rather than doing what you ask. Second, because it learned patterns from human text rather than following rules someone wrote down, it can produce something that sounds exactly like a correct answer while being false. It has no separate sense of truth, only a sense of what most plausibly comes next. When a false statement is a likely-sounding one, a plausible but wrong gene function, an off-by-one in a loop, a codon table with a quiet mistake, the model may hand it to you with total confidence. This has a name in the AI world: hallucination.

The reason that AIs have revolutionized programming is that, even with their occasional hallucinations, they come a long way towards something truly amazing:

translating a problem stated in plan language into computer code that solves that problem on your computer. The more precisely you can specify a problem and the more completely you can verify the code the AI returns, the more empowering AI it is. If you only one thing from this chapter, remember this:

An AI is useful to you up to the limit of what you can specify and verify.

Understanding Python lets you specify problems more precisely, and lets you test that the code produced does exactly what you want it to.

Exercise 10-1

SOLO

Before you touch the assistant, close the diagram and write the five rungs of the ladder out in order from memory, from the raw bits at the bottom to plain language at the top. Beside each one, write a single word: **faithful** if that rung is a translator that always means exactly what the rung above it said, and **plausible** if it is not. Four of the five get the same word and one does not. Being able to say which one, and why, is the entire argument of this course compressed into five lines, and you now have it before you have written a single line of Python.

Exercise 10-2

AI: EXPLAINER

Open the assistant in your browser and ask it a small factual question you can check, for example, what the three stop codons of the standard genetic code are. Read its answer. Then find a way to verify it that does not involve asking another AI: a textbook, a trusted database, or, later in this course, a tiny program of your own. Was it right? How did you know it was right, independently of the AI telling you so? Note what the badge on this exercise permits and what it does not. You asked it to explain something that already exists, and you settled the question somewhere else. That is the whole of rung one, and it is the only rung you have.

Exercise 10-3

AI: EXPLAINER

Now ask the assistant to explain the difference between a compiler and itself: *what does a compiler guarantee about the code it produces that you cannot guarantee about the code you produce?* Read the answer against what this note just told you. Two things are worth noticing, and they pull in opposite directions. The answer will probably be good, because this is a well-worn question and there is a great deal of text about it. And the answer was produced by exactly the process it is describing, one plausible next word after another, which means a fluent account of its own unreliability is not evidence of anything. It cannot check itself any more than it could check the stop codons.

Write two or three sentences in your logbook: what you asked, what it said, and how you checked. This is your first entry, and by the end of the course you will

have a running record of exactly where the machine helped you and where it quietly misled you, and, more importantly, of your own growing ability to tell the difference.

You now have both pictures. A computer is switches, made into numbers, made into files of instructions, run by a CPU on what memory holds, all managed by an operating system, and Python is a trustworthy translator from words you can read down to those instructions. An AI is a next-word guesser that is very good at what it does and whose output is plausible rather than guaranteed. Everything else in this book is you building the one ability that lets you use the second safely: the ability to read what it writes, run it, and see for yourself.

Week 2

11 Control flow

This chapter is about how you make your program do different things under different circumstances. Making functionality dependent on data is what makes programs useful.

The `if` statement

So far, the small programs you have written run the same sequence of statements (lines). Imagine if you could control which statements were run depending on the circumstances. Then, you would be able to write more flexible and useful programs. Cue the music - and let me introduce the “if-statement”.

Write the following carefully into a file. It is a small program that monitors bus passenger status. Notice the colon ending the if-statements. Also, note that the lines below each if-statement are indented with precisely four spaces. While writing the program, figure out what the if-statement does. Then, run the code and see what happens.

```
bus_seats = 32
passengers = 20
bags = 20

print(passengers, "people ride the bus")

if bus_seats >= passengers + bags:
    print("Smiles, everyone has room for bags")

if bus_seats >= passengers:
    print('Everyone gets to sit down, no complaints')

if bus_seats < passengers:
    print('Some passengers standing, annoyed')

if bus_seats < passengers / 3:
    print("General dissatisfaction, some swearing too")
```

Try changing the values of `bus_seats`, `passengers`, and `bags` and see how the program executes.

You have probably realized that the if-statements control which prints statements that are evaluated. A statement nested under an if-statement is only evaluated if the expression between the `if` keyword and the `:` reduces to a value Python considers as *true*. This does not happen if the expression between the `if` and `:` reduces to a value Python considers *false*.

When asked to evaluate something as true or false, Python will interpret zero and empty values (like `0` and `''`) as `False` and all other non-zero and non-empty values as `True`.

Exercise 11-1

SOLO

Which of the following letters are printed: A, B, C, D, E, F, G. Make up your mind before you write and run the code.

```
if 0:
    print('A')

if "Banana":
    print('B')

if 3.14159265359:
    print('C')

if False:
    print('D')

if 9 > 5 and 4 < 7:
    print('E')

if '':
    print('F')

if False or "banana":
    print('G')
```

Exercise 11-2

AI: EXPLAINER

What happens if you forget to write the `:` in the if-statement?

```
if 4 > 2
    print('Hi!')
```

Ask the assistant what error this will produce, word for word, before you run it. Then run it, and see whether it got the wording right or only the general idea. Predicting the exact wording is much harder than predicting that something goes wrong, and the difference between the two is worth noticing.

Exercise 11-3

AI: EXPLAINER

What happens if you do not indent the code under the if-statement?

```
if 4 > 2:  
print('Hi!')
```

Same again, but ask a sharper question: ask the assistant why Python cares about indentation here at all, when many other languages do not. Then run the code and check that the error it predicted is the error you got. The two errors in these last two exercises have different names, and you should be able to tell them apart.

“ By now, you probably know that your text editor is intelligent regarding indentation. If you hit Enter after a statement ending with `:`, it will indent the next line with four spaces. Also, if you use the tab in Python code, it will produce four spaces for you. ”

FAQ

Q: Isn't "If" a poem by Rudyard Kipling?

A: Yes.

Else-statement

Sometimes you not only want your program to do something if an expression reduces to `True`, you also want it to do something *else* if it is `False`. It is as simple as it looks:

```
cookies = 3  
  
if cookies > 0:  
    print("Uh, I wonder if we have some milk too...")  
else:  
    print("Sigh!")
```

Remember to put a `:` after the `else` keyword. Write the code and change the value of `cookies` to `0`.

Exercise 11-4

SOLO

Test your understanding about which expressions that reduce to a `True` or `False` value. Write the code below and then see how it responds to different values of `x`. Try to come up with other variations yourself.


```

x = 0.0
# x = '0'
# x = ' '
# x = ''
# x = not 0
# x = 'zero'

if x:
    print('x is substituted with True in the if-statement')
else:
    print('x is substituted with False in the if-statement')

```

FAQ

Q: Isn't "Else" a poem by Rudyard Kipling?

A: No.

Exercise 11-5

SOLO

What do you think this code prints? Notice how you can nest if and else-statements under other if and else-statements. This way, you can make your program include only some statements when certain combinations of conditions are met. Just remember that the code below each `if` or `else` is indented by four spaces. Try to change the True/False values of `milk` and `cookies`.

```

milk = False
cookies = True
if milk:
    if cookies:
        status = 'Good times!'
    else:
        status = 'Not thirsty, thanks or asking'
else:
    if cookies:
        status = 'How does something like this happen?'
    else:
        status = 'Whatever...'

print(status)

```

Blocks of code

In the examples above, some lines are indented more than others, and you probably already have some idea of how Python interprets this. Indentation defines blocks

of code. The `if` and `else` statements control each code block's evaluation when your code runs. The following three rules define individual blocks of code:

1. All statements in a code block have the *same indentation*. That is, they line up vertically.
2. A block of code *begins* at the first line of code at a line that is indented more than the one before it.
3. A block *ends* when it is followed by a less indented line or at the last line of code.

This way, a block can be nested inside another block by indenting it further to the right, as shown in Figure 11.1. Compare the example in Figure 11.1 to the code example above. Note how a colon at the end of a statement means “this applies the block of code below”. Make sure you understand which print statements are controlled by which `if` and `else` statements.

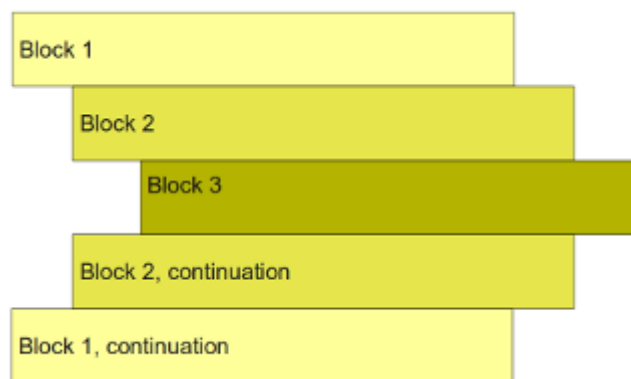


Figure 11.1: The amount of indentation defines blocks of code

Elif-statement

Say you need to test several mutually exclusive scenarios. For example, if a base is equal to A, T, C, or G. You can do that like in the example below, but it is very verbose and shifts your code further and further to the right.

```
base = 'G'

if base == 'A':
    print('This is adenine')
else:
    if base == 'T':
        print('This is thymine')
    else:
        if base == 'C':
            print('This is cytosine')
        else:
            print('This is guanine')
```

This is where an `elif` statement can be helpful. It is basically short for “else if.” The correspondence is hopefully obvious if you compare it to the example below.

```
base = 'G'

if base == 'A':
    print('This is adenine')
elif base == 'T':
    print('This is thymine')
elif base == 'C':
    print('This is cytosine')
else:
    print('This is guanine')
```

Here, we put an else-statement at the end to capture all cases not covered by the if-statement and the two elif-statements.

Exercise 11-6

SOLO

You can use logical operators (`and` , `or` , `not`) in the expressions tested in an if-statement. Can you change the program from exercise 11-5 so that there are no nested if-statements - in a way that the program still does the same? You can use `if` , `elif` , and `else` and test if, e.g., `milk` and `cookies` are true using `and`.

Exercise 11-7

SOLO

The snippet of code below has three blocks with three statements. Which statements belong to which block? Which statements are executed?

```
x = 5
if x > 4:
    y = 3
    if x < 1:
        x = 2
        y = 7
        z = 1
    x = 1
z = 4
```

Exercise 11-8

SOLO

Can you see four blocks of code? If not, read the three rules above again. Which statements are executed?

```
x = 5
if x > 4:
    y = 3
    if x < 1:
        x = 2
        y = 7
    else:
        x = 1
        y = 9
z = 4
```

General exercises

Exercise 11-9

SOLO

Will this print `You are a superstar!` ?

```
if -4 and 0 or 'banana' and not False:
    print("You are a super star!")
```

Exercise 11-10

SOLO

Will this print `You are a superstar!` ?

```
if -1 + 16 % 5 == 0 :
    print("You are a super star!")
```

Exercise 11-11

SOLO

Assign values to two variables, `x` and `y`. Then, write code that prints `OK` if (and only if) `x` is smaller than five *and* `y` is larger than five. Do it using *two* `if` statements:

```
x = 3 # or something else
y = 7 # or something else

# rest of code here...
```

Now solve the same problem using only *one* `if` statement.

Exercise 11-12

SOLO

Assign values to two variables, `x` and `y`. Then, write some code that prints `OK` if and only if `x` is smaller than five or `y` is larger than five. Do it using *two* `if` statements:

```
x = 3 # or something else
y = 7 # or something else

# rest of code here...
```

Now solve the same problem using only *one* `if` statement and *one* `elif` statement.

Exercise 11-13

SOLO

Assign values to two variables, `x` and `y`. Then, write some code that prints `OK` if either `x` or `y` is zero but not if both are zero (this is tricky).

```
x = 3 # or something else
y = 7 # or something else

# rest of code here...
```

Exercise 11-14

SOLO

Which value of `x` makes the code below print Banana?

```
x =
s = ''
if x**2 == 16:
    s = s + 'Ba'
if x + 6 == 2:
    s = s + 'na'
if 7 == x - 3:
    s = 'na' * 2
else:
    s = s + 'na'
print(s)
```

Exercise 11-15

SOLO

Make three exercises that require the knowledge of programming so far. Have your fellow students solve them.

12 Functions

This chapter is about organizing your code into chunks that you can call upon to perform well-defined tasks in your program.

Functions

Buckle down for the most powerful and useful thing in programming. Functions! Functions serve as mini-programs that perform small, well-defined tasks in your program.

I have started to write a song about functions:

```
print("Functions are super, Functions are cool")
print("When writing a program they are a great tool")
print("La la dim du da da di")
print("Skubi dubi dumdi di")
print("Bing di dubi dum da di")

print("Functions are used to package some code")
print("They are not so strange that your head will explode")
print("La la dim du da da di")
print("Skubi dubi dumdi di")
print("Bing di dubi dum da di")
```

①

① Last line

I will add many more verses, and I do not want to write the entire chorus every time. So what would be more natural than to make a function named `chorus` that takes care of that for us? That way, we can write our song the way lyrics with a chorus are usually written:

```
def chorus():
    line1 = "La la dim du da da di"
    line2 = "Skubi dubi dumdi di"
    line3 = "Bing di dubi dum da di"
    return line1 + '\n' + line2 + '\n' + line3

print("Functions are super, Functions are cool")
print("When writing a program they are a great tool")
print( chorus() )

print("Functions are useful to wrap up some code")
print("They are not so strange that your head will explode")
print( chorus() )
```

First, let us break down the function definition in the top part of this code:

1. We *define* a function with the `def` keyword (which is short for “define” in case you wonder).
2. After `def`, we write the function `chorus`. We could name it something else, but like good variable names, good function names can help you remember what your code does.
3. After the name, you put two parentheses, `()`.
4. Then a colon, `:`.
5. The statements that are part of the function are nested under the `def` statement and are indented with four spaces exactly like we do under if-statements.
6. The return statement ends the function. After the `return` keyword, the expression reduces to a *value* that the function returns.

When Python runs this code, each line is executed one by one, starting from the first line (remember oath two?). So, in this case, python first executes the *definition* of the `chorus` function. The only thing that happened after Python had executed the first five lines of code was that it assigned the name `chorus` to the four indented statements. So Python now “knows” about the `chorus` function (like it “knows” about a variable `x` after we do `x = 4`).

To *use* the function, we “call” it by writing its name followed by parentheses: `chorus()`. When it comes to functions, “use”, “call” and “run” means the same thing. As you can see, we call the function twice in the rest of the code. Each time we do, the following happens:

1. When a function is *called*, each statement in the definition is executed one after the other. If you look at the function definition, you can see that our `chorus` function has four statements.
2. The first statement assigns a string value to the `line1` variable.
3. The second statement assigns a string value to the `line2` variable.
4. The third statement assigns a string value to the `line3` variable.
5. The fourth statement is a return statement. The expression after the `return` keyword in the final statement is reduced to a *value*, and this value is substituted for the function call. In this case, that value is the following string:

```
"La la dim du da di\nSkubi dubi dumdi di\nBing di dubi dum da di"
```

So, the key properties of functions are:

- A function names a piece of code (some statements) just like variables name values like strings and numbers.
- We call a function by writing the function name followed by parentheses: `chorus()`. Just writing the function name will not call the function.
- When a function is *called*, it is substituted by the *value* that the function *returns* – exactly like a variable in an expression is substituted by its value. It is crucial that you remember this.

Exercise 12-1

SOLO

Now that we have a `chorus` function, that part is out of the way, and we can concentrate on our song without worrying about remembering how many “la la”s it has and so on. Try to change the “lyrics” in the chorus a little bit. Notice how you only need to make the change in one place to change all the choruses in the song – cool, right? Without the function, you would have to rely on correctly changing the code in many different places.

Exercise 12-2

AI: EXPLAINER

Try to delete the return statement in the `chorus` function (the last line in the function) and run the code again. You should see something like this:

```
Functions are super, Functions are cool
When writing a program they are a great tool
None
Functions are used to wrap up some code
They are not so strange that your head will explode
None
```

The function call (`chorus()`) is now substituted with `None` . How can that be when we did not return anything? The reason is that when you do not specify a `return` statement, the function returns `None` by default. This is to honor the rule that variables and a function calls are substituted by a value, and `None` is simply the value that Python uses to represent “nothing”. `None` is a value denoting the lack of value. As you just saw, it represents that no value is returned from a function. It can also be assigned to a variable as a placeholder value until another value is assigned:

```
x = None
x = 4
```

Also, `None` is considered false in a logical context:

```
print(not None)
```

Ask the assistant why a function without a `return` statement gives back `None` rather than nothing at all. A good answer connects it to substitution: the function call has to be replaced by *something*. If the assistant explains what happens without ever mentioning that the call is substituted by a value, it has not answered the question, and you should ask again.

Exercise 12-3

SOLO

Try this variant to the chorus function. Go through the code *slowly* and repeat all the steps to break down what happens when a function is called. Remember also to do each substitution and reduction carefully.

```
def chorus():
    line1 = 'La la'
    line2 = 'Du bi du'
    return line1 + '\n' + line2
```

Do the same for this variant:

```
def chorus():
    line1 = 'La la'
    line2 = 'Du bi du'
    chorus_text = line1 + '\n' + line2
    return chorus_text
```

and for this variant:

```
def chorus():
    return "La la\nDu bi du"
```

Exercise 12-4

SOLO

What do you think happens if you move the definition of `chorus` to the bottom of your file? Decide what you think will happen and why (maybe you remember what happens when you try to use a variable in an expression before defining it?). Then try it out.

```
print("Functions are super, Functions are cool")
print("When writing a program they are a great tool")
print( chorus() )

print("Functions are useful to wrap up some code")
print("They are not so strange that your head will explode")
print( chorus() )

def chorus():
    line1 = "La la dim du da da di"
    line2 = "Skubi dubi dumdi di"
    line3 = "Bing di dubi dum da di"
    return line1 + '\n' + line2 + '\n' + line3
```

Ensure you understand how the error you get relates to how Python runs your script (remember oath two?). If you still need help understanding, do the next exercise and then return to this one.

Exercise 12-5

AI: EXPLAINER

Which error do you get here, and why? How is that similar to the error in the previous exercise?

```
print(x)
x = 7
```

Ask the assistant to explain this error, and to say in which order Python reads the two lines. Then run it. This is Oath 2 in disguise, and you already know the answer, so it is a fair chance to catch the assistant hedging.

Exercise 12-6

SOLO

Consider the code below. Do all the substitution and reduction steps in your head. Remember that each function call is substituted by the value that the function returns. Then run it.

```
def lucky_number():
    return 7

x = lucky_number()
y = lucky_number()
twice_as_lucky = x + y
print(twice_as_lucky)
```

Now, change the code to that below. The code makes the same computation but in fewer steps. Do all the substitution and reduction steps.

```
def lucky_number():
    return 7

twice_as_lucky = lucky_number() + lucky_number()
print(twice_as_lucky)
```

Now, change the code to the one below. The code makes the exact computation but in fewer steps. Do all the substitution and reduction steps.

```
def lucky_number():
    return 7

print(lucky_number() + lucky_number())
```

Functions can take arguments

The functions we have written so far are not very flexible because they return the same thing every time they are called. Now write and run this beauty:

```
def square(number):
    squared_number = number**2
    return squared_number

result = square(3)
print(result)
```

Notice how we put a variable (`number`) between parentheses in the function *definition*. This variable is assigned the value we put between the parentheses (`3`) when we *call* the function. So when we call like that (`square(3)`), Python automatically makes the assignment `number = 3`.

Here is another example:

```
def divide(numerator, denominator):
    result = numerator / denominator
    return result

division_result = divide(44, 77)
print(division_result)
```

When the function call `divide(44, 77)`, these two things implicitly happen: `numerator = 44` and `denominator = 77`.

Take note of the following three important points: 1. The *values* that we pass to the function in the function call (like `3`, `44`, and `77`) are called *arguments*. It is crucial to remember that it is *values* and not variables that are passed to functions. 2. The *variables* in the definition line of a function, like `number`, `numerator` and `denominator`, are called *parameters*. They hold the *values* passed to the function when it is called (the arguments). 3. You can define functions with any number of *parameters* if you use the same number of *arguments* when you call the function.

Exercise 12-7

SOLO

Try to call your `divide` function like this `divide(77, 44)`. What does it return, and what do you learn from it? Does the order of arguments and parameters correspond?

Exercise 12-8

SOLO

Try to call your `divide` function like this `divide(44)`. Do you get an error, and what do you learn from that?

Exercise 12-9

AI: EXPLAINER

Try to call your `divide` function like this `divide(44, 77, 33)`. Do you get a different error message, and what do you learn from that?

Paste both error messages, this one and the one from the previous exercise, into the assistant and ask what the difference tells you about when Python counts the arguments. Then test its claim yourself by calling `divide` with one argument too few.

Exercise 12-10

SOLO

Read this code and do all substitution and reduction steps from beginning to end.

```
def square(x):  
    return x ** 2  
  
result = square(9) + square(5)  
print("The result is:", result)
```

Now replace the line `return x ** 2` with `print(x ** 2)`. What is printed now and why?

Exercise 12-11

SOLO

As described above, a `return` statement ends the function by producing the value that replaces the function call. If a function has more than one `return` statement, then the function ends when the first one is executed.

```
def assess_number(x):  
    if x < 3:  
        return 'quite a few'  
    if x < 100:  
        return 'a lot'  
    return 'a whole lot'  
  
nr_apples = 2  
print(nr_apples, "apples is", assess_number(nr_apples))
```

What happens when `x` is 2, 3, 50, 200? Think about it first.

Functions and variables

A function call is the temporary little world that only exists when the function is *called* and until it *returns* its value. It did not exist *before* the function was called, and it does not exist *after* the function returns its value. By necessity, the variables defined in your function are also temporary.

This means variables defined inside a function are private to each function call. It also means that variables defined inside functions are unavailable to code *outside* the function. Running the following example should help you understand this:

```
def make_greeting():
    greeting = 'Guten tag'
    name = 'Heinz'
    message = greeting + " " + name
    return message

greeting = 'Buongiorno'
name = 'Giovanni'
print( make_greeting() )
print(greeting + " " + name)
```

Notice how Heinz and Giovanni are greeted in their native languages. This means that the variable definitions inside the function do not overwrite the Italian versions already defined outside the function. This is because the variables defined in the function are temporary and private to the function, even if they have the same names as variables outside the function. This is why the function call `make_greeting()` in the `print` statement does not change the variables' values printed in the last line.

Now try to “comment out” the line `greeting = 'Guten tag'` and run the example again. All of a sudden, Heinz is greeted in Italian! The reason is that now Python cannot find a definition of `greeting` inside the function. It then looks outside the function for a definition and finds the Italian version.

Now try to “comment out” the line `greeting = 'Buongiorno'` and run the example again. You get an error, but which one? Python complains that it cannot find a definition of `greeting`. The reason is that once the last print statement is executed, the small world of the function call in the previous line no longer exists.

You should learn two rules from the above example:

1. All variables you define inside a function are *private* to the function. If a variable in a function has the same name as a variable in the main script (like `greeting` above), then these are two *separate* variables that just happen to have the same name.
2. When you use a variable like `greeting` in the function (E.g., `message = greeting + " " + name`), Python checks if the variables have been defined in the function. If that is not the case, then it will look for it outside the function. In the above example, `name` is found in the function, and `greeting` is found outside the function. It is good practice to make your functions “self-contained” in the sense that Python should not have to look outside the function for variables.

Exercise 12-12

SOLO

Try this version of the example above. Now, `name` is defined as a function parameter, but it is still a function variable, just like `greeting`.

```
def make_greeting(name):
    greeting = "Guten tag"
    message = greeting + " " + name
    return message

greeting = 'Buongiorno'
name = 'Giovanni'
print( make_greeting("Heinz") )
print(greeting + " " + name)
```

Exercise 12-13

SOLO

Consider the following example:

```
def double(z):
    return z * 2

x = 7
result = double(x)
print(result)
```

When the function is called (`double(x)`), the `x` is substituted by its value `7`. That *value* is passed as the argument and assigned to the function parameter `z` (`z = 7`). `z` is a private function variable and does not exist before or after the function call. Does this change in any way if we use the variable name `x` instead of `z`, as shown below?

```
def double(x):
    return x * 2

x = 7
result = double(x)
print(result)
```

Do *all* substitutions and reductions for each line of code from top to bottom. Keep the sequence of events in mind and remember that a function definition is merely a template describing a mini-world that is created anew every time the function is called.

Builtin functions

So far we have only talked about functions you write yourself, but Python also has built-in functions that are already available to you. They work just like a function you would write yourself. You already know the `print` function quite well; that is an example of a function that prints something but returns `None`. There are many other useful built-in functions, but I will tell you about another two: `len` and `type`.

Exercise 12-14

SOLO

Try these examples:

```
x = 'Banana'
print("The value of variable x is of type", type(x))
print("The value of variable x has length", len(x))
```

As you can see, `type` returns the type of the value passed as the argument, and `len` returns the length of the value passed as the argument. The `type` function is handy if you wonder what type a value has, but it is not a function we will use in this course. The `len` function, however, is your new best friend. You will see why soon enough.

Exercise 12-15

SOLO

Try to change the value of the `x` in exercise 12-14 to an integer or a float and see what happens when you run it. Do you get an error? Does it make sense that not all types of values can meaningfully be said to have a length?

Exercise 12-16

SOLO

What happens if you pass an empty string (`""`) as the argument to the `len` function?

Exercise 12-17

SOLO

What is printed here? Think about it first, and then try it out. Remember to do the substitution and reduction steps.


```
return_value = print("Hello world")
print(return_value)
```

Exercise 12-18

SOLO

What is printed here? Think about it first, and then try it out. Remember to do the substitution and reduction steps.

```
print(print("Hello world"))
```

General exercises

The following exercises treat the areas we have worked on in this and previous chapters. They are meant to train your familiarity with if-statements and functions. Remember that the purpose of the exercises is not to answer the questions but to train the chain of thought that *allows* you to answer them. Play around with the code for each example and see what happens if you change it a bit.

Exercise 12-19

SOLO

Consider this function definition that takes a single number as the argument:

```
def square(n):
    return n**2
```

What does it do? What does it return? What number does `square(2)` then represent?

Below, I have used it in some printed expressions. Make sure you understand what each expression evaluates to. Make the explicit substitutions and reductions on paper before you run it. Remember that we substitute a function call (like `square(2)`) for the value it returns, just like a variable `x` substitutes for the value it points to.

```
print(square(3))
print(square(2 + 1))
print(square(2) * 2 + square(3))
print(square(square(2)))
print(square(2 * square(1) + 2))
```

Exercise 12-20

SOLO

What does this function do? How many parameters does it have? How many statements does the function have? What does the function print? Which value does it return?

```
def power(a, b):  
    print("This function computes {}".format(a, b))  
    return a**b  
  
print(power(4, 2))
```

Try (possibly strange) variations of the code like the ones below better to understand the contribution of each line of code. What is the difference between return and print? What happens when Python gets to a `return` statement in a function? What happens when the function does not have a `return` statement?

Variation 1:

```
def power(a, b):  
    print("This computes", a, "to the power of", b)  
    print(a**b)  
  
result = power(4, 2)  
print(result)
```

Variation 2:

```
def power(a, b):  
    print("This computes", a, "to the power of", b)  
    return a**b  
  
result = power(4, 2)  
print(result)
```

Variation 3:

```
def power(a, b):  
    print("This computes", a, "to the power of", b)  
    a**b  
  
print(power(4, 2))
```

Variation 4:

```
def power(a, b):
    return a**b
    print("This computes", a, "to the power of", b)

print(power(4, 2))
```

Exercise 12-21

SOLO

Define a function called `diff`, with two parameters, `x` and `y`. The function must return the difference between the values of `x` and `y`.

Example:

```
def diff(x, y):
    ...

diff(8, 2) # should return 6
diff(-1, 2) # should return -3
```

Save the value returned from the function in a variable. Then, test if the function works correctly by comparing the result to what you know is the true difference (using `==`).

Exercise 12-22

SOLO

Define a function called `all_equal` that takes five arguments and returns `True` if all five arguments have the same value and `False` otherwise. The function should work with any input, for example:

```
all_equal("Can", "Can", "Can", "Can", "Can")
all_equal(0, 0, 0, 0, 0)
all_equal(0.5, 0.5, 0.5, 0.5, 0.5)
all_equal(True, True, True, True, True)
```

Hint: You test equality with `a == b`. Now, think back to what you learned about logic. Which operator can you use to test if `a == b` and `b == c`?

Exercise 12-23

SOLO

Define a function called `is_even`, which takes one argument and returns `True` if (and only if) this is an even number and `False` otherwise (remember the modulo operator?).

```
is_even(8) # should return True
is_even(3) # should return False
```

Exercise 12-24

SOLO

Define a function called `is_odd`, which takes one argument and returns `True` if (and only if) the argument is an odd number and `False` otherwise.

```
is_odd(8) # should return False
is_odd(3) # should return True
```

Can you complete this exercise using the `is_even` you defined in exercise 12-23? How? Why is that a good idea?

Exercise 12-25

SOLO

Here is a function that should return `True` if given an uppercase (English) vowel and `False` otherwise:

```
def is_uppercase_vowel(c):
    c == 'A' or c == 'E' or c == 'I' or c == 'O' or c == 'U'

char = 'A'
if is_uppercase_vowel(char):
    print(char, "is an uppercase vowel")
else:
    print(char, "is NOT an uppercase vowel")
```

Now you can just type the code exactly as shown and run it. Do you get what you expect? Does the code work? If not, try to figure out why. Try to print the value that the function returns. Do you know if that gives you any hints about the cause of the problem?

Exercise 12-26

SOLO

Define a function called `is_nucleotide_symbol`, which takes one argument and returns `True` if this is either A, C, G, T, a, c, g or t, and `False` in any other case.

Name your parameter something sensible like `symbol`.

```
is_nucleotide_symbol("A") # should return True
is_nucleotide_symbol("B") # should return False
is_nucleotide_symbol("Mogens") # should return False
is_nucleotide_symbol("") # should return False
```

Exercise 12-27

SOLO

Define a function called `is_base_pair` which takes two parameters, `base1`, `base2`, and returns `True` if `base2` is the complementary of `base1`, and `False` otherwise.

```
is_base_pair("A", "G") # should return False
is_base_pair("A", "T") # should return True
is_base_pair("T", "A") # should return True
is_base_pair("Preben", "A") # should return False
```

Exercise 12-28

SOLO

Did you find the bug in exercise 12-25? You were supposed to find that the function did not have a return value. This makes the function return `None` by default. Do you think the `None` value is considered true or false in an if-statement?

Exercise 12-29

SOLO

Define a function called `celcius2fahrenheit` that converts from celsius to Fahrenheit. You can do this because you know the linear relationship between the two. On Figure 12.1 you can see that the slope is $9 / 5$ and the intercept is 32 . The function should have one parameter `celsius`. Inside the function, you should define the variables `slope` and `intercept` and give them the appropriate values. Then you can calculate the conversion to Celcius using these variables and return the result.

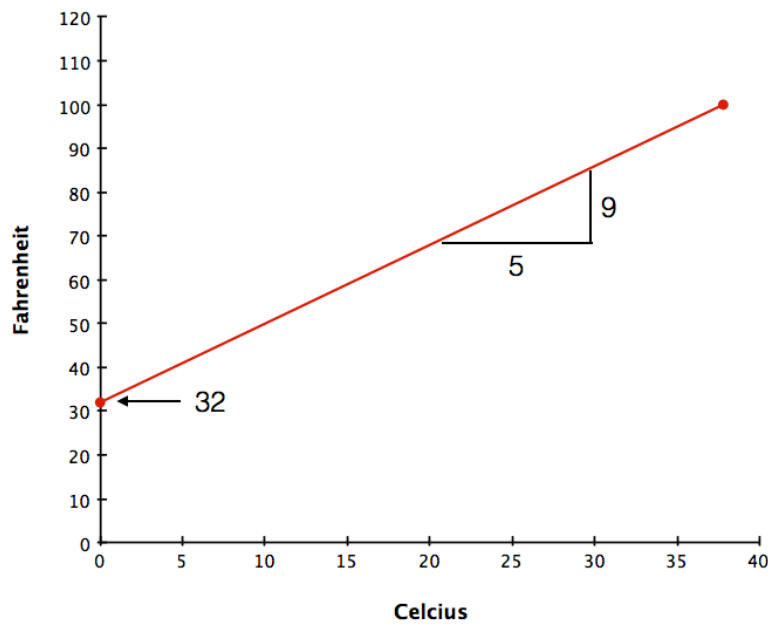


Figure 12.1: Temperature conversion

Exercise 12-30

SOLO

Try to change your conversion function so it takes three arguments, corresponding to celsius, slope and intercept so you can call it like this to convert 27 degrees celsius: `conversion(27, 9 / 5, 32)`. Now you have a function that can do any linear conversion that you can put inside another function like this:

```
def celcius2fahrenheit(celsius):
    return conversion(celsius, 9 / 5, 32)
```

Exercise 12-31

SOLO

Now try to extend this to a different problem: It has been found that the height and weight of a person are related by a linear equation with slope = 0.55 and intercept = -25. Define a function called `predict_weight` which takes just one argument, the height of a person, and returns the estimated weight of the person.

Exercise 12-32

SOLO

By now you know that some of the words in your code have specific purposes. `def` defines functions, `return` returns value from a function, `and` is a logical operator etc. Here is a list of the ones you will see in this course: `and`, `assert`, `break`,

`continue` `def` , `del` , `elif` , `else` , `False` , `for` , `from` , `if` , `import` , `in` , `is` ,
`not` , `or` , `pass` , `return` , `while` , `True` , `None` (you can see a full list [here](#))

These words are reserved for their special purposes in Python and you will not be allowed to assign values to them. Try this to see for yourself:

```
None = 4
```

or this:

```
and = 2
```

FAQ - Frequently Asked Questions

Q: Can function names be anything?

A: Just about. The rules that apply to variable names also apply to function names. Good function names are lower case with underscores (`_`) to separate words, like in the examples above.

13 The AI predicts, the widget proves

The week-two AI beat: a claim about what code does is a prediction to be checked.

Session length

Estimated 3 hours, run as this week's TA session. Bring last week's precedence and substitution/reduction notes; every round below runs on that material.

The routine for this whole session is the same three-way move, repeated on harder and harder expressions until it is automatic. You predict what an expression evaluates to, by hand, doing the substitution and reduction steps yourself. You ask the assistant what it evaluates to, and read its answer, including whatever reasoning it gives. Then you run it through the `%%steps` widget, which is the only one of the three that actually reduces the expression step by step and cannot be talked around. Three answers, one witness. When your prediction and the assistant's answer agree, that is pleasant and proves nothing. When they disagree, or when either of you turns out to be wrong, that is the session working exactly as intended.

The three-way move, every time

1. **Predict.** Write down your answer and the substitution/reduction steps that got you there, before you type anything into the assistant.
2. **Ask.** Give the assistant the bare expression and ask what it evaluates to and why. Read its reasoning, not just its final answer.
3. **Prove.** Run the expression through `%%steps`. Its trace is the verdict. Compare it against both of your answers and note any disagreement.

Every round below carries the `AI: Explainer` badge, which is the rung of the ladder you were given in week one and the only one you have so far. You are putting something in front of the assistant that already exists and asking it to explain that thing; you are not asking it to write anything, and you are not asking it to choose between two versions. What makes this session demanding is not the permission, which is the smallest one there is, but how hard you are going to work it. Rung one, twenty-two times, until checking a claim against a machine is what your hands do without being told.

Part 1 — Warm-up: arithmetic precedence (30 min)

Straightforward operator precedence, to get the three-way move running smoothly before anything tricky shows up.

The ladder so far

Explainer

Still to come: Translator, Illustrator, Comparer, Drafter, Worker, Collaborator, Developer

Exercise 1 13-1

AI: EXPLAINER

```
4 + 6 * 2 - 3
```

Exercise 2 13-2

AI: EXPLAINER

`2 ** 3 ** 2` (exponentiation groups right-to-left — decide what that means for this one before you predict)

Exercise 3 13-3

AI: EXPLAINER

`10 - 2 - 3` (subtraction groups left-to-right — same caution as above)

Exercise 4 13-4

AI: EXPLAINER

```
(3 + 2) * (8 - 6) / 2
```

Exercise 5 13-5

AI: EXPLAINER

```
7 % 3 + 2 * 5 // 2
```

Part 2 — Comparisons and boolean logic (35 min)

Chained comparisons and `not` / `and` / `or` precedence are a place fluent-sounding wrong answers are common, because the rules read as obvious right up until you meet a case that breaks the obvious reading.

Exercise 6 13-6

AI: EXPLAINER

`1 < 2 < 3` (this is a chained comparison, not two separate comparisons combined by magic — predict what Python actually does with it before asking)

Exercise 7 13-7

AI: EXPLAINER

```
1 < 2 > 3
```

Exercise 8 13-8

AI: EXPLAINER

```
not 1 == 1 == 2
```

Exercise 9 13-9

AI: EXPLAINER

```
True and False or True
```

Exercise 10 13-10

AI: EXPLAINER

```
not True and not False
```

Part 3 — Short-circuit values, not just True/False (35 min)

`and` and `or` do not return `True` or `False`; they return whichever operand decided the outcome. This is the single richest source of a plausible-but-wrong AI answer in this whole session, because “and/or return booleans” is a very natural, very common, very wrong simplification.

Exercise 11 13-11

AI: EXPLAINER

`'ATG' and 'GCC'` (predict the actual value returned, not just whether it is “truthy”)

Exercise 12 13-12

AI: EXPLAINER

```
' ' or 'GCC'
```

Exercise 13 13-13

AI: EXPLAINER

```
0 or '' or 'default'
```

Exercise 14 13-14

AI: EXPLAINER

```
'ATG' and '' and 'GCC'
```

Exercise 15 13-15

AI: EXPLAINER

Ask the assistant, before predicting or proving anything, to state in one sentence what `and` and `or` return in Python. Only after you have its claim in writing, run Exercises 11–14 through the widget and check its general claim against all four actual results. Does its one-sentence rule survive contact with all four?

Part 4 — Mixed types and strings (30 min)

Precedence and truthiness interact with strings, repetition, and coercion in ways worth a dedicated round.

Exercise 16 13-16

AI: EXPLAINER

```
'AT' * 3 + 'G'
```

Exercise 17 13-17

AI: EXPLAINER

```
'AT' * 2 == 'ATAT'
```

Exercise 18 13-18

AI: EXPLAINER

```
bool('False')
```

 (a classic trap — predict carefully)

Exercise 19 13-19

AI: EXPLAINER

`3 == 3.0 and 3 is 3.0` (if your Python raises a `SyntaxWarning` on `is` with a literal, that is itself worth reading and predicting around)

Exercise 20 13-20

AI: EXPLAINER

`len('ATG') == 3 and 'G' in 'ATG'`

Part 5 — Break the AI (30 min)

Now you write the expressions. Working alone or with a partner, construct **two** expressions of your own, each combining precedence, comparison, and short-circuit logic, specifically designed to be the kind of case a fluent guesser would get wrong: something where the obvious-looking simplification (like “and/or return booleans”) fails. Trade your two expressions with another pair. For each expression you receive, do the full three-way move: predict, ask, prove.

Exercise 21 13-21

SOLO

Before trading anything, write down in one sentence *why* each of your two expressions is designed to trip up a plausible-but-wrong guess. This is the harder half of the exercise: constructing a good trap requires understanding the rule well enough to know exactly where it is easy to misapply.

Exercise 22 13-22

AI: EXPLAINER

Run the full three-way move on the two expressions you received from the other pair. Report back to them, in one sentence each, whether their trap actually caught the assistant, caught you, caught neither, or caught both.

Wrap-up (10 min)

For your logbook this week, report your personal tally across the whole session: how many of the roughly twenty expressions the assistant got wrong, how many you got wrong, and whether the two of you tended to get the *same* ones wrong or *different* ones. That last question matters more than the raw count — it tells you whether your instincts and the assistant’s failure modes overlap, which is exactly the thing you need to know to check it well.

Week 3

14 How a model produces code, and why it can be wrong

A plain, non-technical account of what a model is doing when it writes code.

Back in week one you met the idea that an assistant guesses its next word rather than reasoning about truth, and you saw it guess a stop codon out of a sentence rather than out of an actual sequence. Two weeks of predicting expressions and confirming explanations later, it is worth asking the same question about the thing you will lean on most for the rest of the course: what is the assistant actually doing when it hands you a function?

The answer is: exactly the same thing, token by token, except the tokens now spell Python instead of English. Ask for a function that translates a codon into an amino acid, and the model does not consult a codon table, does not run anything, and does not check its answer against biology. It produces the most statistically plausible next piece of code, given everything you have said and everything like it in its training, one token at a time, until the function looks finished. `def`, then a plausible name, then a plausible parameter, then a plausible body, each choice made the same way a plausible next word was chosen when it was writing you a sentence.

This is exactly why it is so good at looking right. Function names, docstrings, tidy indentation, a sensible-looking dictionary of base pairs, all of these are things that appear constantly in real, correct code, so the model reproduces that surface fluently and by default. What it has no access to at all is whether the specific logic inside that familiar-looking surface actually computes what the name promises. Nothing in the process of predicting the next plausible token includes running the function and checking the result, because the model is not running anything. It is guessing what correct code tends to look like, not verifying that this particular code is correct.

That gap between looking right and being right is not a flaw that will be patched out of some future version. It is a direct consequence of how the model works, in the same way that a coin does not become less random with practice. A more capable model will guess better, more often, on harder problems, but “guessing better, more often” is still guessing. It can still be fluent, confident, and wrong on exactly the case you needed right, and it will hand you that wrong answer with the same tidy formatting it hands you a correct one. There is no warning light. Nothing about the way an assistant presents a wrong function is different from the way it presents a right one, because from the inside, generating one was no different from generating the other.

This is also why an assistant is not simply the newest, friendliest compiler, even though the ladder in week one might have tempted you toward that reading. A compiler is a deterministic translator: the same source always produces the same machine code, and that machine code means exactly what the source said, because the translation follows fixed rules a person wrote down and proved correct. Ask

The ladder so far

Explainer · Translator ·
Illustrator

Still to come: Comparer,
Drafter, Worker,
Collaborator, Developer

the same model the same question twice and you can get two different functions back, and neither one comes with a guarantee that it means what you asked. That single difference, guaranteed meaning versus plausible meaning, is why every rung below the assistant on that ladder can be trusted the moment it compiles, and the assistant cannot be trusted until you have checked it yourself.

Hold on to the word *plausible* for the rest of the term; it is carrying a great deal of weight. It is not an insult to the tool, which is often extraordinarily good, and it is not a reason to avoid it. It is an exact description of what its output is, and exact descriptions are what let you build the right habit around them. A plausible answer deserves your attention and often deserves your trust once you have checked it. It never deserves your trust before you have.

Two new rungs on the ladder

Since week one you have had exactly one permission, **AI: Explainer**, and you have worked it hard. This week you get two more, and it is worth noticing what they have in common: neither of them lets the assistant write code you intend to keep. That is still several weeks away. Both of these are about *reading*, which is the skill everything above them is built on.

Translator. You may ask the assistant to turn code into plain language, or plain language into code. The catch, and it is the whole exercise, is that you check the translation in *both* directions. Ask what a line does, get a sentence, then take that sentence into a fresh conversation and ask for the code. If what comes back is not the line you started with, one of the two translations was wrong, and finding out which is the point. A translation that only runs one way is not a translation; it is a guess you happened to agree with.

Illustrator. You may ask for more examples of something you half understand, which is a genuinely good use of a machine that has seen a great deal of code. The catch here is the same one as always: you predict what each example does before you run it. An example you cannot predict is not illustrating anything to you yet — it is just more code on the screen.

Both new badges are, of course, still governed by the rule you took in week one. The assistant translates and illustrates; the machine says whether it was right.

Exercise 14-1

AI: TRANSLATOR

Take this line, which uses nothing you have not met:

```
print('ATGGCC'[1:4].lower())
```

Write down in one English sentence what it does, before you run it and before you ask anything. Then give the assistant the line and ask it for one English sentence. Put the two sentences side by side: do they agree, and is one of them vague exactly where the other is precise? Now open a *fresh* conversation, give it your sentence

and nothing else, and ask it for the Python that does what the sentence says. Run what comes back. Is it the line you started with?

Exercise 14-2

AI: ILLUSTRATOR

Slicing is the thing most people half understand at this point in the course, so use it. Ask the assistant for five short examples of slicing the string `'ATGGCC'`, each showing something different, and ask it for the code only, with no output and no explanation. Do not run them yet. Predict all five first and write your predictions down. Then run all five. You now have two scores: how many of its examples did what it said they would, and how many you called correctly. The second number is the one to put in your logbook.

Exercise 14-3

AI: EXPLAINER

Ask the assistant for a one-line Python function that returns `True` if a DNA strand is a palindrome (reads the same as its own reverse complement) and `False` otherwise. Before you run anything, read the function and decide, using only your eyes, whether you believe it is right. Then ask the assistant, in a second message, to explain step by step why its own function is correct. Notice that the explanation is generated the same way the code was, one plausible next token at a time, and is not a proof. Does the explanation change how confident you feel, and should it?

For your logbook this week, describe one moment so far where a fluent-sounding answer from the assistant turned out, on checking, to be wrong, or note that you have not caught one yet, and say what that would tell you about how carefully you have actually been checking.

15 Objects

This chapter introduces the notion of an object, one of the most central aspects of Python. Once you catch your breath, you will love that all Python values are objects.

Methods

In Python, a value like an integer or string not only holds data. It is also packaged with a lot of useful functionality relevant to the particular type of value. When a value is packaged with such relevant functionality and meta information, programmers call it an object—and in Python, all values are objects.

The associated functionality comes in the form of *methods*. You can think of methods as functions that are packaged together with the value. For example, string values have a method called `capitalize`. Try it out:

```
x = "banana".capitalize()
print(x)
```

To call the method on the string value, you connect the string value and the method call with a dot. So, to call a method on a value, you do the following:

1. Write the value (or a variable name that substitutes for a value).
2. Then write a `.`
3. Then write the name of the method (like `capitalize`).
4. Then, write two parentheses to call the method. If the method takes any arguments other than the value it belongs to, then you write those additional values between the parentheses with commas in between, just as when you call a function.

You can see that the method call looks just like a function call, and in many ways, calling a method works much like calling a function. The difference is that when we call a function, we say: “*Hey function, capitalize this string!*”. When we call a method, we say: “*Hey string, capitalize yourself!*”

So why do we need methods? Why do we need them when we have functions? It is very handy to have some relevant and ready-to-use functionality packaged together with the data it works on. You will start to appreciate that sooner than you think.

Methods are almost always used with variables. So remember to make any substitutions and reductions required. When we put a method call after a *variable* like below, the variable is first substituted for its value, and *then* the method is called on the value. Consider the second line of this example:

```
x = "banana"
print(x.capitalize())
```

Here, `x` is first substituted by `"banana"` and *then* the method is called on that value, like this: `"banana".capitalize()`.

Now write and run these examples:

```
message = "Methods Are Cool"
print(message)

shout = message.upper()
print(shout)

whisper = message.lower()
print(whisper)

new_message = message.replace("Cool", "Fantastic")
print(new_message)
```

You can see what these methods do. For example: `upper` returns an uppercased copy of the string.

Exercise 15-1

SOLO

Write and run the following code. What do you think it does?

```
line = '\n\tSome text\n'
print(">{<".format(line))

line = line.strip()
print(">{<".format(line))
```

Make sure you do the substitution and reduction steps in your head. Be especially careful about the third line of code. Also, what do you think the special `\t` character is?

Exercise 15-2

SOLO

The string methods you have tried so far have all returned a new string. Try this example:

```
'ATGACGCGGA'.startswith('ATG')
```

and this

```
'ATGACGCGGA'.endswith('ATG')
```

What do the methods do, and what do they return?

Using the Python documentation

Now that you are well underway to becoming a programmer, you should know your way around the Python documentation. Especially the part called the Python standard library. There is a *lot* of details in there that we do not cover in this course. These are mainly tools and techniques for writing more efficient, extensible, robust, and flexible code. The parts we cover in this course are the minimal set that will allow you to write a program that can do *anything*.

Exercise 15-3

SOLO

There is a string method that returns a secret agent:

```
print('7'.zfill(3))
```

You can look it up in the Python documentation.

Exercise 15-4

AI: EXPLAINER

Browse through all the string methods to get an impression of all the functionality that is packaged with string objects.

Then pick three methods whose names you cannot guess the meaning of, and ask the assistant what each one does and to give you a one-line example of each. Run all three examples. Documentation you have run is worth more than documentation you have read, and the assistant is a fast way of turning one into the other.

String formatting

You have already tried string formatting in exercise 15-1. String formatting is a simple but powerful technique that lets you generate pretty strings from pre-computed values. You may have noticed that many decimals are shown every time we print a floating-point number. It is not very pretty if you are only interested in two decimals. You use the `format` method (surprise) to format a string. In its most straightforward use, `format` replaces occurrences of `{}` with the arguments that are passed to it - like this:

```
taxon = "genus:{}, species:{}".format('Homo', 'sapiens')
```

Exercise 15-5

SOLO

What happens if you try this?

```
question = "Was {} {} Swedish?".format('Carl', 'Linnaeus')
```

and this?

```
question = "Was {} Swedish?".format('Carl', 'Linnaeus')
```

and this?

```
question = "Was {} {} Swedish?".format('Carl Linnaeus')
```

In the two last examples, the number of `{}` did not match the number of arguments to `format`. What happens when there are too few and when there are too many?

Exercise 15-6

SOLO

Consider this code:

```
s = "{} is larger than {}".format(4, 3)
print(s)
```

What will happen if you run this code? Write the code and see for yourself once you think you know. If you were wrong, make sure you understand why.

Exercise 15-7

SOLO

Consider this code:

```
language = 'Python'
invention = 'sliced bread'
s = '{} is the best thing since {}'.format(language, invention)
print(s)
```

What will happen if you run this code? Write the code and see for yourself once you think you know. If you were wrong, make sure you understand why.

Exercise 15-8

SOLO

Consider this code:

```
my_template = '{} is the best thing since {}'  
language = 'Python'  
print(my_template.format(language, 'sliced bread'))  
print(my_template.format(language, 1900 + 89))
```

What will happen if you run this code? Do the substitution and reduction steps in your head.

Exercise 15-9

SOLO

Think back to exercise 7-3, where you calculated how many cookies you could buy for 30 kr. The bars are 7 kr. So your program looked something like this:

```
nr_bars = 30 / 7  
print('I can buy', nr_bars, 'chocolate bars!')
```

and it ran like this: `python chocolate.py`

I can buy 4.285714285714286 chocolate bars!

String formatting lets you rewrite the program like this:

```
nr_bars = 30 / 7  
message = "I can buy {} chocolate bars!".format(nr_bars)  
print(message)
```

Try to replace `{}` with `{:.2f}`. `format` reads the stuff after the colon in each set of curly brackets and uses it as directions for formatting the value it inserts. Try it out and see what happens if you change the number `2` to `3`, `4`, `5` or `10`.

Exercise 15-10

AI: EXPLAINER

See if you can find the documentation for the `format` function in the Python documentation. It can do wondrous things, for this course we will only try to control the number of digits and padding with spaces. Look at the examples below. Maybe you can figure out how it works.

```
pi = 3.14159265359
print("{}*".format(pi))
print("{:.3f}*".format(pi))
print("{:.6f}*".format(pi))
print("{:>5.3f}*".format(pi))
print("{:>10.3f}*".format(pi))
```

Ask the assistant to explain what each part of `{:>10.3f}` means, one character at a time. Then test the explanation by changing one character at a time yourself: if it says that `>` right-aligns the number, replace it with `<` and see whether the number jumps to the other side.

Exercise 15-11

AI: EXPLAINER

This is bonus info rather than an actual exercise. How do you think Python can figure out that adding strings is supposed to work differently than adding numbers? Remember that `'1' + '2'` is `'12'` not `3`. The answer is that all values you can add with the `+` operator have a secret method called `__add__` that defines how adding works for that type of value:

```
s1 = "11"
s2 = "22"
n1 = 11
n2 = 22
print(s1 + s2)
print(s1.__add__(s2))
print(n1 + n2)
print(n1.__add__(n2))
```

This is one of many examples of how objects allow Python to implement functionality that fits each value type. This was to show how Python does this. Like yellow and black stripes in nature means “don’t touch me!” – double underscores (`__`) is Python’s way of saying “do not use this!”. You are supposed to use the `+` operator, not the `__add__` method.

Ask the assistant which other operators have a secret method like this one, and what those methods are called. Then check two of its answers the way this exercise did: call the dunder directly and see whether it gives the same result as the operator. If the assistant invents one that does not exist, you get an `AttributeError`, which is exactly the kind of thing you are learning to catch.

Indexing and slicing strings

Another feature of string objects is that they allow you to extract individual parts of the string.

Each character in a string is identified by its *index* in the string. To access a character in a list, you write brackets after the string. Between those brackets, you specify the index of the character you want. The first character has an index of 0; the second has an index of 1, and so on.

```
codon = 'ATG'
print("first base is", codon[0])
print("second base is", codon[1])
print("third base is", codon[2])
```

You may wonder why the index of the first character is zero and not one. That is simply the convention in programming and is so for good reason. Over time you will begin to find this useful rather than annoying. You should think of the index as the offset from the start of the string.

That also means that the index is not the length of the string but the length minus one:

```
amino_acids = 'ARNDCQEGHILKMFPSTWYV'
last_index = len(amino_acids)-1
print("Last amino acid is", amino_acids[last_index])
```

If you want a sub-string from a larger string (we call that a *slice*), you specify a start index and an end index separated by a colon:

```
print(amino_acids[1:4])
```

When you run that, you can see that `amino_acids[1:4]` is substituted for `'RND'`, so the slicing operation produces a sub-string of `amino_acids`. You may wonder why the value at index 4 is not in the resulting sub-string. That is another programming convention: intervals are *ends exclusive*. So when you specify an interval with a start index of 1 and an end index of 4, it represents all the characters starting from 1 and up to, *but not including*, 4. So, the slice `1:4` corresponds to the characters at indexes 1, 2, and 3. The reason programmers handle intervals in this way is that it makes it easier to write clear and simple code as you will see in the exercises.

Exercise 15-12

SOLO

What does this expression reduce to?

```
"Futterwacken"[7]
```

Exercise 15-13

SOLO

What is printed here? Do all the substitution and reduction steps and compare to the exercise above.

```
s = "Futterwacken"  
print(s[7])
```

Exercise 15-14

SOLO

What is printed here? Do all the substitution and reduction steps – and do it *twice*. Next week you will be happy you did.

```
dna = 'TGAC'  
i = 0  
print(dna[i])  
i = 1  
print(dna[i])  
i = 2  
print(dna[i])
```

Exercise 15-15

SOLO

What do you think happens here? Make up your mind and try out the code below:

```
s = "Futterwacken"  
s[6] = 'W'
```

Did you see that coming? Strings are *immutable*, which means that you cannot change them once you have made them. If you want "FutterWacken" you need to produce a new string with that value. Try to figure out how to do that with the `replace` method of strings.

Exercise 15-16

SOLO

When you do not specify a slice's start and/or end, Python will assume sensible defaults for the start and end indexes. What do you think they are? Make up your mind and try out the code below:


```
s = 'abcdefghijklmnopqrstuvwxyz'
print(s[:11])
print(s[11:])
print(s[:])
```

Exercise 15-17

AI: EXPLAINER

Find the documentation for how the slicing of strings works.

Then ask the assistant what `'ATGTAG'[::-1]` does and why, before you run it. Slicing is one of the places where a confident wrong answer looks exactly like a confident right one, so read the explanation, decide what you expect to see, and let Python settle it.

Exercise 15-18

SOLO

What do you think happens when you specify an index that does not correspond to a value in the list:

```
alphabet = 'abcdefghijklmnopqrstuvwxyz'
print(alphabet[99])
```

Read and make sure you understand the error message. You can try to Google the error message.

Exercise 15-19

SOLO

Do you think you also get an error when you specify a slice where the end is too high? Try it out:

```
alphabet = 'abcdefghijklmnopqrstuvwxyz'
print(alphabet[13:99])
```

and this:

```
alphabet = 'abcdefghijklmnopqrstuvwxyz'
print(alphabet[10000:10007])
```

I guess that is worth remembering.

Exercise 15-20

SOLO

Which character in a string named `alphabet` does this expression reduce to?

```
alphabet[len(alphabet)-1]
```

Exercise 15-21

SOLO

Because intervals are “ends exclusive”, we can compute the length of a slice as `end - start`:

```
dna = "ATGAGGTCAAG"
start = 1
end = 4
print("{} has length {}".format(dna[start:end], end-start))
```

Figure out what this code would look like if ends were included in intervals.

Exercise 15-22

SOLO

Another advantage of “ends exclusive” intervals is that you only need one index to split a string in two:

```
s = 'Banana'
idx = 3
beginning = s[:idx]
end = s[idx:]
print(beginning + end)
```

Figure out what indexes you would need to use to split a sequence in two if ends were included in intervals.

Exercise 15-23

SOLO

Did you look up the details of how slicing works in exercise 15-17? Then you should be able to explain what happens here:

```
s = 'zyxvutsrqponmlkjihgfedcba'
print(s[::-1])
```

General exercises

Exercise 15-24

SOLO

Will this print `Bananas rule!` ? Do all the substitutions and reductions.

```
if 'na' * 2 == "Banana"[2:]:  
    print("Bananas rule!")
```

Exercise 15-25

SOLO

Will this print `Bananas rule!` ? Do all the substitutions and reductions.

```
if "{}s".format('Banana'[1:].capitalize()) == 'Ananas':  
    print("Bananas rule!")
```

Exercise 15-26

SOLO

Write a function called `even_string` that takes a string argument and returns `True` if the length of the string is an even number and `False` otherwise. E.g. `even_string('Pear')` should return `True` and `even_string('Apple')` should return `False` (remember the `modulo` operator?).

Exercise 15-27

SOLO

Look at the code below and decide what is printed at the end. Then, write the code and test your prediction. If you are wrong, figure out why by revisiting the chapter about functions.

```
def enigma(x):  
    if x == 4:  
        return x  
  
result = enigma(5)  
print(result)
```

Exercise 15-28

SOLO

Inspect the code below and determine why it does not print that you are a super star. Test the function using various inputs and identify the mistake.

```
def even_number(x):  
    if x % 2:  
        return False  
  
if even_number(4):  
    print('You are a super star!')
```

16 Lists

This chapter is about lists and dictionaries that are Python values that can contain other Python values. Lists and dictionaries let you build relationships between values, which is what data structures represent.

Lists

For many kinds of data, the order of things is important. Just like the order of characters is important for the meaning of the text in a string, we sometimes want to specify the order of other things because the relative order of items in the list has some meaning. It could be a grocery list where you have listed the things to buy in the order you get to them in the supermarket. This is where Python lists are helpful. When you print a list, it nicely shows all the values it contains.

```
grocery_list = ["salad", "canned beans", "milk", 'beer', 'candy']
print(grocery_list)
```

Unlike strings that can only store the order of characters, lists can contain *any* kind of values, and you can mix different types of values in any way you like. Here is a list that contains an integer, a boolean, a string, and a list:

```
mixed_list = [42, True, 'programming', [1, 2, 3] ]
```

By now, you have probably guessed you will make a list with two square brackets. Between them, you can put values with commas in between. A list is a *container* of other values, and the value of the list itself does not depend on the values it contains. This makes sense. Otherwise, an empty list would not have a value:

```
my_list = []
```

You can add single values to the end of a list using the `append` method of lists. Try it out:

```
desserts = []
print(desserts)
desserts.append('Crepe suzette')
print(desserts)
desserts.append('Tiramisu')
print(desserts)
desserts.append('Creme brulee')
print(desserts)
```

If you have a list you want to add to the end of another list, you use the `extend` method:

```
cheeses = ['Gorgonzola', 'Emmentaler', 'Camembert']
desserts.extend(cheeses)
print(desserts)
```

Notice how `append` and `extend` *modifies* the existing list instead of producing a new list with the added element.

Exercise 16-1

SOLO

Do you think this will work?

```
cheeses = ['Gorgonzola',
           'Emmentaler',
           'Camembert']
print(cheeses)
```

Surprised? Code inside parentheses, brackets, and braces can span several lines, sometimes making your code easier to read.

Exercise 16-2

SOLO

You use the `' in '` operator to test if a value is in a list. Try this:

```
print('Tiramisu' in desserts)
print('Meatloaf' in desserts)
```

Exercise 16-3

SOLO

You can concatenate two lists to produce a new joined list. Make sure you figure out how this works before you try it. Then, experiment with changing the lists. Can you concatenate two empty lists?

```
some_list = [1, 2, 3]
another_list = [7, 8, 9]
merged_list = some_list + another_list
print(merged_list)
```

This is yet another example of how the functionality of Python objects lets them “know” how to behave under different circumstances, such as when adding two objects (see exercise 15-11).

Exercise 16-4

SOLO

What do you think is printed here? Make sure you figure out how you think this works before you try it out. What does the `append` method return?

```
my_list = []
x = my_list.append(7)
print(x)
print(my_list)
```

Indexing and slicing lists

Now you know how to make lists, but to work with the values in lists, you must also know how to access the individual values a list contains. Fortunately, indexing lists work just like indexing strings: Each value in a list is identified by an *index* exactly like each character in a string:

```
numbers = [7, 4, 6, 2, 8, 1]
print("first value is", numbers[0])
print("second value is", numbers[1])
print("third value is", numbers[2])
```

Notice that the function `len` can also compute the length of a list. So you also get the last value in a list like this:

```
numbers = [7, 4, 6, 2, 8, 1]
last_index = len(numbers)-1
print("Last element is", numbers[last_index])
```

If you want a sub-list of values from a list (we also call that a *slice*), you specify a start index and an end index separated by a colon, just like with strings:

```
print(numbers[1:4])
```

When you run that, you can see that `numbers[1:4]` is substituted for `[4, 6, 2]`, so the slicing operation produces a new list of the specified values.

Exercise 16-5

SOLO

What do these two expressions reduce to?

```
[11, 12, 13, 14, 15, 16, 17][2]
```

Exercise 16-6

SOLO

What is printed here? Do all the substitution and reduction steps and compare them to the exercise above.

```
l = [11, 12, 13, 14, 15, 16, 17]
print(l[2])
```

Exercise 16-7

SOLO

What is printed here? Do all the substitution and reduction steps — and do them twice. Next week, you will be happy you did.

```
numbers = [1,2,3]
i = 0
print(numbers[i])
i = 1
print(numbers[i])
i = 2
print(numbers[i])
```

Exercise 16-8

SOLO

What do you think happens here? Make up your mind and try out the code below:

```
l = [11, 12, 13, 14, 15, 16, 17]
l[4] = "Donald"
print(l)
```

Were you surprised by what happened? Compare to exercise 15-15. Lists are not immutable like strings, and you can replace values by assigning a new value to an index in the list.

Exercise 16-9

SOLO

With your knowledge of slicing, what do you think is printed below:

```
l = [11, 12, 13, 14, 15, 16, 17]
print(l[:3])
print(l[3:])
print(l[:])
```


Exercise 16-10

AI: EXPLAINER

What do you think happens when you specify an index that does not correspond to a value in the list:

```
l = [11, 12, 13, 14, 15, 16, 17]
print(l[7])
```

Read and understand the error message. Does it ring a bell?

Then ask the assistant to explain the error and to tell you the largest index this list will accept. Check that number by running it, and run one larger too. An answer that is off by one is the most likely way for the assistant to be wrong here, and it is also the easiest to catch.

Exercise 16-11

SOLO

Do you also get an error when you specify a slice where the end is too high? Try it out:

```
l = [11, 12, 13, 14, 15, 16, 17]
print(l[4:99])
```

I guess that is also worth remembering.

Exercise 16-12

SOLO

Which value in a list named `l` does this expression reduce to?

```
l[len(l)-1]
```

Exercise 16-13

SOLO

If you do not like Emmentaler, you can delete it. What does the `del` keyword do?

```
cheeses = ['Gorgonzola', 'Emmentaler', 'Camembert']
print(cheeses)
del cheeses[1]
print(cheeses)
```

Exercise 16-14

SOLO

Because intervals are “ends exclusive,” we can compute the length of a slice as `end - start`:

```
l = [7, 4, 6, 2, 8, 1]
start = 1
end = 4
print("{} has length {}".format(l[start:end], end-start))
```

Consider what this code would look like if ends were included in intervals.

Exercise 16-15

SOLO

Another advantage of “ends exclusive” intervals is that you only need one index to split a list in two:

```
numbers = [1, 2, 3, 4, 5, 6]
i = 3
beginning = numbers[:i]
end = numbers[i:]
print(beginning + end)
```

If ends were included in intervals, this would be more complex.

Exercise 16-16

SOLO

Do *all* the substitution and reduction steps in your head (or on paper) before you write any of the following code. Think carefully and decide what you think will be printed below. Remember that the *value* of a list is a container that holds other *values* in it. Then, write the code and see if you are right. If you were not, figure out what led you to the wrong conclusion.

```
x = 'A'
y = 'B'
z = 'C'
lst = [x, y, z]
print(lst)

x = 'Preben'
print(lst) # what is printed here?

lst[0] = 'Mogens'
print(lst) # what is printed here?
```

Exercise 16-17

SOLO

Do you remember this trick from string slicing?

```
l = [1, 2, 3, 4, 5]
print(l[::-1])
```

Exercise 16-18

SOLO

You can produce a list by splitting a long string into smaller parts. Think: *“Hey string, split yourself on this smaller string”*. Try these variations to figure out how it works.

```
"Homo sapiens neanderthalensis".split(" ")
"Homo sapiens neanderthalensis".split('en')
'ATGCTCGTAACGACACTGCACTACTACAATAG'.split('')
"1, 2, 3, 5, 3, 2, 5, 3".split(', ')
"1,2,3,5,3,2,5,3".split(',')
'ATGCTCGTAACGACACTGCACTACTACAATAG'.split()
"Homo sapiens neanderthalensis".split()
```

Notice that the method has a default behavior when no argument is passed to it.

Exercise 16-19

SOLO

You can produce a string by joining the elements of a list (if all the elements are strings, of course). Think: *“Hey string, put yourself between all the strings in this list”*.

```
"-".join(['Homo', 'sapiens', 'neanderthalensis'])
"...".join(['Homo', 'sapiens', 'neanderthalensis'])
"".join(['A', 'T', 'G'])
```

Notice how you can join something on an empty string. This is a very useful technique to turn a list of characters into a string.

Exercise 16-20

AI: TRANSLATOR

This is the new badge from Chapter 14, and it comes with a rule: you check the translation in *both* directions.

Take this line:

```
print(len(['ATG', 'GCC', 'TAA'][:1]))
```

First write down, in one English sentence and without running anything, what it does. Then give the line to the assistant and ask it for one English sentence. Put the two side by side. Do they agree? Is one of them vague exactly where the other is precise? Now open a fresh conversation, give it *your* sentence and nothing else, and ask for the Python that does what it says. Run what comes back, and run the original. If they disagree, one of the two translations was wrong, and working out which one is the whole exercise.

General exercises

What does this expression reduce to? `'aaaaa'`, `'BaBaBa'`, or `'Banan'`. Make up your mind, and then run the expression to check.

```
'a'.join('Banana'.split('a')[:3] * 4)[-5:]
```

17 Dictionaries

This chapter is about dictionaries that, like lists, is another Python value that can contain other Python values. Dictionaries let you build relationships between values, which is what data structures represent.

Dictionaries

Lists are useful for storing values when the order of the values is important, but they have one drawback: you can only access a value in a list using its index.

A dictionary called `dict` in Python, is a much more flexible data type. Like a list, a dictionary is a container for other values, but dictionaries do not store values in sequence. They work more like a database that lets you store individual *values*. When you store a value, you assign it to a *key* that you can use to access the stored value. Now, create your first dictionary:

```
person = {'name': 'Robert Redford', 'height': 179, 'job': 'Actor'}
```

This dictionary has three values (`'Actor'` , `'Robert Redford'` and `179`) and each value is associated with a key. Here `'height'` is the key for the value `179` . So, when defining a dictionary, you should note the following:

1. You make a dictionary using braces.
2. you put key-value pairs separated by a colon between your braces.
3. Commas separate the key-value pairs.
4. To make an empty dictionary, write the braces with nothing between them: `{}` .

To access a value in the dictionary, you put its key in square brackets after the dictionary:

```
"{} is a {} cm {}".format(person['name'], person['height'], person['job'])
```

Here we used strings as keys, but you can also use many types of values as keys (Python will give you an error if you try to use a type that is not allowed):

```
misc_dict = {42: "Meaning of life", "pi": 3.14159, True: 7}
```

A dictionary stores key-value pairs but does not keep track of their order. So, when you print a dictionary, the order of the key-value pairs is arbitrary.

If you have a dictionary, you can add key-value pairs in this way:

```
person['job'] = 'Retired'
person['hair'] = 'uniquely combed'
print(person)
```

Notice that if you assign a value (**71**) to a key that is already in the dictionary (**'age'**), then the old value (**70**) is replaced.

Exercise 17-1

SOLO

What does this expression evaluate to?

```
{ 'name': 'Robert Redford', 'height': 179, 'job': 'Actor' }['name']
```

Exercise 17-2

SOLO

Assuming the definition of the **person** dictionary above, what does this expression evaluate? Compare this to the expression in the previous exercise.

```
person['name']
```

Exercise 17-3

SOLO

The **in** operator also works with dictionaries. Look at what these expressions reduce to and then try to figure out what **in** does when applied to a dictionary:

```
'name' in person
'height' in person
'job' in person
84 in person
'Actor' in person
'Robert Redford' in person
```

Exercise 17-4

AI: EXPLAINER

Write and run this code with different values of **key** and read any error messages.

```
key = 3
# key = 'banana'
# key = 3.14159
# key = True
# key = {}
# key = []
d = {}
d[key] = 7
```

Are any of the values not allowed as keys?

Before you uncomment each line, ask the assistant to predict which of these keys Python will refuse and what the error will say. Then work through them and keep score. This is a good early test of whether the assistant knows a rule or is guessing from the shape of the code.

Exercise 17-5

SOLO

Do you think this will work?

```
person = {'name': 'Robert Redford',
          'height': 179,
          'job': 'Actor'}
print(person)
```

Exercise 17-6

AI: ILLUSTRATOR

The other new badge from Chapter 14. Nested dictionaries are usually the thing people half understand at this point, so use it on those.

Ask the assistant for five short examples of a dictionary whose values are themselves dictionaries, each one about genes, codons or species, and each one reaching into the nesting in a different way. Ask for the code only: no output, no explanation. Now do the work. Predict what each of the five prints, write your predictions down, and only then run all five.

You end up with two numbers: how many of its examples did what it claimed, and how many you called correctly before you ran them. The second one is the interesting number. An example is only illustrating something to you if you can say what it does before the machine tells you.

General exercises

Start by making dictionaries for (some of) the Trump family:

```
donald = {'name': 'Donald Trump', 'age': 70, 'job': 'President' }
melania = {'name': 'Melania Trump', 'age': 70, 'job': 'First lady' }
tiffany = {'name': 'Tiffany Trump', 'age': 23, 'job': 'Internet personality' }
ivanka = {'name': 'Ivanka Trump', 'age': 35, 'job': 'Top aide' }
```

Exercise 17-7

SOLO

What do you think the following code produces? Do *all* of the substitution and reduction steps in your head, and only then try out the code.

```
donald['child'] = tiffany
melania['husband'] = donald

print(melania)
print(melania['husband']['child'])
```

Exercise 17-8

SOLO

A dictionary can contain *any* kind of Python values, even lists or dictionaries. Consider the code below, where we add a list of ex-wives to the Trump persona. Can you see why we need to check the `'ex-wives'` key before we add it to the list of ex-wives?

```
donald = {'name': 'Donald Trump', 'age': 70, 'job': 'President' }

if 'ex-wives' not in donald:
    donald['ex-wives'] = []
donald['ex-wives'].append('Marla Maples')
donald['ex-wives'].append('Ivana Trump')

print(donald)
```

Exercise 17-9

SOLO

In case you wonder what the *type* of value a list is, or a dictionary, try this:

```
print("A list has type:", type([]))
print("A dictionary has type:", type({}))
```

Now the types `list` and `dict` are your friends too.

Exercise 17-10

SOLO

Lists can also contain any type of value. Consider this example. What do you think the following code produces? Do *all* the substitution and reduction steps in your head, and only then try out the code.

```
trump_family = [donald, melania, ivanka, tiffany]
print(trump_family)
print(trump_family[1]['job'])
```

Exercise 17-11

SOLO

Write and run this code

```
amino_acids = {}
amino_acids['ATG'] = 'met'
amino_acids['TCT'] = 'ser'
amino_acids['TAC'] = 'tyr'

codon = 'TCT'
print("{} encodes {}".format(codon, amino_acids[codon]))
```

“ You have probably noticed that the interpretation of length is different for each type of value. In a string, it is the number of characters; in a list, it is the number of values in the list; in a dictionary, it is the number of key-value pairs. How do you think Python knows which length interpretation to use when the `len` function is called? This is where objects shine. `len(x)` returns the value that `x.__len__()` returns. So the `len` function is defined roughly like this:

```
def len(x):
    return x.__len__()
```

Similarly, the `in` operator calls a secret `__contains__` method.

”

18 Tuples

Grouping values

A tuple is a sequence of values, just like a list. However, unlike a list, the elements of a tuple can not be changed. You cannot append to a tuple, either. Once a tuple is made, it is immutable (or unchangeable). To make a tuple, you just use round brackets instead of square brackets:

```
fruits = ("apple", "banana", "cherry")
```

It may seem strange that Python has both tuples and lists. One reason is that tuples are more efficient, whereas lists are more flexible. We will not use tuples often, but you must know what they are.

You can do most of the operations on a tuple that you can also do on a list. The following exercises should be easy if you remember how to do the same thing on lists:

Exercise 18-1

SOLO

Find the number of elements in the `fruits` tuple using the `len` function.

Exercise 18-2

SOLO

Extract the second element of the `fruits` tuple (`"banana"`) using indexing.

Exercise 18-3

SOLO

Try to change the second element of the `fruits` tuple to `"apple"` and see what happens. It should be something like this:

```
Traceback (most recent call last):
```

```
  File "script.py", line 2, in <module>
```

```
    fruits[3] = "apple"
```

```
TypeError: 'tuple' object does not support item assignment
```

You cannot change elements of a tuple because they are immutable (once made, they stay that way).

Tuple assignment

Python lets you assign a tuple of values to a tuple of variables like this:

```
father, mother, son = ("Donald", "Ivana", "Eric")
```

It does the same as the following three assignments:

```
father = "Donald"  
mother = "Ivana"  
son = "Eric"
```

When a tuple is made, the values are “packed” in sequence:

```
family = ("Donald", "Ivana", "Eric")
```

Using the same analogy, values can be “unpacked” using tuple assignment:

```
father, mother, son = family
```

The only requirement is that the number of variables equals the number of values in the tuple.

Once in a while, it is useful to swap the values of two variables. With conventional assignment statements, we have to use a temporary variable. For example, to swap *a* and *b*:

```
tmp = a  
a = b  
b = tmp
```

Exercise 18-4

AI: EXPLAINER

Try this and read the error message:

```
family = ("Donald", "Ivana", "Eric")  
father, mother = family
```

Paste the error into the assistant and ask it to say, in one sentence, what Python was trying to do when it failed. Then run the fixed version, with three names on the left, to confirm that the sentence was right.

Exercise 18-5

AI: EXPLAINER

Try this and read the error message:

```
family = ("Donald", "Ivana", "Eric")  
father, mother, son, daughter = family
```

Compare to the error message in the previous exercise.

Ask the assistant to explain this one too, and then ask it whether it is the same error as the previous one. If it tells you that they are the same, decide in what sense that is true: the type of error is the same, the message is not, and knowing which of the two you are being told matters more than it sounds.

Exercise 18-6

SOLO

Say you want to swap the values of two variables, `a` and `b`. To do that, you would need to keep one of the values in an extra variable like this:

```
temp = a  
a = b  
b = temp
```

Using what you have learned in this chapter, can you devise a simple and pretty way of swapping `a` and `b` in one statement? It may occur to you before you realize how it works, so make sure you can connect your solution to the rules of tuples and tuple assignment.

19 Turtle, week 3: an object that keeps its own state

This week you met objects and their methods. You learned that a string is a thing that knows how to do something to itself, that `'atgc'.upper()` is the way you ask it, and that the same idea explains lists, dictionaries and tuples. Everything in this session is that same idea seen from a second angle, because a turtle is an object too, and the ways it differs from a string are exactly the ways beginners get objects wrong.

A turtle is a little machine that lives on a canvas, holds a position and a direction, carries a pen that is either down or up, and obeys methods you call on it. When you tell it to move forward it leaves a line behind, and after it has moved it is somewhere else. That is the whole of it, and it is enough to make one distinction very concrete. A string method hands you a new string and leaves the original string exactly as it was. A turtle method changes the turtle and hands the turtle back. Both are methods, both are called with a dot, and they behave in opposite ways, which is why having both in front of you at once is worth an hour.

The turtle also gives you somewhere to put the containers you met in the same week. A drawing is a natural thing to describe with a list of distances, a dictionary from a letter to a colour, or a tuple of two coordinates, and you will do all three before the end of this session. The turtle's own answer to the question of where it is comes back as a tuple, so you will meet tuple unpacking on real output rather than on an invented example.

Getting a turtle on the screen

Turtle exercises are ordinary notebook cells and not `%%exercise` scripts, because the drawing appears underneath the cell that made it. The rule that follows from this is simple and you should adopt it now: everything belonging to one drawing goes in one cell, including the line that creates the turtle. If you create a turtle in one cell and try to drive it from the next, you will get no picture, because the picture was already drawn and finished at the end of the first cell.

If you are working in the browser version of the notebooks, install the widget once with the following line. If you are working in your own environment where it is already installed, skip it.

```
%pip install -q turtle-widget
```

Then, in every cell that draws something, start by importing and making a turtle.

```
from turtle_widget import Turtle

t = Turtle()
t.forward(100)
```

Run that. The turtle starts in the middle of the canvas facing right, and the line it leaves behind is the evidence that it moved. Notice that you did not have to write anything on the last line of the cell to make the drawing appear. The turtle shows itself when the cell ends.

Exercise 19-1

SOLO

Decide, before you run it, where the turtle will end up and which way it will be facing. Then run it and see whether the picture agrees with you.

```
from turtle_widget import Turtle

t = Turtle()
t.forward(100)
t.left(90)
t.forward(100)
t.left(90)
t.forward(100)
```

The method `left` turns the turtle counter-clockwise by the number of degrees you give it, and there is a matching `right`. Neither of them moves the turtle. A turn changes only which way the turtle is facing, and it needs saying out loud because a great many wrong drawings come from forgetting it.

Exercise 19-2

SOLO

Here is a string doing what strings do. Decide what each of the two `print` calls will show before you run the cell.

```
dna = 'atgctagc'
dna.upper()
print(dna)
print(dna.upper())
```

You have seen this before, and the lesson is that `dna.upper()` produces a new string and leaves `dna` alone, so calling it on a line by itself accomplishes nothing at all. The result was made and immediately thrown away.

Now do the same thing to a turtle and watch it behave in the opposite way.

```
from turtle_widget import Turtle

t = Turtle()
t.forward(100)
print(t.position())
```

Calling `t.forward(100)` on a line by itself is not a wasted line. It is the whole point. The turtle did not hand you a new turtle that had moved. It moved.

Commands and questions

Turtle methods come in two kinds, and telling them apart is the skill this section builds. A command tells the turtle to do something, and its effect is a change in the turtle. Methods like `forward`, `backward`, `left`, `right`, `penup`, `pendown` and `goto` are commands. A question asks the turtle about itself and hands you back an answer, and the turtle is unchanged by being asked. Methods like `position`, `xcor`, `ycor`, `heading`, `isdown` and `isvisible` are questions.

The string methods you met this week are all questions in this sense, even the ones that sound like commands. When you ask a string to capitalise itself, it does not change. It works out what a capitalised version would look like and hands you that instead. A string cannot be changed at all, which is what immutable means, and it is the reason every useful string method returns something you have to catch. The turtle is the other case. It can be changed, and its commands change it.

Exercise 19-3

SOLO

What does a turtle command hand back? Run this and look at what the second and third lines draw.

```
from turtle_widget import Turtle

t = Turtle()
t.forward(80).left(120).forward(80)
t.left(120).forward(80)
```

That works because each command hands the turtle itself back, already changed, so the next dot in the line has a turtle to talk to. It is the same turtle every time, and not a copy. Chaining like this is legal, and you should recognise it when you see it, but for anything longer than a couple of steps a separate line for each command is far easier to read and far easier to trace in your head.

Exercise 19-4

AI: EXPLAINER

Ask the assistant to explain, in three sentences and without code, why `dna.upper()` on a line by itself does nothing useful while `t.forward(100)` on a line by itself does everything you wanted. Then decide whether its explanation is right, and confirm the part of it you are least sure about by writing a few lines and running them. An explanation stays provisional until the machine has agreed with it, and this week the machine is a turtle.

Asking the turtle where it is

The questions are the part of the turtle that will matter most later in the course, so get used to them now. The turtle's canvas has its origin in the middle, x growing to the right and y growing upwards. A heading of zero means facing right, and the heading grows as the turtle turns counter-clockwise, so ninety is straight up.

Exercise 19-5

SOLO

Write down the three numbers you expect before you run this. Then run it.

```
from turtle_widget import Turtle

t = Turtle()
t.forward(100)
t.left(90)
t.forward(50)
print(t.xcor())
print(t.ycor())
print(t.heading())
```

If any of the three surprised you, do the substitution in your head one line at a time until you can say where the turtle was after each command. The picture and the numbers are two reports of the same fact, and if they disagree in your head then one of them is wrong and it is worth finding out which.

Now ask the same question a second way. The method `position` answers with both coordinates at once, packed into a tuple. Predict the two lines of output before running.

```
from turtle_widget import Turtle

t = Turtle()
t.forward(120)
t.left(90)
t.forward(60)
```



```
here = t.position()
print(here)
x, y = t.position()
print(y)
```

The last two lines are tuple unpacking, exactly as you met it this week. The tuple has two items, the two names on the left take one each, and the order is the order in the tuple.

Exercise 19-6

SOLO

This one is meant to fail, and the error is the point. Predict which line raises the error and roughly what it will say, then run it and read the message carefully.

```
from turtle_widget import Turtle

t = Turtle()
t.forward(100)
here = t.position()
here[0] = 0
print(here)
```

A tuple cannot be changed after it is made, so there is no way to assign into one. Notice what this means for the turtle. The tuple you got back is a report of where the turtle was when you asked, and rewriting the report would not move the turtle even if Python let you. To move the turtle you have to give the turtle a command.

Exercise 19-7

SOLO

The pen is a piece of the turtle's state just like its position, and there is a question for it. Predict all three printed values before running.

```
from turtle_widget import Turtle

t = Turtle()
print(t.isdown())
t.forward(80)
t.penup()
print(t.isdown())
t.forward(80)
t.pendown()
t.forward(80)
print(t.isdown())
```

Look at the picture as well as the numbers. The turtle travelled the same distance three times and the canvas shows only two lines. The middle journey happened, and left no trace of itself, which is a small preview of something that becomes important in week eight.

Two names, one turtle

This week you saw that giving a list a second name does not give you a second list. Turtles behave the same way, and because a turtle is visible the demonstration is unusually blunt.

Exercise 19-8

SOLO

Decide what the picture will look like and what the final line will print, then run it.

```
from turtle_widget import Turtle

t = Turtle()
other = t
other.forward(100)
other.left(90)
other.forward(100)
print(t.position())
```

There is one turtle on that canvas and it has two names. Everything you did through `other` happened to `t`, because `other` and `t` were never two things. This is the same fact you met with `a = [1, 2, 3]` followed by `b = a`, and it is worth having met it twice, in two places where the consequences look completely different.

Lists and dictionaries that describe a drawing

A drawing is data before it is a picture, and this section is where you start writing the data down. You do not yet have loops, so you will index into your containers by hand, which is a perfectly good way to see what is in them.

Exercise 19-9

SOLO

Fill in the three missing lines so that the turtle walks all four distances in the list, turning ninety degrees to the left after each one. Take each distance out of the list by its index rather than typing the number again.

```

from turtle_widget import Turtle

distances = [120, 60, 120, 60]

t = Turtle()
t.forward(distances[0])
t.left(90)

```

When it works, change one number in the list, run the cell again, and watch the drawing follow the data. Nothing in the drawing commands mentions a size any more. The sizes all live in one place, and that is the beginning of a habit that will carry you a long way.

Exercise 19-10

SOLO

A dictionary is the natural way to say that each base gets its own colour. Predict which three colours appear, in which order, then run it.

```

from turtle_widget import Turtle

colour = {'A': 'green', 'T': 'red', 'G': 'black', 'C': 'blue'}
dna = 'GAT'

t = Turtle()
t.pensize(6)
t.pencolor(colour[dna[0]])
t.forward(80)
t.pencolor(colour[dna[1]])
t.forward(80)
t.pencolor(colour[dna[2]])
t.forward(80)

```

Read the expression `colour[dna[0]]` from the inside out, exactly as you would reduce any other expression. First `dna[0]` reduces to `'G'`, then `colour['G']` reduces to `'black'`, and only then does the method get called. Two lookups of two different kinds sit in that one pair of brackets, one into a string by position and one into a dictionary by key, and being able to take them apart calmly is most of what indexing is.

Next week the turtle gets loops, and everything you just typed out three times will collapse into three lines that work for a strand of any length.

Week 4

20 Ask it to explain, then confirm by running

The assistant as Explainer: an explanation is useful but provisional.

Session length

Estimated 3 hours, run as this week's TA session. Bring code from your own exercises this term — several rounds below ask you to supply your own broken or puzzling code rather than a prepared example.

The ladder so far

Explainer · Translator ·
Illustrator

Still to come: Comparer,
Drafter, Worker,
Collaborator, Developer

The move for this whole session: paste something, a traceback or a piece of code, ask the assistant to explain it, read the explanation, and only then run the actual code and read what really happens. An explanation you have not confirmed is a plausible story, not knowledge. Every round below ends the same way: does what actually happened match what the assistant said would happen, in every detail, or only in the parts that were easy to guess?

The move, every time

1. **Paste.** Give the assistant the error or the code, with no extra hints about what you already suspect.
2. **Read the explanation.** Note the specific claim it makes: which line is at fault, what value caused it, what the code will print.
3. **Confirm.** Run the code yourself. Compare what actually happened, line for line, against the specific claim, not just the general vibe of it.

Part 1 — Syntax errors (30 min)

Each snippet below fails before it ever runs. Read the assistant's explanation of *why*, then run the snippet yourself and confirm the error message matches its story exactly, not approximately.

Exercise 1 20-1

AI: EXPLAINER

```
def gc_content(dna:
    return dna.count('G') / len(dna)
```

Exercise 2 20-2

AI: EXPLAINER

```
sequences = ['ATG', 'GCC', 'TAA']  
print(sequences)
```

Exercise 3 20-3

AI: EXPLAINER

```
for base in 'ATGC':  
    print(base)
```

Exercise 4 20-4

AI: EXPLAINER

```
name = 'Preben'  
print('Hello, ' name)
```

Part 2 — Runtime errors (40 min)

These run, and then fail partway through. Read the traceback yourself first — note the line and the error name — before you ask the assistant anything, so you have your own prediction of the cause to compare against its explanation.

Exercise 5 20-5

AI: EXPLAINER

```
counts = {'A': 4, 'T': 3, 'G': 2}  
print(counts['C'])
```

Exercise 6 20-6

AI: EXPLAINER

```
bases = ['A', 'T', 'G', 'C']  
print(bases[4])
```

Exercise 7 20-7

AI: EXPLAINER

```
def average_length(sequences):  
    total = sum(len(s) for s in sequences)  
    return total / len(sequences)  
  
print(average_length([]))
```

Exercise 8 20-8

AI: EXPLAINER

```
def combine(dna, count):  
    return dna * count  
  
print(combine('ATG', '3'))
```

Part 3 — Explaining code that runs fine (35 min)

Not every explanation is about a failure. Here the code works; the question is whether the assistant's account of *what it does* is actually right, which you confirm by predicting the printed output yourself before running it.

Exercise 9 20-9

AI: EXPLAINER

```
dna = 'ATGGCCTAA'  
print(dna[::-1])
```

Exercise 10 20-10

AI: EXPLAINER

```
counts = {}  
for base in 'ATGGCCA':  
    counts[base] = counts.get(base, 0) + 1  
print(counts)
```

Exercise 11 20-11

AI: EXPLAINER

```
sequences = ['ATG', 'GCCTAA', 'TT']
longest = max(sequences, key=len)
print(longest)
```

Part 4 — Catch the wrong explanation (35 min)

This one is curated so the assistant’s explanation, while fluent and structured exactly like the correct ones above, gets something specific and checkable wrong. Do not skip the confirm step here even if the explanation sounds completely convincing — especially if it sounds completely convincing.

Exercise 12 20-12

AI: EXPLAINER

```
def add_base(bases=[]):
    bases.append('A')
    return bases

print(add_base())
print(add_base())
```

Ask the assistant to explain, line by line, what each call to `add_base()` prints, and why. Run it. Where exactly does the explanation and the real output part ways, and what does that tell you about default argument values in Python?

Exercise 13 20-13

AI: EXPLAINER

```
original = ['A', 'T', 'G']
copy = original
copy.append('C')
print(original)
```

Ask for an explanation of what `print(original)` will show, predicting first yourself. Where the assistant’s explanation and the actual output disagree, or where it gets the right output but for a reasoning that would fail on a different example, is exactly the case this exercise exists to surface.

Part 5 — Bring your own (20 min)

Find one error message or one piece of code from your own work this term, ideally from this week’s exercises, that confused you at the time.

Exercise 14 20-14

SOLO

Before asking the assistant anything, write down, in your own words, what you now believe was actually happening, using what you have learned since.

Exercise 15 20-15

AI: EXPLAINER

Ask the assistant to explain the same error or code. Compare its explanation to the one you wrote yourself in Exercise 14. Confirm both against the actual running code. Which explanation was more exact, and which, if either, missed a detail that mattered?

Wrap-up (10 min)

For your logbook this week, describe the one explanation from today's session that sounded most convincing and turned out to be wrong, or incomplete, once you confirmed it by running the code. Say specifically what detail the explanation glossed over, and how you would rewrite it so it would have survived confirmation.

21 Iteration

This chapter is about how you repeat the same code for many different values – and the many reasons why this is useful.

The for-loop

Programs often need to do repetitive things. Consider this example below, where `x` is assigned a value that is then printed:

```
x = 1
print(x)
x = 5
print(x)
x = 3
print(x)
x = 7
print(x)
```

You can see that we do the same thing four times, with the only difference being that the variable `x` takes a new value each time. Now, carefully write the alternative version below and compare what is printed to what was printed in the above example.

```
for x in [1, 5, 3, 7]:
    print(x)
```

It should be the same. What you just wrote is called a *for-loop*. It is called a for-loop because it does something *for* each of many values – in this case, for each value in our list.

The statements nested under the for-loop are run as many times as there are values in our list, and every time they are run, `x` is assigned a new value. The first time the statements are run, `x` is assigned the *first* value in the list. The second time they run `x`, the *second* value is assigned to the list. This continues until `x` has been assigned all the values in the list.

The semantics of a for-loop is as follows:

1. First, you write `for`.
2. Then, you write the name of the variable that will be assigned a new value for each iteration of the loop (`x` in the above case).
3. Then you write `in`.
4. Then you write the name of an *iterable* or an expression that reduces to one. In the above case, it was the list `[1, 5, 3, 7]`.

5. The statements nested under the for loop are indented with four spaces, just like with if-statements. These statements are executed once for every value in the *iterable*.

What is an *iterable*, you may ask? It is any Python value that knows how to serve one value at a time until there are none left. Only objects with an `__iter__` method can do this. You will get an error if you try to iterate over a value that does not have an `__iter__` method. Try the code below and see how Python complains that “‘int’ object is not iterable”:

```
for x in 4:
    print(x)
```

Try these variations of the for-loop above and notice how the rules 1-5 apply in each case:

```
for x in [1, 5, 3, 7]:
    print(x)

list_of_numbers = [1, 5, 3, 7]
for x in list_of_numbers:
    print(x)

for x in [1, 5] + [3, 7]:
    print(x)
```

In each case, the expression after `in` reduces to the *value* `[1, 5, 3, 7]`, which then serves as the *iterable*.

It is not only lists that are iterable. Strings are, too. Their `iter` method of a string tells it that it should serve one character at a time. Try this:

```
for character in 'banana':
    print(character)
```

Neat, right?

In programming, you often need to iterate over integer values and sometimes quite a few (like the 250 million bases of the human chromosome one). It would be quite annoying if you had to manually make long lists of integers, so Python provides a built-in function called `range` that helps you out. It returns a special *iterator* value that lets you iterate over a specified range of numbers. Try the two examples below and compare what is printed:

```
total = 0
for number in [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]:
    total += number
print(total)

total = 0
```

```
for number in range(10):
    total += number
print(total)
```

You can see that using `range` works just like using a list of numbers, but the cool thing about the range is that it does not return a list. It just serves one number at a time until it is done. This is also why you will not see a list if you try to print what `range` returns:

```
number_iterator = range(10)
print(number_iterator)
```

The `range` function needs three values to know which values to iterate over: “start”, “end” and “step”. It will assume sensible defaults if you do not give it all three arguments. Try this:

```
for i in range(0, 10, 1):
    print(i)

for i in range(0, 10):
    print(i)

for i in range(10):
    print(i)
```

You can see that the first and last arguments default to 0 and 1. If you give it two arguments, it will assume that they are “start” and “end”. If you only give it one argument, it will assume that it is the “end”.

Exercise 21-1

SOLO

What do you think the third argument to `range` specifies? Try these variations and see if you can figure it out:

```
for i in range(0, 10, 1):
    print(i)

for i in range(0, 10, 2):
    print(i)

for i in range(0, 10, 3):
    print(i)
```

Check the documentation once you have decided.

Exercise 21-2

SOLO

What will happen here:

```
for x in []:  
    print(x)
```

and here:

```
for x in range(0):  
    print(x)
```

and here:

```
for x in range(10, 10):  
    print(x)
```

Exercise 21-3

SOLO

The two examples below print the same. Make sure you understand why. Write and experiment with the code on your own.

```
list_of_words = ['one', 'two', 'three']  
  
# example 1  
for word in list_of_words:  
    print(word)  
  
# example 2  
list_length = len(list_of_words)  
for index in range(list_length):  
    print(list_of_words[index])
```

Exercise 21-4

SOLO

Finish the code below so all the even numbers go into one list, and all the odd numbers go into the other (hint: remember the modulo operator?)

```

numbers = [4, 9, 6, 7, 4, 5, 3, 2, 6]
even = []
odd = []
for n in numbers:
    # your code here ...

```

Exercise 21-5

SOLO

You can put any statements under the for loop. Here, it includes an if-statement that lets you generate a list of all the `a` characters in banana (in case you need that).

```

result = []
fruit = 'banana'
for character in fruit:
    if character == 'a':
        result.append(character)
print(result)

```

Now change the code so you instead get the *indexes* of the 'a' characters: `[1, 3, 5]`. Here are some hints:

1. You need a for-loop over a list of numbers.
2. `range(len(fruit))` may be relevant numbers :-).
3. `fruit[1]` substitutes for `'a'`.

Exercise 21-6

SOLO

Imagine you want to throw a big party and have rented a place with space for 100 people. Now, you want to start inviting people. What kind of error do you get here, and why?

```

friends = ["Mogens", "Preben", "Berit"]
invited = []
for index in range(100):
    invited.append(friends[index])

```

Exercise 21-7

SOLO

You can also put a for loop under another for loop, and the rules for each for loop are the same as those explained above. The statements nested under the for loop are indented with four spaces, just like with if statements. These statements are executed once for every value in the iterable. Think carefully about what you think is printed in the example below before you try it out.

```
for i in range(3):  
    for j in range(3):  
        print(i, j)
```

Make sure you understand i, j pairs are printed in the order they are.

22 Testing

This chapter is about how you figure out if the code you wrote solves the problem in the way you intended. You will be surprised how often that is not the case – even for seasoned programmers.

Why test your code?

There are tons of reasons why you should test your code. Here are what I think are the two most important ones:

1. **Makes you think:** Testing forces you to slow down and think about exactly what the code is supposed to do. By deciding what tests to do before you start coding, you try to anticipate errors and cases that need to be covered by how you want to solve the problem. The notion of falsification is important in science and in coding, too. You should try to prove that your idea is wrong and consider it valid only if this process fails. So, testing motivates you to think about ways to break their code, thereby helping you solve your programming problem in a general and robust way so that it does exactly what you expect it to do.
2. **Gives you peace of mind:** Testing increases your confidence that a function you have written works as it is supposed to so that you can now stop thinking about how it is implemented and focus on using it as a component in solving a larger, more complicated problem. Having set up a series of tests also allows you to change and improve your function's implementation without worrying that it stops working the way it is supposed to. As long as it passes all the tests, it should be ok.

Testing of code is a *big* thing in programming. Professionals consistently test their code. You will do it in time, too, but in this course, you will only do the basic testing yourself. Instead, you will have access to readymade testing suites made especially for each of your programming projects.

Basic testing

Say that you are asked to make a function that takes a string argument and returns `True` if that string is a palindrome and `False` otherwise (hypothetical example). Then you start thinking about which strings should make the function return `True` and which return `False`. Once you have defined your `is_palindrome` function, you can set up some fairly obvious tests like this:


```
print(is_palindrome('123321') == True)
print(is_palindrome('ATGGTA') == True)
print(is_palindrome('ATGATG') == False)
print(is_palindrome('XY') == False)
```

But if you keep thinking, maybe you will come up with more tests to cover all the different types of cases you may encounter:

```
print(is_palindrome('12321') == True) # uneven length
print(is_palindrome('121') == True) # uneven length
print(is_palindrome('AA') == True)
print(is_palindrome('A') == True) # single char
```

The project testing utility

To keep you focused on the programming part, each of the programming projects you will do in this course comes with a ready-made suite of tests of the functions you are asked to implement. So, for each function, you can run tests to ensure it implements the behavior it is supposed to.

Each project comes with two files that you can download from the course page. They have their names for a good reason, so do not change them. In the first project about translating DNA, they are called `translationproject.py` and `test_translationproject.py`.

To be able to test your functions, you *must* write your code in the file called `translationproject.py`. To run your code, you type this in the Terminal as usual:

Terminal

```
python translationproject.py
```

To test the functions as you complete each one, you can run the test script `test_translationproject.py` like this:

Terminal

```
python test_translationproject.py
```

The code in `test_translationproject.py` reads your code in `translationproject.py` and performs a series of tests of each function. When you run the test script, four things may happen depending on the state of your code:

Case 1: If you have not yet implemented all the functions, the test script will remind you (once for each test) that you did not implement the functions with the names required.

Terminal

ATTENTION! The following functions are not defined:

```
translate_codon
split_codons
translate_orf
```

These functions are either not correctly named (spelled) or not defined at all. They will be marked as FAILED. Check your spelling if this is not what you intend.

Ran 16 tests in 0.000s

OK (skipped=14)

If you have implemented a function but misspelled its name, you will also get this type of reminder. The reminders are meant to ensure that you do not hand in the assignment with missing or misspelled function definitions.

Case 2: If a test for one of the functions you have written fails, the testing is aborted, and the script prints some information to help you understand what the problem could be. Say you wrote the function `translate_codon` wrongly so that it always returns `M` for some reason:

```
def translate_codon(codon):
    return 'M' # crazy
```

then you would get this message:

Terminal

```
FAILED TEST CASE: test_translate_codon_2
```

```
MESSAGE:
```

```
    The call:
```

```
        translate_codon('TAA')
```

```
    returned:
```

```
        'M'
```

```
    However, it should return:
```

```
        'x'
```

```
=====
Ran 4 tests in 0.001s
```

```
FAILED (failures=1)
```

It is now left to you to figure out why your function returns the wrong value when called with these arguments.

Case 3: If you defined all functions correctly and they all work the way they are supposed to, then the test script just prints:

```
Ran 14 tests in 0.140s
```

```
OK
```

23 Turtle, week 4: loops, and the first tests

This week brought the for-loop, `range`, loops inside loops, and the first proper discussion of testing. The turtle is a good place to practise all four, for one reason above all others: when a loop goes wrong the picture tells you which loop went wrong. A drawing that comes out as a fan instead of a square, or as one row instead of a grid, is a diagnosis and not merely a complaint.

That same quality is a trap, and this session sets the trap deliberately in its second half. A picture is a fast and truthful report about the run you just made. It is not a report about any other run, and it says nothing at all about the parts of the turtle that the picture does not show. So after you have used loops to draw a few things, you will start writing down what you expected in a form Python can check, in exactly the style you met in the testing note this week, and you will find that the turtle can answer questions about itself that the canvas never showed you.

Remember the practical rule from last week. One drawing, one cell, including the line that makes the turtle.

The same thing, several times

Exercise 23-1

SOLO

Here is a square written out the long way. Run it first so you know what you are aiming at, then rewrite it using a for-loop and `range` so that the pair of commands appears only once. Predict, before running your version, whether the picture will be identical.

```
from turtle_widget import Turtle

t = Turtle()
t.forward(120)
t.left(90)
t.forward(120)
t.left(90)
t.forward(120)
t.left(90)
t.forward(120)
t.left(90)
```

If you wrote `for i in range(4):` and never used `i` inside the loop, use the underscore instead, as in `for _ in range(4):`. The underscore is an ordinary name and Python does not treat it specially. It is a message to whoever reads

the code that this loop repeats a fixed number of times and does not care which repetition it is on.

Exercise 23-2

SOLO

Turn your square into a polygon with any number of sides. Decide first what the turn has to be for the shape to close, then write it. Test it with three, five and eight before you read on.

```
from turtle_widget import Turtle

sides = 5
size = 90

t = Turtle()
for _ in range(sides):
    t.forward(size)
    t.left(360 / sides)
```

A closed shape is one where the turns add up to a full circle, which is why the turn is `360 / sides` and not something you have to look up. Try `sides = 3`, then `sides = 12`, then `sides = 60`, and notice what a polygon with enough sides starts to look like.

Exercise 23-3

SOLO

Now let the loop variable do some work. Predict the shape of the result before running.

```
from turtle_widget import Turtle

t = Turtle()
for step in range(10, 200, 20):
    t.forward(step)
    t.left(90)
```

Then change the third number in `range` and predict again before each run. The three numbers are start, stop and step, and the value at `stop` is never used, which is the single most common source of an off-by-one drawing.

Exercise 23-4

SOLO

A loop can walk a list, and it can walk a string, because both hand out one item at a time. Decide how many lines will be drawn, then run it.

```
from turtle_widget import Turtle

angles = [60, 120, 60, 120, 60, 120]

t = Turtle()
for angle in angles:
    t.forward(70)
    t.left(angle)
```

Then replace the list with a strand of DNA and turn left for one base and right for another, so that `for base in dna:` draws a path whose shape is decided by the sequence. Choose a strand of at least twelve bases and look at what comes out.

Loops inside loops

A loop inside a loop is where most beginners lose their footing, and it is where the turtle earns its place in this course. The inner loop finishes all of its repetitions every single time the outer loop goes round once. That sentence is the whole thing, and the next two exercises are designed to make you believe it rather than merely accept it.

Exercise 23-5

SOLO

Before you run this, say out loud how many squares will appear and how many times `t.forward(40)` will be called in total.

```
from turtle_widget import Turtle

t = Turtle()
t.speed(10)
for _ in range(5):
    for _ in range(4):
        t.forward(40)
        t.left(90)
    t.penup()
    t.forward(60)
    t.pendown()
```

Notice the three lines after the inner loop. They belong to the outer loop and not to the inner one, and the only thing that says so is the indentation. Move them one level further in, predict what will happen, and then run it to see. This is Oath 2 doing its work: the lines run top to bottom, and the indentation decides which lines are inside which repetition.

Exercise 23-6

SOLO

Here is a grid of dots. Predict the number of dots, then swap the two `range` calls so that the outer loop counts columns and the inner one counts rows, and predict what changes about the picture before running it again.

```
from turtle_widget import Turtle

t = Turtle()
t.speed(0)
t.penup()
for row in range(4):
    for col in range(6):
        t.goto(col * 40 - 100, row * 40 - 60)
        t.dot(10)
```

One of the two versions draws the grid row by row and the other draws it column by column, and the finished pictures are identical. That is not a small observation, and it is the first time in this course that two visibly different programs have left the same evidence behind. Hold on to it. Week eight is built on it.

Exercise 23-7

AI: EXPLAINER

Paste the nested loop from the exercise before last into the assistant and ask it two things. Ask how many times in total the body of the inner loop runs, and ask which of the two loops is responsible for the gap between the squares. Then confirm both answers with the machine rather than taking its word for either. For the first, count the calls yourself by adding a counter variable that starts at zero and goes up by one inside the inner loop, and print it after the loops have finished. For the second, change the number in the outer loop's `t.forward(60)` and look.

Checking with the machine instead of your eyes

In the testing note this week you wrote claims about a function in a form Python can check, by writing the call and the expected answer with `==` between them and printing the result. Every one of those lines prints `True` when the claim holds and `False` when it does not, and a column of `True` is a much better piece of evidence than a feeling that the output looked right.

The turtle can be checked in exactly the same way, because it will answer questions about itself. This is where the questions you met last week stop being curiosities.

Exercise 23-8

SOLO

Write down, before you run this, what each of the two claims should print if the square is really a square.

```
from turtle_widget import Turtle

t = Turtle()
for _ in range(4):
    t.forward(120)
    t.left(90)

print(t.heading() == 0)
print(t.position() == (0.0, 0.0))
```

A shape that closes properly brings the turtle back to where it started, facing the way it started. Those are two claims about the drawing that the drawing itself never displayed, because the canvas shows you four lines and says nothing whatever about which way the turtle ended up pointing.

The second claim almost certainly printed `False`, and the picture is a perfectly good square. Nothing is wrong with the drawing and nothing is wrong with your loop. Print `t.position()` on its own and look at the numbers. What is wrong is the claim, because working out where a turn of ninety degrees points requires a fraction the machine cannot store exactly, so the turtle lands a vanishingly small distance from where it started. Every language that stores fractions this way has this property, and it is the reason that comparing two calculated numbers with `==` is almost always the wrong test.

Exercise 23-9

SOLO

Since exact equality is not available, you have to decide how close counts as the same place, and `round` is the blunt way to say what you decided. Predict what all six of these print before running.

```
from turtle_widget import Turtle

t = Turtle()
for _ in range(7):
    t.forward(90)
    t.left(360 / 7)

print(t.position() == (0.0, 0.0))
```



```
print(round(t.xcor(), 6) == 0.0)
print(round(t.ycor(), 6) == 0.0)
print(round(t.heading(), 6) == 0.0)
print(round(t.xcor(), 12) == 0.0)
print(round(t.xcor()) == 0)
```

Notice that the answer depends on a number you chose, and that nobody can choose it for you, because how close is close enough is a question about what you are doing and not about arithmetic. Six decimal places is a reasonable default for turtle work in this course and you should be able to say why you picked it if asked. Deciding what counts as equal is part of writing the test rather than a detail you can skip, and this is the first time in this course that the decision has been yours.

Exercise 23-10

SOLO

The turtle keeps a running total of how far it has travelled, in `t.total_movement`, and it counts your turns in `t.nr_left` and `t.nr_right`. None of these three is a method, so you write them without brackets. Predict all three numbers for the square below, and write your prediction as three checks that should all print `True`.

```
from turtle_widget import Turtle

t = Turtle()
for _ in range(4):
    t.forward(120)
    t.left(90)

print(t.total_movement == 480)
print(t.nr_left == 4)
print(t.nr_right == 0)
```

Then break the loop on purpose. Change the four in `range` to three and add one more `t.forward(120)` on its own line after the loop, so that there are four straight sides and only three turns.

```
from turtle_widget import Turtle

t = Turtle()
for _ in range(3):
    t.forward(120)
    t.left(90)
t.forward(120)

print(t.total_movement == 480)
print(t.nr_left == 4)
print(t.heading() == 0)
```

Look at the picture first, and decide whether you would have spotted anything. You will not have, because the picture is a perfect square in the same place as before, drawn in the same order, ending in the same corner. The turtle travelled the same total distance, so the first check still passes. What changed is that the turtle is left facing a quarter turn away from where it started, and the two checks that ask about that are the only reports you have of it. Nothing you could have done by looking harder would have told you, because the fact was never in the image. Remember that this happened. It is the whole subject of week eight.

Week 5

24 Practical AI use

A short, tool-light guide to using the assistant in the browser for this course. Shared by weeks four and five.

You now know what the assistant is doing under the hood, and you have practiced predicting, explaining, and confirming with it a few times. This note is smaller than those and more useful in the moment: some ordinary habits for actually typing at the thing, so that the answers you get back are worth the checking you are about to give them. None of this is about which button to click. Buttons change every few months; the habits below do not.

Say what you want, not what you are thinking about. “Something about DNA sequences” is not a request, it is a mood. “A function that takes a DNA string and returns the count of each base as a dictionary” is a request. The assistant fills gaps you leave with a plausible guess, not a clarifying question, so a vague prompt does not get you a cautious answer, it gets you a confident answer to a question you did not quite ask. The fix costs you nothing but a few extra words: name the input, name the output, and if there is an edge case you already care about, such as lowercase letters or an empty string, say so up front.

Give it the context it actually needs, and no more. If you want help with one function inside a larger project, paste that function and the one or two things it depends on, not the whole file. Too little context and the assistant guesses at conventions you already decided; too much and it starts guessing at which of forty things you actually meant, which is its own kind of vagueness. A good rule of thumb: give it what a classmate would need to help you, no more, no less.

Read the reply before you trust the tone. An assistant’s reply is fluent and confident whether it is right or wrong, so confidence is not evidence of anything and you should stop letting it feel like evidence. Read a reply the way you would read a claim from a classmate who is occasionally brilliant and occasionally makes things up with total sincerity: interesting, worth considering, not yet worth acting on.

Ask for the reasoning, but do not mistake it for proof. You can ask the assistant to explain a line, walk through an example, or justify a choice, and doing so often surfaces a mistake that a bare answer would have hidden. But remember what an explanation is: another plausible-sounding piece of text, generated the same way the original answer was. It can make a wrong answer sound more convincing just as easily as it can reveal that the answer was wrong. Treat an explanation as a second thing to check, never as the checking itself.

Tell helpful apart from merely fluent. A helpful answer is one you can act on because you have verified it, by running it, testing it, or checking it against something you already know. A merely fluent answer is one that reads well and that you have not yet checked. The two are indistinguishable by feel, which is the entire point of this course, and distinguishable only by the running, testing, or checking you actually do.

The ladder so far

Explainer · Translator ·
Illustrator

Still to come: Comparer,
Drafter, Worker,
Collaborator, Developer

Exercise 24-1

AI: EXPLAINER

Ask the assistant the same underspecified question two different ways: first as something thoroughly vague, such as “write me something to look at a DNA sequence,” and then as a specific request naming the input, the output, and one edge case you care about. Compare the two replies. Which one would you have had to guess at least, and which one can you actually check against what you asked for?

For your logbook this week, write the single worst prompt you have given the assistant so far in this course, in your own memory or your prompt history, and rewrite it as it should have been.

25 Data structures

This section will further train your ability to create such data structures using iteration. Data structures in Python are fundamental tools that allow you to organize, store, and manipulate data effectively. They provide a way to represent and manage data in a structured and organized manner, making it easier to perform various operations on the data. Python offers a variety of built-in data structures, such as lists and dictionaries, with different properties that are useful for different needs. Sometimes, a single list or dictionary is all you need. Still, sometimes you need to combine many lists and dictionaries to make an elaborate data structure. As your programming skills progress, you will find that mastering the use of different data structures is crucial for becoming a proficient Python programmer.

Exercise 25-1

SOLO

Imagine you want to count how many times each nucleotide appears in a DNA string like this: `'ATGCCGATTAA'`. One way to proceed with an account of this is by using a dictionary where the keys represent the different values we want to count (in this case `'A'`, `'T'`, `'C'` and `'G'`). The value associated with each key is the number of times we have seen that key (nucleotide). So, we want to end up with a dictionary like this one (not necessarily with key-value pairs in this order):

```
{ 'G': 2, 'C': 2, 'T': 3, 'A': 4 }
```

Remind yourself how you assign a value to an existing key in a dictionary. Here is some code to get you going:

```
dna = 'ATGCCGATTAA'
counts = { 'G': 0, 'C': 0, 'T': 0, 'A': 0 }
for base in dna:
    # Your code here...
```

Exercise 25-2

SOLO

In exercise 25-1 we started by initializing a dictionary with a key for each nucleotide and the number zero for each key to show that we had not seen any of those nucleotides yet. *Then* we iterated over the values we wanted to count using the for-loop. This approach of course only works if know which values we will encounter

in the iteration. When counting nucleotides, this works because we know there are only four different nucleotides.

Often, however, we are faced with one or both of the following problems:

- The number of different values to count is very large. If we were to count English words, we would have to initialize a dictionary with more than 170.000 key-value pairs (which would be somewhat impractical). If we were to count numbers, this would be impossible (as there are infinitely many of those).
- We do not know what values we are going to count. It goes without saying that we can only initialize a dictionary with keys if we know what they are.

We can solve this problem by only adding keys for the values we see in the iteration. To do this, we need to change our approach from before in two ways:

1. We start with an *empty* dictionary.
2. For every value we iterate over, we check if that value is a key in our dictionary. If it is not, then we need to add it and pair it with the value `0`. You can test if a key is not in a dictionary using the `not in` operator:

```
if number not in counts:
    counts[number] = 0
```

Now use these hints to complete the code below so that it counts how many times each number appears in `number_list`.

```
counts = {}
number_list = [13, 51, 3, 51, 6, 42, 3]
for number in number_list:
    # Your code here...
```

When you are done you should have a dictionary like this (possibly with key-value pairs in a different order):

```
{3: 2, 42: 1, 51: 2, 13: 1, 6: 1}
```

Exercise 25-3

SOLO

The counting technique you developed in exercise 25-2 lets you count pretty much anything that can be a key in a dictionary. Try using the same approach to count the number of each thing in this list:

```
stuff = ['sofa', 42, 42, 3.14159, 'sofa', 'Dragon']
```

Exercise 25-4

SOLO

Instead of counting values, we sometimes need to split the values we iterate over into categories. Here, we build a dictionary where the keys are the first letter of each word we iterate over, and the value associated with each key is a list of all the words that start with that letter. So, if we iterate over the words in this list:

```
names = ['apricot', 'banana', 'anas', 'apple', 'cherry']
```

We should end up with this dictionary (not necessarily with key-value pairs in that order):

```
{ 'c': ['cherry'], 'a': ['apricot', 'anas', 'apple'], 'b': ['banana'] }
```

Now, figure out how to reorder and indent the statements below to produce code that performs this task:

```
names = ['apricot', 'banana', 'anas', 'apple', 'cherry']
words[first_letter].append(fruit)
first_letter = fruit[0]
for fruit in names:
words = { 'a': [], 'b': [], 'c': [] }
```

Exercise 25-5

SOLO

Produce the same dictionary as in exercise 25-4 by reordering and indenting the following statements:

```
first_letter = fruit[0]
words = {}
words[first_letter].append(fruit)
if first_letter not in words:
names = ['apricot', 'banana', 'anas', 'apple', 'cherry']
words[first_letter] = []
for fruit in names:
```

Compare the solution to that in exercise 25-4. How is it different? Is it more generic? How does the difference relate to the difference between exercise 25-1 and exercise 25-2?

Exercise 25-6

SOLO

Look at this code and decide for yourself what will be printed. Do we greet one person in three languages first, or do we greet all people in one language first? Then, try it out to see if you were right.

```
greetings = ['Hi', 'Hola', 'Ciao']
names = ['Mogens', 'Preben', 'Henning']
for greeting in greetings:
    for name in names:
        print(greeting, name)
```

Exercise 25-7

SOLO

Now consider the code below. How is it different from the code in exercise 25-6? How does that alter the order of what is printed?

```
greetings = ['Hi', 'Hola', 'Ciao']
names = ['Mogens', 'Preben', 'Henning']
for name in names:
    for greeting in greetings:
        print(greeting, name)
```

Exercise 25-8

SOLO

Decide what you think the code below does and why you think so. Do *every* step in your head, including all the substitutions and reductions. Then, write the code carefully and run it.

```
nr_list = [10, 20, 30]
combinations = []
for a in nr_list:
    for b in nr_list:
        pair = [a, b]
        combinations.append(pair)
```

The `combinations` list becomes:

```
[[10, 10], [10, 20], [10, 30],
 [20, 10], [20, 20], [20, 30],
 [30, 10], [30, 20], [30, 30]]
```

Here, I broke over three lines to make it fit on the page. You should print your own `combinations` list to ensure you got the code right.

Exercise 25-9

SOLO

The code in exercise 25-8 printed all combinations of numbers in the list – including those where the two numbers are the same. Change the code above so these pairs are *not* printed. You should end up with this list:

```
[[10, 20], [10, 30], [20, 10], [20, 30], [30, 10], [30, 20]]
```

Exercise 25-10

SOLO

Can you solve the same task as in exercise 25-9, by modifying the code below? You can begin by understanding why it produces the same list as that in exercise 25-8. If you need help with that, then have another look at exercise 21-7.

```
nr_list = [10, 20, 30]
combinations = []
for i in range(len(nr_list)):
    for j in range(len(nr_list)):
        pair = [nr_list[i], nr_list[j]]
        combinations.append(pair)
```

Exercise 25-11

SOLO

The code in the exercise above printed all combinations of *different* numbers in the list. But you can see that each pair of numbers still appears twice if you do not take their order into account (e.g., [10, 30] are the same two numbers as [30, 10]). Change the code you wrote for the previous exercise so these pairs are not printed. You should end up with a list like this:

```
[[10, 20], [10, 30], [20, 30]]
```

Hint: the easiest way to do it is to use the value of `i` to change the range of numbers you iterate over in the second for-loop.

Exercise 25-12

SOLO

Sometimes programmers (like you) work with matrices of numbers like the one below:

```
[[0, 1, 2, 3, 4],  
 [0, 1, 2, 3, 4],  
 [0, 1, 2, 3, 4],  
 [0, 1, 2, 3, 4],  
 [0, 1, 2, 3, 4]]
```

Here, I nicely wrote the list so you can see that a matrix is just *a list of lists*. When you print it looks like this:

```
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]
```

Can you write some code that produces this matrix? If you let out a sigh just now, then reread the sections on lists and for-loops. You may think you absorbed all the information you could when you read it the first time, but with your practice since then, you may be able to understand it at a deeper level the second or third time you read it.

The code below produces the matrix above. There are several tricky parts that you need to make sure you understand. In line three, we add an empty list to the list of lists. In line five, we add the value of `j` to the list at index 'i' in the list. Go over this many times in your head and with pen and paper.

```
matrix = []  
for i in range(5):  
    matrix.append([])  
    for j in range(5):  
        matrix[i].append(j)
```

Exercise 25-13

SOLO

If you understand how you created the matrix in exercise 25-12, you should be able to produce the matrix below using a small modification to the code from exercise 25-12.

```
[[1, 1, 1, 1, 1],  
 [2, 2, 2, 2, 2],  
 [3, 3, 3, 3, 3],  
 [4, 4, 4, 4, 4],  
 [5, 5, 5, 5, 5]]
```

Exercise 25-14

SOLO

Now produce this matrix:

```
[[0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0]]
```

Exercise 25-15

SOLO

Can you write some code that produces this matrix?:

```
[[ 0, -1, -2, -3, -4],
 [-1,  0,  0,  0,  0],
 [-2,  0,  0,  0,  0],
 [-3,  0,  0,  0,  0],
 [-4,  0,  0,  0,  0]]
```

General exercises

“ By now, you have learned a lot, and the general exercises, which serve to keep it all current, get more complicated. But remember: even though the code may mix lists, for-loops, and functions, the *rules* for lists, for-loops, and functions are *not* mixed. The separate simple rules for a list, a for-loop, and a function are *still* the same. If you get confused, it is time to revisit the sections about each separate topic. You may have to do that many times during the course. ”

Exercise 25-16

SOLO

Write a function, `square_numbers`, that takes a list of numbers as argument and returns a new list with the numbers squared.

```
# write your function definition here ...

numbers = [1, 5, 3, 7]

# then you can call it like this:
squared = square_numbers(numbers)
```

Exercise 25-17

SOLO

Write a function `count_characters`, which takes a string argument and returns a dictionary with the counts of each character in the string. When you call the function like this:

```
count_characters('banana')
```

it must return (not necessarily with key-value pairs in that order):

```
{ 'n': 2, 'b': 1, 'a': 3 }
```

The technique you should use is the one you learned in exercise 25-2. Here, we iterate over a string of characters instead of a list of numbers. Here is a bit of code to help you along.

```
def count_characters(text):  
    counts = {}  
    # fill in the missing code ...  
  
    return counts
```

Exercise 25-18

SOLO

Use the function you made in the previous exercise to construct the following data structure below from this list: `['banana', 'ananas', 'apple']`.

```
{ 'banana': { 'b': 1, 'a': 3, 'n': 2 },  
  'apple': { 'a': 1, 'e': 1, 'p': 2, 'l': 1 },  
  'ananas': { 'a': 3, 's': 1, 'n': 2 } }
```

Here is some code to help you along:

```
my_database = {}  
for word in ['banana', 'ananas', 'apple']:  
    my_database[word] = # you figure this out...
```

Once you are done, what value do you think `my_database['banana']` represents? I.e., what will it be reduced to if used in an expression? And what value does `my_database['banana']['a']` represent?

Exercise 25-19

SOLO

Read the code below and make sure you understand each step before you write any of it. If necessary, revisit previous sections and look in the Python documentation. Then, write and run the code—and enjoy that it was exactly what you expected.

```
def get_words(text, search_string):
    hits = []
    for word in text.split():
        if search_string in word:
            hits.append(word)
    return hits

s = 'eenie meenie minie moe'
nie_words = get_words(s, 'nie')
m_words = get_words(s, 'm')

print(' '.join(nie_words))
print(' '.join(m_words))
```

Exercise 25-20

SOLO

This larger will take you through some of the most common string manipulations. A palindrome is a string that is spelled the same way, backward and forward.

Write a function, `is_palindrome`, which takes one argument:

- A string.

The function must return:

- `True` if the string argument is a palindrome and `False` otherwise.

Example usage:

```
is_palindrome('abcba')
```

should return `True` and

```
is_palindrome('foo')
```

should return `False`

One approach to this is to run through `s` from the first to the middle character, and for each character, check if the character is equal to the character at the same index from the right rather than the left. Remember that the first character of a string is at index `0` and the last at index `-1`, the second character is at index `1` and the second last at index `-2` and so forth.

Since you need to run through the string from the first to the middle character, you must first figure out how many characters that corresponds to. Say your palindrome is "ACTGTCA", then the number of indexes you need to loop over with a for loop is:

```
s = "ACTGTCA"
nr_indexes = len(s)//2
```

Figure out how to make `range()` return indexes to access the characters in the first half of the sequence. Then make a for loop where you iterate over the indexes you get from `range()`. Try to make the for-loop print out the first half of the characters, just to make sure you are using the right indexes.

Once you get this far, you must compare each character from the first half of the corresponding ones, starting from the other end of the palindrome. Figure out how to change each index used for the first half to the corresponding index for the other half so you can compare the relevant pairs. (You need to compare index 0 with -1, 1 with -2 and so on...)

Now, try to make the for loop print both the character from the first half and the corresponding character from the other end. If you get the indexes right, you will see that the A prints with the A from the other end, the C with the C, and so on.

Write an if-statement in the for-loop that tests whether the two corresponding characters are identical. If the string is a palindrome, then each pair is identical. So, as soon as you see a pair that is not identical, you know it is not a palindrome, and you can let your function return `False` like this:

```
if left_character != right_character:
    return False
```

Remember that the function ends when it encounters a `return` statement.

If all pairs pass the test, the string is a palindrome, and the function should return `True` when exiting the for loop.

26 Files

This chapter covers the bare necessities of making your program read data from a file on your computer and how to create a file to write results.

Writing files

To interact with a file on your hard disk, you need to know its name and whether you want to write to it or read from it. Then, you can use the built-in function `open` to create a file object that lets you read or write to that file. The `open` function takes two arguments: The first is a string, which gives the file's name. The second argument is also a string and should be `'w'` for “write” if you want to write to the file or `'r'` for “read” if you are going to read from the file. To keep things simple, we will assume that the file you want to open is always in the same folder (directory) as the Python script that calls the `open` function.

Exercise 26-1

SOLO

Try to write the code below and run it:

```
f = open('workfile.txt', 'w')
f.write("First line\n")
f.write("Second line\n")
f.close()
```

Now open the `workfile.txt` in *VScode* and see what is in it now. It should contain:

```
First line
Second line
```

Let's break down what happened:

1. You used the `open` built-in function to open a file called “workfile.txt” in *writing* mode using the `'w'` as the second argument.
2. You then wrote the string `"First line\n"` to the file using the `write` method of the file object.
3. You wrote another string `"Second line\n"` to the file using the `write` method of the file object.
4. You closed the file using the `close` method of the file object.

Note that if you open a file for writing, a file with that name is created. If a file of that name already exists, it is overwritten.

Exercise 26-2

SOLO

Close `workfile.txt` in *VScode* again and change your program above to this (removing the `\n` characters):

```
f = open('workfile.txt', 'w')
f.write("First line")
f.write("Second line")
f.close()
```

What do you think the content of `workfile.txt` is now? Decide before you open `workfile.txt` in *VScode* again and have a look. What do you think the `\n` character represents?

Exercise 26-3

SOLO

Close `workfile.txt` in *VScode* and change your program above to this:

```
f = open('workfile.txt', 'w')
f.write("First line\nSecond line\n")
f.close()
```

Can you see how that is equivalent to what you did before? Open `workfile.txt` in *VScode* again and have a look.

Exercise 26-4

SOLO

You can also make `print` write to a file instead of the terminal. That way, your output will end up in the file instead of the terminal. To make `print` write to a file, you need to use `print`'s `file` keyword argument to tell `print` which file to write to. The argument must be an object that represents the file you want to write to (`file=f` below). Try to write the code below and run it:

```
f = open('workfile.txt', 'w')
print("First line", file=f)
print("Second line", file=f)
f.close()
```

Compare the code to that in exercise 26-1. Notice how the strings we print end with a newline character `\n`. This is because the default behavior for `print` is to add a new line to the end of what it prints—just like when you print to the terminal.

Reading files

When you want to read from an existing file, you give the `open` function the name of that file and specify `'r'` for reading as a second argument. If the file you name does not exist, Python will tell you it does not exist (it is nice like that). Before you head into the rest of this section, make sure you redo exercise 26-1 so the content of `workfile.txt` is:

```
First line
Second line
```

Exercise 26-5

SOLO

```
f = open('workfile.txt', 'r')
file_content = f.read()
print(file_content)
f.close()
```

Let's break down what happened:

1. You used the `open` built-in function to open a file called `workfile.txt` in *reading* mode using the `'r'` as the second argument.
2. You then read the file's content using the `read` method, which returns the contents as a string.
3. You printed the string.
4. You closed the file using the `close` method of the file object.

Exercise 26-6

AI: EXPLAINER

Try to read from the file after you close it:

```
f = open('workfile.txt', 'r')
f.close()
file_content = f.read()
```

Do you get an error? Which one? Do you understand why?

Ask the assistant what a closed file is, why Python cannot read from one, and what the error will be called, all before you run the code. Then run it. If it named the error correctly, ask one more question: does closing a file twice also raise an error? Predict, then check.

Exercise 26-7

SOLO

You can use the `readline` method to read one line at a time. What do you think happens if you run this code:

```
f = open('workfile.txt', 'r')
line = f.readline()
print(line)
line = f.readline()
print(line)
line = f.readline()
print(line)
```

Once you decide, try it out. What is printed in the last print statement? The thing is, the file object keeps track of how much of the file it has read. Once it ends, you can read as much as you like- nothing is left. If you want to start reading from the top of the file, you can close it and open it again. Try to insert the following two statements at various places in the code above and see what happens.

```
f.close()
f = open('workfile.txt', 'r')
```

Exercise 26-8

SOLO

Look at the code below and figure out for yourself what it does:

```
input_file = open('workfile.txt', 'r')
output_file = open('results.txt', 'w')

for line in input_file:
    line = line.upper()
    output_file.write(line)
```

Then, run it and open `results.txt` in *VScode* and see what it produced.

Were you surprised that the file object can be an iterator in a for-loop? Just like strings can iterate over characters, lists can iterate over values, dictionaries can iterate over keys, and file objects can iterate over the lines in the file.

Try to modify your code to use the `print` function instead of the `write` method (see exercise 26-4).

General exercises

Exercise 26-9

SOLO

Write a function `read_file` that takes the name of a file as an argument. The function should read the content and return it. Like:

```
content_of_file = read_file('some_file.txt')
```

Exercise 26-10

SOLO

Write a function that takes the name of two files as arguments. The file should read the content of the first file and write it to the second file.

```
copy_file('some_file.txt', 'other_file.txt')
```

27 Turtle, week 5: drawings described by data

This week you built things out of dictionaries and lists of lists, counted occurrences into a dictionary, put values into buckets, walked a matrix with two loops, and learned to read from and write to files. All of that is about the same move, which is getting the information out of the code and into a data structure, so that the code says how to do something and the data says what to do it to.

The turtle makes that move visible in an unusually satisfying way, because once a drawing is described by data you can change the picture without touching a single command. You will write a drawing as a list of steps, then as a dictionary from letters to turns, then as a matrix, and by the end of the session you will save a drawing to a file, hand the file to somebody else, and have them redraw it. What is stored in that file is not a picture. It is a description that any turtle can carry out, which is the same relationship a recipe has to a meal.

You will also meet counting again, and this time you can check your count against a count the turtle kept independently, which is a small foretaste of the habit of verifying one thing against another that the rest of the course is built on.

A drawing as a list of steps

Exercise 27-1

SOLO

A step in a path is naturally two numbers, a distance and a turn, so a path is naturally a list of tuples. Predict the shape this draws before you run it.

```
from turtle_widget import Turtle

path = [(100, 90), (60, 90), (100, 90), (60, 90)]

t = Turtle()
for distance, angle in path:
    t.forward(distance)
    t.left(angle)
```

The unpacking happens in the `for` line itself. Each time round the loop, one tuple comes out of the list and its two items go into the two names, which is the same unpacking you did last week with `x, y = t.position()`, done automatically once per repetition.

Now change the data and not the code. Make the path draw a triangle, then a staircase of five steps. If you find yourself editing anything below the `for` line, stop and put the change in the list instead.

A drawing as a dictionary of instructions

A path written as a list of tuples is still fairly close to the turtle. You can go further and invent a tiny language of your own, where a drawing is just a string of letters and a dictionary says what each letter means.

Exercise 27-2

SOLO

Read the dictionary and the string, work out the shape on paper, and then run it.

```
from turtle_widget import Turtle

turn = {'F': 0, 'L': 90, 'R': -90}
program = 'FFLFFLFFLFF'

t = Turtle()
for letter in program:
    t.forward(30)
    t.left(turn[letter])
```

Notice that a turn of `-90` to the left is a turn of ninety to the right, so you did not need a second command. Now extend the language. Add a letter that lifts the pen and a letter that puts it down, and change the loop so that those two letters do not also move the turtle forward. You will need an `if` to do it, and the interesting design question is whether the dictionary is still the right place to keep the meaning of a letter once some letters mean something other than a turn. Answer that question in a sentence before you write the code.

Exercise 27-3

SOLO

Count the letters of the program into a dictionary, in the way you counted this week, and then check your count against a count the turtle kept for itself.

```
from turtle_widget import Turtle

turn = {'F': 0, 'L': 90, 'R': -90}
program = 'FFLFFRFFLFFRFF'

counts = {}
for letter in program:
```

```

    if letter not in counts:
        counts[letter] = 0
    counts[letter] = counts[letter] + 1
print(counts)

t = Turtle()
for letter in program:
    t.forward(30)
    t.left(turn[letter])

print(t.nr_left)

```

Predict, before running, what `t.nr_left` will be, and be careful, because the answer is not the number of `L` letters in the program. The turtle counts every call to `left`, and the loop calls `left` on every letter, including the ones that turn by zero degrees. Two counts of two different things came out of the same run, and neither is wrong. What would have been wrong is assuming that a number called `nr_left` counts the thing you happened to be thinking about.

A drawing as a matrix

Exercise 27-4

SOLO

A list of lists of zeros and ones is a picture in the most literal sense. Predict what appears, then run it.

```

from turtle_widget import Turtle

grid = [
    [0, 1, 1, 1, 0],
    [1, 0, 0, 0, 1],
    [1, 0, 0, 0, 1],
    [1, 0, 0, 0, 1],
    [0, 1, 1, 1, 0],
]

t = Turtle()
t.speed(0)
t.penup()
for row in range(len(grid)):
    for col in range(len(grid[row])):
        if grid[row][col] == 1:
            t.goto(col * 30 - 60, 60 - row * 30)
            t.dot(20)

```

Take the expression `grid[row][col]` apart before you decide it is obvious. The first pair of brackets reduces `grid[row]` to one of the five inner lists, and the

second pair then takes one item out of that list. The row index comes first because a list of lists is a list of rows, and that ordering is a decision somebody made, not a law.

Then look at the arithmetic in the `goto` line and work out why the row uses `60 - row * 30` while the column uses `col * 30 - 60`. Draw the grid with `row * 30 - 60` instead and see what happens to your picture. On a turtle canvas `y` grows upwards, and in a matrix rows grow downwards, so somewhere the two conventions have to be reconciled, and that line is where.

Exercise 27-5

SOLO

Count the bases in a strand into a dictionary and draw the result as a bar chart. Write it yourself, using the counting pattern from the exercise above and the drawing pattern from the matrix exercise, and predict the relative heights of the four bars from the strand before you look at the picture.

```
from turtle_widget import Turtle

dna = 'ATGGCGCTAAGCTTAGCGCGATTAACGGCTA'

counts = {}
for base in dna:
    if base not in counts:
        counts[base] = 0
    counts[base] = counts[base] + 1
print(counts)

t = Turtle()
t.speed(0)
t.pensize(20)
x = -120
for base in 'ACGT':
    t.penup()
    t.goto(x, -100)
    t.setheading(90)
    t.pendown()
    t.forward(counts[base] * 8)
    x = x + 60
```

Notice that the loop walks `'ACGT'` and not `counts`, so the bars come out in an order you chose rather than an order the dictionary happened to give you. If the strand contained no `C` at all this code would fail, and it is worth deciding now whether the right answer is a bar of height zero or an error. Write the version that does whichever you decided.

Saving a drawing and getting it back

A file is how a drawing survives the end of the cell. Everything in this section uses the `with` form of `open`, which is the form you should use from now on, because it closes the file for you as soon as the indented block ends, including when something goes wrong inside it.

Exercise 27-6

SOLO

Write your little language to a file, then look at what you wrote.

```
program = 'FFLFFRFFLFFRFF'

with open('drawing.txt', 'w') as f:
    f.write(program)

with open('drawing.txt') as f:
    print(f.read())
```

The `'w'` means writing, and it empties the file first, so running this twice does not give you the program twice. Opening without a second argument means reading. Change the string, run the cell again, and check that the file changed.

Exercise 27-7

SOLO

Now read the file in one cell and draw it, with no program string written anywhere in the drawing code.

```
from turtle_widget import Turtle

turn = {'F': 0, 'L': 90, 'R': -90}

with open('drawing.txt') as f:
    program = f.read()

t = Turtle()
for letter in program:
    t.forward(30)
    t.left(turn[letter])
```

If your file ends with a newline this will fail, because a newline is a letter as far as the loop is concerned and it is not a key in the dictionary. Read the error, then fix it with `strip`, which removes whitespace from both ends of a string. The lesson is worth more than the fix: what comes out of a file is never quite what you thought you put in, and a program that reads files has to say what it does about that.

Exercise 27-8

SOLO

A file with one number per line is the ordinary way to store a column of data, and iterating over an open file hands you one line at a time. Predict how many bars appear before you run it.

```
from turtle_widget import Turtle

with open('heights.txt', 'w') as f:
    for height in [40, 90, 60, 130, 75]:
        print(height, file=f)

t = Turtle()
t.speed(0)
t.pensize(20)
x = -120
with open('heights.txt') as f:
    for line in f:
        t.penup()
        t.goto(x, -100)
        t.setheading(90)
        t.pendown()
        t.forward(int(line))
        x = x + 60
```

The call `int(line)` is doing two jobs that are easy to miss. It turns text into a number, which you knew, and it quietly tolerates the newline on the end of the line, which you did not. Try `float` instead and it behaves the same way. Now put a blank line in the middle of the file by hand and run it again, and read the error you get, because that is the error you will meet every time you are handed a data file by somebody else.

Exercise 27-9

AI: EXPLAINER

Ask the assistant to explain what `strip` does, and specifically whether `strip` and `int` are doing the same job when they meet a line that ends in a newline. Then confirm the answer with the machine rather than accepting it, by writing four lines that print the length of a line before and after stripping and the value that `int` gives for the unstripped version. If the assistant's explanation and your four lines disagree, the four lines win, and it is worth working out where the explanation went wrong before you move on.

Exercise 27-10

SOLO

Finish by making the turtle record its own journey. Keep a list of the turtle's position after every step, write those positions to a file, then read them back in a second cell and replay the path with `goto`.

```
from turtle_widget import Turtle

t = Turtle()
t.speed(0)
positions = []
for step in range(12):
    t.forward(20 + step * 8)
    t.left(75)
    positions.append(t.position())

with open('path.txt', 'w') as f:
    for x, y in positions:
        print(x, y, file=f)
```

Then write the cell that reads `path.txt` and draws the same path on a fresh turtle. You will have to split each line into two pieces and turn each piece into a number, and `split` is how you do it. When it works, compare the two pictures. They should be the same path, drawn by two programs that have almost nothing in common, one of which never knew there was a loop, an angle or a distance involved. What travelled between them was the data.

Week 6

28 Ask for another way

The assistant as Comparer: judging between alternatives is the student's job.

Session length

Estimated 3 hours, run as this week's TA session, alongside this week's translation project.

There is rarely one correct way to write a function, and an assistant asked for a second solution will usually produce one, cheerfully, whether or not it is actually better. The skill this session builds is not getting a second opinion; it is judging between two things that both run, both look plausible, and may both even pass your tests. That judgment is yours to make, and it depends on understanding both versions well enough to say, specifically, why one is better than the other for a stated reason, not a feeling.

A rubric for judging two solutions

- **Correctness on edge cases.** Do they agree on an empty input, a one-element input, and the nastiest input you can think of? Test both, do not just read them.
- **Readability.** Could someone who has never seen this problem before understand what it does in under a minute?
- **Efficiency.** Does one clearly redo work the other avoids, for reasons you can name, not just guess at?
- **Idiom.** Does one use Python the way this course has taught you to read it, or does it lean on a trick you cannot yet explain?

The ladder so far

Explainer · Translator ·
Illustrator · **Comparer**

Still to come: Drafter,
Worker, Collaborator,
Developer

Part 1 — Warm-up, done together (20 min)

As a group, ask the assistant for two ways to reverse a DNA string: one using slicing, one using a loop. Read both aloud. Run both on `'ATGGCC'` and on `''`. Apply the rubric above out loud as a class before anyone works alone, so everyone has seen the process modeled once.

Part 2 — Two solutions, three problems (50 min)

Working alone or in pairs, for each problem below, ask the assistant for two different solutions in two separate requests (do not ask for “two ways” in one prompt, which tends to produce two trivial variations of the same idea rather than two properly different approaches). Test both against at least one gentle and one nasty input.

Apply the rubric and write one sentence naming your winner and the specific rubric point that decided it.

Exercise 1 28-1

AI: COMPARER

A function that counts how many times the codon `'ATG'` appears in a longer DNA string, **including overlapping occurrences**.

Exercise 2 28-2

AI: COMPARER

A function that checks whether a DNA string is a palindrome, meaning it reads the same as its own reverse complement.

Exercise 3 28-3

AI: COMPARER

A function that takes a list of DNA strings and returns the one with the highest GC content.

Part 3 — Clever versus readable (30 min)

Some second solutions are not just different, they are compressed: a one-liner using a comprehension or a chained method call, offered in place of a plainer loop.

Exercise 4 28-4

AI: COMPARER

Take your solution to Exercise 1 or 2. Explicitly ask the assistant for “a more Pythonic, more compact version” of it. Read the compact version line by line until you can explain every part of it out loud to a partner without looking at the original.

Exercise 5 28-5

SOLO

Decide, honestly, whether you would have understood the compact version if you had been handed it cold, with no plainer version alongside it to compare against. Write down the specific feature, a comprehension, a chained call, a slice trick, that made it harder, if anything did.

Part 4 — Defend your choice, in writing (30 min)

Exercise 6 28-6

AI: COMPARER

Pick whichever of Exercises 1 through 3 produced the closest call between its two solutions. Write a short paragraph, three to five sentences, defending your choice of winner to someone who has not seen either solution: name the specific rubric point that decided it, and name one weakness you can actually point to in the version you did **not** pick, so your defense acknowledges the trade-off rather than presenting one side of it.

Part 5 — Ask for a third way (25 min)

A comparer's job does not stop at picking a winner between two options; sometimes the right move is to ask for better.

Exercise 7 28-7

AI: COMPARER

Take the losing solution from any one of Exercises 1 through 3. Without showing the assistant the winner, tell it specifically what was weak about the loser, using your own rubric-based reasoning from earlier, and ask for a third version that fixes exactly that weakness. Test the third version the same way you tested the first two. Did naming the specific weakness produce a real improvement, or did the assistant just rephrase the same approach?

Wrap-up (10 min)

For your logbook this week, report which rubric point, correctness, readability, efficiency, or idiom, did the most work deciding your choices today, and describe one moment where your first instinct about which solution was “better” changed after you actually tested both on a nasty input.

29 Translating ORFs

This chapter is about translating DNA into protein. If bacteria can do it, so can you.

In this project you will write the code needed to translate an open reading frame (ORF) on a DNA sequence into the corresponding sequence of amino acids.

Project files

Begin by downloading the files you need for the project:

https://munch-group.org/bioinformatics/supplementary/project_files

The file `translationproject.py` is for your code. The file `test_translationproject.py` is the script you use to test the functions you write for this project (see Chapter 22).

In this project you will need a data structure that pairs each codon to the amino acid it encodes. This is an obvious use of a dictionary and at the top of `translationproject.py` I have defined such a dictionary you can use. Defining it outside the functions means that it is visible inside all your functions (unless you define *another* variable called `codon_map` inside a function). Defining variables globally to your program sometimes make sense if some value can be considered a *constant* in your program and is *never* changed.

“ It is normally very bad programming style to access variables outside functions in this way because it may have all kinds of unexpected side effects across function calls. So make it a rule for yourself that code *inside* a function should never to access variables *outside* the function. The reason we define `codon_map` globally in this project is to help you understand that when Python cannot find a variable inside a function, it looks outside the function to find it. In this project functions will find `codon_map` in this way. However, as I already said, you should *never* do this yourself. The chance that you make an unexpected mistake is overwhelming.

”

Translating a single codon

Write a function, `translate_codon` that takes one argument:

1. A string, which is a codon.

The function should return:

- A string of length one (one character). If the string argument is a valid codon then this letter should be an amino acid letter specified by the `codon_map` dictionary. Note that stop codons are represented by a star (`'*'`). If the string argument is *not* a valid codon, the function must return `'?'` .

Example usage:

```
translate_codon('ACG')
```

should return

```
'T'
```

Before you start coding you should always outline for yourself intuitively what you need to do to complete the task at hand. In this case want to translate, or map, between a three letter string, codon, and the corresponding one letter string for the amino acid that the codon corresponds to. Notice that the keys in the `codon_map` dictionary are in upper case, so you must make sure that the keys *you* use are also in upper case. You can translate codon into an upper case version of itself using the `upper()` method.

Try this out first:

```
codon = 'TTG'
amino_acid = codon_map[codon]
print(amino_acid)
```

Now write the function so it uses the string parameter as a key to look up the corresponding amino acid letter and returns this letter. Before you go on, make a function that does only that.

Before you are completely done you need to make your function handle the situation when the argument to the function is not a key in the `codon_map` dictionary. Use an if-else construct to handle the two cases. The boolean expression must test if the function argument is a key in `codon_map` . Remember that you can use the `in` operator to do this.

Splitting an open reading frame into codons

To translate an entire open reading frame into the corresponding amino acid sequence, you need to split the ORF sequence into codons. When we have done that we can translate each codon using the function `translate_codon` you just wrote.

Write a function, `split_codons` , that takes one argument:

1. A string, which is an ORF sequence

The function must return:

- A list of strings. Each string must have length 3 and must represent the-non overlapping triplets in the same sequence as they appear in the string given as argument.

Example usage:

```
split_codons('ATGTATGCCTGA')
```

should return

```
['ATG', 'TAT', 'GCC', 'TGA']
```

Divide the problem into simpler tasks like above. You need to loop over the sequence to perform operations on it. Start by writing a function that prints each character:

```
def split_codons(orf):
    for i in range(len(orf)):
        print(orf[i])
```

Now try to figure out how you can modify the function to make it move over the sequence in jumps of three. Look at the documentation for the `range` function to see how you can make it iterate over numbers with increments of three like this: 0, 3, 6, 9, 12, Modify your function so that it now prints every third character.

What you want is obviously not every third character. You want three characters. I.e. every third character *and* the two characters that come right after. You can use the index in the for loop to get the corresponding codon using slicing. Modify your function so that it prints each codon.

Now all that remains is to put each codon on a list that you can return from the function. You can define a list before your for-loop so you have a list to add codons to.

Translating an open reading frame

Now you can use the two functions `split_codons` and `translate_codon` to write a function that translates an ORF into a protein sequence.

Write a function, `translate_orf`, that takes one argument:

1. A string, which is a DNA sequence.

The function must return

- A string, which is the protein sequence translated from the ORF sequence argument.

Example usage:

```
translate_orf('ATGGAGCTTANCAAATAG')
```

should return

```
'MEL?K*'
```

30 Worked example: solving a problem by composition

A companion to this week's worked-example lecture and its snippet-cast recording.

You have spent five weeks learning the pieces: variables, conditionals, functions, containers, loops. This week you start the first project, and the projects are where a fact you have not yet had to face catches up with you: knowing the pieces is not the same skill as putting them together. This note models that second skill, live, on a small problem, from a completely blank page. Nothing here is more advanced than what you already know. What is new is only the process of assembling it, which is exactly what the snippet-cast recording for this lecture shows in real time and what you should replay at home before you touch this week's project.

The problem

Here is the task, stated the way a task usually arrives: vaguely. "Given a DNA coding sequence, tell me the protein it makes, and warn me if the sequence doesn't end the way a coding sequence should." That sentence is not a program. Before writing a line of code, decide what pieces it needs.

Deciding the pieces

A coding sequence is read three bases at a time, and each triplet, or **codon**, stands for one amino acid, until a **stop codon** ends the protein. That reading-in-threes step feels like one job, and translating a single codon into an amino acid feels like a different job, and reporting the whole result in a sentence feels like a third job again. Three feels right: split the sequence into codons, translate the codons into a protein, and describe the result. Each of those should be small enough to test on its own before the next one leans on it.

```
def split_into_codons(dna):
    "Split a DNA string into a list of three-base codons."
    ...

def translate(dna):
    "Translate a DNA coding sequence into a protein string, using split_into_codons."
    ...

def describe(dna):
    "A sentence reporting the protein and whether the sequence ended in a stop codon."
    ...
```

Three empty functions with docstrings, no bodies yet. This is the plan, and it already contains a decision worth noticing: `translate` depends on `split_into_codons`, and `describe` depends on `translate`, so that is the order they get built and tested in.

Building the first piece

```
def split_into_codons(dna):
    """Split a DNA string into a list of three-base codons."""
    codons = []
    for i in range(0, len(dna), 3):
        codons.append(dna[i:i + 3])
    return codons
```

Predict what `split_into_codons('ATGGCC')` gives back before you run it, then check:

```
print(split_into_codons('ATGGCC'))
```

`['ATG', 'GCC']`. Good. One piece, proven, before anything is built on top of it.

Building the second piece, and hitting a bug

For `translate`, a small codon table is enough to make the example work; a real project would use the full table of sixty-four.

```
codon_table = {
    'ATG': 'M', 'GCC': 'A', 'TGG': 'W', 'TAA': '*', 'TAG': '*', 'TGA': '*',
}

def translate(dna):
    """Translate a DNA coding sequence into a protein string, using split_into_codons."""
    protein = ''
    for codon in split_into_codons(dna):
        protein += codon_table[codon]
    return protein

print(translate('ATGGCCTGA'))
```

! Spot the bug

Run the cell above before reading past this box. It raises `KeyError: '*'`. Read that traceback the way earlier chapters taught you to: find the line it points at, and ask what value was actually being looked up when it failed. This bug is planted on purpose, to give you a real one to diagnose rather than a described one to take on faith.

The traceback points at the line `protein += codon_table[codon]`, and the key it could not find is `'*'`, not one of the DNA codons. That is the giveaway: `codon_table['TGA']` correctly returns `'*'`, and then the loop keeps going and tries to look up `'*'` itself as if it were a fourth codon, because nothing told the loop to stop at a stop codon. The bug is not in the table. It is in the logic: the function translates every codon including the ones after the stop, instead of stopping there.

Fixing it

```
def translate(dna):
    """Translate a DNA coding sequence into a protein string, stopping at the first stop codon.
    protein = ''
    for codon in split_into_codons(dna):
        amino_acid = codon_table[codon]
        if amino_acid == '*':
            break
        protein += amino_acid
    return protein

print(translate('ATGGCCTGA'))
```

MA . Fixed, and proven on the one example so far. A second call, `translate('ATGGCC')`, has no stop codon at all in this small table; deciding what should happen then is exactly the kind of awkward case a real test would pin down before it became a surprise.

The last piece

With `translate` proven, `describe` is now allowed to trust it completely and only has to decide what to report:

```
def describe(dna):
    """A sentence reporting the protein and whether the sequence ended in a stop codon."""
    protein = translate(dna)
    codons = split_into_codons(dna)
    ends_properly = codon_table.get(codons[-1]) == '*'
    return f"Protein: {protein}. Ends in a stop codon: {ends_properly}."

print(describe('ATGGCCTGA'))
print(describe('ATGGCC'))
```

Zooming out

Notice what just happened, because it is the whole point of the exercise and not a detail of this particular protein problem. Nothing above required a construct

you had not already met by week four. What made the problem tractable was the order: decide the pieces before writing any of them, build and prove the piece with no dependencies first, and only then build the piece that leans on it, so that when something breaks, as `translate` did, you know exactly where to look, because everything underneath the broken piece was already trusted. That discipline, decompose, build the foundation first, test before you build on top, is the entire difference between a problem that feels impossible from a blank page and one you can just start.

Exercise 30-1

SOLO

Extend `describe` to also report the sequence's length in bases and its GC content, reusing a `gc_content` function from an earlier chapter if you still have it, or writing a short one now. Decide the pieces and their order before you write anything, the way this note did, and prove each piece before leaning on it.

For your logbook this week, describe the moment in this week's project where a bug first appeared, and say whether reading the traceback, the way this note modeled it, actually told you where to look.

31 Turtle, week 6: building a drawing out of pieces

This week has two halves that fit together. In the worked example on composition you watched a problem get taken apart into three functions before a single body was written, and you saw that deciding the pieces is a separate job from filling them in, done first and on purpose. In the AI note you practised judging between two solutions that both run, using four questions in a fixed order: whether they agree on the awkward cases, whether a stranger could read them, whether one of them clearly redoes work you can name, and whether they use Python in a way you can explain. The translation project asks you to do both on assessed work.

The turtle is a good rehearsal room for this because a drawing decomposes naturally and because a piece of a drawing can be wrong in more than one way at a time. A function that draws a polygon can produce the right shape and still leave the turtle in the wrong place, which means two versions can be equally correct about the picture and not equally correct about anything else. That is the kind of difference the rubric exists to catch, and it is much easier to see with a pen on a canvas than in prose.

One convention runs through the whole session. Every function that draws takes the turtle as its first argument, written `t`, rather than reaching out for a turtle that happens to exist somewhere else. A function that depends on a name it did not receive is a function you cannot move, cannot reuse, and cannot test on a second turtle, and you will want to do all three before the session is over.

Making the pieces

Exercise 31-1

SOLO

Take the polygon loop you wrote in week four and wrap it in a function. Decide what it needs to be told before you write the `def` line.

```
from turtle_widget import Turtle

def polygon(t, sides, size):
    "Draw a regular polygon with the given number of sides."
    for _ in range(sides):
        t.forward(size)
        t.left(360 / sides)

t = Turtle()
```



```
t.speed(0)
polygon(t, 6, 70)
```

Then call it three times in the same cell with different numbers of sides and watch the three shapes land on top of each other. That overlapping mess is a result and not a failure. It is telling you that `polygon` says nothing about where it draws, which is a decision somebody has to make and which `polygon` has decided not to be responsible for.

Exercise 31-2

SOLO

Write the piece that takes responsibility for position. It should move the turtle somewhere without drawing a line on the way, and it should leave the pen as it found it.

```
def move_to(t, x, y):
    "Move the turtle to (x, y) without drawing."
    t.penup()
    t.goto(x, y)
    t.pendown()
```

Now answer a design question in a sentence, in writing, before you write any more code. Should `move_to` also set the heading? There is no right answer, there is only a decision, and the two possible decisions lead to two different functions that behave identically in the first program you write and differently in the third. Write down which one you chose and why, because you will need that sentence again in the last exercise of this session.

Exercise 31-3

SOLO

Now compose. Using `polygon` and `move_to` and nothing else new, draw a row of five hexagons of increasing size. Write the plan first, as one sentence saying what the loop repeats and what changes each time, and only then write the code.

The point of writing the sentence first is not bureaucracy. If you cannot say in one sentence what changes each time round the loop, the loop will not come out right, and you will spend twenty minutes discovering that by trial and error instead of ninety seconds discovering it by thinking.

Exercise 31-4

SOLO

Not every piece of a drawing program draws. Write a function that computes and returns a number and never touches a turtle at all.

```
def polygon_perimeter(sides, size):  
    "Return the total distance a turtle travels drawing this polygon."  
    return sides * size  
  
print(polygon_perimeter(6, 70) == 420)
```

Then use it. Draw a hexagon with your `polygon` function and check the answer against the turtle's own record.

```
from turtle_widget import Turtle  
  
t = Turtle()  
t.speed(0)  
polygon(t, 6, 70)  
print(t.total_movement == polygon_perimeter(6, 70))
```

Notice what just happened. Two independent things agreed: a number you worked out from the specification, and a number the machine accumulated while carrying the drawing out. That agreement is worth more than either number on its own, and this pattern of checking one thing against a second thing that was arrived at differently is the single most useful habit in the whole course.

Judging between two versions

Everything from here uses the four questions from this week's AI note, in the same order: the awkward cases first, then readability, then efficiency you can actually name, then whether you can explain the Python being used. Run both versions before you read either of them.

Exercise 31-5

AI: COMPARER

Here are two functions that draw the same hexagon. Decide which you prefer, for a stated reason, before you run anything.

```
def polygon_a(t, sides, size):  
    for _ in range(sides):  
        t.forward(size)  
        t.left(360 / sides)
```

```
def polygon_b(t, sides, size):
    angle = 360 / sides
    for _ in range(sides):
        t.forward(size)
        t.left(angle)
```

Now apply the rubric properly. For the first question, find an input on which they behave differently, and say so if you cannot find one. For the third question, say exactly what work `polygon_a` redoes and then say whether you can measure it, because a difference you cannot measure is a difference you should not claim. This exercise is here mostly so that you learn what it feels like when the rubric returns a shrug, which it will do often and which is a perfectly good answer.

When you have your own verdict written down, ask the assistant which version it prefers and why. You are comparing reasons, not conclusions: if it agrees with you for a reason you had not thought of, that reason is the thing worth keeping, and if it claims one version is faster, ask it how you would measure that.

Exercise 31-6

AI: COMPARER

Now two versions that really do differ. Read both, predict for each whether the turtle ends where it started, and then run each one twice in a row in the same cell.

```
def flower_a(t, petals, size):
    for _ in range(petals):
        t.circle(size)
        t.left(360 / petals)

def flower_b(t, petals, size):
    for _ in range(petals):
        t.circle(size)
        t.left(360 / petals)
    t.left(17)
```

The second version is the first with one extra line, and one drawing of each is enough to convince anybody that they are the same function. Two drawings of each is enough to prove that they are not. Add this to your rubric answers as a fifth thing worth asking about a function: what does it leave behind. A function that does not put things back where it found them cannot be used twice, and you will not discover that by looking at it once.

Now give the assistant both functions and nothing else, and ask whether they are the same. It will most likely tell you that they differ by a turn of seventeen degrees, which is true and is not the point. Ask the follow-up question yourself: does that matter? Then run each of them twice and see whether its answer survives contact with the screen.

Exercise 31-7

AI: COMPARER

Ask the assistant for another way to draw a regular polygon, one that does not use a loop of `forward` and `left`. It will probably reach for `t.circle`, which takes a `steps` argument and can draw a polygon directly. Get its version, run it beside yours, and judge them with the four questions.

Pay particular attention to the fourth question, which is idiom, and read it strictly. If you cannot explain what the `steps` argument does to somebody who has not seen it, then for you, today, that version is the worse one, however much shorter it is. This is not a permanent judgment about the function. It is a judgment about what you can currently verify, and it will change the week you understand it.

Exercise 31-8

AI: COMPARER

Ask the assistant to write a function that draws a five-pointed star, and hold it to the discipline you have been building. Before you prompt, write down what you want the function to be called, what it takes, and what it should leave behind, including where the turtle should end up and which way it should be facing. Then get the code, read every line of it, and check the promise about what it leaves behind by comparing `t.position()` and `t.heading()` before and after the call.

Almost every star function you get will draw a correct star and leave the turtle rotated. This is not the assistant being careless. It is the assistant answering the question you asked, which was about a picture, while you were also silently caring about something you never said out loud.

Two pens on one canvas

Comparing two versions is easier when you can see both at once, and a canvas can carry more than one turtle. The first turtle owns the canvas, and any further turtle joins it with `new_turtle`.

Exercise 31-9

SOLO

Run your version and the assistant's version side by side in two colours.

```
from turtle_widget import Turtle

red = Turtle()
red.speed(0)
blue = red.new_turtle(color='blue')
blue.speed(0)
```

```
red.pencolor('red')
move_to(red, -120, 0)
polygon(red, 6, 60)

move_to(blue, 40, 0)
polygon(blue, 6, 60)
```

The two turtles take turns rather than moving at the same time, in the order the lines run, exactly as Oath 2 says they should. Now put them both at the same place and run two different implementations of the same drawing, and watch for the step at which the pens stop lying on top of each other. That step is the answer to the question of where the two versions actually differ, and it took you no reading at all to find it.

Be careful about how much you conclude from this. Two pens that stay on top of each other for the whole drawing have told you that the two functions produce the same picture, for these arguments, on this canvas, starting here. They have told you nothing about any other arguments, and nothing about what either function leaves behind.

Comparing designs rather than code

The four questions were written for two versions of a function, and they work just as well one level up, on two ways of cutting a problem into pieces. This is the most useful thing you will take from this session, and it is worth doing carefully.

Exercise 31-10

AI: COMPARER

Choose a drawing with enough parts to argue about, such as a house with a door, three windows and a roof, or a simple bar chart with axes and labels. Write down your own decomposition first, meaning the name of each function, what it takes, what it returns, and one sentence about what it leaves behind. Write no bodies.

Then give the assistant the same problem, ask it for a decomposition and nothing else, and refuse the code if it offers it. Now compare the two lists of function signatures with the four questions, adapted one level up. For correctness on the awkward cases, ask whether each design can express a house with no windows, or a chart with one bar. For readability, ask whether somebody could tell what each function does from its name and arguments alone. For efficiency, ask whether either design forces the same work to be done twice. For idiom, ask whether either design contains a piece you could not test on its own.

Then say which design you are going to build and why, in two sentences, and build it. The reason for writing the two sentences down is that in seven weeks you will be doing exactly this on the hardest problem in the course, with nothing given to you and nobody's decomposition to fall back on but your own.

Week 7

32 Let it draft a small piece you can read and test

The assistant as Drafter, kept safe by the smallness of the piece and a test.

Session length

Estimated 3 hours, run as this week's TA session, alongside this week's folding project and the introduction of classes.

This is the first time this course lets the assistant write code you did not type yourself, and it is safe for exactly two reasons: the piece is small enough to read in full, and a test stands behind it before you are allowed to believe it. Every round below follows the same order, and the order is the whole point: specify first, draft second, read every line third, test fourth. Never let the assistant race ahead of a step you have not done yet.

The order, every time

1. **Specify.** Write the function's name, its inputs and outputs, and one worked example, before prompting anything.
2. **Draft.** Ask for exactly that one function, giving it the specification, nothing more.
3. **Read.** Go through what it returns line by line. If there is a line you cannot explain, stop here.
4. **Test.** Write one gentle test and one nasty test, seeded from your own worked example and from a case designed to break it. Only a passing nasty test earns your belief.

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter

Still to come: Worker,
Collaborator, Developer

Part 1 — Warm-up, done together (25 min)

As a class, specify `gc_content(dna)` : fraction of bases that are `G` or `C` , with the worked example `gc_content('GGCC') == 1.0` . Draft it with the assistant. Read it aloud, line by line. Write `assert gc_content('GGCC') == 1.0` as the gentle test and `assert gc_content('') == 0` or a documented decision not to allow it as the nasty one. Run both. This models the full order once before anyone works alone.

Part 2 — Three small functions (60 min)

For each function below, follow the full order: specify with a worked example first, draft, read every line, write one gentle and one nasty test, run both. Do not move to the next function until the current one's tests pass.

Exercise 1 32-1

AI: DRAFTER

`is_palindrome(dna)` : `True` if the DNA string reads the same as its own reverse complement. Worked example: `is_palindrome('GAATTC') == True` . Nasty case to consider: an odd-length string, and a string containing a base outside `A` , `T` , `G` , `C` .

Exercise 2 32-2

AI: DRAFTER

`count_codon(dna, codon)` : how many times a given three-letter codon appears, including overlaps. Worked example: `count_codon('ATGATGC', 'ATG') == 2` . Nasty case to consider: a codon that does not appear at all, and a codon longer or shorter than three letters.

Exercise 3 32-3

AI: DRAFTER

`most_common_base(dna)` : the single most frequent base in a DNA string. Worked example: `most_common_base('GGGATC') == 'G'` . Nasty case to consider: a tie between two bases, so decide, in writing, before you draft, what you want to happen, since last term's cohort found the assistant will pick silently and inconsistently if you do not.

Part 3 — Catch the silent bug (35 min)

Not every draft that passes a gentle test deserves your belief, which is the entire reason the nasty test exists. This round is curated so the gentle test passes and the nasty one does not, on purpose.

Exercise 4 32-4

AI: DRAFTER

Specify `gc_content(dna)` again, but this time specify it to be **case-insensitive**: it should count lowercase `g / c` the same as uppercase. Worked example: `gc_content('ggcc') == 1.0`. Draft it, read it, and write two tests: `assert gc_content('GGCC') == 1.0` and `assert gc_content('ggcc') == 1.0`. Many drafts pass the first and silently fail the second because they only check the uppercase letters. If yours does, do not just accept a patched version — reread the fixed line until you can say in one sentence why the first draft missed lowercase input.

Part 4 — Drafting a method, not just a function (35 min)

Classes entered this week, and a method is drafted the same way a function is, specified first, read in full, tested before belief.

Exercise 5 32-5

AI: DRAFTER

Specify a small `Codon` class with an attribute `bases` and a method `translate(self)` that returns the corresponding amino acid letter for a short, fixed table of codons you provide (four or five entries is enough). Worked example: `Codon('ATG').translate() == 'M'`. Draft the class, read `__init__` and `translate` line by line, and write one gentle test plus one nasty test using a codon not in your table — decide, before drafting, what should happen then, and check the draft actually does what you decided rather than whatever it happened to do.

Wrap-up (15 min)

For your logbook this week, name the one function or method from today where reading it line by line, before testing, made you suspicious enough to write a nastier test than you otherwise would have. Say what specifically in the code raised your suspicion, and whether the nasty test confirmed it or turned out to be unfounded.

33 Classes

Do you know what type you are? Python objects do. This chapter is about how they know it, and about how you make a type of your very own.

You have been using objects all along

In Chapter 15 you learned that every value in Python is an object: a piece of data with a lot of relevant functionality packaged along with it. You have been bossing those objects around ever since. Write and run this one more time:

```
print("banana".upper())
```

Say it out loud the way we did back then: “Hey string, uppercase yourself!”. The string obliges, and `"banana".upper()` is substituted for `"BANANA"`.

Now here is a question I have been putting off: where does `"banana"` keep that `upper` method? Somewhere in your computer’s memory sits the word banana. Does it also carry around its own private copy of `upper`, and `lower`, and `strip`, and `replace`, and the forty-odd other string methods? That would be an awful lot of luggage for six letters. And your program may hold thousands of strings.

It does not. The methods live somewhere else, and where they live is the key to this whole chapter.

The type knows what an object can do

Every object has a type, and you can ask an object what type it is. You met the `type` function back in exercise 12-14:

```
fruit = "banana"
print( type(fruit) )
```

Run it. Python tells you that the value of `fruit` is of type `str`. You have met a good handful of types by now: `int`, `str`, `float` and `bool` for single values, and `tuple`, `list` and `dict` for containers.

Figure 33.1 shows the arrangement. On the left are two objects, `"Preben"` and `"Mogens"`. They are different objects: they hold different data and they sit in different places in memory. But they are both of type `str`, and so they both point at the *same* description of what a string can do. The methods `upper`, `split` and `strip` are defined once, in one place, and every string in your program borrows them from there.

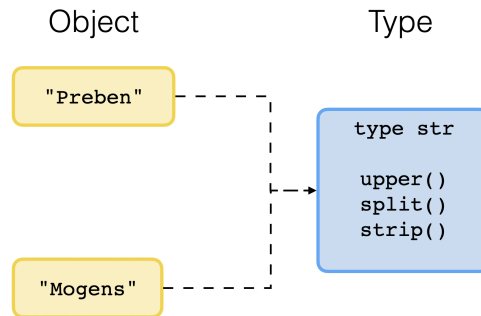


Figure 33.1: Objects and type. Each object carries its own data, but refers to the type for a definition of its methods.

That is the deal: the object holds the *data*, and the type holds the *functionality*. When you write `"banana".upper()`, Python looks at the object, sees that it is a `str`, walks over to `str` to fetch `upper`, and then runs it on your particular banana.

Exercise 33-1

SOLO

Decide what each line prints before you run anything. Then write the code and check.

```

print( type(3) )
print( type(3.0) )
print( type("3") )
print( type([3]) )
print( type({3: 'three'}) )

```

Were you right about all five? The last two are the ones that catch people out.

Exercise 33-2

SOLO

The type decides what an object can do. That means that asking an object to do something its type knows nothing about is an error. Predict what happens here, then run it and read the error message carefully:

```

number = 7
print( number.upper() )

```

The error message is worth learning to read. It names the type it was looking in, and the method it could not find there.

Classes define types

So far you have used the types that came with Python. You are about to make one of your own, and the tool for that is called a **class**.

A class is a description of a type. When you write a class, you define a new type, and the methods you write in the class become available to every object of that type. Python types and your own types work in exactly the same way: `str` is a class too. Someone sat down and wrote it, just as you are about to sit down and write yours.

Why would you want to? Because objects bundle data together with the functionality that belongs to it. Think back over the programs you have written so far. Have you ever had three or four variables that clearly belonged together, and a couple of functions that only ever made sense when applied to exactly those variables? That is a type waiting to be born.

A point in a plane

Let us make something small enough to hold in your head. Figure 33.2 shows two points in a coordinate system. The point `p1` sits at some `x1` and `y1`, and `p2` sits at `x2` and `y2`.

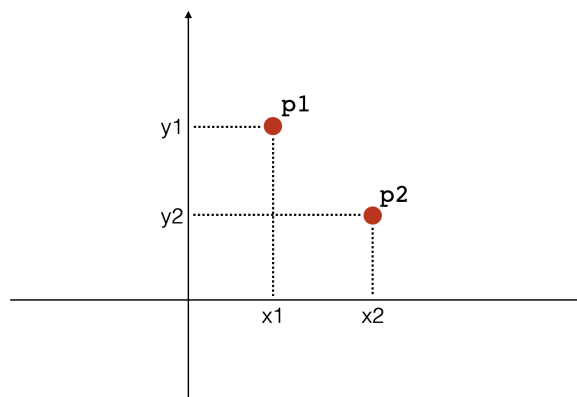


Figure 33.2: Points

You could keep those two points in four variables:

```
x1 = 2
y1 = 5
x2 = 4
y2 = 3
```

and that works fine. Until you need a third point. And a tenth. And a function that takes a point as an argument — which now has to take two arguments and hope that you pass them in the right order, and which cannot return a point at all, because a function returns *one* value.

Exercise 33-3

SOLO

Before you read on, take thirty seconds with pen and paper. If Python let you invent a new type called `Point`, what data would each point object need to carry, and what would you like to be able to ask a point to do? Write down three things. You will meet two of them later in this chapter.

Writing your first class

Here is the whole thing. Write it in a file and run it:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

It seems that nothing happened, but a great deal did. Let us take it apart line by line.

The line `class Point:` says: *from here on I am describing a new type, and its name is `Point`*. Note the colon at the end. You know what a colon at the end of a line means by now: an indented block follows, exactly as after an `if` or a `def` (Chapter 11). Everything indented under `class Point:` belongs to the class.

Inside the block we define a function with `def`, just as in Chapter 12. A function defined inside a class is called a **method** — and now you know why methods and functions have felt so similar all along. A method is a function that lives in a class.

The method is called `__init__`. Those double underscores should ring a bell: you met them in exercise 15-11, where `+` turned out to call a secret method named `__add__`. Same idea here. `__init__` is a method that *you* write but that *Python* calls, on your behalf, every time a new `Point` is made. Its job is to *initialize* the new object: to furnish it with the data it should carry. It does not create the object — Python has already done that by the time `__init__` runs. It moves the furniture in. That is why it is called the initializer.

Not a “constructor”

You will meet people, and AI assistants, who call `__init__` “the constructor”. It is a forgivable slip borrowed from other languages, but it is not what happens in Python. By the time `__init__` runs, the object already exists. `__init__` initializes it.

And then there is `self`, which deserves a section of its own.

What is `self` and why do we need it?

Look again at the two indented lines:

```
self.x = x
self.y = y
```

The right-hand sides are easy: `x` and `y` are just parameters of the method, holding the values that were passed in. The left-hand sides say: *store this in the object, under the name `x`*. And `self` is the name of the object we are storing it in.

Three things to know about `self`:

1. The first parameter of a method is called `self`. This is a convention, not a rule of the language, but it is a convention so universal that breaking it will confuse everyone including you.
2. Python adds the `self` argument automatically when a method is called. You never pass it yourself.
3. `self` points at *the particular object* the method was called on. When you have made a hundred points, the same `__init__` runs a hundred times, and each time `self` points at a different one of them.

That last point is the whole trick. One description of the type; many objects, each with its own data. Exactly the picture in Figure 33.1, only now with points instead of strings.

Making an object

Add a `print` at the bottom of your file and run it again:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

print(Point)
```

Python prints something like `<class '__main__.Point'>`. Read that as: “this is the class `Point`, defined in the file you are running”. The class itself is an object too — of course it is, everything is an object — and this is what it looks like when you print it.

Now make an actual point. Replace the last line:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(4, 3)
print(p)
```

Two things to notice.

First, look at `Point(4, 3)`. It looks exactly like a function call, and it behaves like one: it is an expression, and it reduces to a value. Here is what Python does with it, in order:

1. It makes a new, empty object of type `Point`.
2. It calls `__init__` with that new object as `self`, `4` as `x`, and `3` as `y`.
3. `__init__` stores the data in the object and returns nothing.
4. The whole expression `Point(4, 3)` is substituted for the new object, which is then assigned to `p`.

Notice that you passed two arguments although `__init__` has three parameters. Point 2 is where the third one comes from: Python supplies `self`.

Second, look at what got printed. Something like:

```
<__main__.Point object at 0x1032b4a20>
```

The long number will be different on your machine, and different again the next time you run it. It is roughly where the object sits in your computer's memory. This is Python saying "I have no idea how you want this thing displayed, so here is what I know about it". We will fix that shortly.

Exercise 33-4

SOLO

What do you think these two lines print? Decide first, then run.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(4, 3)

print( type(p) )
print( Point )
```

The two lines print the same thing, and it is worth a moment's thought why. `type(p)` asks the object what type it is, and the answer *is* the class. The class and the type are the same object. That is what “classes define types” means, quite literally.

Getting at the data

The data you stored in the object is available with a dot, just like methods:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(4, 3)

print(p.x)
print(p.y)
```

Run it and you get `4` and `3`. Values stored in an object like this are called **attributes**. The dot means the same thing it has always meant: *go into this object and get me the thing called....* Sometimes what you get is a method you can call, and sometimes it is a value you can use.

Exercise 33-5

SOLO

Attributes are variables, and variables can be assigned to. Predict what this prints, then run it:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(4, 3)
p.x = 100
print(p.x, p.y)
```

Exercise 33-6

SOLO

Now make two points and convince yourself that they really are separate objects with separate data. Predict all four numbers before you run it:


```

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p1 = Point(2, 5)
p2 = Point(4, 3)
p1.x = 100
print(p1.x, p1.y)
print(p2.x, p2.y)

```

This is Figure 33.1 again, with `Point` in the blue box and your two points in the yellow ones.

Exercise 33-7

SOLO

What do you think happens here? Make up your mind, then try it:

```

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(4, 3)
print(p.z)

```

Read the error message. It tells you which type it was looking at and which attribute it could not find. You will see this one a lot, usually because you misspelled something.

How should it look as a string?

Printing a point gave us that unfriendly `<__main__.Point object at ...>`. Let us do something about it, and in doing so meet another double-underscore method.

When you write `print(p)`, Python needs to turn your object into a string, and it asks the object to do it. It does that by calling the object's `__str__` method. Your `Point` class does not have one, so Python falls back on the unhelpful default. Give it one:

```

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return "({}, {})".format(self.x, self.y)

p = Point(4, 3)
print(p)

```

Run it. Now `print(p)` gives you `(4, 3)`.

Look at the method. It takes only `self` — it needs no other information, because everything it needs is stored in the object. It reaches into the object for `self.x` and `self.y`, formats them with the `format` method you learned in Chapter 15, and *returns* the resulting string. It does not print anything. `print` does the printing; `__str__` only supplies the string.

Exercise 33-8

SOLO

Delete the `return` from `__str__` so the method body is just the `"({}, {})".format(self.x, self.y)` line without `return`, and run the program again. What is printed? Why? If you have forgotten, exercise 12-25 is where a function without a `return` was first discussed.

Exercise 33-9

SOLO

Change `__str__` so that a point prints as `Point at x=4, y=3` instead. Predict what your program will print, then run it.

A method of your own

Now for a method that actually computes something. How far is a point from origin? That is the Pythagorean theorem you used in exercise 7-7: the distance is $\sqrt{x^2 + y^2}$, and taking a square root is exponentiating to 0.5.

```

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

```

```
def dist_from_origin(self):
    return (self.x ** 2 + self.y ** 2) ** 0.5

p = Point(4, 3)
dist = p.dist_from_origin()
print(dist)
```

Write it and run it. You should get `5.0`, which is the nicest right-angled triangle there is.

Now look at what you just did. `dist_from_origin` is a *function* that only makes sense for points, so you put it where points can find it: in the class. Every point you ever make from now on can do this. That is what I meant by bundling data together with the functionality that belongs to it, and it is the whole reason classes exist.

Figure 33.3 is the same picture as before, now with your own type in the blue box.

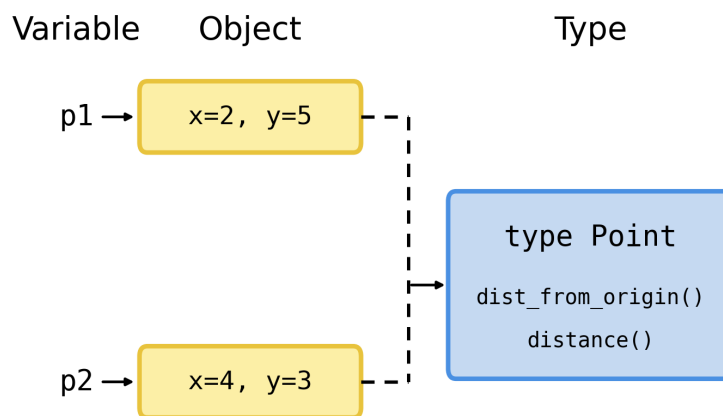


Figure 33.3: Two point objects and their type. The data lives in the objects; the methods live in the type.

Exercise 33-10

SOLO

Predict what this prints, then run it:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def dist_from_origin(self):
        return (self.x ** 2 + self.y ** 2) ** 0.5
```

```
p1 = Point(3, 4)
p2 = Point(6, 8)
print(p1.dist_from_origin())
print(p2.dist_from_origin())
print(p1.dist_from_origin() + p2.dist_from_origin())
```

Do the substitutions in your head first. `p1.dist_from_origin()` is an expression, and like every expression it reduces to a value.

self and method calls

Here is something that will make `self` click, if it has not already.

The `self` variable points at the object that the method was called on. So if you have made a point:

```
p = Point(4, 3)
```

then this:

```
p.dist_from_origin()
```

is actually the same as this:

```
Point.dist_from_origin(p)
```

Try both. They give the same answer, because they *are* the same call. The dot notation is a convenience: `p.dist_from_origin()` means “find the type of `p`, get `dist_from_origin` from it, and call it with `p` as the first argument”. Python does the shuffling so you do not have to.

Nobody writes `Point.dist_from_origin(p)` in real code. But knowing that it means the same thing is what stops `self` from feeling like magic. It is just the first argument.

Ask the machine, not your memory

If you are ever unsure whether a method is being called on the object you think it is, add a `print(self.x, self.y)` as the first line of the method. Methods are ordinary functions and you can debug them in ordinary ways.

Distance to another point

Distance from origin is a special case. What you usually want is the distance between two points. The Pythagorean theorem still applies: the two legs of the triangle are the difference in `x` and the difference in `y`.

Exercise 33-11

SOLO

Fill in the three missing lines. The hint is $c = \sqrt{a^2 + b^2}$, which in Python is `c = (a**2 + b**2)**0.5`.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance(self, other):
        ???
        ???
        return ???

p1 = Point(4, 2)
p2 = Point(3, 7)
dist = p1.distance(p2)
print(dist)
```

Two hints and then you are on your own. First, `distance` has two parameters: `self` and `other`. Python fills in `self` with `p1`, and you supply `other` yourself — it is `p2`. Second, `other` is a `Point`, so you can reach into it with a dot exactly as you reach into `self`.

Take your time with this one. Work out on paper what the answer should be before you run anything, so you know whether to believe the number that comes out.

Exercise 33-12

AI: COMPARE

Once your `distance` method works, open the assistant in the browser, show it your class, and ask it to write a `distance` method for a `Point` class. Do not paste its answer into your editor. Compare it with yours instead. Is it doing the same arithmetic? Did it use different variable names, or fewer lines? Does it handle anything you did not think of? Then run *your* version on `Point(0, 0)` and `Point(3, 4)` and check the answer against what you know the answer must be.

Remember the rule: the assistant predicts, the machine proves.

Here is one solution:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```

def distance(self, other):
    a = self.x - other.x
    b = self.y - other.y
    return (a ** 2 + b ** 2) ** 0.5

p1 = Point(4, 2)
p2 = Point(3, 7)
dist = p1.distance(p2)
print(dist)

```

Exercise 33-13

SOLO

Should `p1.distance(p2)` and `p2.distance(p1)` give the same answer? Decide what you believe, argue to yourself why, and then check it with the machine.

Peeking behind the curtain, again

You have now written two methods with double underscores around their names, and both of them were called by Python rather than by you. There are many more, and they are how Python lets your own types behave like built-in ones.

Remember `__add__` from exercise 15-11? The `+` operator does not know how to add anything. It asks the object on its left to add the object on its right, by calling its `__add__` method. Which means that if you give your class an `__add__` method, `+` works on your objects.

```

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return "({}, {})".format(self.x, self.y)

    def __add__(self, other):
        new_x = self.x + other.x
        new_y = self.y + other.y
        return Point(new_x, new_y)

p1 = Point(4, 2)
p2 = Point(3, 7)
new_p = p1 + p2
print(new_p)

```

Predict what is printed, then run it. Look carefully at what `__add__` returns: not a number, but `Point(new_x, new_y)` — a brand new point. A class can make objects of its own type inside its own methods. That is not a special feature; it is just an expression that reduces to a value, like everything else.

Exercise 33-14

SOLO

What do you think `p1 + p2 + p1` reduces to? Do the substitution and reduction steps one at a time. Then run it.

Exercise 33-15

SOLO

Comparison works the same way. The `==` operator calls a method named `__eq__`. Predict what this prints before you run it:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

p1 = Point(4, 2)
p2 = Point(3, 7)
print(p1 == p2)
```

Then change `p2` to `Point(4, 2)` and predict again. Two points at the same position: are they the same object? Are they equal? Those are two different questions, and `__eq__` is you answering the second one.

Exercise 33-16

SOLO

Delete the `__eq__` method entirely, set both points to `Point(4, 2)`, and run it again. What does `==` mean when you have not said what it should mean? Python's default is to ask whether the two names refer to *the very same object*, which two separately created points never are.

Making your own type feel like a real one

One more, because it is so satisfying. A gene has a sequence, and asking how long a gene is should be an ordinary thing to do. The `len` function calls a method named `__len__`:

```
class Gene:
    def __init__(self, name, sequence):
        self.name = name
        self.sequence = sequence

    def __str__(self):
        return "{} ({} bases)".format(self.name, len(self.sequence))

    def __len__(self):
        return len(self.sequence)

gene = Gene('BRCA1', 'ATGGATTATCTGCTCTTCGCGTT')
print(gene)
print(len(gene))
```

Predict both lines, then run it. Notice that `__len__` does not count anything itself. It hands the question on to the string it is holding, because a string already knows how long it is. Most of the methods you write will be like this: short, and mostly delegating to objects that already know how to do the work.

Exercise 33-17

SOLO

Give `Gene` a method called `gc_content` that returns the fraction of the sequence that is `G` or `C`. You already know everything you need: `count` is a string method, and `len` you have just used twice. Test it on `'GGCC'`, where you know the answer must be `1.0`, and on `'ATAT'`, where you know it must be `0.0`. If your method passes both of those, try it on the real sequence above.

Exercise 33-18

AI: EXPLAINER

Show the assistant your `Gene` class and ask it: “what does `len(gene)` do here?” Then ask it: “what would `gene[0]` do?” One of those two answers is going to be wrong, or at least is going to describe something your class cannot do. Find out which by running both. Write down in your logbook what it claimed and what actually happened.

General exercises

Exercise 33-19

SOLO

Look at the code below and decide what is printed. Then run it. If you were wrong, work out why.

```
class Dog:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "{} says woof".format(self.name)

d1 = Dog('Mogens')
d2 = Dog('Preben')
print(d1.speak())
print(d2.speak())
print(Dog.speak(d1))
```

Exercise 33-20

AI: EXPLAINER

! Spot the bug

The code below is deliberately broken. It is not a typo left in by accident; it is here for you to find.

```
class Dog:
    def __init__(name):
        self.name = name

d = Dog('Henning')
print(d.name)
```

Run it, read the error message, and fix it. The error message is not especially friendly, so read it twice: it is counting arguments, and it is counting one more than you passed.

Once you have found the bug, ask the assistant to explain why the message counts one argument more than you passed. The answer is about `self`, and it is the most useful single thing to understand about methods, so do not accept an explanation you cannot repeat in your own words.

Exercise 33-21

AI: EXPLAINER

! Spot the bug

This one is deliberately broken too, but in a sneakier way: Python complains about a line that is perfectly fine.

```
class Point:
    def __init__(self, x, y):
        x = x
        y = y

    def dist_from_origin(self):
        return (self.x ** 2 + self.y ** 2) ** 0.5

p = Point(3, 4)
print(p.dist_from_origin())
```

Run it and read the error. Python points at `dist_from_origin` and says the point has no attribute `x`. But `dist_from_origin` is written correctly. The mistake is two lines earlier, in `__init__`, and `__init__` ran without complaining at all. Where is the mistake, and why did Python only notice later? This gap between *where the bug is* and *where Python trips over it* is the single most common reason a beginner stares at the wrong line for twenty minutes.

Ask the assistant which line is at fault, and ask it before you tell it anything about your own verdict. Python's complaint points at a line that is perfectly fine, so an assistant that trusts the traceback will point at the wrong line too. Decide for yourself first, then run your fix and see which of you was right.

Exercise 33-22

SOLO

Write a class called `Rectangle` that is initialized with a width and a height, and that has a method `area` and a method `perimeter`. Then write a `__str__` method so that printing a rectangle shows something like `Rectangle 3 x 4`. Predict what each of your methods will return for `Rectangle(3, 4)` before you run anything.

Exercise 33-23

SOLO

Add a method `is_square` to your `Rectangle` that returns `True` or `False`. Do not write an `if` statement. If you are tempted to, reread exercise 7-10: a comparison is already a boolean.

Exercise 33-24

AI: EXPLAINER

Paste the `Point` class with `__add__` into the assistant and ask it to explain, line by line, what happens when Python evaluates `p1 + p2`. Then check its story against what you know: does it get the order of the substitutions right? Does it correctly say what `__add__` returns? Does it call `__init__` a constructor? Score it, and put the score in your logbook.

What you learned

You now know that every object has a type; that the type holds the methods and the object holds the data; that a class is how a type is described; that `__init__` initializes a new object rather than constructing it; that `self` is simply the first argument, filled in for you; and that the double-underscore methods are how your own types get to use Python's ordinary operators.

That is the last big idea in the language. Everything from here on is practice, and using other people's classes — which you can now read, because you know what they are.

Logbook

This week, record one thing the assistant told you about classes that turned out to be right, and one thing that turned out to be wrong or imprecise. Note how you found out. "I ran it" is the answer we are looking for.

34 Primer analysis

Say you have a short sequence that you want to use as a PCR primer. To evaluate if it is a suitable primer, you need to know the melting temperature of the sequence. You also need to make sure that it will not fold up on itself so that it cannot bind to the DNA sequence you want to amplify.

Download the files you need for this project:

https://munch-group.org/bioinformatics/supplementary/project_files

- `foldingproject.py` is an empty file where you must write your code.
- `test_foldingproject.py` is the test program that lets you test the code you write in `translationproject.py`.

Put the files in a folder dedicated to this project. On most computers you can right-click on the link and choose “Save file as...” or “Download linked file”.

Count the number of bases in your candidate primer

Before you can compute the melting temperature, you need to determine how many times each base occurs in the sequence. You can assume that the only characters in the string are A, T, G, and C.

Write a function, `count_bases`, which takes one argument:

1. A string, which is a DNA sequence.

The function must return:

- A dictionary, in which keys are strings that represent bases and values are integers that represent the number of occurrences of each base. If a base is not found in the sequence, its count must be zero.

Example usage: If the function is called like this:

```
count_bases("ATGG")
```

then it should return (not necessarily with key-value pairs in the same order):

```
{"A": 1, "C": 0, "G": 2, "T": 1}
```

Compute the melting temperature

Knowing the base composition in your sequence, you can now calculate the melting temperature the double-stranded DNA that forms when your primer pairs up with the sequence to amplify. If the primer has less than 14 bases the formula for calculating melting temperature is:

$$Temp = (A + T) * 2 + (G + C) * 4$$

and if the primer has 14 bases or more it is calculated like this:

$$Temp = 64.9 + 41 * (G + C - 16.4) / (A + T + G + C)$$

The A, T, C, and G in the formulas represent the numbers of A, T, C and G in the DNA primer.

You must write a function that applies these two formulas appropriately by taking the length of the primer into account.

Write a function, `melting_temp`, which takes one argument:

1. A string, which is a DNA sequence (your primer).

The function must return:

- A number, which represents the melting temperature of the double-stranded DNA corresponding to the DNA string given as argument.

Example usage: If the function is called like this:

```
melting_temp("ATG")
```

then it should return:

```
8
```

Reverse complement the sequence

It is possible that one part of the primer forms base pairs with another part of the primer to form a hairpin structure. To figure out if this can happen to your primer, you need to be able to find the reverse complement of DNA sequence. The reverse complement of a DNA sequence is one where the sequence of bases is first reversed, and then each base is replaced with its Watson-Crick complementary base.

Write a function, `reverse_complement`, which takes one argument:

1. A string, which is a DNA sequence.

The function must return:

- A string, which represents the reverse complement of the DNA string given as argument.

Example usage: If the function is called like this:

```
reverse_complement("AATTC")
```

then it should return:

```
"GAATT"
```

Check for hairpins

You would like to be able to determine if your primer can fold to form a hairpin with some specified minimum number of consecutive base pairs. We assume that hairpin loops are always at least four bases long and that base pairs in the hairpin can only be Watson-Crick basepairs. Here is an example of a hairpin with five basepairs and a loop of four bases (four Cs):

```
  C C
C      C
A - T
T - A
A - T
T - A
A - T
```

To test if a sequence can form a hairpin with at least four consecutive base pairs, you need to test if the sequence contains any subsequence of length four whose reverse complement is identical to another nonoverlapping subsequence. To take into account that the hairpin loop is at least four bases long, any such two subsequences must be separated by at least four bases.

Write a function, `has_hairpin`, which takes two arguments:

1. A string, which is a DNA sequence.
2. An integer, which represents the minimum number of consecutive base pairs in the hairpins to search for.

The function must return:

- `True` if the sequence contains a hairpin of at least the specified length and `False` otherwise.

Example usage: If the function is called like this:

```
print(has_hairpin("ATATACCCCTATAT", 4))
```

then it should return:

```
True
```

This is a hard one, so I will give you a bit of help. Here is the function with some parts missing.

```
def has_hairpin(s, k):  
    looplen = 4  
    for i in range(len(s)-k+1):  
        subs = # Hint A  
        right = # Hint B  
        revcl = reverse_complement(subs)  
        if revcl in right[looplen:]:  
            return True  
    return False
```

Hint A: Here you need to extract a substring of length k starting at i. Hint B: Here you need to extract all the sequence to the right of `substr`.

35 Debugging

A companion to this week's debugging clinic, reused across the project weeks.

Every piece of code you have written so far has broken at least once, and every piece you write for the rest of the course will too, whether you typed it or the assistant did. That is not a sign you are behind; it is what writing code is. What separates a bad afternoon from a five-minute fix is not luck, it is a method, and this note gives you one: read the traceback, locate the failing line, form a hypothesis, confirm it. The bugs below are planted on purpose, curated so that each one teaches a different way code breaks, rather than left to chance the way a real bug is.

The method

1. **Read the traceback from the bottom up.** The last line names the error and usually says what value was involved. Start there, not at the top.
2. **Locate the failing line.** Python tells you the exact line, and the file and function it was in. Find it before you theorize about anything else.
3. **Form a hypothesis.** Given the error and the line, what specific thing must be true about the values at that moment for this error to happen? State it as a sentence you could be wrong about, not a vague feeling.
4. **Confirm it before you fix it.** Print the value you suspect, or step through the line with the codelens widget, or write a small test that isolates just this case. Only once you have seen the actual value that caused the failure do you know your hypothesis was right, and only then do you change the code.

Skipping straight to step four, changing something and hoping, is how a five-minute bug turns into an hour. The method is slower on the first bug and much faster on every one after it, because it is the same four steps every time.

Bug one: an error that stops the program

```
def gc_content(dna):  
    "Fraction of bases that are G or C."  
    gc = dna.count('G') + dna.count('C')  
    return gc / len(dna)  
  
sequences = ['ATGC', 'GGCC', '']  
for seq in sequences:  
    print(gc_content(seq))
```


! Spot the bug

Run it before reading on. It crashes partway through with `ZeroDivisionError: division by zero`. This bug is planted on purpose.

Reading from the bottom: `ZeroDivisionError`, in `gc_content`, on the line `return gc / len(dna)`. The hypothesis writes itself once you look at the line: this fails when `len(dna)` is `0`, which means one of the strings in `sequences` must be empty. Confirm it by looking at the list: yes, the last entry is `''`. The fix is a decision, not just a patch: what should the GC content of an empty sequence even be? Undefined, arguably, so a reasonable fix is to say so explicitly rather than let the program crash:

```
def gc_content(dna):
    "Fraction of bases that are G or C. Undefined for an empty sequence."
    if len(dna) == 0:
        return None
    gc = dna.count('G') + dna.count('C')
    return gc / len(dna)
```

Bug two: no error at all

The bugs that crash are, in one sense, the friendly ones, because Python is shouting at you about exactly where to look. The next kind is quieter and more dangerous.

```
def count_start_codons(dna):
    "Count how many times the start codon ATG appears, including overlaps."
    count = 0
    for i in range(0, len(dna), 3):
        if dna[i:i+3] == 'ATG':
            count += 1
    return count

print(count_start_codons('AATGATGC'))
```

! Spot the bug

Run it. It does not crash, and it returns `1`. Decide for yourself, by counting on paper, whether that is actually correct before you read on. This bug is planted on purpose.

By hand: `AATGATGC` contains `ATG` starting at position 1 and again at position 4, so the correct answer is 2, but a plausible-looking answer of 1 does not announce that it is wrong. There is no traceback here, so the method's first two steps have nothing to bite on, which is itself the lesson: when there is no error, the "failing line" is not a crash site, it is the line whose *assumption* is wrong. The hypothesis:

`range(0, len(dna), 3)` only checks positions 0, 3, 6, stepping by three as if codons could not overlap, but the task explicitly asked for overlaps, so a start codon beginning at position 1 or 2 is never even looked at. Confirm it by printing the positions the loop actually checks: `list(range(0, 8, 3))` gives `[0, 3, 6]`, and position 1 is simply not among them. The fix is to step by one instead of by three:

```
def count_start_codons(dna):
    "Count how many times the start codon ATG appears, including overlaps."
    count = 0
    for i in range(len(dna) - 2):
        if dna[i:i+3] == 'ATG':
            count += 1
    return count
```

Confirming with codelens instead of print

Printing a suspect value works, but sometimes you want to watch a loop's variables change on every single step rather than guess where to insert a print. That is exactly what the codelens widget is for: point it at `count_start_codons('AATGATGC')` with the fix above and step through it one line at a time, watching `i` and `count` in the panel beside the code. Use it whenever your hypothesis is about *how a value changes over several steps* rather than about *what a value is at one point*; a single print statement cannot show you a sequence of changes the way stepping through the loop can.

Exercise 35-1

SOLO

Here is a third planted bug. Use the four-step method, in order, and write down your hypothesis in a sentence before you attempt any fix.

```
def reverse_complement(dna):
    "Return the reverse complement of a DNA strand."
    complement = {'A': 'T', 'T': 'A', 'G': 'C', 'C': 'G'}
    result = ''
    for base in dna:
        result = complement[base] + result
    return result

print(reverse_complement('ATGC'))
print(reverse_complement('atgc'))
```

The first call is correct. The second is not, and it fails the same quiet way bug two did: no traceback at all. Find the failing line, form the hypothesis, confirm it by printing the one value that matters, and only then fix it.

For your logbook this week, describe a bug from your own project code this week that you found using this four-step method rather than by guessing, and say which step actually found it.

36 Turtle, week 7: your own kind of object, and finding the bug

Three things landed this week. You learned to define a class, which is to say to make your own kind of object with its own state and its own methods, using `__init__` and `self`. You learned a method for debugging, four steps applied in the same order every time, which begins with reading the traceback from the bottom and refuses to let you change anything until you have confirmed your hypothesis. And in the AI note you let the assistant write code you intend to keep, for the first time, under two conditions that make it safe: the piece is small enough to read in full, and a test stands behind it before you are allowed to believe it.

All three come together on a turtle. You have been using an object with state and methods since week three, and now you build one of your own that drives it. You have been reading tracebacks all term, and now you get two bugs on purpose, one that stops the program and one that does not. And you will nominate exactly one small method for the assistant to draft, with its gentle and nasty tests written before the prompt is sent.

Keep the convention from last week. A drawing function is given the turtle rather than helping itself to one, and that includes a method on your own class.

An object of your own

Exercise 36-1

SOLO

Here is a class with state and one method. Read it, predict what the last three lines do, and then run it.

```
from turtle_widget import Turtle

class Polygon:

    def __init__(self, sides, size):
        self.sides = sides
        self.size = size

    def draw(self, t):
        for _ in range(self.sides):
            t.forward(self.size)
            t.left(360 / self.sides)

t = Turtle()
```

```
t.speed(0)
hexagon = Polygon(6, 60)
hexagon.draw(t)
```

Two objects are on that canvas, in a manner of speaking, and only one of them is a turtle. The `Polygon` knows how many sides it has and how long they are and nothing else. It does not know where it is, what colour it is, or when it will be drawn, because all of that belongs to the turtle. Deciding what a class knows about is the same job as deciding what a function takes, done once for a whole family of methods.

Notice that `hexagon.draw(t)` calls a method defined as `draw(self, t)` and that you passed one argument rather than two. The object before the dot becomes `self`, automatically, every time. This is the same arrangement you have been using since week three, when `t.forward(100)` put the turtle before the dot into the first parameter of `forward`, and you are now on the other side of it.

Exercise 36-2

SOLO

Make three of them and predict the picture before you run it.

```
from turtle_widget import Turtle

t = Turtle()
t.speed(0)
shapes = [Polygon(3, 80), Polygon(5, 60), Polygon(8, 40)]
for shape in shapes:
    shape.draw(t)
    t.left(120)

print(shapes[0].sides)
print(shapes[1].sides)
```

Three objects, each holding its own numbers, all made from one class. The class is the recipe and each `Polygon(...)` call bakes one cake, and changing what is on one plate does nothing to the others. Confirm that last claim rather than believing it: set `shapes[0].size` to some other number and print `shapes[1].size` afterwards.

Exercise 36-3

SOLO

Add a method that changes the object rather than drawing it, and give the class a `__str__` so that printing one tells you something useful.

```

class Polygon:

    def __init__(self, sides, size):
        self.sides = sides
        self.size = size

    def __str__(self):
        return 'a polygon with ' + str(self.sides) + ' sides of length ' + str(self.size)

    def grow(self, factor):
        self.size = self.size * factor

    def draw(self, t):
        for _ in range(self.sides):
            t.forward(self.size)
            t.left(360 / self.sides)

p = Polygon(5, 40)
print(p)
p.grow(2)
print(p)

```

Look at `grow` and at `__str__` together, because they are the two kinds of method you first met in week three, now written by you. The method `grow` is a command. It changes the object and hands nothing back, which is why `print(p.grow(2))` would be a mistake. The method `__str__` is a question. It works out an answer and returns it and leaves the object exactly as it was. Your own class has the same two categories that the turtle does, because they are a fact about objects rather than a fact about turtles.

Exercise 36-4

SOLO

Write a second class of your own with the same shape. Call it `Star`, give it a number of points and a size, give it a `draw` method that leaves the turtle exactly as it found it, and give it a `__str__`. Then write the check that proves the claim about what it leaves behind.

```

from turtle_widget import Turtle

t = Turtle()
t.speed(0)
before = t.position()
facing = t.heading()
Star(5, 80).draw(t)
print(t.position() == before)
print(t.heading() == facing)

```

If either check prints `False`, the picture is not the thing to look at. Add a `t.left(...)` at the end of `draw` until both print `True`, and then say in one sentence why the number you needed is what it is.

Exercise 36-5

AI: DRAFTER

Nominate one method for the assistant to write, and follow the full order from this week's AI note without cutting any of it.

First specify, in writing and before you prompt: a method `perimeter(self)` on `Polygon` that returns the total distance a turtle would travel drawing it, with the worked example that `Polygon(6, 70).perimeter()` is `420`. Decide, also before you prompt, what it should do for a polygon with zero sides, because that is the awkward case and the assistant will decide it for you silently if you do not.

Then write the two tests, still before prompting.

```
print(Polygon(6, 70).perimeter() == 420)
print(Polygon(0, 70).perimeter() == 0)
```

Then ask for the body, read every line of what comes back, and run both tests. A passing gentle test tells you the method is not catastrophically broken. Only the nasty test passing earns your belief, and if it fails then the interesting question is whether it was answering the question you actually asked, rather than whether it was wrong.

Two bugs, on purpose

The four steps from this week are read the traceback from the bottom, locate the failing line, form a hypothesis you could be wrong about, and confirm the hypothesis before changing anything. The temptation to skip to the last step and change something hopefully is strongest when the fix looks obvious, which is exactly when it is most likely to be a different bug wearing the same clothes.

Exercise 36-6

SOLO

This one stops. Run it, then work the four steps in order, out loud or on paper, before you touch the code.

```
from turtle_widget import Turtle

class Spiral:

    def __init__(self, turns, step):
        self.turns = turns
```

```

        self.step = step

    def draw(self, t):
        for i in range(self.turns):
            t.forward(self.step * i)
            t.left(self.angle)

t = Turtle()
t.speed(0)
Spiral(20, 6).draw(t)

```

Read the last line of the traceback first and say what it names. Then find the line it points at. Then state your hypothesis as a sentence that could turn out to be false, in the form of a claim about what is or is not true of the object at that moment. Then confirm it, by printing something, before you decide what the fix is. There is more than one defensible fix here, and which one is right depends on a decision about what a spiral is, which the error message cannot make for you.

Exercise 36-7

SOLO

This one does not stop. It runs without complaint and draws the wrong thing, which is the harder case because Python has nothing to say about it.

```

from turtle_widget import Turtle

def staircase(t, steps, size):
    "Draw a staircase going up and to the right."
    for _ in range(steps):
        t.forward(size)
        t.left(90)
        t.forward(size)
        t.left(90)

t = Turtle()
t.speed(0)
staircase(t, 5, 40)

```

There is no traceback to read, so the first step of the method is replaced by a different one: say exactly what you expected and what you got, in a way somebody else could check. Then halve the problem. Run it with `steps` set to one and look at what a single step of the staircase actually draws, because a bug that repeats five times is five copies of a bug that happens once, and it is always cheaper to find the one.

Exercise 36-8

SOLO

Confirm your hypothesis about the staircase by watching the state rather than the picture, which is the print-based version of what the codelens widget shows you.

```
from turtle_widget import Turtle

def staircase(t, steps, size):
    for i in range(steps):
        t.forward(size)
        t.left(90)
        t.forward(size)
        t.left(90)
        print(i, t.position(), t.heading())

t = Turtle()
t.speed(0)
staircase(t, 3, 40)
```

Read the printed heading after each step and compare it with the heading you expected. The picture and the numbers were always saying the same thing, but the numbers say it in a form you can be exact about, and being exact is what turns a hunch into a hypothesis. When you have found it, fix it, and delete the `print` line afterwards, because a debugging print left in finished code is a small lie about what the function is for.

Exercise 36-9

AI: DRAFTER

Take the traceback from the spiral exercise, paste it into the assistant along with the class, and ask it what is wrong. Then treat the answer as what it is, which is a hypothesis produced by something that has never run your code.

Do the third and fourth steps of the method yourself. Turn its answer into a sentence that could be false, and confirm it with the machine before you change anything. Then ask yourself the question that matters more than whether it was right: did its answer include the decision about what a spiral is, or did it quietly choose one for you and present the result as the fix?

Exercise 36-10

SOLO

Finish both bugs properly, by adding the check that would have caught each one. For the spiral, write a check that the object has the attribute the method expects, which you can do by drawing it and confirming the turtle moved at all. For the staircase, write a check on the turtle's heading after one step, with the value you decided was correct.


```
from turtle_widget import Turtle

t = Turtle(autoshow=False)
staircase(t, 1, 40)
print(round(t.heading(), 6) == 0.0)
print(round(t.xcor(), 6) == 40.0)
print(round(t.ycor(), 6) == 40.0)
```

The `autoshow=False` is there because this cell is not drawing anything for you to look at. It is asking questions, and it does not need a canvas to do it. That is a small thing now and it is the entire subject of next week, so it is worth noticing that you have just written a check on a drawing without producing the drawing.

Week 8

37 Reading and judging

For most of this course you have been the one writing the code. You typed it, you traced the substitutions and reductions in your head, and you knew what every line did because you had put it there. Over the last few weeks that changed a little: you began letting the assistant draft small pieces for you. This week the situation changes completely. From here on, most of the code you work with will be code you did not write, produced by a machine that is very good at making it look correct. The skill you need now is reading rather than writing, and more than reading, judging: deciding whether a piece of code actually does what it claims, and proving that decision rather than trusting it.

It helps to understand why code from a language model is untrustworthy in a particular way, because it is different from the way a beginner's own code is untrustworthy. When you make a mistake, the result usually looks like a mistake. You get an error, or the output is obviously wrong, or the code is clearly unfinished. A language model does not work this way. It has been trained on an enormous amount of human-written code, and what it produces is text that is statistically similar to correct code. It is a machine for generating plausible continuations. Plausible is not the same as correct. The model does not run the code, does not have a model of what the code will do when executed, and has no way of checking whether its own output is right. What it is excellent at is the surface: sensible function names, tidy structure, confident comments and docstrings that describe what the code is supposed to do. That surface is exactly the part a human reader skims over when they are hoping the code is right. So the failure mode is specific and dangerous: the assistant hands you code that reads beautifully, runs without error, and quietly returns the wrong answer.

Consider a concrete case. Suppose you ask the assistant for a function that returns the reverse complement of a DNA strand. Recall what that means: you read the strand backward and swap each base for its partner, A with T and G with C. Here is a version the assistant might give you. Read it before you run anything.

```
def reverse_complement(dna):  
    "Return the reverse complement of a DNA strand."  
    complement = {'A': 'T', 'T': 'A', 'G': 'C', 'C': 'G'}  
    return ''.join(complement[b] for b in dna)
```

The function has a sensible name. Its docstring states exactly the right task. The four base pairs in the dictionary are correct. Nothing about it looks wrong, and if you are reading quickly, the way people read something they want to be right, you will accept it and move on.

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter

Still to come: Worker,
Collaborator, Developer

Exercise 37-1

SOLO

Before running it, decide for yourself what each of the following should produce, and then what this particular function will actually produce, which may be a different thing:

```
print(reverse_complement('ATGC'))  
print(reverse_complement('AAAA'))  
print(reverse_complement('A'))
```

The reverse complement of ATGC is GCAT, but the function returns TACG. It swapped each base for its partner correctly and then failed to reverse the strand. It computes the complement, not the reverse complement. The name was wrong, the docstring was wrong, and the code ran without a single error to warn you. This is the whole lesson in one example. You read it, you predicted the result, you ran it, and the running machine settled the question that reading alone could not.

Notice carefully why the bug was so easy to miss, because the reason generalizes. The reverse complement of A is T, which this function returns correctly. The reverse complement of AAAA is TTTT, which it also returns correctly, because a strand of identical bases reads the same forward and backward. Any input that happens to be a single base, or a run of one base, or otherwise symmetric, passes straight through and tells you nothing. These are exactly the inputs a hopeful reader reaches for when checking, because they are easy to compute in your head. The inputs that would expose the bug are the awkward ones, like ATGC, where every base is different and the order matters. The general principle is that code you did not write is only as trustworthy as the nastiest input you have tried it on. A gentle input is not a test; it is a comfort. A test is an input chosen because it would break the code if the code were wrong.

Because you cannot spot these faults by staring harder, you need a routine: a short set of questions you ask of every piece of code the assistant produces, every time, so that your suspicion is organized rather than occasional. The first question is whether the code actually does what its name and its comment claim, since those are written by the same unreliable narrator and prove nothing on their own. The second is how it behaves on the awkward inputs: an empty string, a single element, lowercase letters where you expected uppercase, a character that should not be there at all. The third is whether it gets boundaries and order right, the classic off-by-one at the start or end of a range and the reversed-versus-not confusion you just saw. The fourth, and the one most specific to this course, is whether the biology is correct and not merely plausible: whether the base pairs, the codons, the reading frames and the rules of the domain are actually right. The fifth is whether the code could return a wrong answer silently instead of raising an error, which is the most dangerous case of all because nothing will ever draw your attention to it. The sixth is simply whether you can write a test for it, because if you cannot describe in code what the right answer would be, you do not yet understand the problem well enough to trust any solution to it. Run those six questions over the `reverse_complement` function and the very first one already fails: the claim, reverse complement, does not match the behavior, complement only. You did not have to be clever to catch it. You had to be suspicious in an organized way.

The fourth question, about the biology, deserves special attention, because a wrong fact hides far better than a wrong loop, and in a scientific context a wrong fact does more damage. Python will never tell you that a piece of biology is wrong. Consider another function the assistant might offer:

```
def complement(dna):  
    "Return the complementary DNA strand."  
    pairs = {'A': 'T', 'T': 'A', 'G': 'C', 'C': 'G'}  
    return ''.join(pairs[b] for b in dna)
```

The code here is flawless. It runs, it handles every base in its dictionary, and on a strand such as GC it returns a perfectly plausible looking CC. But C does not pair with C. It pairs with G. The bug is not in the code at all; it is in the biology encoded in the dictionary, and no error, no traceback, and no amount of testing that you did not write yourself will reveal it. Only you, holding the output up against what you know about base pairing, will catch it. This is what it means to treat the assistant as an unreliable narrator. It will state biological facts with exactly the same fluent confidence whether they are true or false, because it has no notion of which is which. In a course whose entire purpose is to let you produce scientific results with the help of a machine, the discipline of checking the domain facts, and not only the code, is the difference between a result you can stand behind and one you cannot.

Exercise 37-2

Go looking for a wrong fact rather than a wrong loop. Ask the assistant for a Python dictionary mapping all sixty-four codons to their amino acids, and then check the biology, not the code, against a real codon table from a textbook or a trusted database. Do not skim it; go through it. Two things usually happen. The dictionary is syntactically perfect and will never raise an error whatever is in it, and somewhere in sixty-four entries there is a good chance of at least one assignment you cannot confirm. Whether you find a mistake or not, write down the answer to this: if there had been one, which of your tests would have caught it? The honest answer is none of them, because a test checks that the code agrees with the table, and the table is the thing that was wrong.

Here is a case to work through yourself. The assistant wrote a function that is supposed to return the fraction of bases in a strand that are G or C, a quantity called the GC content:

```
def gc_content(dna):  
    gc = 0  
    for base in dna:  
        if base == 'G' or base == 'C':  
            gc += 1  
    return gc / len(dna)
```

Exercise 37-3

SOLO

Predict the value of `gc_content('GGCC')` and then the value of `gc_content('ggcc')`, and run both. Then decide which of the six questions this code fails, and, more importantly, why the failure is silent.

It does not crash on lowercase input; it simply counts none of the lowercase bases as G or C and returns a confident, wrong answer of zero. A researcher who fed this function a file of lowercase sequence would get a column of zeros and no indication whatever that anything had gone wrong. That silence is the danger, and the only defense against it is the habit of trying the awkward input on purpose.

Once you have found a fault, how you report it back to the assistant decides whether the next version is better or merely broken in a new place. A vague complaint gives the model nothing to work with, so it guesses, and a guess is as likely to introduce a fresh bug as to fix the old one. Telling the assistant that its code “doesn’t work” or asking it to “fix this” is close to useless. A specific report is far better, and a specific report is really just you writing down the judging you have already done. It should contain the exact call you made, the output you actually got, the output you expected instead, and, where you can see it, the reason. Instead of saying the function is broken, you say that `reverse_complement` of ATGC returned TACG but should have returned GCAT, that the bases were complemented correctly but the strand was not reversed, and that it should be reversed as well. An assistant handed the failing case and the reason usually repairs the function in a single step. An assistant told only that something is wrong writes you a different bug. Notice that the specific report costs you nothing extra, because you had to do all of that judging anyway in order to know the code was wrong; the report is simply the record of it.

Exercise 37-4

AI: DRAFTER

Prove that to yourself rather than taking it on trust. Take the broken `reverse_complement` from the start of this note, hand it back to the assistant with nothing but “this doesn’t work, fix it”, and keep whatever it returns without running it. Then start a *fresh* conversation, hand it the same broken function, and this time give it the specific report: the exact call, the output you got, the output you expected, and the reason as far as you can see it. Now run both replies against `reverse_complement('ATGC')`. Compare not just whether each one is right, but whether each one changed anything you did not ask it to change. The vague complaint is the one that tends to rewrite parts that were never broken.

You have been pinning these faults down by hand so that you can see the reasoning, but you already have a tool that does the pinning down without mercy and without hope, which is the project check widget you have been using. When you place a suspect function in a test cell, or run the checks against your file, every cross mark is a claim the assistant made that the machine has refused, and every tick is a claim that survived contact with a real input. That is the loop you will live inside for the

rest of the course and especially during the projects: the assistant drafts a piece of code, you read it against the six questions, and the tests deliver the verdict that neither you nor the assistant is allowed to argue with. The habit to build now is that you never accept code you cannot read and cannot test. If you cannot read it, you cannot judge it, and if you cannot test it, you cannot prove your judgment, so in either case you are simply trusting the unreliable narrator, which is the one thing this week exists to stop you doing.

Exercise 37-5

AI: DRAFTER

To make that habit concrete, try the full loop once here, on a small problem, before you meet it for real on the HIV subtyping project next week. Ask the assistant to write a `reverse_complement` function. Before you believe a word of it, do three things. Read the function against the six questions and decide what you think. Write three checks of your own, and make sure at least one of them is a properly nasty input such as `ATGC`, where the order of the bases matters, rather than a gentle one that would pass no matter what. Then run your checks. Only when your own tests pass are you permitted to believe the function is correct, and even then only for the cases your tests covered. When you have done this, write down in a single sentence what your nasty input was and why you chose it, because that instinct, the reflex for the input that would break the code if the code were wrong, is the entire skill of this week compressed into one habit.

Nothing you did this week was a new kind of program. What you learned was how to withhold trust from a program in an organized, provable way. The assistant is a fluent, confident, and unreliable narrator, both about code and about biology. The six questions are how you cross-examine it, and the running machine is the witness that cannot be talked around. This is the skill that makes the rest of the course safe, because it is by learning to distrust the assistant properly that you earn the right to let it do more.

For your logbook this week, paste one piece of code the assistant wrote for you, describe what your reading or your tests caught, whether that was a bug in the code, an error in the biology, or nothing at all, and record which awkward input revealed it. If the code turned out to be correct, describe instead how you convinced yourself of that, and what you tried that could have proved it wrong but did not.

38 Tests are the contract

You already know how to write a test. In the previous part of the course you learned to make a claim about a function in a form the machine can check, by writing a statement like `assert gc_content('GGCC') == 1.0` and letting Python confirm or deny it. This week that skill stops being a way to check your own code and becomes the single most important tool you have for working with an assistant. The idea to hold on to is short: a test is a specification you can execute, and a specification is a contract. Once you see tests that way, the whole business of getting code out of a machine you cannot fully trust becomes manageable, because the test, and not the assistant and not even you, becomes the thing that decides whether the code is right.

Start with the word specification, because it is doing quiet work. Before you can judge whether a piece of code is correct, you have to have decided what correct would mean, and that decision is separate from, and comes before, any code. If you ask an assistant for a function that finds genes, the request has no truth value at all. There is nothing to be right or wrong about, because you have not said what output the function should produce for what input. A specification is exactly that missing thing: a statement of what the result must be for given inputs. When it is written in prose, as a comment or a docstring, it is a promise about what the code will do. When it is written as a test, it is a promise the machine will enforce. That difference is everything. A comment that says the function returns the reverse complement is a promise the code can quietly break, and you saw last week how casually generated code breaks exactly such promises. A test that says `assert reverse_complement('ATGC') == 'GCAT'` is a promise that cannot be quietly broken, because the moment the code fails to keep it the machine says so.

This gives you a discipline, and the discipline is mostly about order. The natural way to work with an assistant is to ask for the code, receive it, glance at it, and hope. That order puts hope where proof should be. The disciplined order is the reverse: first you decide the specification by choosing concrete examples, then you write those examples as tests, and only then do you ask the assistant for the code, and finally you run the tests to see whether the code it produced actually satisfies the contract you wrote. Writing the tests before you see the assistant's code matters for a reason that is easy to underestimate. If you write the tests afterward, having already seen the plausible looking function the assistant produced, you will, without meaning to, write tests that the function happens to pass. The code will anchor your idea of what correct means, and you will end up checking that the code does what the code does, which proves nothing. Writing the tests first keeps the decision about what correct means where it belongs, with you, made before the assistant had any chance to bias it.

Turning a vague request into a specification is done by choosing examples and working out their answers by hand. Suppose you want the fraction of bases in a strand that are G or C, the GC content. You do not begin by prompting; you begin by writing down that `gc_content('GGCC')` should be 1.0, that `gc_content('ATAT')`

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · **Worker**

Still to come: Collaborator,
Developer

should be 0.0, and that `gc_content('GGAT')` should be 0.5, each of which you can compute yourself with certainty. Each such example is one point of the specification, and each becomes one line of your test. The strength of the contract you are writing is exactly the strength of the examples you choose. If every example is gentle, the contract is weak, and a wrong function can satisfy it while still being wrong, which is the happy path trap from last week wearing a different hat. A strong contract includes the awkward inputs on purpose: the empty string, a single base, lowercase letters, a character that should not appear. The nastier your examples, the stronger the promise the machine will hold the code to.

There is a further reason to write the tests first, which is that the act of writing them forces you to make decisions you would otherwise leave to the assistant to guess. The interesting decisions in a specification almost always live in the awkward cases, and the awkward cases are exactly the ones a fluent, confident assistant will silently decide for you, in whatever way its training happened to lean. Consider a function `is_gc_rich` that is supposed to say whether a strand is GC rich. What should it return for a strand that is exactly half G and C, like GGAT, whose GC content is exactly 0.5? Is a strand that is exactly half GC rich, or not? There is no universal answer; it is a decision, and it is your decision to make. If you write the test first, you are forced to decide it, and you write down `assert is_gc_rich('GGAT') == False`, or `True`, deliberately. If you do not, the assistant will pick one silently, using a greater than where you might have wanted a greater than or equal, and you will have no way of knowing whether its choice matches your intention, because you never formed an intention. The same thing happens with a function that returns the most common base in a strand. What should it return when two bases are tied, as G and C are in the strand GC? Run this and watch:

```
def most_common_base(dna):  
    return max(set(dna), key=dna.count)  
  
print(most_common_base('GC'))
```

Exercise 38-1

SOLO

Before you run that, write down which base you expect to come back, and the reason you expect it. Then run it. Now the real question, which is not about the answer at all: could you have predicted it from the code alone, without knowing how `set` happens to order its elements and how `max` happens to break a tie? If the answer is no, then the behaviour of this function on a tie is not something you decided; it is something that fell out. Write the single `assert` you would have to add to take that decision back, and notice that writing it forces you to make the decision you had been avoiding.

The answer you get back is essentially arbitrary, an accident of how `max` happens to break the tie over a set whose order is not something you should rely on. The point is not which base it returns. The point is that the tie is a real decision about what the function should mean, the assistant will make that decision for you and never mention it, and the only way to take the decision back is to write a test that states what you want to happen on a tie before any code exists. Writing tests first

is, in this sense, not really about testing at all. It is about doing the thinking, in advance, about the cases where thinking is actually required.

This is why it is worth calling a test a contract rather than merely a check. A contract says: I do not care how you do the job, but the result must satisfy these conditions. That is the relationship you want with an assistant that you cannot fully trust and whose inner workings you cannot inspect. You own the contract, you write it in a form the machine enforces, the assistant supplies an implementation, and the contract, not your hope and not the assistant's confidence, decides whether the implementation is acceptable. A useful consequence of this arrangement is that the implementation becomes replaceable. Because the contract is separate from any particular code, you can throw away the assistant's version and write your own, or accept a faster version later, or fix a bug and try again, and the same contract keeps guarding you the whole time. The tests outlive any one attempt at the code, which is exactly what you want when the code is coming from a source that gets things wrong.

You have already been on the receiving end of this idea without writing the contracts yourself. When you ran the check widget on a project, the tests had been written for you, and every tick and cross you saw was a clause of a contract being enforced against your code. This week you begin writing those contracts yourself, and you meet tests in three ways that will recur for the rest of the course. You will keep using the ready made contracts through the widget when you work on the projects. You will start writing your own assert statements to pin down functions you or the assistant have written. And you will run those tests directly with the `pytest` command in the terminal and read its report, which lists exactly which contracts held and which were broken, so that you are no longer dependent on a friendly widget to tell you and can read the raw verdict for yourself.

Put the whole discipline together on a small problem before you rely on it. Here is a specification for `gc_content` written as a contract, before any implementation exists, with the awkward cases decided on purpose:

```
assert gc_content('GGCC') == 1.0
assert gc_content('ATAT') == 0.0
assert gc_content('GGAT') == 0.5
assert gc_content('ggcc') == 1.0      # lowercase counts too: a deliberate decision
```

Only now do you ask the assistant for the function. If it hands you a version that walks through the strand counting bases equal to the capital letters G and C, that version will satisfy the first three clauses and fail the fourth, and the failure is not a nuisance; it is the contract doing its job, catching a decision the assistant made silently that does not match the decision you made deliberately. Without the fourth clause you would never have known the two of you disagreed.

Exercise 38-2

SOLO

Take a vague request and turn it into a contract. Choose a small function you might ask an assistant to write, such as one that counts how many times a given codon

appears in a longer strand, and before writing or requesting any code, write down three concrete examples with the answers worked out by hand, then turn each into an `assert` statement. Make sure at least one of your three examples is an awkward one, such as an overlapping occurrence or a codon that does not appear at all, and be ready to say what decision that awkward example forces. Keep this contract; the next three exercises all use it.

Exercise 38-3

SOLO

Now with no assistant at all, so that the mechanics are firmly yours. Take a function you have already written earlier in the course and write two tests for it: one that you are confident it passes, and one that you expect it to fail, perhaps because it exposes a case the function never handled. Run both. The point is to feel the difference between a test that confirms and a test that accuses, because a contract that only ever confirms is not protecting you from anything.

Exercise 38-4

AI: WORKER

Run the full discipline in order, on the contract you wrote two exercises ago. You have the specification already, so now, and only now, ask the assistant to write the function. Give it the purpose and the signature. Do **not** give it your tests. Run your tests against what it produces. If they all pass, you have earned a limited and specific belief in the code, limited to the cases you actually specified. If one fails, notice that you found the disagreement before the code ever ran in anger, which is the entire reason for writing the contract first.

Exercise 38-5

AI: WORKER

Now aim the contract at a decision rather than at a bug. Write a contract for `is_gc_rich(dna)`, a function that says whether a strand is GC rich, and make it decide two things the request itself leaves open: what happens at exactly one half, as in `'GGAT'`, and what happens for the empty strand. Decide both deliberately and write an `assert` for each. Then ask the assistant for the function using only the prose description, saying nothing about either case. Run your contract. Whatever it guessed, it guessed silently, and the only reason you can now tell whether the two of you agree is that you decided first. Record which of your two decisions it got wrong, if either.

Exercise 38-6

AI: WORKER

Last, watch the contract outlive the code. Take the same specification and ask the assistant for a *second*, deliberately different implementation of the same function, perhaps by asking for one that avoids a loop, or one that a beginner would find easier to read. Run your unchanged tests against it. Two different pieces of code, one contract, the same verdict. That is what it means to say the implementation is replaceable and the specification is not, and it is why the tests are worth more than any particular function you will ever get out of a machine.

For your logbook this week, record one specification you wrote as tests before requesting any code, note in particular which awkward case you had to make a deliberate decision about, and say whether the assistant's eventual code agreed with your decision or quietly disagreed with it. If it agreed on everything, add the one further awkward case you wish you had thought to specify.

39 Pairwise alignment

This chapter is about global pairwise alignment, and you will implement your own Needleman-Wunch algorithm.

Your task is to find an optimal alignment of two sequences. If two such sequences are roughly 140 bases long, there are as many different ways to align them as there are atoms in the visible universe. Finding an optimal alignment among those 10^{80} possibilities is a hard problem, but implementing the Needleman-Wunch algorithm will let you do it.

This project is meant to train your coding abilities and consolidate your understanding of the Needleman-Wunch algorithm. The better you understand the algorithm before you begin, the easier and more rewarding the project will be. So, re-read the book chapter about pairwise alignment and browse the lecture slides.

Download the files you need for this project:

https://munch-group.org/bioinformatics/supplementary/project_files_alignmentproject.py is the file where you must write your code. It already contains a function I wrote for you. - [test_alignmentproject.py](#) is the test program that lets you test the code you write in [alignmentproject.py](#).

Put the files in a folder dedicated to this project. On most computers, you can right-click on the link and choose “Save the file as...” or “Download linked file.”

The project is split into two parts:

1. Filling in a dynamic programming matrix.
2. Reconstructing the optimal alignment by doing traceback through the dynamic programming matrix.

To help you along, the [alignmentproject.py](#) file already contains a function I wrote for you, so you can print your dynamic programming (DP) matrix as you gradually fill it in. You are not expected to understand how this function works. I made it as condensed as possible so it does not take up so much space in your file.

The function is named [print_dp_matrix](#) and takes two arguments:

1. A string, which represents a DNA sequence of some length [m](#).
2. A string, which represents a DNA sequence of some length [n](#).
3. A lists of lists of integers, representing a dynamic programming matrix.

The function returns:

- None, but it *prints* a nice representation of the matrix lined with the bases of the two sequences.

Filling in the dynamic programming matrix

Make a matrix

We start by making a list of lists (a matrix) with the right shape but only holds `None` values. We use the `None` values as placeholders, which you can later replace with scores. You can think of it as an empty matrix into which you can fill scores, just as we did in the lectures. If you want to align two sequences like `AT` and `GAT`, you want a matrix with three rows and four columns. Note that the matrix must have one more row than the number of bases in sequence one and one more column than the number of bases in sequence two.

Write a function, `empty_matrix`, that takes two arguments

1. An integer (which represents the length of sequence one + 1).
2. An integer (which represents the length of sequence two + 1).

The function must return:

- A list of lists. The number of sub-lists must be equal to the first integer argument. Each sublist must contain a number of `None` values equal to the second integer argument.

Example usage:

```
empty_matrix(3, 4)
```

returns a list with *three* lists each of length *four*:

```
[[None, None, None, None], [None, None, None, None], [None, None, None, None]]
```

Even though this is a list of lists, we can *think of it* as a three-by-four matrix:

```
[[None, None, None, None],  
 [None, None, None, None],  
 [None, None, None, None]]
```

If you want to print the matrix in a way that looks like the slides I showed you at the lecture, you can use the `print_dp_matrix` function (again, `None` represents empty cells):

```
      G   A   T  
None None None None  
A None None None None  
T None None None None
```

Important: You can implement `empty_matrix` in a way that superficially looks ok but will cause you all kinds of grief when you start filling it in. When you create the list of lists (e.g., three as above), you must generate and add three *separate* lists. If you add the *same* list three times, you do not have three separate rows in your matrix. Instead, you have *three* references to the *same* row. You can test if you did it right this way – by changing the value of one cell to see what happens:

```
empty = empty_matrix(3, 4)
empty[0][0] = 'Mogens'
print(empty)
```

If this only changed *one* value like below, you are ok:

```
[['Mogens', None, None, None], [None, None, None, None], [None, None, None, None]]
```

If it changed the first value in *all* the lists, it means that all your lists are the same (which is not what you want).

```
[['Mogens', None, None, None], ['Mogens', None, None, None], ['Mogens', None, None, None]]
```

Fill the top row and left column

Now that you can make a matrix with the correct dimensions, you need to write a function that fills in the top row and the left column with multiples of the gap score. E.g., if the gap score is `-2`, you want the matrix to look something like this when you print it with `print_dp_matrix`:

		G	A	T
	0	-2	-4	-6
A	-2	None	None	None
T	-4	None	None	None

Write a function, `prepare_matrix`, which takes three arguments:

1. An integer (which represents the length of sequence one plus one)
2. An integer (which represents the length of sequence two plus one)
3. An integer, which represents the gap_score used for alignment.

The function must return:

- A list of lists. The number of sub-lists must be equal to the first integer argument. The values in the first sub-list must be multiples of the gap score given as the third argument. The first elements of the remaining sub-lists must be multiples of the gap score. All remaining elements of sub-lists must be `None`.

Example usage:

```
prepare_matrix(3, 4, -2)
```

must return:

```
[[0, -2, -4, -6], [-2, None, None, None], [-4, None, None, None]]
```

Hint

You should call `empty_matrix` inside `prepare_matrix` to get a matrix filled with `None`.

Now, all you need to do is replace the right `None` values with *multiples* of the gap score. For example, the third element in the first sub-list is `matrix[2][0]`, for which you would need to assign the value `2` times the gap score. In the same way `matrix[3][0]` should be `3` times the gap score. So, you need to figure out which elements you should replace and which pairs of indexes you need to access those elements. Then, use `range` to generate those indexes and for-loops to loop them over.

Fill the entire matrix

Now that we can fill the top row and left column, we can start thinking about how to fill the whole matrix.

For that, we need a score matrix of match scores. In Python, that is most easily represented as a dictionary of dictionaries like this:

```
score_matrix = {'A': {'A': 2, 'T': 0, 'G': 0, 'C': 0},
                 'T': {'A': 0, 'T': 2, 'G': 0, 'C': 0},
                 'G': {'A': 0, 'T': 0, 'G': 2, 'C': 0},
                 'C': {'A': 0, 'T': 0, 'G': 0, 'C': 2}}
```

That lets you get the score for matching an `A` with a `T` like this: `score_matrix['A']['T']`. Note that the match scores are only for uppercase letters (`A`, `T`, `G`, `C`).

Write a function, `fill_matrix`, which takes four arguments:

1. A string, which represents the first sequence.
2. A string, which represents the second sequence.
3. A dictionary of dictionaries like the one shown above, which represents match scores.
4. An integer, which represents the gap score.

The function must return:

- A list of lists of integers, which represents a correctly filled dynamic programming matrix given the two sequences, the match scores, and the gap score.

Example usage: If `score_matrix` is defined as above, then

```
fill_matrix('AT', 'GAT', score_matrix, -2)
```

must return:

```
[[0, -2, -4, -6], [-2, 0, 0, -2], [-4, -2, 0, 2]]
```

If you print that matrix using `print_dp_matrix` it should look like this:

```
      G  A  T
0  -2 -4 -6
A -2  0  0 -2
T -4 -2  0  2
```

Hint

You should call `prepare_matrix` inside `fill_matrix` to get a matrix with the top row and left column filled. Assuming `seq1` is sequence one and `seq2` is sequence two, then you can do it like this:

```
matrix = prepare_matrix(len(seq1)+1, len(seq2)+1, gap_score)
```

Now, you only need to fill out the rest. You need two nested for-loops to produce the indexes of the elements in the list of lists that you need to assign values to.

```
for i in range(1, len(seq1)+1):
    for j in range(1, len(seq2)+1):
        print(i, j) # just to see what i and j are
```

Examine this code and make sure you understand why we give those arguments to `range`. Each combination of `i` and `j` lets you access an element `matrix[i][j]` in `matrix` (list of lists) that you can assign a value to. The value to assign to `matrix[i][j]` (green cell on the slides) is the maximum of three values (the yellow cells on the slide):

1. The value of the cell to the left (`matrix[i][j-1]`) plus the gap score.
2. The cell above (`matrix[i-1][j]`) plus the gap score.
3. The diagonal cell (`matrix[i-1][j-1]`) plus the match score for base number `i` (index `i-1`) of sequence one and base number `j` (index `j-1`) of sequence two.

Reconstructing the optimal alignment

This is the most challenging part, so I will hold your hand here. Below is first a function that identifies which of three cells (the yellow cells on the slides) some cell (green cell on the slides) is derived from. On the slides, this is the cell pointed to by the red arrow.

```
def get_traceback_arrow(matrix, row, col, match_score, gap_score):

    # yellow cells:
    score_diagonal = matrix[row-1][col-1]
    score_left = matrix[row][col-1]
    score_up = matrix[row-1][col]

    # gree cell:
    score_current = matrix[row][col]

    if score_current == score_diagonal + match_score:
        return 'diagonal'
    elif score_current == score_left + gap_score:
        return 'left'
    elif score_current == score_up + gap_score:
        return 'up'
```

Write (do not copy and paste) this into your file, and make sure that it works and that you understand exactly how it works before you proceed.

Here is a function that uses `get_traceback_arrow` to do the traceback. It reconstructs the alignment starting from the last column, adding columns in front as the traceback proceeds. It is big, so it breaks across three pages.

```
def trace_back(seq1, seq2, matrix, score_matrix, gap_score):

    # Strings to store the growing alignment strings:
    aligned1 = ''
    aligned2 = ''

    # The row and col index of the bottom right cell:
    row = len(seq1)
    col = len(seq2)

    # Keep stepping backward through the matrix untill
    # we get to the top row or the left col:
    while row > 0 and col > 0:

        # The two bases we available to match:
        base1 = seq1[row-1]
        base2 = seq2[col-1]

        # The score for mathing those two bases:
```

```

match_score = score_matrix[base1][base2]

# Find out which cell the score in the current cell was derived from:
traceback_arrow = get_traceback_arrow(matrix, row, col, match_score, gap_score)

if traceback_arrow == 'diagonal':
    # last column of the sub alignment is base1 over base2:
    aligned1 = base1 + aligned1
    aligned2 = base2 + aligned2
    # next cell is the diagonal cell:
    row -= 1
    col -= 1
elif traceback_arrow == 'up':
    # last column in the sub alignment is base1 over a gap:
    aligned1 = base1 + aligned1
    aligned2 = '-' + aligned2
    # next cell is the cell above:
    row -= 1
elif traceback_arrow == 'left':
    # last column in the sub alignment is a gap over base2:
    aligned1 = '-' + aligned1
    aligned2 = base2 + aligned2
    # next cell is the cell to the left:
    col -= 1

# If row is not zero, step along the top row to the top left cell:
while row > 0:
    base1 = seq1[row-1]
    aligned1 = base1 + aligned1
    aligned2 = '-' + aligned2
    row -= 1

# If col is not zero, step upward in the left col to the top left cell:
while col > 0:
    base2 = seq2[col-1]
    aligned1 = '-' + aligned1
    aligned2 = base2 + aligned2
    col -= 1

return [aligned1, aligned2]

```

Once you have *written* it into your file, make sure you understand the correspondence to the sequences of events on the lecture slides.

Now you can write a function that performs the alignment. You get to do that yourself. It just calls `fill_matrix` and then `trace_back` to get the optimal alignment

Write a function, `align`, that takes four arguments:

1. A string, which represents sequence one.

2. A string, which represents sequence two.
3. A dictionary of dictionaries, which represents the match scores (as described above).
4. An integer, which represents the gap score.

The function must return:

- A list of length two. The first element of that list must be a string representing the aligned sequence one. The second element must be a string, representing the aligned sequence two.

Example usage:

```
align('ATAT', 'GATGAT', score_matrix, -2)
```

must return:

```
['-AT-AT', 'GATGAT']
```

Once you have written that function, you can print your alignment like this:

```
alignment = align('ATAT', 'GATGAT', score_matrix, -2)
for s in alignment:
    print(s)
```

40 Turtle, week 8: retiring the turtle

This is the last turtle session, and it ends with the turtle being put away deliberately rather than quietly dropped. Say that out loud, because the reason for retiring it is the lesson. For five weeks the picture has been your evidence. You wrote something, you looked at what appeared, and if it was what you meant then you moved on. This week you are going to find out exactly how far that gets you, and the answer is: further than you would guess on the first try, and nothing like far enough on the fiftieth.

The two AI notes this week are the other half of the argument. In one of them the assistant is an unreliable narrator, producing code and explanations that read beautifully and are sometimes wrong, and your job is to work out how you would know. In the other, tests are the contract, which is to say that the thing you write down before the code is what settles whether the code is right, and not your impression of the output afterwards. The alignment project is where both get used on work that counts.

The turtle earned its place in this course because looking is cheap and immediate. It has to be put away for exactly the same reason. When the artefact is a picture, looking is a direct check on the thing itself. When the artefact is an alignment of two sequences, a table of counts, or a file of gene annotations, looking is a check on a formatted report about the thing, done for one case, by an eye that already knows what it expects to see. Everything below is about the difference.

The two ways looking fails

Looking at output fails in two quite different ways, and it matters which one you have got, because they have different remedies.

The first is sampling. You looked at one run. The code will be run many times, on inputs you have not thought of, and the one you looked at was chosen by you, which means it was chosen by the same understanding that wrote the code. If your understanding has a hole in it, the case that falls in the hole is exactly the case you will not think to try. The remedy for a sampling failure is more cases, chosen adversarially rather than conveniently, which is what a nasty test is.

The second is observability. You looked at the whole output and the output does not contain the fact you needed. No amount of extra cases helps here, because every one of them will be equally silent. The remedy is a different question, asked of a different thing: the state behind the picture rather than the picture. That is what `t.heading()`, `t.total_movement` and `t.isdown()` have been for all along.

The two get confused constantly, so you should be able to say which one you are looking at. A one-case test suite fails in exactly the way one glance fails, and dressing it up as a test does not fix it. A test that asks the wrong question about a hundred cases fails in exactly the way a hundred glances would.

Exercise 40-1

SOLO

A sampling failure, in the simplest form it comes in. Run this with `n` set to four, then five, then six, then eight, then ten.

```
from turtle_widget import Turtle

n = 5
t = Turtle()
t.speed(0)
for _ in range(n):
    t.forward(80)
    t.right(360 // n)
```

Every one of those draws a shape that closes perfectly, and after five of them you would be entitled to conclude that the function is correct. Now run it with `n` set to seven and look at what appears. Then work out what `360 // 7` is, and say in one sentence why the five cases you tried could never have told you.

The values you tried were not chosen carelessly. They are the obvious ones, the ones a reasonable person picks, and that is exactly the problem, because the code was written by the same reasonable person and divides evenly in the same places.

Exercise 40-2

SOLO

Now catch the same fault without looking at anything. This is the week `assert` becomes available to you, and it is the same claim you have been printing since week four with a different consequence when it is false.

```
from turtle_widget import Turtle

def polygon(t, n, size):
    for _ in range(n):
        t.forward(size)
        t.right(360 // n)

for n in [3, 4, 5, 6, 7, 8, 9, 10, 11, 12]:
    t = Turtle(autoshow=False)
    polygon(t, n, 80)
    assert round(t.heading(), 6) == 0.0, 'fails to close for n = ' + str(n)
```

A `print` of a comparison invites you to skim a column of `True` and miss the one `False` in the middle of it. An `assert` stops the program at the first false claim and tells you which one it was, which is the whole difference, and it is why tests are written with `assert` once you have more than about three of them. Note the

message after the comma. Without it you are told that something failed and not which thing, and with ten cases in a loop that is half the information you need.

Notice also what this cell did not do. It drew nothing, looked at nothing, and found in a fraction of a second a fault that would have taken you ten deliberate runs and ten deliberate looks.

Exercise 40-3

SOLO

An observability failure, which no number of extra cases would have caught. Run this and look hard at the star.

```
from turtle_widget import Turtle

def star(t, size):
    for _ in range(5):
        t.forward(size)
        t.right(144)

t = Turtle()
t.speed(0)
star(t, 120)
```

It is a perfect five-pointed star. It closes, the points are even, the turtle is back where it began. Write down, before the next exercise, whether you believe the function is correct, and try to say what you are actually claiming when you say that.

Exercise 40-4

SOLO

Now ask the star the question the picture could not answer, and then draw the consequence.

```
from turtle_widget import Turtle

t = Turtle(autoshow=False)
before = t.heading()
star(t, 120)
print(before, t.heading())
```

Five turns of one hundred and forty-four degrees is seven hundred and twenty degrees, which is two full circles, and yet the heading has moved. Work out why on paper before you accept it, and then confirm your explanation by printing the heading after each of the five repetitions.

Then make the fault visible in the only way it ever becomes visible, by using the function more than once.

```

from turtle_widget import Turtle

t = Turtle()
t.speed(0)
for _ in range(10):
    star(t, 120)

```

Ten stars in a fan, each one tilted a little further than the last. The bug was in the function from the first line you ran, and the first star showed you nothing at all, because the function's fault is in what it leaves behind and a single call leaves you no way to see what was left. Fix `star` so that it puts the heading back, and then write the `assert` that would have caught it on the very first run.

Exercise 40-5

SOLO

The same shape of fault, in the two other places the turtle keeps state that the canvas cannot show. Both of these draw something you would accept without hesitation.

```

from turtle_widget import Turtle

def side_a(t, size):
    t.forward(size - 1)

def side_b(t, size):
    t.penup()
    t.forward(size)

```

Write a cell that calls `side_a` four times with a left turn between them, and another that does the same with `side_b`, and look at both. The first draws a square that is one unit short on each side, which at a size of one hundred and twenty you will not see and could not measure by eye. The second draws nothing whatsoever, which you will certainly notice, so change it to put the pen down again at the end and leave the fault in place somewhere less obvious.

Then check both without looking, using the two facts the turtle keeps for you.

```

from turtle_widget import Turtle

t = Turtle(autoshow=False)
for _ in range(4):
    side_a(t, 120)
    t.left(90)

assert t.total_movement == 480
assert t.isdown()

```


The first assertion fails by four units out of four hundred and eighty, which is under one per cent, and no eye has ever caught a one per cent error in a line length. Now imagine the same function inside a loop of fifty, where the error is two hundred units and the drawing is visibly ruined, and notice that the check you just wrote would have told you at four.

Exercise 40-6

Give the assistant the original `star` function, the one that leaves the turtle rotated, and ask it a question with an assumption buried in it: ask it to explain why this function returns the turtle to its starting state.

Read what comes back carefully. There is a decent chance it will explain, fluently and at length, why the turtle ends as it began, because you told it that it does and it is obliging. This is the failure mode from this week's AI note in its purest form, and it is a failure of arbitration rather than of knowledge, because nothing in the conversation was ever going to contradict it.

Then run the check that does contradict it, paste the actual heading back into the conversation, and watch the explanation change. Record the whole exchange in your logbook, including what you asked and what you got, because this is the clearest example you are likely to produce all term of an answer being shaped by the question rather than by the code.

Looking is a filter, not a verdict

There is an asymmetry in all of this that is easy to miss and needs stating plainly. Looking is very good at finding faults and very bad at confirming correctness. A glance at the seven-sided polygon told you instantly that something was wrong, and it took no setup, no thought about what to assert, and no decision about how close counts as equal. That is valuable, and it is why you should still look.

What a glance cannot do is the opposite job. When the picture looks right, you have learned that this run, on these arguments, produced output containing no fault large enough and visible enough for you to notice. That is a much smaller claim than it feels like from the inside, and the feeling of having checked is the dangerous part, because it is indistinguishable from the feeling of having checked properly.

So the working position for the rest of the course is that looking is the cheap first filter and never the last word. You look because it costs nothing and sometimes saves you an hour. You assert because looking cannot tell you the thing you actually needed to know.

Exercise 40-7

SOLO

Here are two functions that draw a square, and this is the exercise the whole session has been building towards. Run them both and compare the pictures pixel by pixel if you like.

```
from turtle_widget import Turtle

def square_a(t, size):
    for _ in range(4):
        t.forward(size)
        t.right(90)

def square_b(t, size):
    t.goto(size, 0)
    t.goto(size, -size)
    t.goto(0, -size)
    t.goto(0, 0)
```

Run each on its own turtle from the starting position and satisfy yourself that the two pictures are identical, because they are. Now run each again on a turtle you have first moved somewhere else with `move_to(t, -100, 80)`, and look at what happens.

The first function draws a square wherever the turtle happens to be. The second draws a square in one fixed place and gets there by dragging a line across the canvas from wherever the turtle was. The two are not remotely the same function, and the evidence you had was identical, and would have stayed identical for as long as you kept starting at the origin.

Exercise 40-8

SOLO

Stage the same comparison with two pens so the moment of divergence is visible rather than inferred.

```
from turtle_widget import Turtle

red = Turtle()
red.speed(0)
blue = red.new_turtle(color='blue')
blue.speed(0)
red.pencolor('red')

square_a(red, 100)
square_b(blue, 100)
```

The two pens lie exactly on top of each other for the whole drawing, and the canvas is telling you the truth about what happened. Now do it again with both turtles first moved to the same point away from the origin, and watch where they part company.

Then state what the first version of this cell actually established, in one careful sentence, and count how many words of qualification you needed. Every one of those qualifications is a condition you would have to hold constant for the evidence to keep being evidence, and holding conditions constant is not something you can promise about code you have written into a project.

Exercise 40-9

AI: DRAFTER

Write the contract first, then get the code, in that order and with no negotiation about the order.

Specify a function `arc(t, radius, degrees)` that draws part of a circle and, when given three hundred and sixty degrees, closes it. Write down before you prompt what it should leave behind: where the turtle should be and which way it should face when the call returns. Then write the assertions.

```
from turtle_widget import Turtle

t = Turtle(autoshow=False)
before = t.position()
facing = t.heading()
arc(t, 60, 360)
assert round(t.xcor(), 3) == round(before[0], 3)
assert round(t.ycor(), 3) == round(before[1], 3)
assert round(t.heading(), 3) == round(facing, 3)

t = Turtle(autoshow=False)
arc(t, 60, 90)
assert round(t.heading(), 3) == 90.0
```

Now prompt for the body, read every line, and run all four assertions. The first three are the gentle test, in the sense that any reasonable implementation passes them. The fourth is the nasty one, because a partial arc is where implementations differ, and it is also the one you would never have thought to look at, since a quarter circle drawn slightly wrong looks exactly like a quarter circle.

If the assistant's version fails on the fourth, the interesting question is not whether it made a mistake. It is whether your specification said which way the turtle should face after a partial arc, or whether you left that for somebody else to decide and are now unhappy with the decision.

Exercise 40-10

SOLO

Finish by deciding, for each of five claims, what would actually settle it. Write the check for each one, run it, and for any claim where you conclude that looking really is sufficient, say exactly why.

The claims are these. That a function draws a hexagon rather than a pentagon. That a function draws a hexagon of exactly the size it was asked for. That a function can be called twice in a row without the second drawing coming out crooked. That a function works when the turtle does not start at the origin. That a function leaves the pen down.

One of the five really is settled by looking, because the artefact is the picture, you can see the whole of it at once, and the property is the sort of thing an eye is good at, which is counting six of something. The other four are all about parts of the state the picture never contained, or about runs you did not make. Sort them, write the four assertions, and put the sorted list in your logbook, because from next week the artefacts stop being pictures and every single claim you make will be in the second category.

Week 9

41 The AI invents functions; the docs are the test

You have just met BioPython, and the note on reading it as a catalogue of contracts mentioned, in passing, the assistant's weakest habit around libraries, which is that it sometimes invents a method that sounds exactly right and does not exist. Here is that failure in the flesh, so that you recognise it the first time it happens to you for real, and so that you know what to reach for when it does.

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · **Worker**

Still to come: Collaborator,
Developer

A method that does not exist

Ask an assistant for the GC content of a BioPython `Seq` object and you may well get back something like this.

```
from Bio.Seq import Seq

dna = Seq('ATGGCCTGA')
print(dna.gc_content())
```

Read that before you run it. It looks completely unremarkable. The method name matches the concept exactly, it is called the same way `.translate()` and `.reverse_complement()` were in the last note, and it is called on the same kind of object. It has every surface feature of a correct call. Run it anyway.

```
AttributeError: 'Seq' object has no attribute 'gc_content'
```

There is no `.gc_content()` method on a `Seq` object. The assistant did not look this up. It predicted a plausible next method name given everything it has seen written about sequences and GC content, in the same way it predicts a plausible next word, and `gc_content` was a very good guess by that measure. It is simply not the guess BioPython's actual authors made. The real function lives elsewhere, as `Bio.SeqUtils.gc_fraction`, called on the sequence rather than as a method of it.

```
from Bio.SeqUtils import gc_fraction

print(gc_fraction(dna))
```

What actually caught it

Notice exactly what caught this, because it generalises to every library and every interface you will ever touch. It was not that the code looked wrong, because it looked fine right up until it ran. It was not a hunch either. It was the documentation, checked, which had no entry for `Seq.gc_content` and did have an entry for `SeqUtils.gc_fraction`, with a stated contract that matched what you actually wanted.

The real documentation is the test an invented call fails, exactly the way a hand-written `assert` is a test a wrong function fails. It is the only thing that reliably catches this particular failure, because reading the call a second time, more carefully, will not help. It was written to look right.

The same failure outside libraries

The same failure shows up with web services, not just libraries. An assistant will confidently name an endpoint, a query parameter, or a field in a returned record that the actual service does not offer, with exactly the same fluent certainty whether the field exists or not.

The habit that catches it is identical. Before you trust a call the assistant produced against a library or a service you did not write, find that exact function or endpoint in the real documentation and confirm three things, which are that it exists, that it takes what the assistant assumed it takes, and that it returns what the assistant assumed it returns. If you cannot find it, it very likely is not real, no matter how natural it looked.

Exercise 41-1

AI: WORKER

Ask the assistant for a one-line way to reverse-translate a protein string back into a possible DNA coding sequence using BioPython. Before running anything, search the real BioPython documentation for whatever function or method it names. Does it exist, under that name, taking those arguments? Record what you found either way, because a call that turns out to be real is as much a result as one that turns out to be invented.

For your logbook this week, note whether you have caught the assistant inventing a function or parameter yet, in this course or elsewhere, and if not, say specifically what you now intend to check before you next call a library method you did not write yourself.

42 Codon usage

Codon usage bias refers to differences in the frequency of occurrence of synonymous codons in coding DNA. There are 64 different codons (61 codons encoding for amino acids plus 3 stop codons) but only 20 different translated amino acids. The overabundance in the number of codons allows many amino acids to be encoded by more than one codon. Because of such redundancy, it is said that the genetic code is degenerate. Different organisms often show particular preferences for one of the several codons that encode the same amino acid, that is, a greater frequency of one will be found than expected by chance. How such preferences arise is a much-debated area of molecular evolution.

Given an open reading frame (ORF), you must compute the codon usage. Your goal is to create a data structure where you can look up an amino acid and the frequencies of codons in that encode that amino acid the ORF. Here is a made-up example:

```
{ 'A': { 'GCA': 0.0, 'GCC': 0.0, 'GCT': 1.0, 'GCG': 0.0 },
  'C': { 'TGC': 0.0, 'TGT': 1.0 },
  'E': { 'GAG': 0.2, 'GAA': 0.8 },
  'D': { 'GAT': 1.0, 'GAC': 0.0 },
  'G': { 'GGT': 0.3, 'GGG': 0.0, 'GGA': 0.7, 'GGC': 0.0 },
  'F': { 'TTC': 0.0, 'TTT': 1.0 },
  'I': { 'ATT': 1.0, 'ATC': 0.0, 'ATA': 0.0 },
  'H': { 'CAC': 0.0, 'CAT': 1.0 },
  'K': { 'AAG': 0.2, 'AAA': 0.8 },
  '*': { 'TAG': 0.0, 'TGA': 1.0, 'TAA': 0.0 }, 'M': { 'ATG': 1.0 },
  'L': { 'CTT': 0.0, 'CTG': 0.6, 'CTA': 0.0, 'CTC': 0.0, 'TTA': 0.4, 'TTG': 0.0 },
  'N': { 'AAT': 0.5, 'AAC': 0.5 },
  'Q': { 'CAA': 0.6, 'CAG': 0.4 },
  'P': { 'CCT': 0.5, 'CCG': 0.0, 'CCA': 0.5, 'CCC': 0.0 },
  'S': { 'TCT': 0.0, 'AGC': 0.0, 'TCG': 0.0, 'AGT': 0.5, 'TCC': 0.0, 'TCA': 0.5 },
  'R': { 'CGA': 0.5, 'CGC': 0.0, 'AGA': 0.5, 'AGG': 0.0, 'CGG': 0.0, 'CGT': 0.0 },
  'T': { 'ACC': 0.0, 'ACA': 0.0, 'ACG': 0.0, 'ACT': 1.0 },
  'W': { 'TGG': 1.0 },
  'V': { 'GTA': 0.6, 'GTC': 0.0, 'GTT': 0.2, 'GTG': 0.2 },
  'Y': { 'TAT': 1.0, 'TAC': 0.0 }}
```

That data structure is a dictionary with keys corresponding to amino acids (i.e. single letter strings designating an amino acid such as **'R'** for arginine). The value associated with *each* amino acid key is also a dictionary, and the keys of *this* dictionary should be the different codons that encode the amino acid. The value associated with each codon key should be a number, representing the frequency with which that codon is used to encode that amino acid. The final data structure should only include amino acids that are found in the ORF.

Download the files you need for this project:

https://munch-group.org/bioinformatics/supplementary/project_files

- `sample_orfs.txt` contains open reading frame sequences you can work on.
- `codonbiasproject.py` is an empty file where you must write your code.
- `test_codonbiasproject.py` is the test program that lets you test the code you write in `codonbiasproject.py`.

Now open each file in *VScode* and have a look at what is in `sample_orfs.txt`. (Do *not* change it in any way and do not save it after viewing. If VScode asks you if you want to save it before closing, say *no*.) How many sequences are there in each file?

As in the translation project, you will need a data structure that pairs each codon to the amino acid it encodes. This is an obvious use of a dictionary and at the top of `codonbiasproject.py` I have defined such a dictionary you can use. Defining it outside the functions means that it is visible inside all your functions (unless you define *another* variable called `codon_map` inside a function). Defining variables globally like this sometimes make sense if some value can be considered a *constant* in your program and is *never* changed.

“ It is normally very bad programming style to access variables outside functions in this way because it may have all kinds of unexpected side effects across function calls. So make it a rule for yourself that code *inside* a function should never to access variables *outside* the function. The reason we define `codon_map` globally in this project is to help you understand that when Python cannot find a variable inside a function, it looks outside the function to find it. In this project, functions will find `codon_map` in this way. However, as I already said, you should *never* do this yourself. The chance that you make an unexpected mistake is overwhelming. ”

The project is split into four parts:

1. Read an open reading frame (ORF) into your script and count the codons.
2. Group the codon counts by the amino acids the codons encode.
3. Convert counts to frequencies.
4. Build the data structure representing the codon bias information.

Read an open reading frame and count its codons

Read ORFs from a file

You can use this code to read the ORFs into your script:

```
f = open('sample_orfs.txt', 'r')
orf_list = list()
for line in f:
    seq = line.strip()
```

```
    orf_list.append(seq)
f.close()
```

Try to print the list to see it. Then pick out the first ORF in the list so you can use that to test your code:

```
test_orf = orf_list[0]
```

Split the ORF into codons

You need a function that splits the ORF into codons. This one you have already implemented in the exercise about translating DNA – and if, not here it is in my version to get you started.

```
def split_codons(orf):
    codon_list = []
    for i in range(0, len(orf)-2, 3):
        codon_list.append(orf[i:i+3])
    return codon_list
```

Before you go on, make sure you understand/remember how this function works and what it returns.

Count codons in an ORF

Now you need to count the number of times each codon occurs in the ORF.

Write a function, `count_codons`, that take one argument:

1. A string, which represents the open reading frame.

The function must return:

- A dictionary where keys are strings representing codons and associated values are integers representing the number of times each codon occurs in the ORF given as argument.

Example usage:

```
count_codons("ATGTCATCATGA")
```

should return:

```
{ 'CTT': 0, 'ATG': 1, 'ACA': 0, 'ACG': 0, 'ATC': 0, 'AAC': 0,
  'ATA': 0, 'AGG': 0, 'CCT': 0, 'ACT': 0, 'AGC': 0, 'AAG': 0,
  'AGA': 0, 'CAT': 0, 'AAT': 0, 'ATT': 0, 'CTG': 0, 'CTA': 0,
  'CTC': 0, 'CAC': 0, 'AAA': 0, 'CCG': 0, 'AGT': 0, 'CCA': 0,
  'CAA': 0, 'CCC': 0, 'TAT': 0, 'GGT': 0, 'TGT': 0, 'CGA': 0,
  'CAG': 0, 'TCT': 0, 'GAT': 0, 'CGG': 0, 'TTT': 0, 'TGC': 0,
  'GGG': 0, 'TAG': 0, 'GGA': 0, 'TGG': 0, 'GGC': 0, 'TAC': 0,
  'TTC': 0, 'TCG': 0, 'TTA': 0, 'TTG': 0, 'TCC': 0, 'ACC': 0,
  'TAA': 0, 'GCA': 0, 'GTA': 0, 'GCC': 0, 'GTC': 0, 'GCG': 0,
  'GTG': 0, 'GAG': 0, 'GTT': 0, 'GCT': 0, 'TGA': 1, 'GAC': 0,
  'CGT': 0, 'GAA': 0, 'TCA': 2, 'CGC': 0}
```

– though not necessarily with key-value pairs in that order.

In the function, you should use the `split_codons` function to split the ORF into a list of codons. Then create an empty dictionary that you can populate with counts. You want *all* the possible codons to be in your dictionary. That way, the codons you do *not* find in your ORF will have a count of `0`. In this case, such absence is also valuable information. To achieve this you must start by filling the dictionary with a key for each codon and give each a count of `0`. You can do that by iterating over the keys in the dictionary that maps codons to amino acids. Then you must iterate over all the codons in the list of codons produced by the `split_codons` function and add counts to the dictionary as you go.

Group codon counts by amino acid

Having counted how many times each codon appears in the ORF, you need to group the counted codons by the amino acid they encode.

Write a function, `group_counts_by_amino_acid`, which takes one argument:

1. A dictionary, as that returned by `count_codons`.

The function must return:

- A dictionary of dictionaries, which pairs each amino acid with a dictionary with counts of how many times each codon is used to encode that amino acid in the ORF.

Example usage: Assuming `counts` is the dictionary returned by `count_codons` as in the previous example.

```
grouped_counts = group_counts_by_amino_acid(counts)
```

then `group_counts_by_amino_acid` should return:

```
{ 'A': { 'GCA': 0, 'GCC': 0, 'GCT': 0, 'GCG': 0},
  'C': { 'TGC': 0, 'TGT': 0},
  'E': { 'GAG': 0, 'GAA': 0},
  'D': { 'GAT': 0, 'GAC': 0},
  'G': { 'GGT': 0, 'GGG': 0, 'GGA': 0, 'GGC': 0},
  'F': { 'TTC': 0, 'TTT': 0},
  'I': { 'ATT': 0, 'ATC': 0, 'ATA': 0},
  'H': { 'CAC': 0, 'CAT': 0},
  'K': { 'AAG': 0, 'AAA': 0},
  '*': { 'TAG': 0, 'TGA': 1, 'TAA': 0},
  'M': { 'ATG': 1},
  'L': { 'CTT': 0, 'CTG': 0, 'CTA': 0, 'CTC': 0, 'TTA': 0, 'TTG': 0},
  'N': { 'AAT': 0, 'AAC': 0},
  'Q': { 'CAA': 0, 'CAG': 0},
  'P': { 'CCT': 0, 'CCG': 0, 'CCA': 0, 'CCC': 0},
  'S': { 'TCT': 0, 'AGC': 0, 'TCG': 0, 'AGT': 0, 'TCC': 0, 'TCA': 2},
  'R': { 'AGG': 0, 'CGC': 0, 'CGG': 0, 'CGA': 0, 'AGA': 0, 'CGT': 0},
  'T': { 'ACC': 0, 'ACA': 0, 'ACG': 0, 'ACT': 0},
  'W': { 'TGG': 0},
  'V': { 'GTA': 0, 'GTC': 0, 'GTT': 0, 'GTG': 0},
  'Y': { 'TAT': 0, 'TAC': 0}}
```

– though not necessarily with key-value pairs in that order.

So if we pretend the resulting dictionary of dictionaries is called `d`, then `d['S']` will be a dictionary where keys are codons encoding the 'S' amino acid and the values are the counts of those codons. That means that you can access a particular count like this:

```
d['S']['TCA']
```

In the above example, this count is `2`. So the task is really just to distribute counts given as the first argument into groups for each amino acid. Depending on what you call your variables it could look something like this:

```
grouped_counts[acid][codon] = codon_counts[codon]
```

Your function should begin by defining an empty dictionary to add to. Then use a for-loop to run through all codon/amino-acid pairs and populate your dictionary of dictionaries.

Turn counts into frequencies

Now you know how many times each codon represents a certain amino acid, but we would like to know with which *frequency* a certain codon represents an amino acid. So you need to normalize the counts so they become frequencies. You do that by dividing each codon count by the total number of codons encoding the same

amino acid. We split the solution to this problem in two. We first write a helper function that turns codon counts for *one* amino acid into frequencies.

Write a function, `normalize_counts`, which takes one argument:

1. A dictionary, where keys are strings representing codons and values are integers representing the counts of these codons.

The function must return:

- A dictionary, where keys are strings representing codons and values are floats representing the frequency at which each codon appear. That is, the count of that codon divided by the total of all codon counts in the dictionary. The frequencies for codons that encode the same amino acid must of sum to one. That means that in cases where the total count is zero, the function must return `None`.

Example usage:

```
normalize_counts({'ATT': 8, 'ATC': 10, 'ATA': 2})
```

should return:

```
{'ATC': 0.5, 'ATA': 0.1, 'ATT': 0.4}
```

– though not necessarily with key-value pairs in that order.

Now you have solved part of the task, what remains is to now use that function to normalize the codon counts of *each* amino acid in your grouped counts:

Write a function, `normalize_grouped_counts`, which takes one argument:

1. A dictionary of dictionaries, as that returned by `group_counts_by_amino_acid`.

The function must return:

- A new dictionary of dictionaries where the values of the inner dictionaries are frequencies instead of counts as in the example in the introduction. You should not include amino acids for which there are no counted codons.

Example usage: Assuming `gr_counts` is the dictionary of dictionaries returned by `grouped_group_counts_by_amino_acid` in the example above.

```
normalize_grouped_counts(gr_counts)
```

should return:

```
{'*': {'TAA': 0.0, 'TGA': 1.0, 'TAG': 0.0},  
'M': {'ATG': 1.0},  
'S': {'AGC': 0.0, 'TCG': 0.0, 'TCC': 0.0, 'TCT': 0.0, 'AGT': 0.0, 'TCA': 1.0}}
```

– though not necessarily with key-value pairs in that order.

Here is some help to get you started:

```
def normalize_grouped_counts(grouped_counts):  
    grouped_freqs = {}  
    for aa in grouped_counts:  
        counts = grouped_counts[aa]
```

Amino acids with no codon counts should *not* be part of the data structure. Remember that in this case `normalize_counts` returns `None`, so you can simply test if the return value from `normalize_counts` is `None`

Compute the codon usage

Now all that remains is to tie together the functions you have written in a final function that generates your big data structure from an ORF:

Write a function, `codon_usage`, which takes one argument:

1. A string, which is an ORF.

The function must return:

- A dictionary of dictionaries, same as that returned by `normalize_grouped_counts`.

43 Code in other files

Every useful program is spread across many files, and `import` is the one statement that puts them back together.

Everything you have written so far has lived in a single file. That was fine while a program was forty lines long, and it stops being fine somewhere around two hundred. But there is a bigger point hiding behind the practical one. You have been calling `print` and `len` since week one without ever asking where they live. You have been running tests written by somebody else against code written by you, in two different files, and somehow the tests could see your functions. Every one of those things happened because of a single statement, which you have typed a few times already without being told what it does.

This chapter is about what `import` actually does. It is a short chapter, because the answer is short. It is also the chapter that quietly unlocks the second half of the course, because from here on you will be using code that other people wrote, and you should know what is happening when you do.

A module is a file

Make a folder. Put two files in it. Type them both; do not copy them.

seqtools.py

```
def gc_content(dna):  
    "Fraction of the bases that are G or C."  
    gc = dna.count('G') + dna.count('C')  
    return gc / len(dna)
```

analyse.py

```
import seqtools  
  
print(seqtools.gc_content('ATGC'))
```

Now run the *second* one from the terminal, the way you have been running programs all term:

Terminal

```
$ python analyse.py
0.5
```

A function defined in one file was called from another one. That is the whole feature, and you have just used it.

What you wrote is called a **module**. A module is a file with Python in it. That is the entire definition, and there is nothing else to it. `seqtools.py` is a module whose name is `seqtools` — the file name without the `.py`. Because that name becomes a variable in the program that imports it, it has to be a name you could legally give a variable: no dashes, no spaces, and it cannot start with a digit. `seq_tools.py` is fine. `seq-tools.py` will not import, ever, and the error will not be helpful about why.

What `import` actually does

Three things, in this order.

First it **finds** the file. It looks in the folder you are running from, and then in the places where installed libraries live, and it takes the first `seqtools` it finds.

Second it **runs** the file. Top to bottom, every line, exactly as if you had typed `python seqtools.py` yourself. Each `def` runs and creates a function object; each assignment at the left margin runs and creates a variable.

Third it **binds a name**. When the file has finished running, everything it created is packed into a single object, and that object is assigned to the variable `seqtools` in your program.

The third one is the one that makes the rest make sense: `import seqtools` is an assignment statement. Afterwards, `seqtools` is an ordinary variable holding an ordinary object, and you can look at it:

```
import seqtools

print(type(seqtools))
print(seqtools)
```

It reports `<class 'module'>` and then tells you which file it came from. So `seqtools.gc_content` is not new syntax. It is the dot from Chapter 15, doing what it always does: “Hey module, give me your `gc_content`!” A module is an object whose attributes are the things defined in its file.

The middle step, the one where the file *runs*, is the one that surprises people. Try it. Put a line at the left margin of the module, above the `def`:

what you write

```
import seqtools
import seqtools as st
from seqtools import gc_
```


seqtools.py

```
print("seqtools is being imported")

def gc_content(dna):
    "Fraction of the bases that are G or C."
    gc = dna.count('G') + dna.count('C')
    return gc / len(dna)
```

Run `analyse.py` again and that sentence appears, without anything in `analyse.py` asking for it. It happened because importing a module executes it. Anything sitting at the left margin of a module runs the moment somebody imports it, whether they wanted it to or not.

Exercise 43-1

SOLO

Put `print("analyse is starting")` as the very first line of `analyse.py`, and then write `import seqtools` twice, on two lines in a row. Decide what you think the program prints, and in what order, and how many times, before you run it. Then run it. One part of the result is going to be different from what you predicted, and the explanation is that Python keeps a note of every module it has already imported and refuses to run the same file twice.

! Important

Because importing a module runs it, a module should contain *definitions*, not *actions*. Functions, classes, and constants: yes. A loop over your whole genome that takes five minutes: no, not at the left margin, because it will happen to anybody who imports your file for any reason at all. When a file needs to be both — a module other files can import *and* a program you can run directly — the fix is one incantation you may as well learn as a recipe:

[H]
analyse.py

```
import seqtools

def main():
    print(seqtools.gc_content('ATGC'))

if __name__ == '__main__':
    main()
```

Python sets the variable `__name__` inside every file it runs. If you ran the file yourself, it is set to `'__main__'`; if somebody imported the file, it is set to the module's name instead. So the body of that `if` runs when you run the file and stays quiet when somebody imports it. That is all it is: a normal if-statement, asking a normal question about a variable somebody else set for you.

The three ways to write it

```
import seqtools
print(seqtools.gc_content('ATGC'))
```

```
import seqtools as st
print(st.gc_content('ATGC'))
```

```
from seqtools import gc_content
print(gc_content('ATGC'))
```

All three run the whole file. They differ only in the name you are left holding.

The first is the honest one, and it should be your default. Six months from now, `seqtools.gc_content(dna)` still tells the reader where that function came from, and the reader might be you.

The second is the same thing with a shorter name, and you use it when the community has agreed on an abbreviation, the way everyone writes `import pandas as pd`. Do not invent your own abbreviations for well-known libraries; you will only confuse people who already know the conventional one.

The third reaches into the module and copies one name out of it, so you can use it without saying where it came from. That is convenient, and the convenience is exactly the cost: a file full of bare function names gives a reader no way to tell which ones you wrote and which ones arrived from somewhere else. Use it sparingly,

for a handful of names you use constantly, the way `from Bio.Seq import Seq` is written by everybody.

There is a fourth form, `from seqtools import *`, which empties the entire module into your program. It looks like a shortcut and is a trap: any name in the module that happens to match a name of yours silently replaces it, and you find out much later, in the form of a function that does something you did not write. Do not use it.

You have been importing your own code all term

Open the test file from any of the projects. You will find functions that look like this:

```
@requires("find_orfs")
def test_find_orfs(module):
    assert module.find_orfs("ATGAAATAGAAATGAAATAGTAA") == [[0, 6], [11, 17]]
```

There is that dot again, and now you can read the line properly. The test suite **imports your project file as a module**, and hands the resulting module object to each test as the argument `module`. Then `module.find_orfs` is the test asking your module for the function of that name, exactly the way `analyse.py` asked `seqtools` for `gc_content`.

That also explains a message you have seen. When the test report says a function is not written yet, nothing clever is going on: the tests looked up a name in your module, the name was not there, and the report says so. Your file has been a module all along. Nobody mentioned it, because until now you did not need to know.

Modules that came with Python

A large pile of modules arrived on your computer along with Python itself, and you can import any of them without installing anything. This is called the *standard library*, and it is one of the reasons people like Python.

```
import math

print(math.sqrt(16))
print(math.log(100, 10))
print(math.floor(3.7))
```

`random` gives you random choices and shuffles, `statistics` gives you means and medians, `os` and `pathlib` deal with files and folders, and `csv` reads the comma-separated tables from Chapter 54 without pandas being involved at all. You do not need to learn them; you need to know they exist, so that you check before writing something that already exists.

! Important

Do not call one of your own files `math.py` . Or `random.py` , or `csv.py` , or `string.py` .

You now know exactly why, which is the point of putting this warning here rather than earlier. Import looks in *your folder first*. If there is a `math.py` sitting next to your program, that is the `math` that gets imported, yours, and the real one becomes unreachable — and the error you get will be about `sqrt` not existing, which points at the wrong file entirely. It is a genuinely nasty afternoon. Give your files names that are about your problem: `seqtools.py` , `orffinder.py` , `translation.py` .

A package is a folder of modules

Once a library grows past what fits comfortably in one file, it becomes a folder of files. A folder of modules is called a **package**, and the dots in an import statement are steps down into it:

```
from Bio.Seq import Seq
```

Read it left to right. Go into the package `Bio` . Inside it, find the module `Seq` . Out of that module, take the name `Seq` , which here is a class. The fact that the module and the class have the same name is unfortunate and very common; the dots tell you which is which.

That is all a package is. `pandas` is a package. `Bio` is a package. Both are folders full of ordinary `.py` files that somebody else wrote, sitting somewhere on your disk, containing functions and classes defined with exactly the `def` and `class` statements you already know.

Which also demystifies installing. When `pixi` , `conda` or `pip` installs a package, it downloads that folder and puts it somewhere `import` already knows to look. There is no other magic step. It is copying files into a place on a list.

Those two errors are worth being able to tell apart at a glance, and the second one is about to become important. An assistant that has read a great deal of code will happily offer you `from Bio.Seq import reverse_translate` , because that is the kind of thing a sequence library plausibly contains, and plausible is what these models are good at. The module exists, so the import gets that far; the name does not, so it stops there. That `ImportError` is the machine telling you the assistant made something up. Chapter 44 is about learning to catch it a step earlier, by reading what the library actually promises.

Exercise 43-2

AI: WORKER

Two errors worth telling apart

`ModuleNotFoundError: No`

Import went looking for a file and found nothing. Not installed, or misspelled, or you are running from the wrong folder.

`ImportError: cannot impo`

Import found the module. The *name* you asked for is not in it.

Ask the assistant for a function from the `math` module that you suspect does not exist — something plausible, like a function that returns the greatest common divisor of a list, or one that rounds to a given number of significant figures. Ask it for the exact import statement.

Then run the import. Whatever happens, you learn something: either the function exists and you have found a useful one, or you get an `ImportError` naming precisely what was invented. Write both the claim and the outcome in your logbook.

Splitting a project of your own

The same feature that lets you use other people's code lets you organise your own, and it is the same reason you started writing functions in the first place. A function gives a name to a piece of a program. A module gives a name to a group of functions.

Take the ORF project. By the end it holds a codon table, a translation function, several ORF-finding functions and something that runs the whole analysis. Three groups, so three files:

translation.py

```
CODON_MAP = {'TTT': 'F', 'TTC': 'F'}    # rest of the table here...

def translate_orf(orf):
    # rest of code here...
```

orffinder.py

```
def find_start_positions(seq):
    # rest of code here...

def find_orfs(seq):
    # rest of code here...
```

main.py

```

import translation
import orffinder

def report(genome):
    for start, end in orffinder.find_orfs(genome):
        print(translation.translate_orf(genome[start:end]))

if __name__ == '__main__':
    report('ATGAAATAGAAATGAAATAGTAA')

```

Three rules and you have everything you need. The files go in the same folder. Each module holds definitions, not actions. And the arrows point one way: `main` imports the other two, and neither of them imports `main`. If two modules import each other, Python has to start running one of them before the other has finished, and the resulting error is as confusing as it sounds. Keep the arrows pointing downhill and it cannot happen.

What you get for the trouble is what you got from functions, one level up. A name for each part. A file you can read without holding the other two in your head. And a piece you can test on its own, which is going to matter more and more from here.

Exercise 43-3

AI: DRAFTER

Take a program you have already written that has at least two functions in it. Decide *yourself*, on paper, which functions belong together and what the two files should be called — that decision is the part worth doing, and it is the part the assistant cannot do for you, because it does not know what your program is for.

Then ask the assistant to perform the split for you, telling it exactly which functions go where. Run the result. If it does not run, read the error before asking for a fix: nine times out of ten it is a missing import, and being able to say that out loud before anyone tells you is the whole point of this chapter.

What comes next

You now know that the code you import is code, in files, on your disk, no different in kind from the files you write. It was written with the same `def` statements, by people who were also once confused about `import`.

That matters because of what happens next. In Chapter 44 you meet BioPython, a package containing careful, tested versions of several functions you spent a week writing by hand. Handing that work over is not a defeat; doing it once yourself is what makes you able to judge somebody else's version. And the question that chapter asks — how do I know this library does what it claims? — is the same question you will be asking about every piece of code an assistant hands you for the rest of the term.

Exercise 43-4

SOLO

For this week's logbook: name one function you called this week that you did not write, say which module or package it came from, and say how you would find out what it promises. If your answer to the last part is "ask the assistant", write down what you would do to check the answer.

44 Documentation and resources

Reading a library or a web service as a catalog of contracts.

A few weeks ago you wrote your own `translate` function, codon by codon, and you hit a real bug doing it. That was worth doing by hand once, because now you understand exactly what translation involves. You are not going to keep doing it by hand. This week you meet BioPython, a library written and tested by people who work on exactly this kind of problem, and the skill this note actually teaches is not “how to use BioPython.” It is how to read *any* library, or any web API, well enough to use it correctly without ever reading how it works inside.

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · **Worker**

Still to come: Collaborator,
Developer

The idea you already have

You already learned this idea; it just had a different name. A class bundles data with the methods that act on it, and you call those methods without needing to know how they are implemented, because a method is a promise: given an object in a certain state, called with certain arguments, it returns or does a certain thing. That promise is a **contract**, and you trust it the same way you trust `len` or `.append()`, not because you have read their source, but because their contract is documented and you have seen it hold. A library is nothing more than a large collection of such contracts, written by someone else, that you are now allowed to call the same way.

Reading a contract instead of an implementation

Here is BioPython’s `Seq` object, which represents a DNA or protein sequence and behaves a great deal like the strings you already know:

```
from Bio.Seq import Seq

dna = Seq('ATGGCCTGA')
print(dna.translate())
print(dna.reverse_complement())
```

Before you run it, notice what you are being asked to trust: two methods you did not write, `.translate()` and `.reverse_complement()`, doing jobs you built yourself by hand a few weeks ago. You are not going to read BioPython’s source to check that `.translate()` correctly stops at a stop codon the way your own function eventually did. You are going to read its **documentation**, which states the contract: what the method takes, what it returns, and what it assumes about the object it is called on. The documentation says `.translate()` reads the sequence as a coding sequence and stops translating at the first in-frame stop codon by default, exactly

the fix you made to your own bug in the composition note. You did not need to reinvent that fix. You needed to know it already existed and where to look.

This is the actual reading skill: for any function or method you are about to call, find its entry in the documentation, and answer three questions before you call it. What exactly does it take, including anything optional? What exactly does it return, including the type? What does it assume that, if untrue, would make the result meaningless, such as the sequence being in frame, or the case of the letters, or which strand you are reading? Answering those three questions is reading a contract. Running the code and hoping is not.

What you do and do not verify

You will not write a test that checks `.translate()` implements the standard genetic code correctly; that is BioPython's job, done by people with far more time to get it right than you have this term, and treating it as machinery you trust rather than code you must personally verify is not laziness, it is the correct division of labor. What you do instead is **sanity-check** its output against something you can compute or already know: translate a short sequence whose protein you can work out by hand, or a start codon you know should give methionine, and confirm the library agrees. That is a different, lighter kind of checking than testing your own code, and telling the two apart, what you test because you wrote it and what you sanity-check because a library wrote it, is part of the skill this week teaches.

Combining capabilities, and where it goes wrong

The useful move is combining several such contracts to do something none of them does alone: read a file of sequences, translate each one, and report which produces the longest protein before hitting a stop codon, say. Directing an assistant to write code like this is exactly where library work and AI work meet, and it is also exactly where the assistant's weakest habit shows up. An assistant trained on a huge amount of code involving popular libraries will sometimes call a method that sounds exactly like something the library ought to have, with a name and a signature that fit the library's style perfectly, and that simply does not exist. It is guessing a plausible next token again, the same habit from three weeks ago, now aimed at a library's API instead of at Python's own syntax. The documentation is what catches this, because an invented method has no entry to check against, and the next note works through exactly that failure.

One more resource, mentioned and set aside

Later in the course you will meet the **dataframe**, a table-shaped object from a different library, `pandas`, that several of the later projects use to hold and plot sequence data. It is exactly one more resource of the kind this note is about: an object with a documented contract you read rather than code you write, and Chapter 54 introduces it properly when you need it. For now, file it under the same heading as `Seq`: a capability that exists, that you will learn to reach for by reading what it promises, not by reading how it is built.

Exercise 44-1

AI: WORKER

Ask the assistant to write a short function using `Bio.Seq.Seq` that takes a DNA string and returns `True` if its reverse complement, translated, contains no internal stop codon before the end. Before running anything, find `.translate()` and `.reverse_complement()` in the BioPython documentation and check the assistant's use of them against what the documentation actually promises, particularly any optional arguments it did or did not pass. Then run it on a sequence whose answer you can work out by hand, and confirm.

For your logbook this week, name one method or function you used this week purely by trusting its documented contract, without reading its implementation, and say what you checked to make sure you were using it as documented rather than as guessed.

Week 10

45 Plan before you prompt

By now you can specify a single function by writing its contract as tests, and you can read a single function the assistant produced and judge whether it keeps that contract. Both of those skills are about one function at a time. Real problems are not one function. The tasks you will tackle for the rest of the course, and the kind of task you took this course in order to be able to attempt at all, are made of several functions that fit together, and the difficulty is no longer in any one piece but in the fitting together. This week is about how you get an assistant to help you build something of that size without losing control of it, and the whole answer is contained in the title: you plan before you prompt.

Consider first what happens if you do not. The tempting way to use an assistant on a real task is to describe the whole thing in one prompt and ask for the complete program. You will get one. It will be a single block of code, often quite long, that appears to do what you asked. The trouble begins the moment you try to trust it. You cannot read all of it with the care you gave a four line function last week, so you skim, and skimming is how the plausible wrong code you studied slips past you. You cannot test it piece by piece, because it is not in pieces; it is one lump, and your only test is to run the whole thing and see whether the final answer looks right, which tells you nothing about the parts. And when the final answer is wrong, as it usually will be at first, you have no way to find out which part is at fault, because you never separated the parts. You are left staring at a wall of code that a machine wrote and that you do not understand, with a bug somewhere inside it and no way in. What has actually gone wrong here is subtle: by asking for everything at once, you did not merely delegate the typing to the assistant, you delegated the thinking, and the thinking is the part you cannot afford to give away.

The alternative rests on a division of labor that you must keep clear in your head. You drive; the assistant types. The thinking that matters in a program of any size is the decision about what the pieces are, what each piece is responsible for, and how they connect to each other. That thinking is yours and stays yours. What the assistant is good for is filling in the implementation of a piece once you have decided what that piece is and what it must do. It is an extremely fast, extremely fluent, and extremely unreliable typist for parts that you have specified. It is not a designer, and the surest way to get into trouble is to let it act like one.

The thinking you keep for yourself is called decomposition, and it is the same skill you learned when you first met functions, now used at a larger scale. To decompose a task is to break it into functions, each of which does one thing, has a name that says what that thing is, takes some inputs, and returns a result. A good decomposition is not measured by cleverness; it is measured by whether each piece is small enough that you can do three things with it: read it, test it, and understand it entirely on its own without holding the rest of the program in your head. If a piece is too big to test in isolation, or too tangled to understand without knowing what four other pieces are doing, the decomposition is not yet good enough, and

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · Worker ·
Collaborator

Still to come: Developer

no amount of help from the assistant will rescue a bad decomposition, because the assistant will simply fill your unclear pieces with unclear code.

The plan is where this thinking is written down, and it is written before you prompt for anything. A plan is the list of functions the task will need, and for each function, its name, what goes into it, what comes out of it, and one concrete example of an input and its correct output. You do not need a special format for this, because Python already gives you a perfect one. A plan is most naturally written as a set of empty functions with docstrings and no bodies, which is to say, exactly the kind of stub file the projects have been handing you all along. Suppose the task is to take a DNA strand and report its length, its GC content, and whether it counts as GC rich. A plan for it might look like this, written before a single line of working code exists:

```
def gc_content(dna):
    "Fraction of bases that are G or C. Example: gc_content('GGAT') == 0.5"
    ...

def is_gc_rich(dna):
    "True if more than half the bases are G or C, using gc_content. is_gc_rich('GGGA') == True"
    ...

def describe(dna):
    "A sentence reporting the strand's length, its GC content, and whether it is GC rich."
    ...
```

Look at what you have before you have asked the assistant for anything. You have decided that the task divides into three pieces and not some other number. You have named each piece and stated what goes in and what comes out. You have committed to one example per piece that you can check by hand, which means you have started deciding the awkward cases as well, because to write the example for `is_gc_rich` you had to decide what happens at exactly one half. You have even recorded a dependency, that `is_gc_rich` uses `gc_content` and `describe` uses both, so you know the order in which the pieces must be built. All of that is design, all of it is yours, and none of it required the assistant. This is what it means to plan before you prompt.

Writing the plan first, rather than after, matters for the same reason writing the tests first mattered last week. If you prompt for the code first and plan afterward, the assistant's code becomes the thing you react to. Its choice of how to break up the problem, which may be sensible or may be strange, becomes your starting point, and you find yourself adjusting your understanding to fit its structure instead of making it fit yours. Plan first and the order of authority is correct: you decide the shape of the solution, and the assistant is asked only to fill shapes you have already drawn.

With the plan in hand, you build one piece at a time, and you verify each piece before you build the next one on top of it. You prompt the assistant for `gc_content` alone, you read what it gives you against the questions from last week, you run the tests you seeded from your example, and only when that piece is proven do you move on to `is_gc_rich`, which you can now build trusting that the `gc_content` underneath it is sound. Then `describe`, last, on top of two pieces you have already

checked. The reason this is so much safer than the single lump is that it localizes everything. When something is wrong, it is wrong in the piece you are currently working on, because everything beneath it has already passed its tests, so you always know where to look. Integration, the fitting together that was the whole difficulty, becomes the almost boring act of assembling parts you have each already proven, rather than the terrifying act of debugging a whole you never understood.

Notice that the plan and the contract from last week are the same object seen at two scales. Each function's example in the plan is the seed of that function's test, so to plan a program is already to begin specifying it. Planning tells you what the pieces are; the tests tell you what each piece must satisfy; and building piece by piece with the tests as you go is simply the two ideas used together. When you give the assistant its instructions, you give it the context it needs and no license to wander: the plan, the signature of the one function you want now, and the example or test that function must satisfy. You ask for that one function, you verify it, and you move on. You do not let the assistant race ahead and write three functions you have not planned, because a function you did not plan is a function you did not decide you needed, and it will cost you more to understand and check than it would have cost you to write yourself.

This is exactly the method by which you will build the capstone, which is deliberately larger than anything you could write unaided. That is not a threat; it is the point. The plan is what makes a task that is too big to hold in your head at once into a task you can actually do, because with a plan you never face the whole thing. You face one small, named, testable piece at a time, on top of pieces you have already proven, and the assistant does the fast typing for each piece while you do the deciding and the checking. A problem you could not have written alone becomes a sequence of problems each of which you can.

Exercise 45-1

AI: COLLABORATOR

Do the wrong thing once, deliberately, so that you know what it feels like. Take a task made of three or four pieces, such as reading a file of DNA sequences and reporting which one has the highest GC content, and ask the assistant for the whole thing in a single prompt. Do not plan it, do not break it up. Read what comes back and then attempt exactly one thing: test one piece of it in isolation, without running the rest. Write down what stopped you. Nine times out of ten what stopped you is that there was no piece to test, because the assistant returned one lump that only produces an answer at the very end. Keep this code. You will want to compare it with what you build over the next few exercises.

Exercise 45-2

SOLO

Now practise the deciding, with no assistant and no working code at all. Take the same task and write only the plan for it, as a set of empty functions with docstrings and no bodies. The whole exercise is the decomposition: deciding what the pieces are, naming them, and stating what goes in and out of each. When you think you

are done, check your plan against the standard from earlier: is each piece small enough to read, to test, and to understand on its own?

Exercise 45-3

SOLO

Strengthen the plan you just wrote by adding to each function one concrete example of an input and its correct output, worked out by hand. As you do this you will find that some functions are harder to give an example for than others, and that difficulty is information: a function you cannot easily give an example for is usually a function that is doing too much, or that you have not thought through, and it should be split or reconsidered before any code is written for it.

Exercise 45-4

AI: COLLABORATOR

Only now let the assistant near the design, and note carefully what the badge does and does not license. Show it your stubs and your examples and ask it one narrow question: is there a case my examples do not cover, or a piece I have not named? You are not asking it to redesign anything, and you are not obliged to take anything it says. Go through its suggestions one at a time and, for each, write *accept* or *reject* and one line of reason. Rejecting a suggestion you understand is a better outcome than accepting one you do not, and the point of the exercise is to feel the difference between a collaborator, whose opinions you weigh, and a designer, whose decisions you inherit.

Exercise 45-5

AI: COLLABORATOR

Take the discipline all the way through on a single piece. From your plan, choose one function, and only when its plan and its example are written do you allow yourself to prompt the assistant for that one function. Verify the result against your example, using the check widget or your own `assert` s. The rule to feel here is the one in the title of the week: the plan comes first, the prompt comes second, and the verification comes before you are allowed to believe any of it. When it passes, put it beside the single lump from the first exercise and say which of the two you would be willing to defend.

For your logbook this week, include the plan you wrote as stubs for one task, and note which function in it was hardest to write an example for and what that difficulty told you about your decomposition. If you built one of the pieces with the assistant, record whether having planned it first changed how you judged the code it gave you.

46 HIV sub-groups

This chapter is a programming project where you will put your new programming skills to use analyzing an HIV DNA sequences.

You have now been introduced to all the programming rules you will see in this course. You now know all the building blocks required to write *any* program – and I do mean *any*. The reason why computer geeks are good at what they do is not that they know some incomprehensible secrets. It is because they practiced, a lot. With practice, the simple rules you know now will let you write anything from first-person shooter games over jumbo jet autopilots to scripts for simple problems in bioinformatics. In the last three chapters, we will train your ability to solve bioinformatics problems by putting together all the things you have learned.

The programming project in this chapter deals with DNA sequences from HIV viruses. There are two types of HIV: HIV-1, which is by far the most common, and HIV-2, which is mostly found in West Africa. HIV-1 vira are divided into groups M, N, O, and P. The most important group M (for major) is one primarily responsible for the global epidemic. Group M is further divided into subtypes A, B, C, D, F, G, J, K, and CRFs. In this project we will look at sequences from the subtypes A, B, C, and D. You have multiple database sequences for each of these four subtypes and you have one *unknown* sequence from a patient that you need to assign to either subtype A, B, C or D. To do this you will have to write a program that *predicts* the subtype of the unknown sequence. How cool is that?

Download the files you need for this project:

https://munch-group.org/bioinformatics/supplementary/project_files

- `unknown_type.txt` contains an HIV sequence of unknown subtype
- `subtypeA.txt` contains a database of HIV sequences of subtype A
- `subtypeB.txt` contains a database of HIV sequences of subtype B
- `subtypeC.txt` contains a database of HIV sequences of subtype C
- `subtypeD.txt` contains a database of HIV sequences of subtype D

You also need to download the two project files:

- `hivproject.py` is an empty file where you must write your code.
- `test_hivproject.py` is the test program that lets you test the code you write in `hivproject.py`.

Put the files in a folder dedicated to this project. On most computers you can right-click on the link and choose “Save file as...” or “Download linked file”.

Now open each file in *VScode* and have a look at what is in the data files. (Do *not* change them in any way and do not save them after viewing. If VScode asks you if you want to save it before closing, say *no*.) How many sequences are there in each file?

The project is divided into the following parts:

- Compute the similarity of two sequences
- Read the HIV sequences into your program.
- Assess the similarity of your unknown HIV sequence to each of the HIV sequences with known subtype.
- Find the maximum similarity of your unknown sequence to sequences from each subtype.
- Identify the HIV subtype of your sequence as the subtype of that sequence that your sequence is most similar to.

Make sure you read the entire exercise and understand what you are supposed to do before you begin!

Compute the similarity of two sequences

We need to compare our unknown HIV sequence to all the HIV sequences of known subtypes. That way we can identify the sequence of a known subtype that is most similar to your unknown sequence. We will then assume that our unknown sequence has the same subtype as this sequence. To accomplish this we first need to write some code that compares two sequences so we can compare our HIV sequence to each of the other HIV sequences.

Compare two sequences

Write a function `sequence_similarity` that takes two arguments:

1. A string which is a DNA sequence.
2. A string of the same length as argument one, which is also a DNA sequence.

The function must return:

- A float, which is the proportion of bases that are the same in two DNA sequences.

Example usage:

```
sequence_similarity('AGTC' 'AGTT')
```

should return `0.75`.

Start out defining your function like this:

```
def sequence_similarity(seq1, seq2):  
    # your code here...
```

Remember that `range(len(seq1))` generates the numbers you can use to index the string `seq1`. You can use those numbers as indexes to look up positions in both strings. You will need a for-loop in your function and a variable that keeps track of how many similarities you have seen as you iterate through the sequences.

Compare aligned sequences

All sequences, including the unknown sequence, are from the same multiple alignment. This ensures that sequence positions match up across all sequences but also means that a lot of gap characters (`'-'`) are inserted. To compute similarities between such sequences you need to make function much like `sequence_similarity` that does not consider sequence positions where both bases are a gap (`'-'`) characters. In other words, you must not only count the number of characters that are the same, you also need to count how many alignment columns that are `"-"` for both sequences. E.g. the following mini alignment has five such columns and four columns where the bases are the same. So in the following alignment, the similarity is 0.8 (4/5):

```
A-CT-A
A-CTTA
```

Write a function `alignment_similarity` that takes two arguments:

1. A string which is a DNA sequence with gap characters.
2. A string of the same length as argument one, which is also a DNA sequence with gap characters.

The function must return:

- A float, which is the proportion of bases that are the same in two DNA sequences.

```
alignment_similarity('A-CT-A', 'A-CTTA')
```

should return `0.8`.

Hint

Use an if-statement to test if the two characters at some index are equal to `'-'` in both sequences. You can use an expression like this:

```
seq1[i] == '-' and seq2[i] == '-'
```

Once your function has computed both the number of identical bases and the number of alignment columns that are not both `'-'`, you can have it return the similarity as the ratio of the two.

Read the HIV sequences into your program

To use your `alignment_similarity` function to assess similarity between your unknown sequence and the sequences of known subtype, you need to read the sequences into your program. Here is a function that will read the sequences from one of the files you downloaded into a list:

```
def read_data(file_name):
    f = open(file_name)
    sequence_list = list()
    for line in f:
        seq = line.strip()
        sequence_list.append(seq)
    f.close()
    return sequence_list
```

You can use that function to read the unknown sequence into your program:

```
unknown_list = read_data('unknown_type.txt')
```

In this case, the list only contains the one unknown HIV sequence in unknown_type.txt.

You also need to load the typed HIV sequences into your program. Here is a function that returns a dictionary in which the keys are subtypes ('A' , 'B' , 'C' and 'D') and each value is a lists of sequences with that subtype:

```
def load_typed_sequences():
    return { 'A': read_data('subtypeA.txt'),
            'B': read_data('subtypeB.txt'),
            'C': read_data('subtypeC.txt'),
            'D': read_data('subtypeD.txt') }
```

If you use the function like this:

```
typed_data = load_typed_sequences()
```

then you can access the list of sequences of subtype A like this: `typed_data['A']` .

Compare your HIV sequence to HIV sequences of known subtype

To type you HIV sequence you must compare your sequence to all the database sequences to see which group has the best matching sequence.

Write a function `get_similarities` that takes two arguments:

1. A string, which is your unknown HIV sequence.
2. A list of strings, each of which is an HIV sequence of known type.

The function must return:

- A list of floats, which should be the similarities between the unknown sequence given as the first argument and the list of sequences given as the second argument.

Example usage:

```
get_similarities(unknown_list[0], typed_data['A'])
```

should return:

```
[0.8553288474061211, 0.8721742704480066,  
0.854924397221087, 0.8481709291032696,  
0.8498330281159108]
```

The function should use the function `alignment_similarity` to compare your unknown sequence (`unknown_list[0]`) to each of the sequences of some subtype. Start out like this:

```
def get_similarities(unknown, typed_sequences):  
    # Your code here...
```

In your function you need to define a list that you can *append* the similarities you compute to:

```
similarities = []
```

This is the list of results that your function must return. To compute the similarity between you unknown sequence and each of the sequences of known subtype, you can use your `alignment_similarity` function inside a for-loop.

Compute maximum similarity to each subtype

To predict the subtype of the unknown HIV sequence you need to compare the unknown sequence to all the sequences of each of the different subtypes. The subtype of the sequence with the highest similarity to your unknown sequence is then our predicted subtype (or our best guess).

Write a function `get_max_similarities` that takes two arguments:

1. A string, which is your unknown HIV sequence.
2. A dictionary, like the one returned by `load_typed_sequences` .

The function must return:

- A dictionary, in which keys are strings representing each subtype (`'A'` , `'B'` , `'C'` , and `'D'`) and values are floats representing the maximum similarity between the unknown sequence and the sequences of a subtype. The dictionary *could* look like this (it does not, you need to compute the similarities yourself.):

```
{ 'A': 0.89, 'B': 0.95, 'C': 0.82, 'D': 0.99 }
```

To get the highest number in a list of numbers, you can use the `max` function in Python. It works like this:

```
numbers = [3, 8, 53, 12, 7]
print(max(numbers)) # prints 53
```

For example, to get the highest similarity between the unknown sequence and sequences in `typed_data['A']`:

```
subtypeA_similarities = get_similarities(unknown_list[0], typed_data['A'])
subtypeA_max = max(subtypeA_similarities)
```

Identify the HIV subtype

Now for the grand finale! You ultimately want to be able to write code like this:

```
unknown_list = read_data('unknown_type.txt')
typed_data = load_typed_sequences()
subtype = predict_subtype(unknown_list[0], typed_data)
print("Patient HIV is subtype {}".format(subtype))
```

So all you need now is the `predict_subtype` function.

Write a function `predict_subtype` that takes two arguments:

1. A string, which is your unknown HIV sequence.
2. A dictionary, like the one returned by `load_typed_sequences`.

The function must return:

- A string of length one (either `'A'`, `'B'`, `'C'`, or `'D'`) representing the predicted subtype of your unknown HIV sequence.

The function should use `get_max_similarities` to compute the dictionary of max similarities and then extract from that dictionary the key with the highest value (similarity). So the function must return `'A'` if the unknown sequence is most similar to a sequence of subtype A, `'B'` if the unknown sequence is most similar to a sequence of subtype B and so on.

47 Recursion

A function that calls itself sounds like a joke or a disaster. It is neither. It is the sharpest tool in this book, and learning it will teach you what a function really is.

What happens when you call a function

Before we do anything alarming, let us go back over something you already know. Write this and run it:

```
def greeting(name):  
    salute = "Hello"  
    return salute + " " + name  
  
message = greeting("Mogens")  
print(message)
```

Now walk through it the way you swore to in Oath 2, one line at a time, top to bottom:

1. Python reads `def greeting(name):` and the indented block under it. It does *not* run them. It just remembers that there is a function called `greeting`, and what its body looks like.
2. Python reaches `message = greeting("Mogens")`. Now it runs the body — a temporary little world springs into being, with `name` holding `"Mogens"` and, a line later, `salute` holding `"Hello"`.
3. The `return` hands back `"Hello Mogens"`, and the whole expression `greeting("Mogens")` is substituted for that string. The temporary world evaporates. `name` and `salute` are gone.
4. `message` is assigned `"Hello Mogens"`.

Nothing new there. But notice how much of it you are now taking for granted: that the body is a template, not a thing that runs when you write it; that calling it creates a fresh little world every single time; that the world is thrown away when the function returns.

Hold on to those three facts. They are all you need for what follows.

Recursion

Apparently complicated problems can often be solved *recursively*, if the whole problem can be split into smaller problems of exactly the same kind.

That is the whole idea. Read it again, because the important word is *identical*. Not “smaller problems”. Smaller problems *of the same kind*, so that whatever solves the big one also solves the small ones.

And if the same thing solves both, you only have to write it once — and then have it call itself.

Factorial

Here is the classic example. The factorial of 4, written $4!$, is

$$4 \times 3 \times 2 \times 1$$

Look at that product and squint. Inside it, tucked at the right, sits $3 \times 2 \times 1$, which is $3!$. And inside *that* sits 2×1 , which is $2!$. So:

$$4! = 4 \times 3!$$

which is the same as saying that if you already knew the factorial of 3, computing the factorial of 4 would be one multiplication of work. And you would know the factorial of 3 if you knew the factorial of 2. And so on.

Two lines describe the whole thing:

$$\text{Recursion: } f(x) = x \times f(x - 1)$$

$$\text{Basis: } f(1) = 1$$

The first line says how to make the problem smaller. The second says when to stop. You need both. The first one alone is a machine for making ever-smaller problems that never actually get solved; it is the basis that eventually gives a real answer for the whole tower of calls to be built on.

x	$x!$
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
9	362880
10	3628800
11	39916800
12	479001600
13	6227020800
14	87178291200
15	1307674368000
16	20922789888000
17	355687428096000
18	6402373705728000

Exercise 47-1

SOLO

Write a Python function called `factorial` that takes one argument and computes the factorial recursively. You have everything you need: `def`, `if`, `return`, and the two lines of maths above. The table in the margin is there so you can check yourself — start with `factorial(4)`, where you know the answer must be 24.

Do this one yourself before you read on or ask anyone, human or otherwise. It is four lines long and it is the most valuable four lines you will write this term.

Here is one solution:

```
def factorial(x):  
    if x == 1:  
        return 1  
    return x * factorial(x-1)
```

Read it against the two lines of maths. The `if` is the basis. The last line is the recursion. There is nothing else in the function, and there is nothing else that needs to be there.

If your reaction to that last line is “but you cannot call `factorial` inside `factorial`”, it is not finished yet” — good. That reaction is exactly what we are going to take apart.

Why it works

Go back to the three facts I asked you to hold on to.

A function definition is a *template*. When Python reads `def factorial(x):`, it does not run anything; it just remembers the body. So there is no problem writing a call to `factorial` inside the body — by the time that line actually runs, the definition is long finished.

And calling a function creates a *fresh temporary world*. So when `factorial(3)` reaches `return x * factorial(x-1)` and calls `factorial(2)`, a brand new world appears, with its own `x` holding `2`. The outer `x` is untouched. It is sitting there, patiently, waiting for its multiplication to finish.

Figure 47.1 shows all of it at once for `factorial(3)`. Each box is one call, one temporary world, with its own `x`.

Follow it downwards first. `factorial(3)` cannot return until it knows `factorial(2)`. `factorial(2)` cannot return until it knows `factorial(1)`. And `factorial(1)` does not need anybody: `x == 1` is `True`, so it returns `1` straight away.

Now follow it back up, and read it as pure reduction, the way you have been reading expressions since Chapter 8. In the middle box, `factorial(1)` is substituted for `1`, so `return 2 * factorial(1)` becomes `return 2 * 1`, which reduces to `2`. In

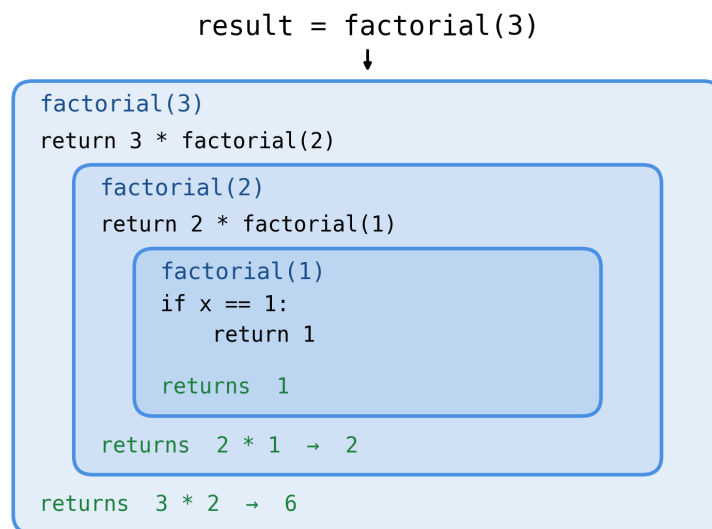


Figure 47.1: Three calls to `factorial`, one inside the next. Each box is a temporary world with its own `x`. The innermost one hits the basis and returns a real number, and then the answers are multiplied on the way back out.

the outer box, `factorial(2)` is substituted for `2`, so `return 3 * factorial(2)` becomes `return 3 * 2`, which reduces to `6`.

Substitution and reduction. That is all recursion is. You already knew how to do this.

Exercise 47-2

SOLO

Add a print to the top of the function so you can watch the worlds being created:

```

def factorial(x):
    print("starting factorial({})".format(x))
    if x == 1:
        return 1
    return x * factorial(x-1)

print(factorial(4))

```

Before you run it: how many lines will be printed, and in what order will they appear relative to the final answer? Decide, then run.

Exercise 47-3

SOLO

Now watch them close down again. Predict the *entire* output of this, in order, before you run it:

```
def factorial(x):
    print("entering factorial({})".format(x))
    if x == 1:
        print("basis: returning 1")
        return 1
    result = x * factorial(x-1)
    print("factorial({}) returns {}".format(x, result))
    return result

print(factorial(4))
```

If your prediction was wrong, do not just re-read the output until it looks reasonable. Draw the boxes of Figure 47.1 for `factorial(4)` on paper and walk through them with your finger.

Exercise 47-4

SOLO

Try this:

```
print(factorial(100))
```

and then this:

```
print(factorial(1000))
```

One of them prints an absurdly long number. The other one does not print anything at all; instead you get an error you have not seen before. Read it. Python is telling you that there is a limit to how deep this can go — each of those temporary worlds takes up room, and Python refuses to stack up more than about a thousand of them.

That limit is not a flaw. It is a smoke alarm. A recursion that runs away is one of the easiest bugs to write, and this is Python catching it for you.

Exercise 47-5

SOLO

! Spot the bug

The `factorial` above is deliberately fragile. It is not a typo; it is a missing case.

What do you think `factorial(0)` does? Decide first. Then run it, and explain the error you get in terms of the basis case. Then fix the function so that `factorial(0)` returns `1`, which is what mathematicians have agreed $0!$ should be. There is more than one way to fix it, and they are all one character or one line.

Divide and conquer

Factorial is the traditional first example, but it is a slightly dishonest one, because a `for`-loop would have done the job just as well. Here is a problem where a loop will not save you.

A **tree** is one of those things you can only really describe in terms of itself. Look at Figure 47.2. At the very top sits a node. It has two things hanging under it. Each of those is either a *leaf* — the end of the line, with a name on it — or another node with two more things hanging under it.

`[ [ [ 'A' , 'C' ] , 'B' ] , [ 'E' , 'D' ] ]`

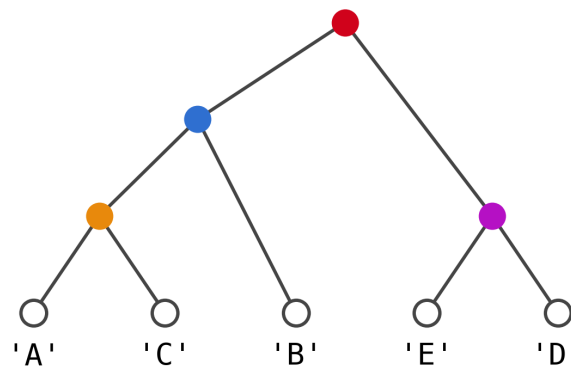


Figure 47.2: A tree with five leaves, and the same tree written as nested lists. Each coloured pair of brackets is one node in the tree.

In Python we can write a tree as nested lists. The tree in Figure 47.2 is:

```
tree = [[[ 'A' , 'C' ], 'B' ], [ 'E' , 'D' ]]
```

Take a moment with the colours in the figure. Each pair of matching brackets in the nested list is one node in the drawing, and the colours pair them up. The red brackets are the outermost list and the node at the top. Inside the red list are two things: the blue list and the magenta list — which is exactly what the drawing shows hanging under the red node.

Exercise 47-6

SOLO

Look at the nested list and *only* the nested list, without looking at the drawing. Which two things are inside the blue list? Which two are inside the orange list? Then check yourself against Figure 47.2.

Exercise 47-7

SOLO

Write the nested list for a tree where the top node has a leaf 'X' on the left and, on the right, a node with leaves 'Y' and 'Z'. Then draw it.

Counting the leaves

Now the question: how many leaves does a tree have?

You could try to write a loop. Go on, think about it for a minute. How many levels of nested loops would you need? The answer depends on how deep the tree is, and you do not know how deep the tree is, and even if you did, you would need to rewrite the program for every new tree. This is where loops run out.

Exercise 47-8

SOLO

Answer these three questions on paper, before writing any code:

1. How could you find the number of leaves on a tree using recursion?
2. How can you split the problem into smaller problems of exactly the same kind?
3. What is the basic case that has an obvious, immediate answer?

Look at Figure 47.3 if you get stuck. It shows the answer written next to every node: 1 at each leaf, and at every other node the sum of the two numbers below it.

Exercise 47-9

SOLO

Write a function called `count` that takes a tree and returns the number of leaves on it. Test it on `[[['A', 'C'], 'B'], ['E', 'D']]`, where you know from Figure 47.3 that the answer must be 5.

One hint, and it is the only one you get. You need to be able to tell a leaf from a node. A leaf is a string and a node is a list, so you can test for it like this:

```
if type(tree) is str:
    # what to do when it is a leaf...
```

Here is one solution:

```
[[['A', 'C'], 'B'], ['E', 'D']]
```

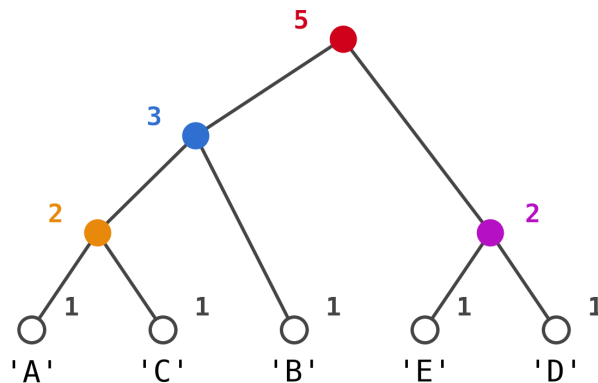


Figure 47.3: The same tree, with the number of leaves written at each node. Every node's number is the sum of the two numbers below it.

```
def count(tree):
    if type(tree) is str:
        return 1
    return count(tree[0]) + count(tree[1])

tree = [['A', 'C'], 'B'], ['E', 'D']]

print(count(tree))
```

Three lines. Compare them with the three questions you answered on paper: the `if` is the basic case, and the last line is the split into two identical smaller problems.

Notice that the last line calls `count` *twice*. There is no rule that a recursive function may only call itself once, and this is what makes recursion so much more powerful than a loop here. Each call branches into two more, and the branching follows the shape of the tree without you ever having to know how deep it goes.

Figure 47.4 shows every call that happens when you run it. The colours match Figure 47.2: each call gets exactly the part of the tree that its node is responsible for.

That is the thing worth staring at. You did not write the shape of that diagram. The tree did. Your function only ever described what to do at *one* node, and the shape came out of the data.

Exercise 47-10

SOLO

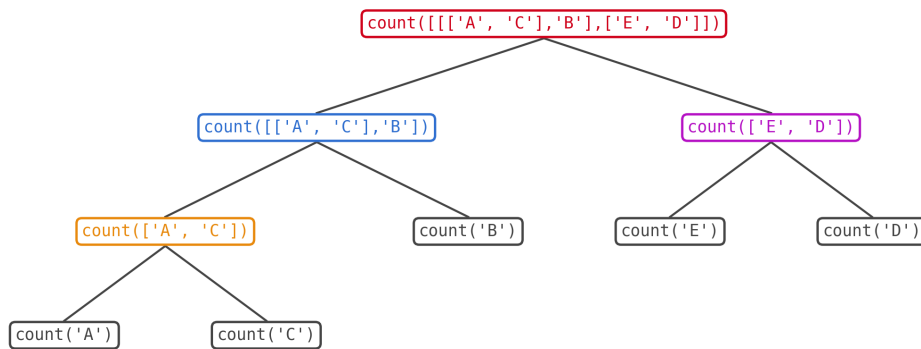


Figure 47.4: Every call made by `count` on the example tree. The call tree has the same shape as the tree itself.

Predict what happens with each of these, then run them:

```
print(count('A'))
print(count(['A', 'B']))
print(count([['A', 'B'], ['C', 'D']]))
```

The first one is worth a moment's thought. Is a single leaf a tree?

Exercise 47-11

SOLO

Add a print to `count` so it announces every call, like this:

```
def count(tree):
    print("count({})".format(tree))
    if type(tree) is str:
        return 1
    return count(tree[0]) + count(tree[1])

print(count([['A', 'C'], 'B'], ['E', 'D']]))
```

Run it and compare the printed lines with Figure 47.4. Do they come out in the order you expected? Which call finishes first, and which finishes last?

Exercise 47-12

Open the assistant in the browser, give it the `count` function and the tree, and ask it to list every call that gets made, in the order they are made. Do not run anything yet. Write its answer down.

Now run the version with the print in it and compare, line for line. Assistants are good at recursion in general and surprisingly slippery about the exact order of calls in a particular one. Where did it agree with the machine, and where did it not?

Put the result in your logbook. This is the rule of the whole course in one exercise: the assistant predicts, the machine proves.

Exercise 47-13

SOLO

Write a function that returns a *list* of all the leaves on a tree, rather than the number of them. On the example tree it should return `['A', 'C', 'B', 'E', 'D']`.

The same hint applies for telling a leaf from a node. And one more: `count` returned a number and added the two results together with `+`. Your function returns a list. What does `+` do to two lists?

Exercise 47-14

SOLO

Write a function that returns the *depth* of a tree — the number of steps from the top node down to the deepest leaf. A single leaf has depth `0`. On the example tree the answer is `3`.

You will need the built-in function `max`, which takes two numbers and returns the larger of them. Predict the answer for `['A', 'B']` before you run anything.

What you learned

Recursion is not a special feature of Python. There is no `recursive` keyword. It is nothing more than a function calling a function, which you have done a hundred times — it just happens that the function it calls is itself.

Which is why recursion is such a good test of whether you really understand functions. If a recursive call looks impossible to you, it is because some part of you still thinks that a function is one thing that happens once. It is not. A definition is a template, and every call builds a fresh temporary world from that template, and every one of those worlds has its own variables and its own patiently-waiting half-finished expressions. Recursion only works because that is true, and once you have watched it work, you will never doubt it again.

Logbook

This week, record what the assistant got right and wrong about the order of the calls in Figure 47.4, and how you found out. If you also asked it to write a recursive function for you, note whether it remembered the basis case — that is the one they most often get wrong, and it is the one that turns a program into an error message.

48 Worked example: decomposition modeled live

A companion to this week's lecture on breaking a task into pieces, pairing with Chapter 45.

In the plan-before-you-prompt note, you were set an exercise: without writing any code, plan a task that reads a file of DNA sequences and reports which one has the highest GC content. This note is that exercise, done properly and out loud, the way the lecture models it live. If you already wrote a plan of your own, keep it beside you and compare it to the decisions made here; disagreeing with a decision below is a perfectly good outcome, as long as you can say why.

Naming the pieces

The task, stated in full: given several named DNA sequences, report the name of the one with the highest GC content. Said that way, it already suggests its own pieces. Something needs to compute the GC content of one sequence, since you cannot compare what you cannot measure. Something needs to look across all the sequences and find the best one, once each can be measured. And something needs to turn that result into an actual report, since “find the best one” and “tell the user which one” are not quite the same job. Three pieces, and notice the shape: the second piece cannot be built without the first, and the third leans on the second. That dependency order is itself a decision, and it is the order they get built and tested in.

```
def gc_content(dna):
    "Fraction of bases in a strand that are G or C."
    ...

def most_gc_rich(sequences):
    "Given a dict of name to DNA string, return the name with the highest GC content."
    ...

def report(sequences):
    "A sentence naming the most GC-rich sequence and its GC content."
    ...
```

Naming the shape of the data, not just the functions

There is a decision hiding above that is easy to walk past: `most_gc_rich` takes “a dict of name to DNA string.” Nothing forced that shape. The sequences could have arrived as two parallel lists, one of names and one of strings, or as a list of two-element tuples. A dictionary was chosen because a name naturally maps to

one sequence and you will want to look sequences up by name later, but this is exactly the kind of decision a decomposition has to make explicit and in writing, because if you leave it implicit, the assistant will guess a shape for you, silently, and it may not be the one the rest of your program expects.

One worked example per piece

A plan is not finished until each piece has a concrete input and output you worked out by hand, because that is what turns a vague name into something you could test.

```
def gc_content(dna):
    "Fraction of bases in a strand that are G or C. gc_content('GGCC') == 1.0"
    ...

def most_gc_rich(sequences):
    "Given a dict of name to DNA string, return the name with the highest GC content.
    most_gc_rich({'a': 'GGCC', 'b': 'ATAT'}) == 'a'"
    ...

def report(sequences):
    "A sentence naming the most GC-rich sequence and its GC content.
    report({'a': 'GGCC', 'b': 'ATAT'}) == 'a is the most GC-rich sequence, at 1.0.'"
    ...
```

Writing the `most_gc_rich` example is where a real decision surfaces, the same way it did for `is_gc_rich` in the plan-before-you-prompt note: what happens on a tie, two sequences with exactly the same GC content? The example above simply does not have a tie in it, which means the decomposition has quietly left that case unspecified. That is worth noticing rather than hiding. A stronger plan would add a second example that does contain a tie, and decide, in writing, which name wins or whether both should be reported.

From plan to code, one piece at a time

The plan above is not decoration; it is what you hand the assistant, one function at a time, in the order the dependencies demand. `gc_content` first, since nothing else can be built without it:

```
def gc_content(dna):
    "Fraction of bases in a strand that are G or C."
    gc = dna.count('G') + dna.count('C')
    return gc / len(dna)

assert gc_content('GGCC') == 1.0
```

Only once that test passes do you move to `most_gc_rich`, which is now allowed to call `gc_content` and trust it completely:

```
def most_gc_rich(sequences):
    "Given a dict of name to DNA string, return the name with the highest GC content."
    return max(sequences, key=lambda name: gc_content(sequences[name]))

assert most_gc_rich({'a': 'GGCC', 'b': 'ATAT'}) == 'a'
```

And last, `report`, on top of two pieces already proven:

```
def report(sequences):
    "A sentence naming the most GC-rich sequence and its GC content."
    name = most_gc_rich(sequences)
    return f"{name} is the most GC-rich sequence, at {gc_content(sequences[name])}."

print(report({'a': 'GGCC', 'b': 'ATAT'}))
```

What the plan bought you

Look back at how that went. At no point did you face “write the whole program.” You faced, in order, three small named problems, each with a worked example that told you exactly what correct looked like before you or the assistant wrote a line. When you hand a piece like `most_gc_rich` to the assistant, you can hand it the one-line docstring and the worked example together, which is a request specific enough to check rather than a mood. That is the entire payoff of decomposition: it turns one large, unverifiable request into several small, verifiable ones.

Exercise 48-1

SOLO

Decide, in writing, what `most_gc_rich` should do when two sequences tie exactly, add a second worked example that exercises that tie, and update the function so it satisfies both examples on purpose rather than by accident of how `max` breaks ties.

For your logbook this week, name one function from your own project plan where writing the worked example was harder than you expected, and say what that difficulty told you about the decomposition.

Week 11

49 Directing to resources: clinic

A companion clinic on building on machinery you did not write, rehearsing the move the codon-bias project and the finale both ask of you.

You now have the two halves separately: reading a library's documentation as a set of contracts, and catching the assistant when it invents one that does not exist. This clinic puts both halves to work on an actual small task, live, the way you will need to on this week's project and, eventually, on the finale. The task: given a DNA coding sequence, report whether its reverse complement, read as a coding sequence, translates cleanly to a protein with no internal stop codon.

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · Worker ·
Collaborator

Still to come: Developer

Deciding what the task needs before asking for anything

Before typing a single prompt, name the capabilities the task actually requires, the way you would decompose a task into functions. This one needs a reverse complement of a DNA sequence, a translation of a DNA sequence into a protein, and a check for whether a stop codon, shown by BioPython as `*`, appears anywhere before the final position of that protein. The first two are exactly the `Seq` methods you met two notes ago. The third is not a library operation at all, it is a small piece of logic over the result, which matters: you are about to direct the assistant to combine library calls, and knowing in advance which parts are the library's job and which part is yours is what keeps you from asking it to reinvent `.translate()` badly, or from quietly accepting logic it wrote for the one part you should have written yourself.

Checking the contracts before prompting

Two methods, two contracts to confirm from the actual documentation, not from memory or from what the assistant assumes. `Seq.reverse_complement()` takes no required arguments and returns a new `Seq`. `Seq.translate()` takes an optional `to_stop` argument; read its entry and you will find that when `to_stop=True`, translation stops at the first in-frame stop codon and the `*` is simply omitted from the result, rather than translation continuing past it the way the default behaves. That single optional argument changes which of two different questions the code answers, and choosing between them is a decision you make before prompting, not one you leave for the assistant to make silently.

Directing the assistant

With the contracts in hand, the prompt can be exact: "Using `Bio.Seq.Seq`, write a function `translates_cleanly(dna)` that takes a DNA coding sequence, returns

its reverse complement, translates it with `to_stop=False` so any internal stop codon stays visible in the result, and returns `True` if the resulting protein contains no `*` before its final character.” Notice how much of that sentence is contract, not code: which method, which argument, which behavior you are relying on. A vague version of the same request, “check if the reverse complement translates okay,” leaves the assistant to guess `to_stop`’s value the same way it once guessed a tie-breaking rule, and you would have no way to know which guess it made without reading the code closely enough that you might as well have specified it yourself.

Verifying the result, twice

The assistant’s answer gets checked in the two different ways this course has been building toward. First, against the documentation: did it call `.reverse_complement()` and `.translate()` with real arguments that mean what you asked, or does anything in the call look invented, the way `.gc_content()` did? Second, against a test, because a documented call can still be assembled into wrong logic:

```
from Bio.Seq import Seq

def translates_cleanly(dna):
    """True if the reverse complement's translation has no internal stop codon."
    protein = Seq(dna).reverse_complement().translate(to_stop=False)
    return '*' not in protein[:-1]

assert translates_cleanly('ATGGCCTGA') in (True, False)  # runs at all
print(translates_cleanly('ATGGCCTGA'))
```

Work out by hand what `'ATGGCCTGA'`’s reverse complement is, and what that translates to, and confirm the function’s answer against your own. That hand-worked case is the sanity check standing in for the test you cannot write against a library you trust as machinery: you are not testing that `.translate()` implements the genetic code correctly, you are checking that the pieces were assembled the way you specified.

Why this is the rehearsal it claims to be

Nothing about this clinic was larger than a single function. What it rehearsed was the shape of every later task that leans on a library: decide what capabilities you need before prompting, confirm their contracts in the real documentation, direct the assistant with the contract instead of a mood, and verify twice, once against the documentation and once against a hand-worked case. The codon-bias project this week, and the finale later, are this same shape, just with more pieces.

Exercise 49-1

AI: COLLABORATOR

Direct the assistant to write a function `longest_orf_protein(dna)` that finds the longest open reading frame in a DNA sequence starting at any `ATG` and translates it, using `Bio.Seq.Seq` for the translation step only; the search for `ATG` and the comparison of lengths is logic you specify, not a library call. Before prompting, decide and write down which part is the library's job and which is yours, the way this clinic did.

For your logbook this week, name one library call from this week's project that you verified against real documentation before trusting it, and one you are not fully sure you checked closely enough.

50 Delegate the full problem

You now hold three separate skills. You can specify a function by writing its contract as tests before any code exists. You can read a function the assistant produced and judge it against a short list of questions, refusing to believe it until the machine agrees. And you can take a task larger than one function, break it into named and testable pieces, and write that plan down before you prompt for anything. The capstone is where these three stop being separate exercises and become one continuous way of working, applied to a task that is deliberately larger than anything you could sit down and write on your own. That last point is not an accident or a cruelty; it is the entire purpose of the exercise. The role you are stepping into this week is the one the whole course has been climbing toward, which is to delegate a real job to the assistant and remain completely responsible for the result.

Be exact about what beyond your unaided reach actually means, because it is easy to misread it as a demand that you suddenly become a better programmer than you are. It is the opposite. You are not expected to be able to write the capstone from a blank page. You are expected to be able to produce it, correct and understood, by driving the assistant through a plan you designed and by verifying every piece it hands back. The gap between those two things, between writing a program yourself and producing a correct program you did not type, is the gap this course exists to close. A professional programmer would write the capstone unaided. You will instead decompose it, specify it, delegate the typing of each piece, and check everything, and if you do that well the finished program will be just as correct as the professional's, and you will understand it just as well, even though you could not have produced it keystroke by keystroke. That is a considerable and useful ability, and it is the one you are being asked to demonstrate.

The way you do it is simply the full loop assembled from the parts you already have, run once for each piece of the plan. You begin, before prompting anything, by decomposing the task into named functions with stated inputs and outputs, exactly as you practiced, so that you are holding a plan made of small pieces each of which you could read, test, and understand on its own. Then you take the pieces in dependency order, and for each one you run a short cycle. You write its contract as one or a few tests, seeded from the example in your plan and stiffened with the awkward cases you have learned to fear. You prompt the assistant for that one function, giving it only what it needs, which is the function's purpose, its inputs and outputs, and the tests it must satisfy. You read what it returns against your judging questions. You run the tests and let the machine deliver the verdict. If the tests pass, that piece is done and you move to the next, building it on top of a piece you have already proven. If the tests fail, you do not start over and you do not despair; you write a specific change request, naming the failing input, the wrong output, the right output, and the reason as far as you can see it, and you send it back and run the tests again. Nothing in that cycle is new. The only new thing is that you are running it many times, on a task whose finished form you

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · Worker ·
Collaborator · **Developer**

could not have written, and watching a program you understand assemble itself out of pieces you have each checked.

The iteration inside a single piece deserves a word, because it is where students most often lose their footing. When a piece fails and you send the assistant a specific change request, the loop should converge: the next version should be closer, and after one or two rounds the tests should pass. If instead the assistant keeps failing in the same way, or fixes your named case while breaking another, that is not a reason to keep hammering the same prompt harder. It is a signal, and it usually means one of two things. Either your specification is wrong or incomplete, so the assistant is optimizing for a target you did not actually mean, in which case you fix the tests and not the code. Or the piece is small enough and you now understand it well enough that you should simply write it yourself, because you have reached the point where delegating costs more than doing. Knowing when to stop asking the assistant and take the keyboard back is part of the skill, not a failure of it. The assistant is there to save you effort on pieces you have specified, not to be argued with indefinitely about a piece you could finish in five minutes.

Two things about verification change scale in the capstone and you should meet them deliberately. The first is that the project comes with a suite of tests already written, the same kind you have been running through the check widget, and it is tempting to treat passing that suite as the finish line. It is not. The provided tests are a floor, not a ceiling. They are the conditions the graders and the author knew to check, and passing them is necessary, but it is not proof, because no suite tests every input and the assistant is expert at producing code that satisfies the visible tests while failing the invisible ones. So you add your own tests, especially the nasty cases you have learned to distrust, and you treat the provided suite as the beginning of your checking rather than the end of it. The second is that passing tests, even your own, is still necessary rather than sufficient. A green result tells you the code is correct on the cases you thought of. It cannot tell you the code is correct on the cases you did not. The backstop for everything the tests miss is your own understanding of the pieces, which is why the rule that you never accept code you cannot read matters even more here than it did on a single function. If a function passes every test but you cannot explain why it works, you have not finished with it, because the day it meets an input none of your tests covered, your understanding is the only thing standing between a wrong answer and a wrong result you publish.

That last point is really about responsibility, and responsibility is the part of delegation that does not transfer. You will hand pieces of typing to the assistant, and that is fine, but you cannot hand it the ownership of the result. When you submit the capstone, you are submitting it, not the assistant, and you must be able to stand behind every function in it and explain what it does and why it is correct. If a reviewer, or a marker, or your future self points at a function and asks how it works, the answer the AI wrote it is not an answer, it is a confession that you delegated the thinking along with the typing. The whole architecture of the course, the specifying and the judging and the planning, exists so that you can use the assistant heavily and still own the outcome completely. Delegation done well is not the absence of responsibility. It is responsibility exercised through verification instead of through typing.

To make your verification visible, both to yourself and for assessment, you keep a

prompt journal alongside the capstone as you build it. Where the weekly logbook is a broad reflection, the prompt journal is narrow and practical: a running record, piece by piece, of what you asked the assistant for, where its answer was wrong, how you caught it, and what you did about it. Keeping it costs a little effort in the moment and pays back twice. It forces you to actually notice the failures rather than skimming past a bug you half saw, and by the end of the project it is a concrete record of your own judgment at work, which is exactly the thing the capstone is meant to demonstrate. A capstone that runs is worth less, in this course, than a capstone that runs and comes with a frank account of the half dozen places the assistant got it wrong and how you knew.

You will do most of this on the real project, so the exercises here are small and meant to sharpen particular moves in the loop.

Exercise 50-1

AI: DEVELOPER

Run one full cycle on one piece, in the order that makes it safe. Before you prompt for anything, write the acceptance criteria for a single function from your plan: what it must return for which inputs, including the awkward cases, in a form the machine can check. Then prompt the assistant for that function, giving it the purpose, the signature, and nothing you have not decided. Read what comes back. Run the criteria. The thing to notice is that you wrote the criteria while you still had no code in front of you to anchor your idea of what correct meant, which is the only condition under which delegating a whole piece is anything other than hoping.

Exercise 50-2

AI: DEVELOPER

Take a single function from your plan, prompt the assistant for it, and when its first attempt fails one of your tests, write the change request you would send back, containing the failing input, the wrong output, the correct output, and the reason. Notice how much of that you already knew from having written the test first. Send it. If the second version passes, the loop converged; if the third does not, stop, and go to the next exercise.

Exercise 50-3

SOLO

Take the keyboard back. Find a piece where the loop did not converge in two rounds, or where you can already see the answer, and write it yourself. Time both if you can. The skill this exercise trains is not programming, it is the judgment about when delegating has stopped paying, and it is a skill in exactly the same sense that specifying and judging are. An assistant argued with for twenty minutes over a function you could have typed in five is not saving you anything, and noticing that quickly is part of using it well.

Exercise 50-4

SOLO

Take a function that has just passed the provided tests and write one more test of your own, a nasty case the provided suite did not include, and run it. If it also passes, you have strengthened the contract. If it fails, you have just caught a bug that would otherwise have shipped, and you have proved to yourself that the provided suite was a floor and not a ceiling.

Exercise 50-5

AI: DEVELOPER

Finally, the exercise that decides whether you may keep the badge. Pick the function in your capstone that you delegated most heavily, the one you are least sure you could have written yourself, and write one paragraph explaining how it works, in your own words, with the assistant closed. Not what it does, which you know from its tests, but *how* it does it, line by line. If you cannot, that function is not finished, whatever its tests say, because you cannot own a result you cannot explain. Do the reading now and write the paragraph again. This is what it means to say that delegation transfers the typing and not the responsibility.

Exercise 50-6

SOLO

Write one prompt journal entry for a piece you built, recording where the assistant was wrong and how the machine, or your reading, revealed it.

For your logbook this week, step back from the individual pieces and describe the experience of producing something you could not have written unaided: which part of the loop, the planning, the specifying, the judging, or the iterating, did the most work in making the task possible, and name one function in your capstone that you are confident you could now explain in full to someone else, and one that you are not yet sure you could, because the second kind is where your remaining work is.

51 Sequence trees

This chapter is about clustering sequence based on the evolutionary distance between them.

Download the files you need for this project:

https://munch-group.org/bioinformatics/supplementary/project_files

- `seqdistproject.py` is an empty file where you must write your code.
- `test_seqdistproject.py` is the test program that lets you test the code you write in `translationproject.py`.

Put the files in a folder dedicated to this project. On most computers you can right-click on the link and choose “Save file as...” or “Download linked file”.

Measuring sequence distance

Clustering is based on the distances between all pairs of sequences. So before you can build your tree you must compute those distances and fill them into a table like that in the book. Here we break that task into three parts:

1. Compare two sequences
2. Make the Jukes-Cantor correction
3. Generate a (lower triangular) distance matrix

Compare two sequences

The first function you must write is one that finds the proportion of different bases between two sequences:

Write a function, `sequence_difference`, which takes two arguments:

1. A string, which represents a DNA sequence.
2. A string, which represents a DNA sequence of the same length as argument one.

The function must return:

- A float, which represents the proportion of different bases between the two sequences.

Example usage:

```
sequence_difference('AAATTAAA', 'AAAAAAA')
```

should return

0.25

Make the Jukes-Cantor correction

To take into account that some substitutions may fall on top of others you must do the Jukes-Cantor correction you read about in the book. The formula is like this:

$$K = -\frac{3}{4} \ln(1 - \frac{4}{3} * D)$$

Where D is the proportion of differences as returned by `sequence_diff` and K is the Jukes-Cantor corrected distance. In the top of `seqdistproject.py` it already says:

```
from math import log
```

That line makes the `log` (logarithm) builtin function from the `math` python library available to your program. Try to find its Python documentation to see how you use it.

Write a function, `jukes_cantor`, which takes one argument:

1. A float, which represents a proportion of different bases between two sequences.

The function must return:

- A float, which represents the Jukes-Cantor corrected distance corresponding the proportion of differences given as argument.

Example usage:

```
jukes_cantor(0.1)
```

should return

```
0.10732563273050497
```

Lower triangular distance matrices

This project is all about distances between pairs (of sequences), and what would be more natural than to put all the distances in a matrix so you can look up the distance between the sequences with indexes `i` and `j` as the matrix element in row `i` and column `j`. You already know how matrices can be represented by lists of lists. E.g. a matrix like this:

can be expressed as a list of lists like this:

```
[[0.0, 0.1, 0.3, 0.3, 0.1, 0.2],
 [0.1, 0.0, 0.2, 0.4, 0.1, 0.1],
 [0.3, 0.2, 0.0, 0.4, 0.2, 0.3],
 [0.3, 0.4, 0.4, 0.0, 0.2, 0.1],
 [0.1, 0.1, 0.2, 0.2, 0.0, 0.1],
 [0.2, 0.1, 0.3, 0.1, 0.1, 0.0]]
```

Notice how the diagonal is all zeros because these distances represent the distance of a sequence to itself. Also, notice that the part above the diagonal is a mirror the part below the diagonal (in bold). This is all a bit redundant, especially in this project where you will have to reduce the matrix as you group (or cluster) sequences together. We want something nice and lean where we only have the numbers we need – and that is the *lower triangular matrix*:

```
0.1
0.3 0.2
0.3 0.4 0.4
0.1 0.1 0.2 0.2
0.2 0.1 0.3 0.1 0.1
```

In Python this is still just a list of lists, only, each sublist now has the same length as its index in the big list (E.g. `[0.3, 0.2]` has index `2` in the list and has length `2`):

```
matrix = [[],
           [0.1],
           [0.3, 0.2],
           [0.3, 0.4, 0.4],
           [0.1, 0.1, 0.2, 0.2],
           [0.2, 0.1, 0.3, 0.1, 0.1]]
```

Here I am just writing it nicely. If you where to print that list of lists it would look like this:

```
[[], [0.1], [0.3, 0.2], [0.3, 0.4, 0.4], [0.1, 0.1, 0.2, 0.2], [0.2, 0.1, 0.3, 0.1, 0.1]]
```

Say your sequences had names: A, B, C, D, E, and F, then the above data structure represents distances between each pair like this:

```
A
B 0.1
C 0.3 0.2
D 0.3 0.4 0.4
E 0.1 0.1 0.2 0.2
F 0.2 0.1 0.3 0.1 0.1
  A   B   C   D   E   F
```

There is only one drawback with this reduced representation of the full square matrix: In the full matrix you can get the distance between the sequences with indexes `i` and `j` as both `matrix[i][j]` and `matrix[j][i]` because the part above and below the diagonal are the same. Using the lower triangular matrix, you must always use the *largest* index first. Using the smaller one first will give you an `IndexError`. So if you want the distance between sequences with index `2` and `4`, you must use the bigger index first (as the row index): `matrix[4][2]`.

Generate a distance matrix

Write a function, `lower_trian_matrix`, which takes one argument:

1. A list of strings. All strings have equal length and represent DNA sequences.

The function must return:

- A list of lists of floats, which represent the lower triangular matrix of Jukes-Cantor distances between DNA sequences given as argument.

Example usage:

```
sequences = ['TAAAAAAAAA',
             'TTAAAAAAAA',
             'AAAAAAAAAGG',
             'AAAAAAGGGG']
lower_trian_matrix(sequences)
```

here `lower_trian_matrix` should return:

```
[[],
 [0.08833727674228764],
 [0.30409883108112323, 0.4408399986765892],
 [0.6081976621622466, 0.8239592165010822, 0.18848582121067953]]
```

You should use `sequence_difference` to compute the proportion of differences between each pair of sequences and `jukes_cantor` to produce the corrected distance to fill into the matrix.

Start by figuring out what pairs of indexes you need and then figure out how you can make two nested for-loops generate them. Remember that the length of each sublist is equal to its index in the big list.

Clustering

Now that you have the distance matrix you are ready for the actual clustering. There are three steps to that:

1. Find the pair you want to join

2. Compute the distances between the joined pair and all other elements (linkage)
3. Keep going until you only have one left

Depending on how you choose which pair to join and how you compute the new distances for the joined pair determines what kind of clustering you do. Here we will try a centroid-like linkage called WPGMA. It does not work as well as UPGMA but is a bit easier to implement (you can look up WPGMA on wikipedia).

Find the pair to join

Here you want to be able to find the pair with the smallest distance. To do that we identify the cell in the matrix with the smallest value:

Write a function, `find_lowest_cell`, which takes one argument:

1. A list of lists, which represents a lower triangular distance matrix as returned by `lower_trian_matrix`.

The function must return:

- A list of two integers, which represent the row and column index of the cell with the smallest value in the matrix.

Remember that the row index is always smaller than the column index. The two indexes tell you which two elements to join next.

Example usage: Assuming that `matrix` is the lower triangular matrix returned by `lower_trian_matrix` in the previous example, then

```
find_lowest_cell(matrix)
```

Should return

```
[1, 0]
```

Decide on a linkage method

You also need a function that computes a new distance from two original ones using the the centroid-like linkage we have decided to use.

Write a function, `link`, that takes two arguments:

1. A float, which represents a matrix element.
2. A float, which represents another matrix element.

The function must return:

- A float, which is the average of the two arguments

Example usage:

```
link(0.4, 0.2)
```

Should return:

```
0.3
```

Perform the clustering

The three functions that do the actual clustering are complicated but you should be able to follow what they do. The first one updates the table to reflect that you join a pair. The second updates the list of sequence names (labels) to reflect that you joined a pair. The last one uses the two other functions to cluster pair until there is only one cluster left.

Your task is to carefully type the code for each function and to understand what every line of code does.

Updating labels

The function `update_labels` takes three arguments:

1. A list of strings representing sequence names.
2. An integer representing the index of a sequence name.
3. An integer representing the index of another sequence name.

The function does not return anything, but it updates the list of names to reflect that you joined a pair. If your list looks like this *before* you call the function:

```
labels = ['A', 'B', 'C', 'D']
```

Then *after* you call the function like this `update_labels(labels, 1, 0)`, the list will look like this:

```
['(A,B)', 'C', 'D']
```

Here is the function:

```
def update_labels(labels, i, j):  
  
    # turn the label at first index into a combination of both labels  
    labels[j] = "({},{})".format(labels[j], labels[i])  
  
    # Remove the (now redundant) label in the first index  
    del labels[i]
```


Updating the matrix

The function `update_table` takes three arguments:

1. A list of lists, which represents a lower triangular distance matrix.
2. An integer representing the index of one of the elements to join.
3. An integer representing the index of the other element to join.

The way this function is implemented, it is assumed that the second argument is always larger than the third argument. I.e. the second argument is a row index and the third argument is a column index.

The function does not return anything, but it updates the matrix to reflect that a pair has been joined. If your matrix looks like this *before* you call the function:

```
m = [[], [0.1], [0.3, 0.4], [0.6, 0.8, 0.2]]
```

Then *after* you call the function like this `update_table(m, 1, 0)`, the matrix will look like this:

```
[[], [0.35], [0.7, 0.2]]
```

Here is the function:

```
def update_table(table, a, b):

    # For the lower index, reconstruct the entire row (ORANGE)
    for i in range(0, b):
        table[b][i] = link(table[b][i], table[a][i])

    # Link cells to update the column above the min cell (BLUE)
    for i in range(b+1, a):
        table[i][b] = link(table[i][b], table[a][i])

    # Link cells to update the column below the min cell (RED)
    for i in range(a+1, len(table)):
        table[i][b] = link(table[i][b], table[i][a])

    # Delete cells we no longer need (lighter colors)
    for i in range(a+1, len(table)):
        # Remove the (now redundant) first index column entry
        del table[i][a]
    # Remove the (now redundant) first index row
    del table[a]
```

The colors refer to cell colors on the slide you I showed you at the lecture.

Do the clustering

Now onto the real task, the actual clustering. The function `cluster` takes two arguments:

1. A list of strings representing DNA sequences of equal length.
2. A list of strings representing sequence names.

The function returns:

- A string representing the generated clustering.

Here is the function:

```
def cluster(sequences, names):

    table = lower_trian_matrix(sequences)
    labels = names[:]

    # Until all labels have been joined...
    while len(labels) > 1:
        # Locate lowest cell in the table
        i, j = find_lowest_cell(table)

        # Join the table on the cell co-ordinates
        update_table(table, i, j)

        # Update the labels accordingly
        update_labels(labels, i, j)

    # Return the final label
    return labels[0]
```

Here is a simple example of how you can use your new clustering code:

```
names = ['A', 'B', 'C', 'D']
sequences = ['TAAAAAAAAA',
            'TTAAAAAAAA',
            'AAAAAAAAAGG',
            'AAAAAAGGGG']

tree = cluster(sequences, names)
print(tree)
```

On your own

From here on you are on your own. If you find a FASTA file with aligned (ungapped) homologous sequences, you can use the function below to read it into your program and try your code out on real-world sequences. I will leave it to you to figure out how it works.

```

def read_fasta(filename):
    f = open(filename, 'r')
    record_list = []
    header = ""
    sequence = ""
    for line in f:
        line = line.strip() ## get rid of whitespace and newline
        if line.startswith(">"):
            if header != "": ## if it is the first header
                record_list.append([header, sequence])
                sequence = ""
            header = line[1:]
        else:
            sequence += line
    record_list.append([header, sequence])
    return record_list

```

Week 12

52 Learning with an AI; how you know a fact is true

The shift from using the assistant to produce code to using it to learn.

Back in week one, this course mentioned a second mode the assistant can work in, the **Study and Learn agent**, and asked you to file it away for later. Later is now. Everything so far has been about the assistant as a producer: of code, of explanations, of drafts you read and test. This week the assistant's job changes to helping you learn something, biology as much as programming, and that shift needs its own version of the one habit you have been building all along.

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · Worker ·
Collaborator · Developer

A tutor, not an answer machine

Ask the ordinary assistant a factual question and, true to form, it predicts a plausible, fluent, confident answer and hands it straight to you. The Study and Learn agent is built to resist doing that. Ask it something and, more often than not, it answers with a question of its own, or a smaller example, or a hint about where to look, nudging you toward working the answer out rather than receiving it. This is not the assistant being unhelpfully coy. It is a deliberate design, because an answer you were handed and an answer you derived leave you in different places: one is a fact you are holding onto because a machine said it, the other is understanding you can rebuild yourself. The whole reason to reach for this mode is that the second kind is what you actually came here for.

The verification principle, applied to facts

You already have the habit for code: a claim about what a function does is plausible until a test or the running machine confirms it. Facts, including biological ones, work the same way, and the parallel is worth stating in full, because it is not a metaphor, it is the same principle wearing different clothes. A test is to a function's correctness what evidence is to a fact's truth. Code you did not write and did not test is plausible, not correct. A biological claim you read, from the assistant or anywhere else, and did not check against evidence or your own knowledge is plausible, not known. The assistant states biological facts with exactly the same fluent confidence whether they are true or false, for exactly the same reason it can hand you a wrong function with total confidence: it is predicting a plausible continuation, not consulting a source and reporting what it found.

Try this concretely. Ask the assistant, in its ordinary mode, why the third position of a codon is often called the "wobble" position. You will get a fluent, plausible-sounding paragraph. Do not file it away yet. Find it in a source outside the assistant, a textbook or a reputable reference, and check whether the explanation the assistant gave, base pairing flexibility at that position of the anticodon, matches what the

source actually says, and whether any specific claim, which bases wobble with which, was stated more narrowly, more loosely, or differently than the assistant put it. That checking step is not optional extra credit. It is the entire difference between a fact you were told and a fact you know.

What to do when the tutor mode still just tells you

The Study and Learn agent nudges rather than answers, but it is still the same underlying model, still predicting plausible text, and nothing about tutor mode makes it incapable of confidently stating something false while it nudges you. A hint can be a wrong hint. The verification habit does not relax just because the assistant is behaving more like a teacher; if anything it matters more, because a wrong hint delivered patiently is more persuasive than a wrong answer delivered bluntly. Whatever you derive with its help, you still check the result against a real source or against running the code, exactly as before.

Exercise 52-1

AI: COLLABORATOR

Open the Study and Learn agent and pick one thing from this course you have not fully settled in your own mind, a concept from the Python side or a piece of biology. Note the badge: you are not asking it to explain the thing to you, which is what week one would have had you do. You are working on it together. Tell it what you already understand, tell it exactly where your understanding runs out, and let it nudge from there. Write down each hint it gave you and what you did with it. Then verify the understanding you arrived at against a source that is not the assistant itself: a textbook, a documented reference, or, where it applies, a small program you write and run.

Exercise 52-2

SOLO

Now close the assistant. Tomorrow, with nothing open in front of you, write the same explanation out again from memory in your own words, and only then compare it with what you wrote yesterday. The second version is the one that counts, and the gap between the two is the honest measure of whether you learned the thing or merely watched it being explained well. Keep both versions. In Chapter 60 you will do exactly this under exam conditions, and it will go a great deal better if this is not the first time you have tried it.

For your logbook this week, describe one fact, biological or otherwise, that you checked against a real source this week rather than accepting from the assistant, and say whether the source confirmed it, corrected it, or left it more uncertain than either you or the assistant first assumed.

53 Finding genes

This chapter is a programming project where you will find open reading frames in the genome of a particularly virulent strain of E. coli.

In this project, you will analyze DNA to identify the open reading frames (ORFs) and the proteins they encode.

Download the files you need for this project:

https://munch-group.org/bioinformatics/supplementary/project_files

- `e_coli_O157_H157_str_Sakai.fasta` contains the full genome of the *Escherichia coli* O157:H7 Sakai strain (yes the full genome).

You also need to download the two project files:

- `orfproject.py` is an empty file where you must write your code.
- `test_orfproject.py` is the test program that lets you test the code you write in `orfproject.py`.

Put all three files in a folder dedicated to this project. On most computers you can right-click on the link and choose “Save file as...” or “Download linked file”.

For your convenience, the file `orfproject.py` already contains three global *constants* (variables that must never be changed by the code). One is a dictionary `codon_map`, which maps codons to letters that represent amino acids. The other two are a string, `start_codon`, and a list, `stop_codons`. You can refer to these three variables in your code, but obviously, never change them.

The project has three parts.

1. In the first part of the project, you will write the code necessary to identify ORFs in a long genomic sequence.
2. In the second part, you will use the code you wrote on the programming project about translating DNA to translate each ORF into a protein sequence.
3. In the third part, you will use the code from parts one and two together to find candidate proteins encoded by ORFs in a genomic sequence.

Start by reading through the exercise before you do anything else. That way you have a good overview of the tasks ahead. Here is a mind map of how we split the larger problem into smaller bits and how they fit together:

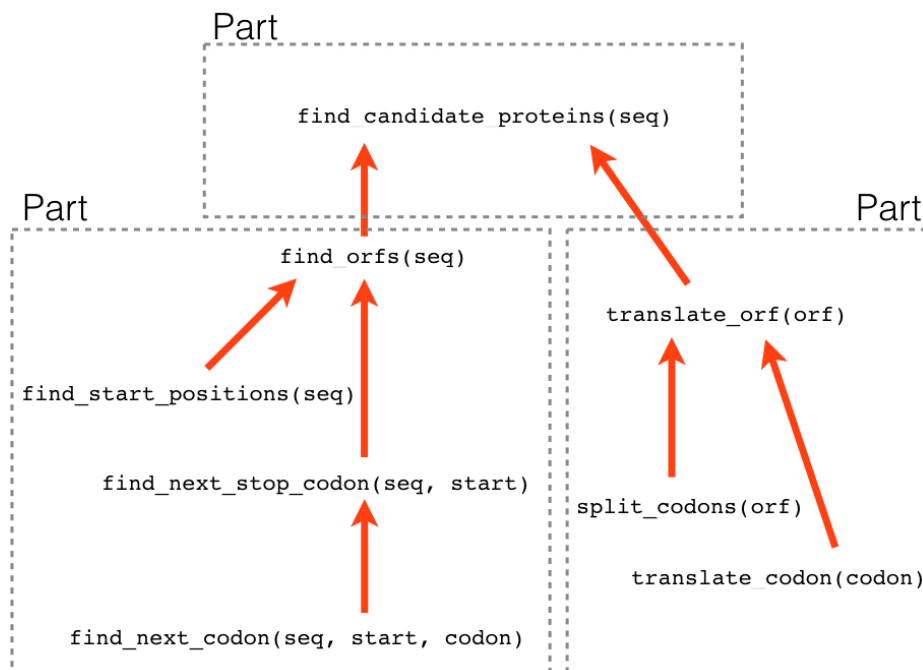


Figure 53.1: Overview

Finding Open Reading Frames

Find the start positions of ORFs in a DNA sequence

The first task is to write a function that finds all the possible positions where an ORF can begin.

Write a function, `find_start_positions`, which takes one argument:

1. A string, which is a DNA sequence.

The function must return:

- A list of integers, which represent the indexes of the first base in start codons in the DNA sequence argument.

Example usage:

```
find_start_positions('TATGCATGATG')
```

should return

```
[1, 5, 8]
```


Your function should contain a for-loop that iterates over all possible positions in a DNA string where a codon can begin. Not surprisingly, these are all the positions except for the last two. So start out with this:

```
def find_start_positions(seq):
    for i in range(len(seq) - 2):
        print(i)
```

Now, instead of just printing `i`, try and make it print the three bases following `i` using the slicing technique:

```
triplet = seq[i:i+3]
```

I.e. if your sequence is `'TATGCATGATG'` it should first print `'TAT'` then `'ATG'` then `'TGC'` and so on.

When you have this working you should change the code so that triplets are only printed if they are start codons. You can use an if-statement that tests if each triplet is equal to `start_codon`.

Then try and make your function print `i` only when `i` is the first base of a start codon.

Finally, modify the function so all the relevant values of `i` are collected in a list using the same technique as in the `split_codons(orf)` function, and then return this list from the function.

Finding the next occurrence of some codon in an ORF

Now that you can find where the ORFs begin in our sequence you must also be able to identify where each of these end. As you know, an ORF ends at any of three different stop codons in the same reading frame as the start codon. So, starting at the start codon of the ORF, we need to be able to find the next occurrence of some specific codon. I.e. you should look at all codons after the start codon and find the first occurrence of some specified codon. If the function does not find that codon in the string it should return `None`.

Write a function, `find_next_codon`, that takes three arguments:

1. A string, which is the DNA sequence.
2. An integer, which is the index in the sequence where the ORF starts.
3. A string, which is the codon to find the next occurrence of.

The function must return:

- An integer, which is the index of the first base in the next in-frame occurrence of the codon. If the function does not find that codon in the string it should return `None`.

Example usage:

```
find_next_codon('AAAAATTTAATTTAA', 1, 'TTT')
```

should return

```
10
```

Your function should contain a for-loop that iterates over all the relevant starts of codons. Remember that no valid codon can start at the last two positions in the sequence. E.g. if the second argument is 7 and the length of the sequence is 20 then the relevant indexes are 7, 10, 13, 16.

Start by writing a function just with a for-loop that lets you print these indexes produced by `range`. Figure out how to make the `range` function iterate over the appropriate numbers.

```
def find_next_codon(seq, start, codon):  
    for idx in range( ?? ):  
        print(idx)
```

When you have that working, use the slicing technique to instead print the codons that start at each index.

Finally, add an if-statement that tests if each codon is equal to `codon`. When this is true, the function should return the value of `idx`.

Finding the first stop codon in an ORF

Now that you can find the next occurrence of any codon, you are well set up to write a function that finds the index for the beginning of the next in-frame *stop codon* in an ORF.

Write a function, `find_next_stop_codon`, that takes two arguments:

1. A string, which is the DNA sequence.
2. An integer, which is the index in the sequence where the ORF starts.

The function must return:

- An integer, which is the index of the first base in the next in-frame stop codon. If there is no in-frame stop codon in the sequence the function should return `None`.

Example usage:

```
find_next_stop_codon('AAAAATAGATGAAAA', 2)
```

should return

```
5
```

Here is some inspiration:

1. You should define a list to hold the indexes for the in-frame stop codons we find.
2. Then we loop over the three possible stop codons to find the next in-frame occurrence of each one from the start index. You can use `find_next_codon` for this. Remember that it returns `None` if it does not find any. If it does find a position you can add it to your list.
3. At the end, you should test if you have any indexes in your list.
4. If you do, you should return the smallest index in the list. I.e the ones closest to the start codon.
5. If you did not find any stop codons the function must return `None` to indicate this.

Finding ORFs

Now you can write a function that uses `find_start_positions` and `find_next_stop_codon` to extract the start and end indexes of each ORF in a genomic sequence.

Write a function, `find_orfs`, that takes one argument:

1. A string, which is a DNA sequence.

The function should return:

- A list, which contains lists with two integers. The list returned must contain a list for each ORF in the sequence argument. These lists each contain two integers. The first integer represents the start of the ORF, the second represents the end. The function should handle both uppercase and lowercase sequences.

Example usage:

```
find_orfs("AAAATGGGGTAGAATGAAATGA")
```

should return

```
[[3, 9], [13, 19]]
```

Start by using `find_start_positions` to get a list of all the start positions in sequence:

```
def find_orfs(seq):
    start_positions = find_start_positions(seq)
```

When you have that working, add a for-loop that iterates over the start positions. Inside the for-loop, you can then get the next in-frame stop codon for each start position by calling `find_next_stop_codon`. Try to print the start and end indexes you find to make sure the code does what you think.

Finally, you need to add a `[start, stop]` list for each ORF to the big list that the function returns. To append a list to a list you do write something like this:

```
orf_coordinate_list.append([start, stop])
```

Test your function. Chances are that some of the end positions you get are None. This is because some of the start codons were not followed by an in-frame stop codon. Add an if-statement to your function that controls that only start-stop pairs with a valid stop coordinate are added to the list of results.

Translation of open reading frames

We need to translate the reading frames we find into the proteins they may encode. So why not use the code you already wrote in the programming project where you translated open frames? Copy the content of `translationproject.py` into `orfproject.py`. Now you can use the function `translate_orf` to translate your ORFs.

Put everything together

Read in genomic sequences

The file `e_coli_O157_H157_str_Sakai.fasta` contains the genome that we want to analyze to find open reading frames. This is an especially nasty strain of *Escherichia coli* O157:H7 isolated after a massive outbreak of infection in school children in Sakai City, Japan, associated with consumption of white radish sprouts.

You can use the function below to read the genome sequence into a string.

```
def read_genome(file_name):
    f = open(file_name, 'r')
    lines = f.readlines()
    header = lines.pop(0)
    substrings = []
    for line in lines:
        substrings.append(line.strip())
    genome = ''.join(substrings)
    f.close()
    return genome
```

Now for the grand finale: Using `read_genome`, `find_orfs` and `translate_orf` you can write a function that finds all protein sequences produced by open reading frames in the genome.

Write a function, `find_candidate_proteins`, that takes one argument:

1. A string, which is a genome DNA sequence.

The function must return

- A list of strings, which each represent a possible protein sequence.

Note that this is a full genome so finding all possible proteins will take a while (~5 min.). You can start by working on the first 1000 bases:

Example usage:

```
genome = read_genome('e_coli_0157_H157_str_Sakai.fasta')
first_1000_bases = genome[:1000]
find_candidate_proteins(first_1000_bases)
```

should return

```
['MSLCGLKKESLTAASELVTCRE*', 'MKRISTTITTTITTTITTTITTTGNGAG*',
'MQNVFCGLPIFWKAMPGRGRWPPSSLPPPKSPTTWWR*', 'MPGRGRWPPSSLPPPKSPTTWWR*',
'MLYPISAMPNVFLPNF*', 'MPNVFLPNF*', 'MSCMALVC*', 'MALVC*']
```

The function should call `find_orfs` to get the list of start-end pairs. For each index pair, you must then slice the ORF out of the sequence (remember that the end index represents the first base in the stop codon), translate the ORF to protein, and add it to a list of proteins that the function can return.

i Hint

To check your result note that all returned sequences should start with a start codon `'M'`, end with a stop codon `'*'` and contain no stop codons in the middle.

On your own

This is where this project ends, but *you* can continue if you like. Given a long list of candidate proteins of all sizes, what would you do to narrow down your prediction to a smaller set of very likely genes? If you have some ideas, then try them out.

- Maybe you can rank them by length? What is the expected minimum length of proteins?
- Maybe you can look for a Shine-Delgarno motif upstream of the start codon? You know how to do that from the lectures.
- You can also try to BLAST them against the proteins in Genbank. The true ones should have some homologs in other species.

54 Dataframes: a table you can plot

Your data finally gets a container that fits it: a table with named columns, that you can question and draw.

You have just come out of the ORF project in Chapter 53 with a program that walks through a bacterial genome and finds every open reading frame in it. Thousands of them. And then what? You can print them, which gives you a wall of text that nobody, including you, will ever read. You can collect them in a list of lists, which you know how to do, and then write a for-loop every single time you want to know something as ordinary as how long the average one is.

There is a better container for this, and it is the last new data type this course hands you. It is called a **dataframe**, and it is exactly what it sounds like: a table, with rows and columns, where the columns have names. It comes from a library called pandas. The name has nothing to do with the animal, sadly. It is short for *panel data*.

Nothing you already know stops being true here. Square brackets still look things up. Methods are still things you ask an object to do, in the “Hey table, do this to yourself!” sense from Chapter 15. A method call is still substituted by the value it returns, so you can keep asking questions of what comes back. A dataframe is just one more object with a lot of methods.

This chapter runs. Download the notebook and the table below, put them in the same folder, and run every cell as you read.

The table is just a text file

Before pandas gets its hands on it, let us look at what we actually have. It is a file, and you know how to open a file and read lines out of it from Chapter 26.

```
with open('data/orfs.csv') as f:
    for line_number, line in enumerate(f):
        print(line, end='')
        if line_number == 4:
            break
```

```
orf,strand,frame,start,length,gc_content,stop_codon
orf0001,+,2,353,2463,0.528,TGA
orf0002,+,0,2817,933,0.563,TAA
orf0003,+,0,3750,1287,0.531,TAA
orf0004,+,2,7382,408,0.559,TGA
```

That is the whole mystery. The first line is a row of column names, every line after it is one open reading frame, and the fields are separated by commas. A file like this is called a *CSV file*, for comma-separated values, and it is the most boring and most useful file format in science.

You could turn this into something workable yourself. Read the lines, `split(',')` each one, throw the first line in a dictionary of column names, convert the numbers with `int` and `float`. Every piece of that you have already written at some point in this course. Doing it once teaches you something. Doing it every time you meet a table is a waste of a perfectly good life.

Note

The columns are `orf` (a name), `strand` (which of the two DNA strands the reading frame is on), `frame` (which of the three reading frames), `start` (where in the genome it begins), `length` (how many bases long it is), `gc_content` (the fraction of its bases that are G or C), and `stop_codon` (which of the three stop codons ended it). These are real numbers from the real *E. coli* O157:H7 Sakai genome, produced by the same kind of program you wrote in the project.

Handing the file to pandas

```
import pandas as pd
```

The `as pd` part gives the library a shorter name for the rest of the notebook, so you can write `pd.read_csv` instead of `pandas.read_csv`. You are allowed to pick any name you like. Do not. Everyone in the world writes `pd`, and code you read for the rest of your life will assume you did too.

Now the entire file, in one line:

```
orfs = pd.read_csv('data/orfs.csv')
orfs.head()
```

	orf	strand	frame	start	length	gc_content	stop_codon
0	orf0001	+	2	353	2463	0.528	TGA
1	orf0002	+	0	2817	933	0.563	TAA
2	orf0003	+	0	3750	1287	0.531	TAA
3	orf0004	+	2	7382	408	0.559	TGA
4	orf0005	+	1	8254	954	0.517	TAA

`pd.read_csv` opened the file, worked out the column names from the first line, worked out by itself that `start` and `length` hold whole numbers while `gc_content` holds decimal numbers, and handed back one value. That value is now sitting in `orfs`.

Then we said `orfs.head()`. In the anthropomorphising voice: “Hey table, show me your head!” The head of a table is its first five rows. We ask for the head rather than the whole thing because the whole thing is seven thousand rows long and your notebook would not thank you.

What you just made is called a **dataframe**. Let us ask it about itself.

```
print(type(orfs))
print(orfs.shape)
print(list(orfs.columns))
```

```
<class 'pandas.core.frame.DataFrame'>
(7189, 7)
['orf', 'strand', 'frame', 'start', 'length', 'gc_content', 'stop_codon']
```

`shape` is the number of rows and the number of columns, as a tuple, in that order — you met tuples as the type Python reaches for when it wants to hand you two things at once. `columns` is the column names.

Notice that `head()` has parentheses and `shape` does not. That difference matters and it is not arbitrary. `head` is a *method*: something the table **does** when you ask it to, and like every function call it needs parentheses even when there is nothing to put inside them. `shape` is an *attribute*: something the table simply **is**, a value sitting on the object waiting to be read. Asking for `orfs.shape()` will fail, because a tuple is not something you can call.

Tip

When you get `TypeError: 'tuple' object is not callable`, you have put parentheses after something that was never a method. Take them off and try again.

Exercise 54-1

SOLO

Decide what you think each of these gives you before you run it. `orfs.head(3)`. Then `orfs.tail()`. Then `orfs.tail(1)`. Write your prediction down, then run all three and see whether the machine agrees with you.

Pulling out a column

A table is a lot of things at once. Most questions are about one column.

```
orfs['length']
```



```

0      2463
1      933
2     1287
3      408
4      954
...
7184    522
7185    870
7186    489
7187    717
7188    354
Name: length, Length: 7189, dtype: int64

```

Square brackets with a name inside them. You have seen this exact syntax before, in Chapter 17, where the thing inside the brackets was a key and what came back was the value stored under it. It is the same idea here: the column name is the key, and the column is what comes back.

Look at what got printed. On the left is the row number, on the right is the length of that ORE, and at the bottom Python tells you two things about the column as a whole: `Name: length`, because the column remembers what it is called, and `dtype: int64`, because the column remembers that everything in it is a whole number.

A single column has a name of its own. It is called a **Series**. A dataframe is a table; a Series is one column of it.

```

lengths = orfs['length']
print(type(lengths))
lengths.head()

```

```
<class 'pandas.core.series.Series'>
```

```

0      2463
1      933
2     1287
3      408
4      954
Name: length, dtype: int64

```

Do the substitution in your head, because it is the whole trick to reading pandas code. `orfs['length']` is *substituted by* a Series. Once it is a Series, you can ask that Series to do anything a Series knows how to do — including `head()`, which is why `orfs['length'].head()` works without you ever needing a variable in between.

Asking a column a question

```
print(lengths.mean())
print(lengths.max())
print(lengths.min())
```

```
811.8465711503686
14412
300
```

So the average open reading frame in this genome is about eight hundred bases long, the longest is a monster of over fourteen thousand, and the shortest is three hundred, because the program that made the table threw away everything shorter than that.

You have written `mean` before, or near enough. A total, a counter, a for-loop adding things up, a division at the end. `mean()` is that loop, written once by somebody else, and correct. That is the entire argument for using a library.

The numbers come out with more decimals than anybody wants. `round` is an ordinary built-in function and it works here like it works anywhere:

```
print(round(lengths.mean(), 1))
```

```
811.8
```

Exercise 54-2

SOLO

The `gc_content` column holds the fraction of each ORF's bases that are G or C. The *E. coli* genome as a whole sits at about 0.51. Predict, before you run anything, whether the mean GC content of the ORFs will come out above or below that, and why coding sequence might differ from the genome average. Then work out the mean, the smallest and the largest, and see whether your reasoning survived contact with the data.

When you want all of that at once, there is a method for it:

```
orfs['gc_content'].describe()
```

```
count    7189.000000
mean      0.521254
std       0.051067
min       0.267000
25%       0.502000
50%       0.529000
75%       0.552000
max       0.701000
Name: gc_content, dtype: float64
```

`count` is how many values there are, `mean` and `std` describe the middle and the spread, `min` and `max` are the extremes, and the three lines with percentages are the quarter marks: a quarter of the ORFs have a GC content below the 25% figure, half of them are below the 50% figure, and so on. `describe()` is the first thing to reach for when you have a column you have never looked at.

Counting the categories

Not every column is a number. `stop_codon` holds one of three short strings, and the question you want to ask of a column like that is not what its average is but how often each value turns up.

You have done this before, by hand, in exercise 25-2: make an empty dictionary, loop over the values, and add one to the count under each key. Here is that loop, again written once by somebody else:

```
orfs['stop_codon'].value_counts()
```

```
stop_codon
TAA      3836
TGA      2620
TAG       733
Name: count, dtype: int64
```

And here the method earns its keep, because that result is biology. All three stop codons end a protein equally well as far as the ribosome is concerned, and yet `TAA` shows up here more than five times as often as `TAG`. That is a real and well-documented bias in *E. coli*, and you have just measured it in one line from a file on your own laptop.

Exercise 54-3

AI: COMPARER

Ask the assistant which of the two DNA strands it expects most open reading frames in a bacterial genome to be found on, and to explain its reasoning. Write down its answer. Then run `value_counts()` on the `strand` column and see what the genome says. If the assistant hedged, decide whether the hedge was honest or evasive. Note the outcome in your logbook.

More than one column at a time

If one column name in the brackets gives you one column, a list of column names gives you several:

```
orfs[['length', 'gc_content']]
```

	length	gc_content
0	2463	0.528
1	933	0.563
2	1287	0.531
3	408	0.559
4	954	0.517
...	...	...
7184	522	0.552
7185	870	0.531
7186	489	0.507
7187	717	0.509
7188	354	0.441

Those double brackets are the single most confusing piece of punctuation in pandas, and they stop being confusing the moment you do the substitution properly. The outer brackets are the lookup, exactly as before. The inner brackets are a *list*, which is what you are handing to the lookup. Reduce the inside first: `['length', 'gc_content']` is a list of two strings, and `orfs[ that list ]` is a lookup of two columns.

Two columns is still a table, so what comes back is a dataframe, not a Series. And a dataframe has a `mean()` too:

```
orfs[['length', 'gc_content']].mean()
```

```
length      811.846571
gc_content    0.521254
dtype: float64
```

One mean per column, handed back as a Series with the column names down the side.

FAQ

Q: Can I not just write `orfs.mean()` and get the mean of everything?

A: Try it. It will complain, because `orf`, `strand` and `stop_codon` hold text, and pandas has no idea how to average a word. Ask for the columns that are numbers and it will do exactly what you asked. This is a well-behaved error message: read it rather than guessing.

Putting the table in order

```
orfs.sort_values('length', ascending=False).head()
```

	orf	strand	frame	start	length	gc_content	stop_codon
369	orf0370	+	0	587409	14412	0.612	TAG
847	orf0848	+	0	1295841	8421	0.543	TAA
3505	orf3506	+	1	5381575	6342	0.529	TGA
3502	orf3503	+	2	5367233	6324	0.488	TAA
7139	orf7140	-	2	5416458	5103	0.492	TAA

Two things are happening on that line and both are worth slowing down for.

`ascending=False` is a keyword argument, the same kind of thing as the `end=' '` you have been passing to `print`. Without it you get the smallest first, which is the sensible default.

And then `.head()` is stuck on the end. This is called *chaining*, and it needs no new rules to understand — only the substitution habit you already have. `orfs.sort_values('length', ascending=False)` is a method call, and a method call is substituted by the value it returns, which here is a **new** table, sorted. Once it is a table, you can ask that table for its head. So the line reads: sort the table, then take the head of the result.

Notice the word *new*. Sorting did not rearrange `orfs`. Run `orfs.head()` again if you do not believe me. Almost everything in pandas works this way: you get a new value back and the original is left alone, which is a mercy, because it means no experiment you run on a table can quietly ruin it.

The top row of that sorted table is a real open reading frame over fourteen thousand bases long. Whether it is a real gene is another question entirely, and a much better one.

Exercise 54-4

SOLO

Predict what `orfs.sort_values('gc_content').head()` will show you, in words, before you run it: which end of the GC range, and roughly what numbers. Then run it. Then work out how to see the *other* end of the same column without writing `ascending=False` — there is a method you already know that will do it.

Your own function, applied to a column

Everything so far has been you asking the library for things it already knew how to do. Here is the join between the library and you.

Write an ordinary function. Nothing new, nothing pandas-flavoured, just a function that takes one length and returns one word:

```
def size_class(length):
    if length < 500:
        return 'short'
    if length < 1500:
        return 'medium'
    return 'long'

print(size_class(300))
print(size_class(1000))
print(size_class(5000))
```

```
short
medium
long
```

Now hand that function to a column, and the column will run it once for every value in it:

```
orfs['size'] = orfs['length'].apply(size_class)
orfs.head()
```

	orf	strand	frame	start	length	gc_content	stop_codon	size
0	orf0001	+	2	353	2463	0.528	TGA	long
1	orf0002	+	0	2817	933	0.563	TAA	medium
2	orf0003	+	0	3750	1287	0.531	TAA	medium
3	orf0004	+	2	7382	408	0.559	TGA	short
4	orf0005	+	1	8254	954	0.517	TAA	medium

There is a new column on the right-hand side of the table. Assigning to a column name that does not exist yet creates it, the same way assigning to a key that does not exist yet creates it in a dictionary.

Look very carefully at what got passed to `apply`. It is `size_class`, not `size_class()`. No parentheses. You are not calling your function and handing over its result; you are handing over the function *itself*, so that `apply` can call it seven thousand times on your behalf. A function is a value like any other, and this is the first time in this course that being a value has actually bought you something.

! Important

`apply(size_class())` is the mistake everyone makes once. It calls your function immediately, with no argument, which raises a `TypeError` about a missing argument before `apply` ever gets started. If you see that error, count your parentheses.

Exercise 54-5

AI: DRAFTER

Write a function `gc_class(fraction)` that returns `'low'` for a GC content below 0.45, `'high'` for one above 0.55, and `'medium'` in between. Apply it to the `gc_content` column, put the result in a new column called `gc_class`, and count the classes with `value_counts()`.

Then ask the assistant to write the same function for you and compare it to yours. Look in particular at what each version does with the exact values 0.45 and 0.55: do the two functions agree? If they do not, neither of you is necessarily wrong, but one of you has made a decision without noticing. That is the kind of thing that goes in the logbook.

From a table to a picture

Seven thousand rows of numbers do not fit in a human head. A picture of seven thousand rows does.

You met `iplot` briefly in Exploring a table with `iplot`. This is where it earns its place. You hand it the whole table and it gives you dropdown menus: what goes on the x-axis, what goes on the y-axis, what splits the colours, what to break into separate panels, and what kind of plot to draw.

```
from iplot_widget import iplot  
  
iplot(orfs)
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

HBox(children=(HBox(children=(Box(children=(Dropdown(description='X:', options=('none', 'orf

Output()

Output()

Output()

Output()

Try these, in this order, pressing “Show plot” each time:

Set `x` to `length` and the plot type to `hist`. That is the distribution of ORF lengths, and the shape of it is the reason nobody trusts a short ORF: there are enormous numbers of tiny ones, and they are mostly noise, stop codons happening to fall a certain distance apart by chance.

Set `x` to `gc_content` and `y` to `length`, with the type `scatter`. Now you are asking whether longer reading frames have a different base composition from short ones. Look at it and decide for yourself what it shows.

Set `x` to `stop_codon` and the type to `count`. That is the `value_counts()` result you computed above, as a picture.

Set `x` to `size` and `y` to `gc_content` with the type `box`, and then set `hue` to `strand`. The `size` column only exists because you wrote `size_class` yourself a few cells ago, and it is now steering a figure.

i Note

Under the plot there is a fold labelled “**Show code for plot**”. Open it. Inside is the seaborn code that would produce the figure you are looking at, written out in full.

That fold is the point of this whole section. The widget is a piece of software writing code for you. So is the assistant. In both cases the honest response is the same one: read what it wrote, work out what each argument is doing, and decide whether it says what you meant. The difference is that the widget can only ever produce a plot of the table you handed it, so it is a very safe place to practise reading generated code before you have to do it where the stakes are higher.

Exercise 54-6

AI: COMPARER

Pick one of the four views above and get the widget to draw it. Copy the code out of the “Show code for plot” fold and keep it.

Now, in the browser, describe the same figure to the assistant in words — the columns, what is on each axis, what kind of plot — and ask it to write the seaborn code for you. Do not tell it what the widget produced.

Put the two versions side by side. Which arguments do they agree on? Where they differ, run both and look at the two figures. One of them may simply be a different taste in defaults, and one of them may be wrong. Deciding which is which is the skill. Logbook entry: what the assistant produced, how it differed, and how you knew which to keep.

What we have deliberately left out

This has been a small chapter about a very large library, and you should know the shape of what is missing, so you recognise it when you meet it.

You cannot yet pick out *rows* by a condition — every ORF longer than a thousand bases, say. You cannot yet group the table by one column and compute something per group, which is how you would get the mean length for each stop codon in one go. You cannot yet join two tables together on a shared column, and you have not met what pandas does when a value is missing. Every one of those is a normal thing to want and every one of them has a one-line answer.

What you can do is the part that comes first anyway, and that most people skip: get a real table into memory, find out what is in it, ask each column what it looks like, put your own function to work on it, and draw it. When you need one of the missing pieces, you now have somewhere to put the answer, whether you find it in the documentation or ask for it.

Exercise 54-7

SOLO

Your logbook entry for this week. In three or four sentences, describe one thing you found in this table that you did not expect, how you found it, and what you would need to check before you believed it. The last part is the one that matters.

Week 13

55 Finale: formulating and planning your own tool

The week-thirteen kickoff, modeled live.

i Draft stub — planned contents

A student formulates a small tool that is useful in molecular biology, something their own background lets them pose, understand, and sanity-check, and plans it from a scaffolded blank page: deciding the pieces, naming them, stating inputs and outputs, and writing the first tests before prompting anything. Describes the floor, a tool over the supplied gene CSV held in a dataframe, and the ceiling, a tool built on a public resource such as BioPython or a REST API. Refers to the dataframes note and the documentation note.

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · Worker ·
Collaborator · Developer

56 Finale build clinic

You arrive at this point holding a plan and a first set of tests for a problem you chose yourself. Nothing about the building that follows is new. It is the loop you have already run many times, applied now to your own decomposition rather than to one somebody handed you, and repeated once for every piece in your plan. What is different is that nobody will tell you which piece to build next, nobody will tell you when a piece is finished, and nobody will notice on your behalf when an answer is quietly wrong. Those three judgments are the whole of the work, and they are yours.

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · Worker ·
Collaborator · **Developer**

Take the pieces in dependency order

Start with a piece that depends on nothing else. In almost every plan there is at least one, usually a small computation over a value passed straight in, and it is the right place to begin because when it fails there is nowhere else for the fault to hide. Once it is proven you build the next piece on top of a foundation you already trust, and you keep doing that until the piece that faces you, the one that reports or plots or prints, is the last thing standing.

Resist the temptation to build the interesting piece first. The interesting piece is usually the one furthest up the dependency chain, and building it early means testing it against pieces that are not yet proven, which means that when something goes wrong you cannot tell whose fault it is. Build boring first. The whole reason the loop stays comfortable is that at any moment exactly one thing in your program is unproven, and it is the thing you are looking at.

The cycle for a single piece

For each piece the cycle is the same five steps, in the same order.

You write the contract as tests, seeded from the hand-worked example already sitting in your plan's docstring, and stiffened with at least one awkward case. You prompt the assistant for that one function and nothing else, giving it the function's purpose, its inputs and outputs, and the tests it has to satisfy. You read what comes back against your judging questions, and you refuse to run anything you cannot read. You run the tests and let the machine deliver the verdict. And when the tests pass, you write the journal entry for that piece before moving on, because a failure you fixed ten minutes ago is already half forgotten.

The order matters at both ends. Tests before the prompt, so that the assistant is aiming at a target you set rather than one it invents. Journal after the pass, so that the record is written while you still remember what actually went wrong rather than reconstructed at the end of the week from a program that now works.

Prompting for one piece and only one piece

The most common way a build clinic goes sideways is that the assistant is allowed to run ahead. You ask for one function and it returns that function plus two helpers it decided you needed, or plus the caller that ties three pieces together, or plus a rewritten version of a piece you had already tested and proven.

Do not accept that. A function you did not plan is a function you did not decide you needed, and every line of it is code you now have to read, understand, test and stand behind, bought at a price you did not agree to pay. If the assistant hands you more than you asked for, take the piece you wanted, discard the rest, and prompt again for the next piece when you actually reach it. If while doing that you find yourself thinking that one of the extra pieces was in fact a good idea, that is a change to your plan, and it belongs in the plan first, with a name and stated inputs and outputs and a hand-worked example, before you accept a line of its code.

When a piece will not converge

You write a precise change request, naming the failing input, the wrong output, the right output, and the reason as far as you can see it. The next version should be closer. If two or three rounds of that leave you no nearer, stop and read the situation rather than prompting harder.

Usually one of two things is true. Either your specification is wrong or incomplete, and the assistant is faithfully hitting a target you did not mean, in which case the thing to fix is the test and not the code. Or the piece has become small and familiar enough that you should simply write it yourself, because you now understand it well enough that delegating costs more than doing. Taking the keyboard back is a decision, not a defeat, and it is worth recording in the journal as one.

Verification follows authorship

Keep the two kinds of checking apart as you build, because they are not the same activity and confusing them is how projects end up both over-tested and under-verified.

Code you wrote, or directed the assistant to write for you, gets tests. It is yours, you are responsible for it, and a test is how you discharge that responsibility. Output that came out of a library gets a sanity check against a value you already know. You do not write tests for BioPython's genetic code or for a dataframe's grouping. You take one small case whose answer you can work out yourself, run it through the library, and confirm the library agrees with you.

The failure that this split is designed to prevent is subtle. It is the one where a function of yours produces a wrong answer, but a trusted library was involved somewhere inside it, and the presence of the trusted name quietly persuades you that the whole thing must be sound. The library being correct tells you nothing about whether you called it with the arguments you meant, on the strand you meant, in the frame you meant. Your code around the library is still your code.

The journal as you go

The prompt journal is not a report you write at the end. It is a running record kept while you build, piece by piece, of what you asked for, where the answer was wrong, how you caught it, and what you did about it. Written as you go it costs a minute per piece. Reconstructed afterwards it is guesswork, and it will be visibly guesswork.

The entries that matter most are the ones where something was wrong and you found it. A build in which the assistant never made a mistake is not a build that went unusually well, it is a build in which the checking was not sharp enough to find the mistakes, and a journal recording no failures says more about the verification than about the assistant.

Exercise 56-1

AI: DEVELOPER

Build the first piece in your dependency order, all the way through the five-step cycle, and do not move on until its tests pass and its journal entry is written. What you hand the assistant is the piece as your plan states it: the name, the inputs and outputs, and the hand-worked example, and nothing else. What you judge is not how the code reads but whether it passes the tests you wrote before you asked. Then say, in one sentence, what you would have to change if the piece turned out later to be wrong, and notice whether that sentence is short because the piece is well isolated or long because it is not.

Exercise 56-2

SOLO

Find one moment in your build where the assistant gave you more than you asked for, and write down what the extra code was, whether you kept it, and why. If you kept it, state where in your plan it now appears, with its name, its inputs and outputs, and its hand-worked example, because keeping it without doing that is exactly the thing this exercise exists to catch.

Exercise 56-3

SOLO

Take one place in your tool where library output flows into your own logic, and write down two separate things: the test that covers your logic, and the sanity check that confirms the library is doing what you assumed. Run both. If you cannot state them as two separate things, the boundary between your code and the library's is not yet clear enough in your own head, and that is worth fixing now rather than during the finishing.

For your logbook this week, describe one piece where the loop did not converge and what you did about it, whether you fixed the specification or took the keyboard back, and say what tipped you toward that decision.

57 Finale finishing and interface testing

At this point the pieces of your tool exist, each of them tested, each of them built on pieces that were tested before it. What remains is the part that most projects skip on, which is the finishing: making the whole thing work end to end, testing the one function you have been quietly avoiding, and then stepping back far enough to see what you have actually demonstrated. That last step is not decoration. The point of the finale was never the tool. It was to find out whether you can produce something correct, understood, and defensible in a domain you know, using a machine you do not fully trust, and the finishing is where you find out.

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · Worker ·
Collaborator · Developer

The function that faces you

Every tool has a piece at the top that faces the person using it, the one that prints the report, draws the plot, writes the output file, or answers the question the tool was built to answer. It is almost always the last one built and almost always the least tested, for a reason that sounds convincing and is not: you can see it working. It ran, output appeared, the output looked like a report, and so it must be fine.

Look at what that reasoning actually covers. It confirms that the function produced output without crashing. It says nothing about whether the numbers in that output are the right numbers, whether the labels are attached to the values they describe, whether a filter that was supposed to exclude something actually excluded it, or whether a count is off by one at the edges. Every one of those failures produces output that looks exactly like success, because looking right is a property of the formatting and not of the content.

So test it like anything else. Give it an input small enough that you can write the entire correct output by hand, run it, and compare character by character or value by value. If the function prints, capture what it prints and compare it against the exact string you expect. If it plots, arrange for it to also return the values it is plotting and test those, because a plot you can only check by squinting at it is not a plot you have checked. If it writes a file, write a tiny input, run it, read the file back, and compare it against what you wrote out by hand beforehand.

The awkward inputs deserve the same attention here as anywhere else, and at the interface they are usually about emptiness and about the edges. What does your tool report when the input contains no rows that pass the filter? When the input is empty entirely? When two results tie for the answer you asked it to report? Each of those is a decision, and if you do not make it deliberately your tool will make it for you, silently, at the worst possible moment.

Exercise 57-1

SOLO

Take the function in your tool that faces you and write a test for it against an input small enough that you worked out the whole correct output by hand first. Then add one test for an awkward case at the interface, whether an empty result, a tie, or a filter that excludes everything, and state in a comment which decision that test is pinning down.

Running it end to end on real data

With every piece tested individually, run the tool on the real input it was built for, and then do the thing that separates a finished tool from one that merely runs. Take one result out of the output and verify it by hand, independently, from the original data.

Pick a row, a gene, a sequence, whatever unit your tool reports on, find it in the source data yourself, compute the answer your tool claims for it using nothing but your own head and perhaps a calculator, and check that the two agree. If they do, you have evidence that the pieces are not only individually correct but correctly connected, which is a different claim and one that unit tests on the pieces cannot make for you. If they disagree, you have found an integration fault, and you have found it now rather than after presenting the result.

Do this at least twice, and choose the second case deliberately from somewhere odd: the shortest sequence, the row with a missing value, the entry at the boundary of whatever your tool filters on. The middle of the data is where everything works.

Exercise 57-2

SOLO

Run your tool on its real input and verify two of its results entirely by hand from the source data, one ordinary and one from an awkward corner of the data. Record both, with the hand calculation written out, because the calculation is the evidence and the agreement alone is not.

What you actually built

Now step back. Look at the finished tool and ask honestly who did what.

You brought the biology. You knew which question was worth asking, what the data meant, which answer would be suspicious, and what an off-by-one on a coordinate or a read of the wrong strand would do to the result. Nobody supplied that, and the assistant could not have. You also brought the structure, because you decided what the pieces were, what each one promised, and how they fitted together, and you wrote that down before anything was typed.

The assistant brought fluent typing of pieces you had already specified. It filled in shapes you drew. On several of them it was wrong, and your journal records where, and the tests or your own reading caught it. That is the honest accounting, and it is worth stating plainly because it is neither of the two stories people usually tell. It is not that you wrote a program with a bit of help, and it is not that the machine wrote your program. You produced something correct that you could not have typed unaided, by knowing what correct meant and refusing to accept anything you had not checked.

That is augmentation, and it is the thing this course was actually about. A machine that types faster than you is only useful to someone who can tell right from plausible. The reason the term spent so long on what a program is, on substitution and reduction, on reading errors, on writing tests, was never that you would need to write everything yourself. It was that none of the checking works without it. The people for whom these assistants are most dangerous are the ones who cannot evaluate the answer, and the whole of your ability to use one safely rests on the understanding you built before you were allowed to lean on it.

The limits that do not go away

Finishing the tool is a good moment to be clear-eyed about what your careful work does not buy you, and Chapter 59 treats this at length. Three things are worth holding in mind as you close the project.

Your tests cover the cases you thought of, and there are always cases you did not think of, so the tests are a floor under your confidence and not a proof. Your own understanding of each piece is what stands behind them, which is why the rule about never accepting code you cannot read matters more at the end of a project than at the start. And no test you write about the code can catch a mistake in the biology the code assumes, because a flawless implementation of a wrong premise passes every test you can devise. Domain facts get checked against domain sources, not against the assistant that stated them.

Above all, responsibility stayed with you the whole way. You are submitting this tool, and if someone points at any function in it and asks how it works, the answer has to be yours.

Exercise 57-3

SOLO

Apply the test in Chapter 60 to your own finale. Close the assistant, pick the two functions in your tool you are least confident about, and for each one either rebuild it from its docstring alone or explain it in full, out loud, to someone who was not there when you wrote it. Where that goes badly you have found what is left to learn, and there is still time to learn it.

For your final logbook entry on this project, write the honest accounting: what you brought, what the assistant brought, where it was wrong and how you knew. Then name the one function in your tool you would be most comfortable defending

in front of someone who wanted to find a fault in it, and say what makes you comfortable about that one in particular.

58 Genome assembly

This chapter is a programming project where you will assemble a small genomic sequence from a set of short sequencing reads.

In genome assembly, many short sequences (reads) from a sequencing machine are assembled into long sequences – ultimately chromosomes. This is done by ordering overlapping reads so that they together represent genomic sequences. For example, given these three reads: **AGGTCGTAG** , **CGTAGAGCTGGGAG** , **GGGAGGTTGAAA** , ordering them based on their overlap like this

```
AGGTCGTAG
  CGTAGAGCTGGGAG
    GGGAGGTTGAAA
```

produces the following genomic sequence:

```
AGGTCGTAGAGCTGGGAGGTTGAAA
```

Real genome assembly is, of course, more sophisticated than what we do here, but the idea is the same. To limit the complexity of the problem, we make two simplifying assumptions:

1. There are no sequencing errors, implying that true overlaps between reads can be identified as perfectly matching overlaps.
2. No reads are nested in other reads. I.e., They are never a smaller part of another read.

The second assumption implies that overlaps are always of this type:

```
CGTAGAGCTGGGAG
  GGGAGGTTGAAA
```

and never of this type:

```
CGTAGAGCTGGGAG
  AGAGCTG
```

In this project, you will be asked to write functions that solve the problem of assembling a genomic sequence. Each function solves a small problem, and you may need to call these functions inside other functions to put together solutions to larger subproblems.

Download the files you need for this project:

https://munch-group.org/bioinformatics/supplementary/project_files

- `sequencing_reads.txt` contains the sequencing reads.

You also need to download the two project files:

- `assemblyproject.py` is an empty file where you must write your code.
- `test_assemblyproject.py` is the test program that lets you test the code you write in `assemblyproject.py`.

Put the files in a folder dedicated to this project. On most computers you can right-click on the link and choose “Save file as...” or “Download linked file”.

Now open each file in your editor and look at what is in `sequencing_reads.txt`. (Do *not* change it in any way, and do not save it after viewing. If your editor asks you if you want to save it before closing, say *no*.) How many sequences are there in each file?

The project is split into four parts:

1. Read and analyze the sequencing reads.
2. Compute the overlaps between reads.
3. Find the right order of reads.
4. Reconstruct the genomic sequence.

Here is an overview of the functions you will write in each part of the project and of which functions are used by other functions.

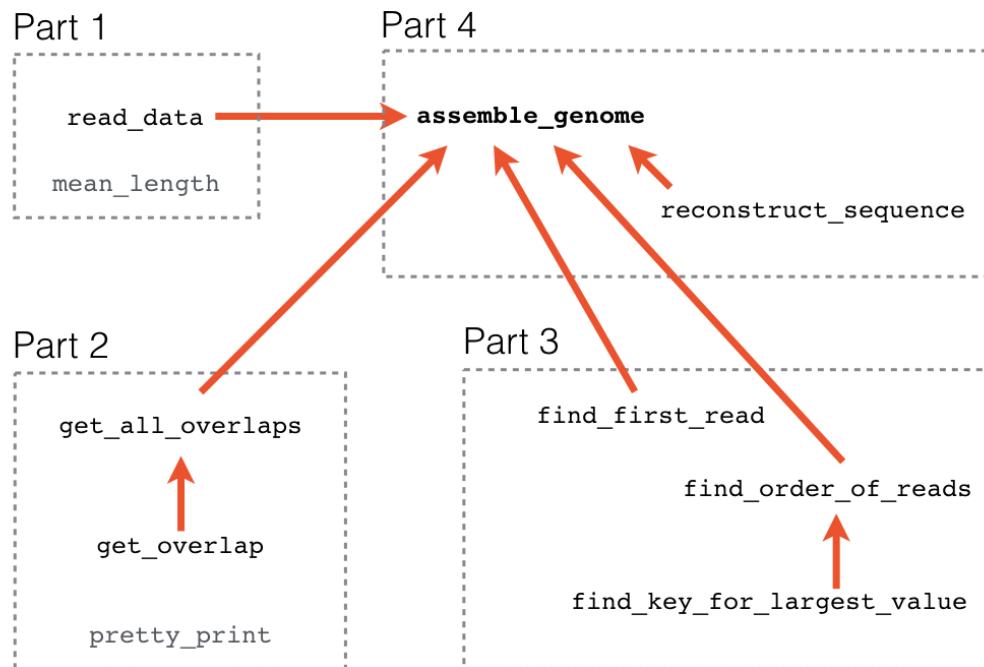


Figure 58.1: Overview

Make sure to read the entire exercise and understand what you are supposed to do before you begin!

Read and analyze the sequencing reads

The first task is to read and parse the input data. The sequence reads for the mini-assembly are in the file `sequencing_reads.txt`. The first two lines of the file look like this:

```
Read1 GGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTCGTCCAGACCCCTAGC
Read2 CTTTACCCGGAAGAGCGGGACGCTGCCCTGCGCGATTCCAGGCTCCCCACGGG
```

Each line represents a read. The first field on each line is the name of the read, and the second field is the read sequence itself. So for the first line, `Read1` is the name, and `ATGCG...` is the sequence.

Read the sequencing reads into your program

Write a function, `read_data`, that takes one argument:

1. A string, which is the name of the data file.

The function must return

- A dictionary, where the keys are the *names* of reads and the values are the associated read *sequences*. Both keys and values must be strings.

Example usage:

```
read_data('sequencing_reads.txt')
```

should return a dictionary with the following content (maybe not with key-value pairs in that order)

```
{ 'Read1': 'GGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTCGTCCAGACCCCTAGC',
  'Read3': 'GTCTTCAGTAGAAAATTGTTTTTTCTTCCAAGAGGTCGGAGTCGTGAACACATCAGT',
  'Read2': 'CTTTACCCGGAAGAGCGGGACGCTGCCCTGCGCGATTCCAGGCTCCCCACGGG',
  'Read5': 'CGATTCCAGGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTC',
  'Read4': 'TGCGAGGGAAGTGAAGTATTTGACCCTTTACCCGGAAGAGCG',
  'Read6': 'TGACAGTAGATCTCGTCCAGACCCCTAGCTGGTACGTCTTCAGTAGAAAATTGTTTTTTCTTCCAAGAGGTCGGAGT' }
```

Here is some scaffolding code to get you started:

```
def read_data(file_name):
    input_file = open(file_name)
    # ...
    for line in input_file:
        # ...
    # ...

    input_file.close()
```

The `line` variable in the for loop holds each line in the file, including the `\n` newline character at the end. You can use the `'split'` method of strings to split each line into the name of the read and the read sequence. You can see the documentation for that method by typing `pydoc str.split` in your terminal.

Compute the mean length of reads

After writing that function, we would like to get an idea about the length of the reads. There are often too many reads to look at manually, so we need to make a function that computes the mean length of the reads.

Write a function, `mean_length`, that takes one argument:

1. A dictionary, in which keys are read names and values are read sequences (this is a dictionary like that returned by `read_data`).

The function must return

- A float, which is the average length of the sequence reads.

One way to do this is to loop over the keys in the dictionary like this:

```
def mean_length(reads):
    count = 0
    total = 0
    for name in reads:
        seq = reads[name]
        # ...
        # ...
    # ...
    return total / count
```

Remember that you can use the `len` function to find the length of a read.

Compute overlaps between reads

The next step is to determine which reads overlap each other. We need a function that takes two read sequences and computes their overlap to do that. Remember that in the input data, none of the reads are completely nested in another read.

Compute the overlap between two reads

We know there are no sequencing errors so that the sequence match will be perfect in the overlap. To compute the overlap between the 3' (right) end of the left read with the 5' (left) end of the righthand read, you need to loop over all possible overlaps, honoring that one sequence is the left one and the other is the right one. In the for loop, start with the largest possible overlap (`min(len(left), len(right))`) and evaluate smaller and smaller overlaps until you find an exact match.

Write a function, `get_overlap`, that takes two arguments

1. A string, which is the lefthand read sequence.
2. A string, which is the righthand read sequence.

The function must return

- A string, which is the overlapping sequence. If there is no overlap, it should return an empty string.

Example usage:

```
s1 = "CGATTCCAGGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTC"
s2 = "GGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTCGTCCAGACCCCTAGC"
get_overlap(s1, s2)
```

should return the string

```
'GGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTC'
```

and `get_overlap(s2, s1)`

should return the string

```
'C'
```

From these two examples, it seems that `s1` and `s2` overlap and that `s1` is the left and `s2` is the right. Treating `s2` as the left one and `s1` as the right one only gives an overlap of one base (we expect a few bases of overlap even for unrelated sequences).

Compute all read overlaps

When you have written `get_overlap`, you can use it to evaluate the overlap between all pairs of reads in both left-right and right-left orientations.

Write a function, `get_all_overlaps`, that takes one argument:

1. A dictionary with read data as returned by `read_data`.

The function must return

- A dictionary of dictionaries specifying the number of overlapping bases for a pair of reads in a specific left-right orientation. Computing the overlap of a read to itself is meaningless and must not be included. Assuming the resulting dictionary of dictionaries is called `d`, then `d['Read2']` will be a dictionary where keys are the names of reads that have an overlap with read `'Read2'` when `'Read2'` is put in the left position, and the values for these keys are the number of overlapping bases for those reads.

Example usage: assuming that `reads` is a dictionary returned by `read_data` then:

```
get_all_overlaps(reads)
```

should return the following dictionary of dictionaries (but not necessarily with the same ordering of the key-value pairs):

```
{ 'Read1': { 'Read3': 0, 'Read2': 1, 'Read5': 1, 'Read4': 0, 'Read6': 29 },
  'Read3': { 'Read1': 0, 'Read2': 0, 'Read5': 0, 'Read4': 1, 'Read6': 1 },
  'Read2': { 'Read1': 13, 'Read3': 1, 'Read5': 21, 'Read4': 0, 'Read6': 0 },
  'Read5': { 'Read1': 39, 'Read3': 0, 'Read2': 1, 'Read4': 0, 'Read6': 14 },
  'Read4': { 'Read1': 1, 'Read3': 1, 'Read2': 17, 'Read5': 2, 'Read6': 0 },
  'Read6': { 'Read1': 0, 'Read3': 43, 'Read2': 0, 'Read5': 0, 'Read4': 1 } }
```

i Hint

You can use the `get_overlap` function you just made to find the overlap between a pair of reads. To generate all combinations of reads, you need two for-loops. One looping over reads in the left position, and another (inside the first one) looping over reads in the right position. Remember that we do *not* want the overlap of a read to itself, so there should be an if-statement checking that the left and right reads are not the same.

Print overlaps as a nice table

The dictionary returned by `get_all_overlaps` is a bit messy. We want to print it in a nice matrix-like format so we can better see which pairs overlap in which orientations.

This `pretty_print` function should take one argument:

1. A dictionary of dictionaries as returned by `get_all_overlaps`.

The function should *not* return anything but must *print* a matrix exactly as shown in the example below with nicely aligned and right-justified columns. The first column must hold names of reads in left orientation. The top row holds names of reads in right orientation. The remaining cells must each have the number of overlapping bases for a left-right read pair. The diagonal corresponds to overlaps with the read itself. You must put dashes in these cells.

Example usage: assuming that `overlaps` is a dictionary of dictionaries returned by `get_all_overlaps` then:

```
pretty_print(overlaps)
```

should print exactly

	Read1	Read2	Read3	Read4	Read5	Read6
Read1	-	1	0	0	1	29
Read2	13	-	1	0	21	0

Read3	0	0	-	1	0	1
Read4	1	17	1	-	2	0
Read5	39	1	0	0	-	14
Read6	0	0	43	1	0	-

This function is hard to get completely right. So, to spare you the frustration, this one is on me:

```
def pretty_print(d):
    print(' ', end='')
    for j in sorted(d):
        print("{: >6}".format(j), end='')
    print()
    for i in sorted(d):
        print("{: >6}".format(i), end='')
        for j in sorted(d):
            if i == j:
                s = ' - '
            else:
                s = "{: >6}".format(d[str(i)][str(j)])
            print(s, end='')
        print()
```

Make sure you understand how it works. You can look up in the documentation what `"{: >6}".format(i)` does.

Find the correct order of reads

Now that we know how the reads overlap, we can chain them pair by pair from left to right to get the order in which they represent the genomic sequence. To do this, we take the first (left-most) read and identify which read has the largest overlap at its right end. Then we take that read and find the read with the largest overlap to the right end of that - until we reach the rightmost (last) read.

Find the first read

The first thing you need to do is to identify the first (leftmost) read so we know where to start. This read is identified as the one that has no significant (>2) overlaps to its left end (it only has a good overlap when positioned to the left of other reads). In the example output from `pretty_print` above, the first read would be read `'Read4'` because the `'Read4'` column has no significant overlaps (no one larger than two).

We break the problem in two and first write a function that gets all the overlaps to the left end of a read (i.e., when it is in the right position):

Write a function, `get_left_overlaps`, that takes two arguments:

1. A dictionary of dictionaries as returned from `get_all_overlaps`.

2. A string, which represents the name of a read.

The function must return

- A sorted list of integers, which represent the overlaps of other reads to its left end.

Example usage: assuming that `overlaps` is a dictionary of dictionaries returned by `get_all_overlaps` then.

```
get_left_overlaps(overlaps, 'Read1')
```

should return

```
[0, 0, 1, 13, 39]
```

i Hint

Once you have made a list of left overlaps, you can use the built-in function `sorted` to make a sorted version of the list that you can return from the function.

OK, now that we have a function that can find all the overlaps to the left end of a given read, all we need to do is find the particular read with no significant (>2) overlaps to its left end.

Write a function, `find_first_read`, that takes one argument:

1. A dictionary of dictionaries as returned from `get_all_overlaps`.

The function must return

- A string containing the name of the first read.

Example usage: assuming that `overlaps` is a dictionary of dictionaries returned by `get_all_overlaps` then.

```
find_first_read(overlaps)
```

should return

```
'Read4'
```

Find the order of reads

Now that we have the first read, we can find the correct order of the reads. We want a list of the read names in the right order.

Given the first (left) read, the next read is the one that has the largest overlap to the right end of that read. We use our dictionary of overlaps to figure out which read that is. If the first read is `'Read4'`, then `overlaps['Read4']` is a dictionary of reads with overlap to the right end of read `'Read4'`. So, to find the name of the read with the largest overlap, you must write a function that finds the key associated with the largest value in a dictionary. We do that first:

Write a function, `find_key_for_largest_value`, that takes one argument:

1. A dictionary.

The function must return the key associated with the largest value in the dictionary argument.

Having written `find_key_for_largest_value`, you can use it as a tool in the function that finds the order of reads:

Write a function, `find_order_of_reads`, that takes two arguments:

1. A string, which is the name of the first (left-most) read (that is returned by `find_first_read`).
2. A dictionary of dictionaries of all overlaps (that returned by `get_all_overlaps`).

The function must return

- A list of strings, which are read names in the order in which they represent the genomic sequence.

Hint

You know the first read is given by the first argument to the function. You also know that you can find the next read in the chain of overlapping reads by using the `find_key_for_largest_value` function. You should keep adding reads to the chain as long as the overlap is larger than two (you can use a for loop with an if statement inside to check that the overlap is larger than 2).

Example usage: assuming that `overlaps` is a dictionary of dictionaries returned by `get_all_overlaps` then:

```
find_order_of_reads('Read4', overlaps)
```

should return:

```
['Read4', 'Read2', 'Read5', 'Read1', 'Read6', 'Read3']
```

Before you implement the function, make sure you understand why this is the right list of read names.

Reconstruct the genomic sequence

Now that you have the number of overlapping bases between reads and the correct order of the reads, you can reconstruct the genomic sequence.

Reconstruct the genomic sequence from the reads

Write a function, `reconstruct_sequence`, that takes three arguments:

1. A list of strings, which are the names of reads in the order identified by `find_order_of_reads`.
2. A dictionary, with read data as returned from `read_data`.
3. A dictionary of dictionaries with overlaps as returned from `get_all_overlaps`.

The function must return

- A string, which is the genomic sequence.

Example usage: assuming that `order` is the list of strings returned by `find_order_of_reads`, that `reads` is the dictionary returned by `read_data` and that `overlaps` is a dictionary of dictionaries returned by `get_all_overlaps` then:

```
reconstruct_sequence(order, reads, overlaps)
```

should return one long DNA string (I had to break it in three to make it fit on the page):

```
TGCGAGGGAAGTGAAGTATTTGACCCTTTACCCGGAAGAGCGGGACGCTGCCCTGCGCGATT  
CCAGGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTCGTCCAGACCCCTAGCTGGTA  
CGTCTTCAGTAGAAAATTGTTTTTTCTTCCAAGAGGTCGGAGTCGTGAACACATCAGT
```

Hint

Iterate over the reads in order and use the overlap information to extract and join the appropriate parts of the reads.

Putting the whole thing together

Now that you have written functions to handle each step, you can write one last function that uses them to complete the entire assembly.

Write a function, `assemble_genome`, that takes one argument:

1. A string, which is the name of a file with sequencing reads in the format described at the beginning of this project description.

The function must return

- A string, which is the genome assembled from the sequencing reads

Example usage:

```
assemble_genome('sequencing_reads.txt')
```

should return the assembled genome:

```
TGCGAGGGAAGTGAAGTATTTGACCCTTTACCCGGAAGAGCGGGACGCTGCCCTGCGCGATT  
CCAGGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTCGTCCAGACCCCTAGCTGGTA  
CGTCTTCAGTAGAAAATTGTTTTTTTCTTCCAAGAGGTCGGAGTCGTGAACACATCAGT
```

Week 14

59 Surviving AI

The whole of this course has been an argument that you can trust the assistant further because you have learned to check it. That argument is true, and it is the right way to work, but it would be a misleading place to stop, because checking well does not remove every limit, and some of the most important judgments you will make are about when the assistant should not be used at all. This last part is about the limits that remain even for someone who specifies, reads, judges, and tests carefully, and about the responsibility that stays with you no matter how good your checking becomes.

Begin with a fact about people rather than about machines, because it is the one most likely to undo you. Humans tend to trust the output of an automated system more than they should, and the more fluent and confident that output looks, the stronger the pull. This is a well documented tendency, and it has a name, automation bias, and it does not spare people who know about it. The danger for you is not that you will fail to learn the checking discipline; it is that once you have it, it will quietly erode. The first weeks of using the assistant you will check everything, because it is new and you are wary. Then it will be right several times in a row, and you will relax, and you will start accepting a function because it looks like the ones that were fine before, which is exactly the plausible surface you were taught to distrust. The better the code looks and the longer the assistant's recent record, the more suspicious you should become, not less, because that is the situation in which your guard is lowest and its confident wrongness is most expensive. The checking is not a phase you pass through on the way to trusting the machine. It is the permanent condition of using it well.

The second limit is one you have already met but must now state sharply, because it is easy to half learn. Passing tests is necessary, and it is not sufficient. A green result tells you that the code produced the right answer on the particular inputs you thought to try. It tells you nothing whatever about the inputs you did not try, and there are always inputs you did not try, because a handful of tests cannot cover the infinity of possible inputs. This means a wrong function can pass a weak suite, and a function that passes even a strong suite can still be wrong on a case nobody imagined. Tests are a floor under your confidence, not a proof of correctness, and the assistant is specifically good at producing code that clears the visible floor while hiding a fault just out of sight of it. The only thing that stands behind the tests is your understanding of what the code does, which is why the rule that you never accept code you cannot read is not a beginner's caution you will one day outgrow. It is the last line of defense for every case your tests forgot.

The third limit is the one that matters most in your field, and it is the one no amount of testing can reach, because it is not about the code at all. The assistant states facts about biology with exactly the same fluent confidence whether those facts are true or false, because it has no way of knowing which they are. It will tell you a base pairing, a codon assignment, a gene function, a statistical assumption, and it will sound equally certain when it is right and when it is inventing. A wrong

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · Worker ·
Collaborator · Developer

fact of this kind is more dangerous than a wrong loop for two reasons. It hides better, because there is no error and no failing test to draw your eye to it, and it does more damage, because a false premise propagates silently into every result built on top of it. No test you write about the code will ever catch a mistake in the biology the code assumes, because the code may be a flawless implementation of a wrong idea. The only defense is to check the domain facts against something other than the assistant: your own knowledge, a textbook, a database, a primary source. In a scientific context this is not an optional nicety. A result that rests on a biological claim you accepted from the assistant without checking is a result you cannot defend, however green its tests.

These limits together sharpen a question the course has mostly answered by demonstration and should now answer directly, which is when you should not use the assistant at all. The clearest case is when you do not understand the problem well enough to check the answer. The assistant amplifies judgment you already have; it cannot supply judgment you lack. If you could not tell a correct solution from a plausible wrong one, then delegating the problem does not get you a solution, it gets you a guess you are unable to evaluate, and you are simply trusting blind, which is the one thing this course was built to stop. A second case is when the stakes are high and you cannot verify. If a wrong answer would matter and you have no way to check the answer, the assistant's speed is worth nothing, because speed toward an unverifiable result is not progress. A third case is more ordinary and more common than students expect: when the task is small enough that writing it yourself is faster and surer than specifying it, prompting for it, reading it, and testing it. The whole apparatus of delegation has a cost, and for a three line function you understand completely, paying that cost is slower than just typing the three lines. And a fourth case is the one that matters while you are still learning: when using the assistant would rob you of understanding you actually need to acquire. There are things you must be able to do yourself, not because the assistant cannot do them, but because your ability to check its work on everything else depends on your having learned them. Handing those to the machine early is a false economy that leaves you unable to judge it later.

Underneath all of these is a single principle that does not bend, which is that responsibility does not transfer. Whatever the assistant produces, the moment you submit it, run it, or publish a result that depends on it, it is yours. The AI wrote it is not a defense for a wrong answer, in this course or in the science you will go on to do, any more than a calculator's arithmetic excuses a wrong sum you copied without checking. Delegating the typing never delegates the responsibility, and the entire structure you have learned, the specifying and reading and planning and testing, exists so that you can accept that responsibility with your eyes open while still using the machine heavily. This has a practical companion, which is honesty about what the assistant did. Recording where and how you used it, in your logbook, in your prompt journal, in the notes on a piece of work, is not an admission of weakness and it is not something to hide. Concealing the assistant's involvement is the dishonest act; disclosing it is simply an accurate account of how the work was done, and in a scientific setting an accurate account of method is not optional. The habit of honest attribution you build now is the same habit that will keep your later research defensible.

It is worth ending where the course began. In the first week the idea was offered that instructing a machine has passed through a sequence of layers, from switches

to machine code to compiled languages to interpreted languages like Python, and that an assistant is one more layer, a way of describing what you want in something close to ordinary language and having code come back. The difference, the one that has organized everything since, is that a compiler is a faithful translator and an assistant is not. A compiler always produces code that means exactly what you wrote; an assistant produces code that is merely plausible, and may mean something other than what you wanted, and cannot tell you which. That single difference is why this new layer, unlike the ones beneath it, cannot be trusted and must be verified, and it is why you had to learn what a program is, and what correct means, and how to specify and read and test, in order to use the newest layer safely. The syntax you learned this term may fade with disuse, and the assistants will keep improving, but the durable skill, the one that will still be yours when the tools have changed, is the judgment: knowing what correct means, being able to say it exactly, being able to read what you are given, and always, always checking. The machine is a powerful new way to instruct a computer. It is your job to instruct it well, and to never stop verifying what it does.

Exercise 59-1

SOLO

Audit your own guard. Look back through your logbook at the weeks since you were first allowed to let the assistant draft code, and count two things: how many pieces of its code you accepted after running a test you wrote yourself, and how many you accepted because they looked right. Do not adjust the second number to be flattering; the whole value of the count is that it is honest. Then say in one sentence what changed between the week you learned to distrust a draft and the week you stopped bothering, if it changed, and what you would need to put in place to stop it changing again. Automation bias does not announce itself. A count is the only way to see it.

Exercise 59-2

SOLO

Identify one task from the course where, looking back, you should not have used the assistant, either because you could not really check its answer, or because leaning on it cost you an understanding you later needed. Say what you would do differently, and say which of the four cases in this note it falls under.

Exercise 59-3

SOLO

Now the other direction. Identify one task where the assistant let you produce something correct that you could not have written unaided, and be specific about which part of your checking made that safe. Not “I tested it”: name the test, and name the case it covered that you would otherwise have missed. If you cannot name it, then what made the task safe was luck, and it is worth knowing that now rather than later.

Exercise 59-4

Take one biological claim the assistant makes, in an explanation, in a comment, or inside a table it writes for you, and verify it against a source outside the assistant rather than against the assistant itself. Record whether it held. Then do the thing that makes this different from every other verification you have done this term: ask the assistant whether its own claim was correct, and watch it agree with you either way. A machine that will confirm both a true statement and a false one, with the same fluency, is not a witness. That is why the source had to come from outside, and it is the last time this course will say so.

For your last logbook entry, look across the whole term. Describe how your use of the assistant changed as your understanding grew, from the first weeks when it could only explain things to you, through the point where you let it draft and learned to distrust the draft, to the capstone where you delegated real work and kept the responsibility. Name the single habit from this course you most want to still have in three years, and say why.

60 The AI-off test: how you know you have learned it

The closing AI beat: verification turned on your own learning.

Every verification habit this course has built points outward: at a function, at a claim about biology, at a call to a library you did not write. This last one points at you. It is the same principle, evidence is to a fact what a test is to a function, applied to the plainest fact of all: whether you actually learned something, or only watched the assistant produce something that worked.

The ladder so far

Explainer · Translator ·
Illustrator · Comparer ·
Drafter · Worker ·
Collaborator · Developer

The test itself

Here it is, stated so plainly there is nothing to misread: you have learned something when you can do it, or explain it, with the assistant switched off. Not “with the assistant available but unconsulted.” Off. Browser tab closed, no rereading a previous answer, no reaching for a hint half-remembered from a chat. Pick a function from your own capstone, one you built with real help from the assistant, close everything, and either rewrite it from a blank page or explain to another person, in your own words, exactly what it does and why each part is there. If you can, the understanding was yours, and the assistant was a tool you used to get there faster. If you cannot, something is missing, no matter how correct and well-tested the function sitting in your project already is.

That gap, between a working function and an explainable one, is the entire reason this test exists. Passing tests, as you know by now, tells you the code is correct on the cases you thought to try. It says nothing at all about whether you are the one who could produce it again. A capstone full of green tests and functions you could not rebuild or explain is a capstone that ran once, with help, and that is a real accomplishment but not quite the one this course is asking for.

Why this is the line, not a rule about honesty

It would be easy to hear “test yourself with the assistant off” as a policing rule, a way of catching cheaters, and to feel a little defensive about it. That is not what it is for. It is a diagnostic you run on yourself, for yourself, and the result belongs to no one but you. Producing something you formulated, understand, and can rebuild is not cheating, no matter how much of the typing the assistant did; that is what every note since week one has been training you to do safely. Producing something you cannot explain and could not rebuild, and presenting it as understood, is where the line actually sits, and the AI-off test is simply how you find out, honestly, which side of it you are standing on before anyone else asks.

This is also the reason SOLO exercises have run alongside the badged ones all term, and the reason your prompt journal on the capstone asked you to record where the assistant was wrong and how you caught it. Both are the same test, run continuously in small doses, rather than saved up as one nerve-wracking check at the very end. If you have been doing the SOLO work honestly and keeping the journal honestly, the AI-off test at the end should mostly confirm what you already suspected about your own understanding, not surprise you.

Running it on purpose

Do not wait for this to happen to you by accident. At the end of the capstone, choose two or three functions you are least sure about, close the assistant, and try each one: rebuild it from the docstring alone, or explain it out loud to a classmate who was not there when you wrote it. Where it goes well, you have real evidence of learning, not a hopeful feeling. Where it does not, you now know exactly what to go back and actually learn, while the project is still open in front of you and there is still time to close the gap.

Exercise 60-1

SOLO

Pick one function from this week's project that the assistant helped you write. With the assistant closed, rewrite it from a blank page, using only its docstring and your own understanding. Compare your version to the original: where they differ, decide whether the difference is a real gap in your understanding or simply a different valid way of writing the same thing.

For your logbook this week, name the function from your capstone you are most confident you could explain in full right now with the assistant off, and the one you are least confident about, and say what you plan to do about the second one before you consider the project finished.

Cheatsheets

61 The tools you check your work with

This course comes with a small family of notebook tools. Each of them exists because there is a question you will want to ask the machine, and asking it in ordinary Python either does not work or buries the answer. This chapter is the place to come back to when you have forgotten which tool answers which question. Every tool gets the same treatment: what it does, how you invoke it, and when you should reach for it.

All of them are cell magics, which means you write the tool's name on the very first line of a cell, starting with two percent signs, and the rest of the cell is the code the tool works on. A cell magic has to be the first line of the cell, with nothing above it, not even a comment.

The tools are already installed in the course environment. Importing them is what registers the magics, so run the cell below before you run anything else on this page.

```
import steps_widget
import puzzle_widget
import codelens_widget
import sandbox_widget
from turtle_widget import Turtle
```

Seeing the substitution steps with `%%steps`

The course keeps telling you to do the substitution and reduction in your head. This is the tool that shows you the steps the machine actually took, so you can hold them next to the ones you took.

Put `%%steps` on the first line of a cell and write ordinary Python underneath. The code runs as usual, and below the output you get the evaluation laid out one reduction at a time: each variable replaced by its value, each operation replaced by its result, until a single value is left.

Predict what the cell below prints before you run it. Then run it and read the steps.

```
%%steps
x = 2
y = 3 * x + 1
print(y)
```

Reach for `%%steps` whenever an expression surprised you. Precedence questions, where you thought the multiplication happened after the addition. Comparison chains. Logical operators with `and` and `or`, where the short-circuit means part of

the expression was never evaluated at all and the steps will show you that it was skipped. If you can predict the steps for an expression you have never seen, you can read Python.

The same machinery works outside a notebook. In a script you write, put a comment `# PRINT STEPS` on the line you want traced and run the file through the `print-steps` command in the terminal.

Exercise 61-1

SOLO

Write a `%%steps` cell containing `result = 2 + 3 * 4 ** 2` and then `print(result)`. Before running it, write down the order in which you think the three operations happen and what the final value is. Run it and compare. If you were wrong, you have just found something worth knowing.

Rebuilding a program with `%%puzzle`

Reading code and writing code are different skills, and there is a third one in between: knowing what order statements have to go in. That is what `%%puzzle` drills.

You write a small program in the cell and give the magic the value the program should end up producing. The widget shuffles the lines and presents them to you out of order, and you drag them back into an order that runs and gives the stated result. It is the same puzzle every time and it is never the same puzzle twice, because you can always shuffle again.

The result you give the magic has to be a plain Python value, a number, a string, a list, something you could type as a literal. The last line of your program has to be an expression on its own, because its value is what gets compared against the result you promised. And every line has to be a complete statement with no indentation, because the widget has to be able to shuffle them into any order without producing something that will not even parse.

```
%%puzzle 12
a = 3
b = 4
a * b
```

Reach for `%%puzzle` when you know what the pieces of a program mean but keep getting the order wrong, which is an extremely common place to be in the first weeks. Assignments before use, the loop before what it depends on, the function defined before it is called. Because the widget checks the result for you, you can work through these on your own without waiting for anyone to tell you whether you got it right.

Watching memory with `%%code lens`

`%%steps` shows you an expression collapsing. `%%code lens` shows you the whole machine: which variables exist right now, what each one refers to, which line is about to run, and what has been printed so far. You step forward and backward through the execution one line at a time.

Put `%%code lens` at the top of a cell and write the code underneath. Nothing runs when the cell executes in the ordinary sense; instead the code is traced, and the widget below lets you walk through the trace.

```
%%code lens
bases = ['A', 'T', 'G']
for base in bases:
    print(base)
```

The part of this tool you will care about most arrives later, when you meet lists and dictionaries. Values like numbers and strings are drawn inside the variable box. Lists, dictionaries, sets, function objects and instances of your own classes are drawn off to the side, with an arrow from the variable to the object. That picture is the answer to the single most confusing thing in the first half of the course: what happens when two variables refer to the same list, and why changing it through one of them changes it through the other.

Reach for `%%code lens` for anything involving a reference, a function call frame, a loop whose variable you have lost track of, or a nested structure you cannot picture. It shows execution and printed output only. It cannot show you what is in a file on disk or anything happening outside the code you gave it.

Exercise 61-2

AI: EXPLAINER

Put these four lines in a `%%code lens` cell: `a = [1, 2, 3]`, then `b = a`, then `b.append(4)`, then `print(a)`. Predict what the last line prints. Then ask the assistant to explain what happens when you write `b = a` with a list on the right-hand side. Finally step through the trace and decide whether the assistant's account matches the arrows you can see. Note the outcome in your logbook.

A fresh interpreter with `%%sandbox`

A notebook remembers. Everything you have run is still sitting in the kernel's memory, which is convenient right up until the moment it is the reason your code appears to work. A script has no such memory: it starts from nothing every time.

`%%sandbox` gives you a script inside your notebook. The cell body runs in a brand new Python interpreter of its own, which is thrown away as soon as it finishes. Nothing from the notebook is visible inside it, and nothing defined inside it survives into the notebook. Run the cell twice and you get exactly the same result both times, because the second run knows nothing about the first.


```
%%sandbox
codon = "ATG"
print(codon * 3)
```

Reach for `%%sandbox` when you want to know whether a piece of code stands on its own, which is the question that matters if it is eventually going into a `.py` file. It is also the safe place to run something that might fail: a sandbox cell reports the error in its own output box rather than raising it into your notebook, so a deliberate failure is contained.

Most of the small exercises in these notes are sandbox cells for exactly that reason. The magic is also available under the name `%%exercise`, which does the identical thing, so you will see both spellings.

Drawing with Turtle

The turtle is not a magic. It is an ordinary Python object you make and give instructions to, and it draws as it goes.

You make one with `Turtle()`, then tell it to move: `forward`, `backward`, `left`, `right`, `goto`, `circle`. You lift and lower the pen with `penup` and `pendown`, set its thickness with `pensize`, and set colors with `pencolor` and `fillcolor`. The drawing appears below the cell on its own, animated in the order the commands were given, so you do not need to display anything explicitly.

```
t = Turtle()
t.speed(6)
for _ in range(4):
    t.forward(120)
    t.left(90)
```

The reason the turtle earns a place in a course about molecular biology is that its output is visible. When you write a loop that draws a square and get a triangle, you do not need a test to tell you something is wrong. That makes it the first safe place to let the assistant draft code for you, because you can judge the result at a glance, which is the whole idea you will spend the rest of the term applying to code whose output is not a picture.

Passing `show_code=True` when you make the turtle puts your source next to the canvas and highlights each line as it executes, which turns a drawing into a trace of your own loop.

Exercise 61-3

AI: DRAFTER

Ask the assistant for turtle code that draws an equilateral triangle. Before you run what it gives you, read it and predict what will appear. Then run it. If it drew a triangle, work out why the angle it used is the one that works. If it drew something

else, work out where its reasoning went wrong. Either way you have verified rather than trusted, and either way it goes in the logbook.

Checking a project with `im_pytest`

From week six onward you work on weekly projects, and each project comes with a set of tests that decide whether your functions do what they are supposed to. The same tests appear to you in three progressively less friendly forms across the term, and that escalation is the point.

At first you call `im_pytest.check("translationproject")` in a cell, or put `%%test` on the first line of a cell with the project name after it. What comes back is a panel with one line per function, a check mark or a cross, the assertion that failed if one did, and a note about which functions you have not written yet. No test source and no traceback, because in week six you have not yet been told what a test is.

Later you run the same file with the real tool from the terminal, as `pytest test_translationproject.py`, and learn to read what pytest actually prints. Same tests, same results, no cushioning.

Later still you write the tests yourself, using the provided ones as your model. This is where the course arrives at the thing it has been aiming at all along: a test you wrote is how you find out whether code somebody else produced, including an assistant, does what you asked for. A function that passes a test you designed is checked. A function that merely looks right is not.

Narrating code with `snippet-cast`

`snippet-cast` turns a piece of Python into a narrated video. You annotate the lines of a snippet with trailing comments that begin with `#:`, and each annotated line becomes one beat of the recording: the code is revealed up to that line, the line is highlighted, the narration is spoken, and a panel at the side shows the variables as they change.

You will mostly meet this as a consumer, because the worked examples from the lectures are recorded this way and you can replay them at home as many times as you need. Watching somebody decide a decomposition, write a function, hit a bug and read the error is the single most useful thing for the part of programming that is hardest to teach.

Later in the term you will make one yourself. Narrating your own code, line by line, out loud, is unforgiving in a useful way: any line you cannot say a sentence about is a line you did not really write.

Exploring a table with `iplot`

In the last weeks, when you are working with real data tables rather than lists you typed yourself, `iplot` gives you a plot with dropdown menus. You hand it a dataframe and pick which column goes on which axis, what splits the colors, and what kind of plot you want, without writing new plotting code for every view you want to look at. It is a way of looking around a dataset quickly, before you commit to the one figure you actually want to make.

Which tool answers which question

If the question is what does this expression evaluate to and in what order, the answer is `%%steps`.

If the question is what order do these statements have to go in, the answer is `%%puzzle`.

If the question is what does memory look like right now, or what is this variable actually pointing at, the answer is `%%codelens`.

If the question is would this still work in a file on its own, the answer is `%%sandbox`.

If the question is did my loop do what I meant, and you would like to see the answer rather than reason about it, the answer is a turtle.

If the question is does my function do what it is supposed to do, the answer is a test, run through `im_pytest` at first and through `pytest` later.

Every one of those questions is a version of the same one. Something claimed to be true, and you would like it proved. The claim might come from your own reading of the code, and it might come from an assistant. The tool does not care which, and neither should you.

Exercise 61-4

SOLO

For each of these situations, name the tool you would reach for, without looking back up the page. Your function returns `None` and you do not know why. You wrote `x = 5` and `y = x * 2 + 1` and cannot work out why `y` is not what you expected. You copied a working cell into a new file and it stopped working. You changed a list through one variable and a different variable changed too. Then check yourself against the section above.

62 Widget demo

videos/hello.mp4

```
# Run this in JupyterLite to install the widgets used below
%pip install -q anywidget codelens-widget puzzle-widget steps-widget turtle-widget
```

Sandbox cell magic

```
import sandbox_widget
```

```
%%sandbox
```

```
variable = 'hello world'
print(variable)
```

<sandbox_widget.widget.ExerciseOutputWidget object at 0x12a561fd0>

Codelens cell magic

```
import codelens_widget
```

```
%%codelens
```

```
def insertion_sort(a):
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a
```

```
data = [5, 2, 9, 1]
insertion_sort(data)
```

<codelens_widget.CodeLens object at 0x12a5630e0>

Puzzle cell magic

```
import puzzle_widget
```

```
%%puzzle 44
```

```
total
total = total + z
z = 2
total = x * y
y = 6
x = 7
```

```
<puzzle_widget.widget.PuzzleWidget object at 0x12a563e00>
```

```
%%puzzle 'HELLO, WORLD'
```

```
greeting.upper()
greeting = 'hello, ' + name
name = 'world'
```

```
<puzzle_widget.widget.PuzzleWidget object at 0x12a5c4e10>
```

Steps cell magic

```
import steps_widget
```

```
%%steps
```

```
s = 'Hej {}'
l = [1, 2, 3]
greeting = s.format('Kasper').upper()[0:3] # PRINT STEPS
total = l[0] + l[1] + l[2] # PRINT STEPS
```

```
<steps_widget.widget.StepsWidget object at 0x13a0402f0>
```

```
%%steps
```

```
3 + 2 * 4 + 9 # PRINT STEPS
```

```
20
```

```
<steps_widget.widget.StepsWidget object at 0x12a5c6850>
```

```
%%steps
x = 0
y = 3
result = x and y or 7 # PRINT STEPS
```

<steps_widget.widget.StepsWidget object at 0x12a5c6350>

Turtle cell magic

```
from turtle_widget import Turtle
```

```
t = Turtle()
t.speed(6)
for _ in range(4):
    t.forward(120)
    t.left(90)
```

<turtle_widget.widget.Turtle object at 0x13a040440>

```
import vscodebn
import seaborn as sns
data = sns.load_dataset('penguins')
data.head()
```

<IPython.core.display.HTML object>

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

Interactive plotting with Seaborn

```
from iplot_widget import iplot
```

```
iplot(data)
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

HBox(children=(HBox(children=(Box(children=(Dropdown(description='X:', options=('none', 'spec

Output()

Output()

Output()

Output()

```
import seaborn as sns
sns.set_style({'style': 'darkgrid', 'palette': 'viridis'})
g = sns.displot(data, x='bill_depth_mm', y='bill_length_mm', hue='sex', col='species', palette=
g.set_titles(col_template="{col_name}", row_template="{row_name}")
g.tight_layout() ;
```

